Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Kubernetes and Cloud Native Associate (KCNA) Training



WSQ Course · 5 Days · 40 Hours (37h training + 3h assessment)

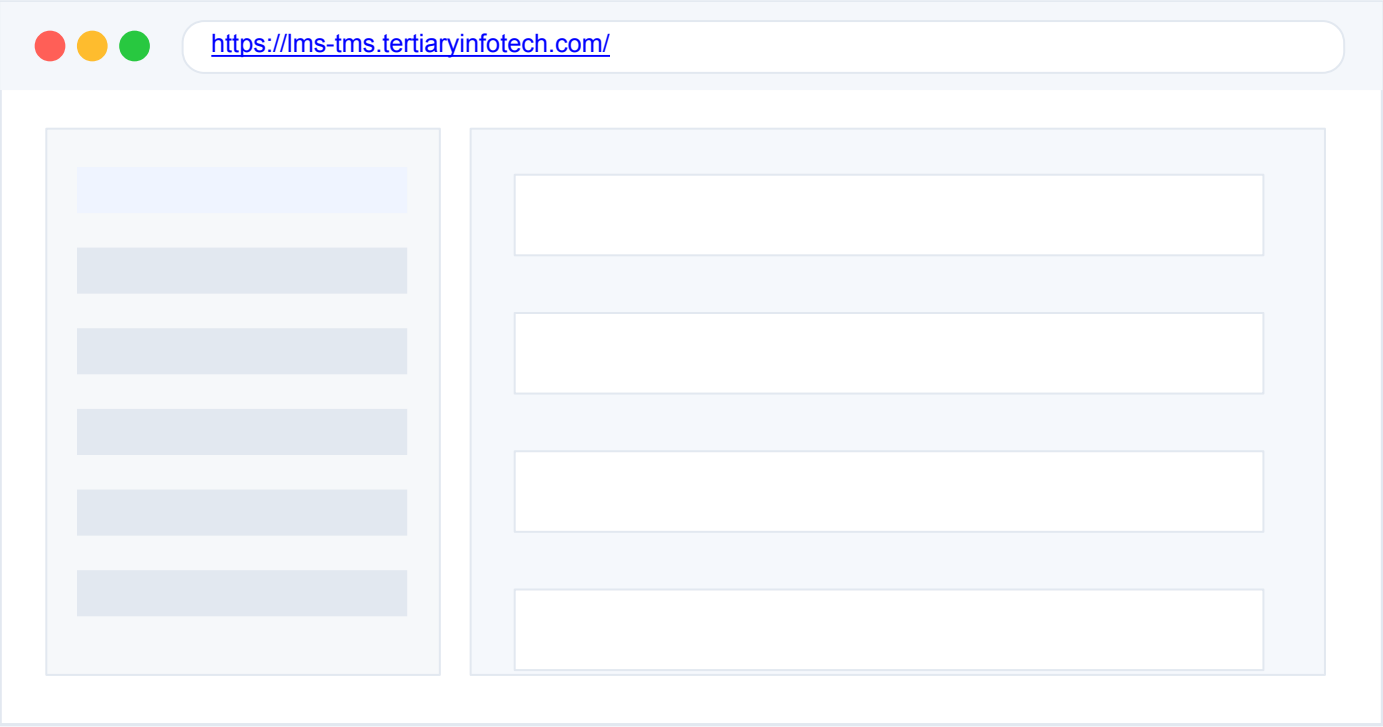
TGS REF NO	VERSION	TRAINER
TGS-2023039343	v6.1	Dr. Alfred Ang

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

Attendance and Learner Feedback

Attendance and TRAQOM feedback serve different purposes.



RESOURCE OVERVIEW

- Attendance records document participation in this course run.
- TRAQOM captures feedback on course quality and outcomes.
- Assessment attendance is a separate course record.
- The Learner Guide and trainer provide the run-specific procedure.

TRAQOM: TPGateway Training Quality and Outcomes Measurement initiative.

https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

ABOUT THE TRAINER

About Your Trainer

Template — to be completed by the delivering trainer.



[Trainer Name]

[Designation]

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

ABOUT THE TRAINER

Dr. Alfred Ang

Founder & Principal Trainer, Tertiary Infotech Academy



Dr. Alfred Ang

Founder & Principal Trainer, Tertiary Infotech Academy

ROLE

Founder & Principal Trainer, Tertiary Infotech Academy Pte Ltd

QUALIFICATIONS

PhD (NUS); ACTA-certified adult educator

AREAS OF EXPERTISE

Cloud native platforms, Kubernetes, DevOps, data engineering

DELIVERS

WSQ cloud, container and data engineering programmes

INDUSTRY EXPERIENCE

20+ years across engineering, consulting and technical training

CONTACT

enquiry@tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

LET'S KNOW EACH OTHER

Learner Introduction

Five days is long enough to learn each other's names — let's start there.

- Your name, and the organisation you work with.
- Your current role — developer, ops, SRE, architect, analyst or student.
- How much Kubernetes or container experience you already have (none is completely fine).
- One thing you want to be able to do by Friday evening.
- Whether you intend to sit the official KCNA exam.

© 2026 Tertiary Infotech Academy Pte Ltd

GROUND RULES

How We Will Work Together

Six working agreements for the week.

1 Be on time

Sessions restart on the clock. Attendance is recorded digitally and 75% is the funding threshold.

2 Ask immediately

If a concept did not land, say so at once. Every later topic is built on the earlier ones.

3 Hands on keyboards

The labs are where the learning actually happens. Type the commands, do not watch.

4 Break things deliberately

Several labs ask you to cause a failure. Reading real error output is an examinable skill.

5 Phones on silent

Take calls outside the room so the session is not interrupted.

6 Your own namespace

Each lab runs in its own namespace. Never delete objects you did not create.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE OUTLINE

The Five Days at a Glance

37 instructional hours plus 3 hours of assessment, across 5 days.

Day	Hours	Focus	Practical
Day 1	8 h	Cloud Native Foundations & Kubernetes Core Concepts	5 labs · Lab 01–Lab 05
Day 2	8 h	Workloads, Scheduling & Container Orchestration	5 labs · Lab 06–Lab 10
Day 3	8 h	Services, Networking & Cluster Security	6 labs · Lab 11–Lab 16
Day 4	8 h	Storage, Cluster Architecture & Application Delivery	5 labs · Lab 17–Lab 21
Day 5	5 h	Observability, Troubleshooting & Assessment	4 labs · Lab 22–Lab 25
Day 5	3 h	Assessment — Written Assessment (SAQ) + Practical Performance (PP)	WA 1 h · PP 2 h

CHOOSE

Every day ends with hands-on labs you keep and can re-run after the course.

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Will Be Able To Do

Registered WSQ outcomes for Solution Architecture (ICT-DES-4006-1.1). These are what the assessment measures.

1	**LO1**	Develop a Kubernetes architectural proof of concept.
2	**LO2**	Identify the technical and practical requirements in a Kubernetes setup.
3	**LO3**	Develop a solution architecture within Kubernetes.
4	**LO4**	Prepare a technical blueprint for a Kubernetes-based solution for security and storage.
5	**LO5**	Demonstrate Kubernetes solution for a specific business problem.
6	**LO6**	Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.

© 2026 Tertiary Infotech Academy Pte Ltd

BRIEFING FOR ASSESSMENT

How You Will Be Assessed

Read this before the assessment slide — you should know exactly what is coming.

1 Two instruments

A Written Assessment (SAQ) of 7 short-answer questions, and a Practical Performance of 6 hands-on tasks.

2 Timing

WA 1 hour, then PP 2 hours — 3 hours in total, on the afternoon of Day 5.

3 Open book

You may use your Learner Guide, the lab folders and the Kubernetes documentation.

4 Competency based

Graded Competent (C) or Not Yet Competent (NYC). There is no percentage mark and no ranking.

5 Everything is taught

Nothing is assessed that is not covered in these slides and the labs you will have completed.

6 If you are NYC

You will be counselled and offered a re-assessment. It is a gap to close, not a failure.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT & FUNDING

Assessment, Attendance and Funding

Attendance and assessment requirements apply alongside the published funding eligibility and terms.

Written Assessment (SAQ)	7 open-ended questions · 60 minutes
Practical Performance (PP)	6 hands-on tasks · 120 minutes
Total assessment time	3 hours
Grading	Competent (C) / Not Yet Competent (NYC)
Minimum attendance	75% of instructional hours

Attendance system	Course-run attendance record
Submission	Upload answers on the LMS
Certification	Statement of Attainment (SOA) from SSG on competency
Course reference	TGS-2023039343
Competency standard	Solution Architecture (ICT-DES-4006-1.1)

Source: Course page: https://www.tertiarycourses.com.sg/wsq-kubernetes-and-cloud-native-associate-kcna-training.html

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT OVERVIEW

Assessment Records and Evidence

The Learner Guide contains the detailed assessment procedure.

1 TRAQOM feedback

Course quality and outcomes feedback; separate from attendance.

2 Attendance

Course participation and assessment-session records.

3 Assessment

Written Assessment: 60 minutes. Practical Performance: 120 minutes.

4 Submitted evidence

Candidate responses and practical evidence on the LMS.

5 Assessor record

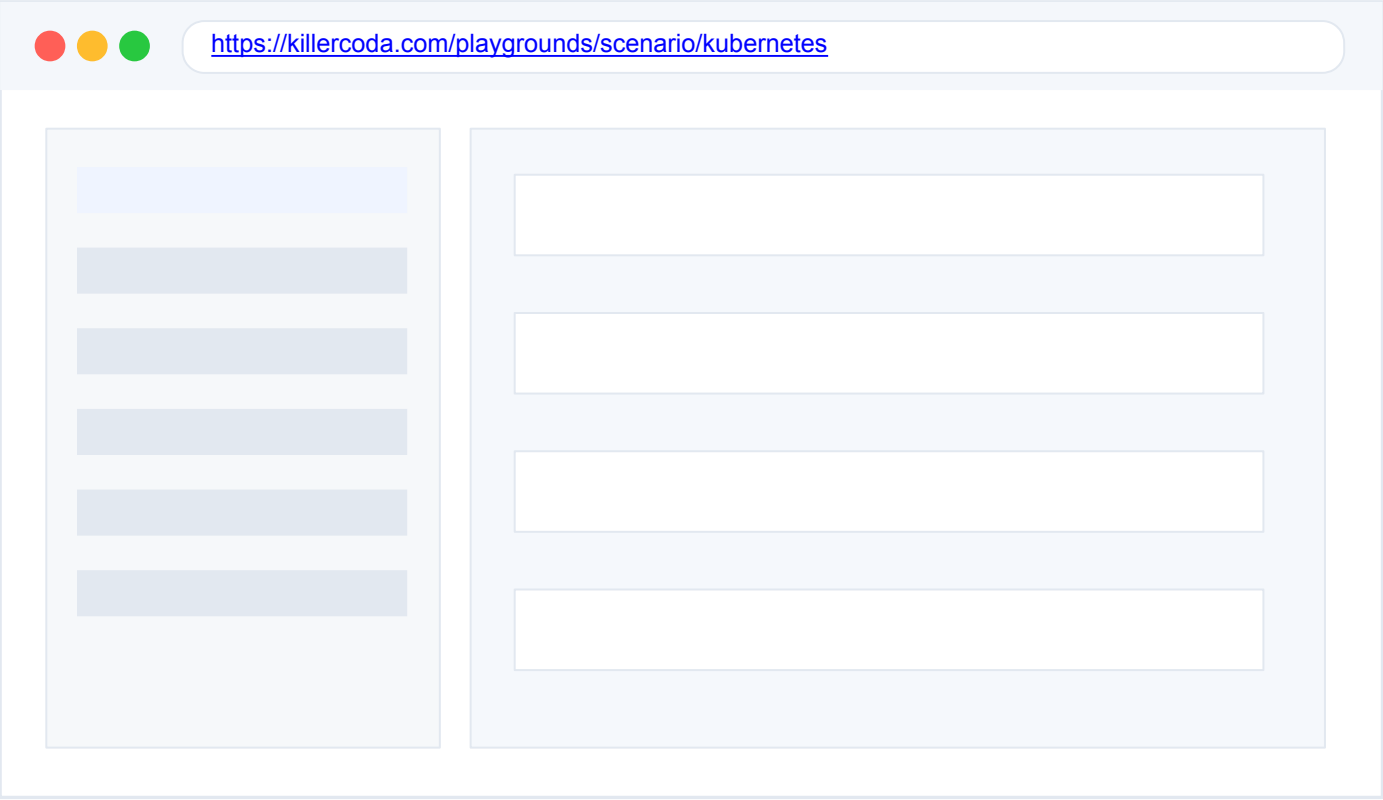
Competency decisions and the Assessment Summary Record.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

Your Kubernetes Playground

The full commands and detailed procedures are in the Learner Guide and lab READMEs.



RESOURCE OVERVIEW

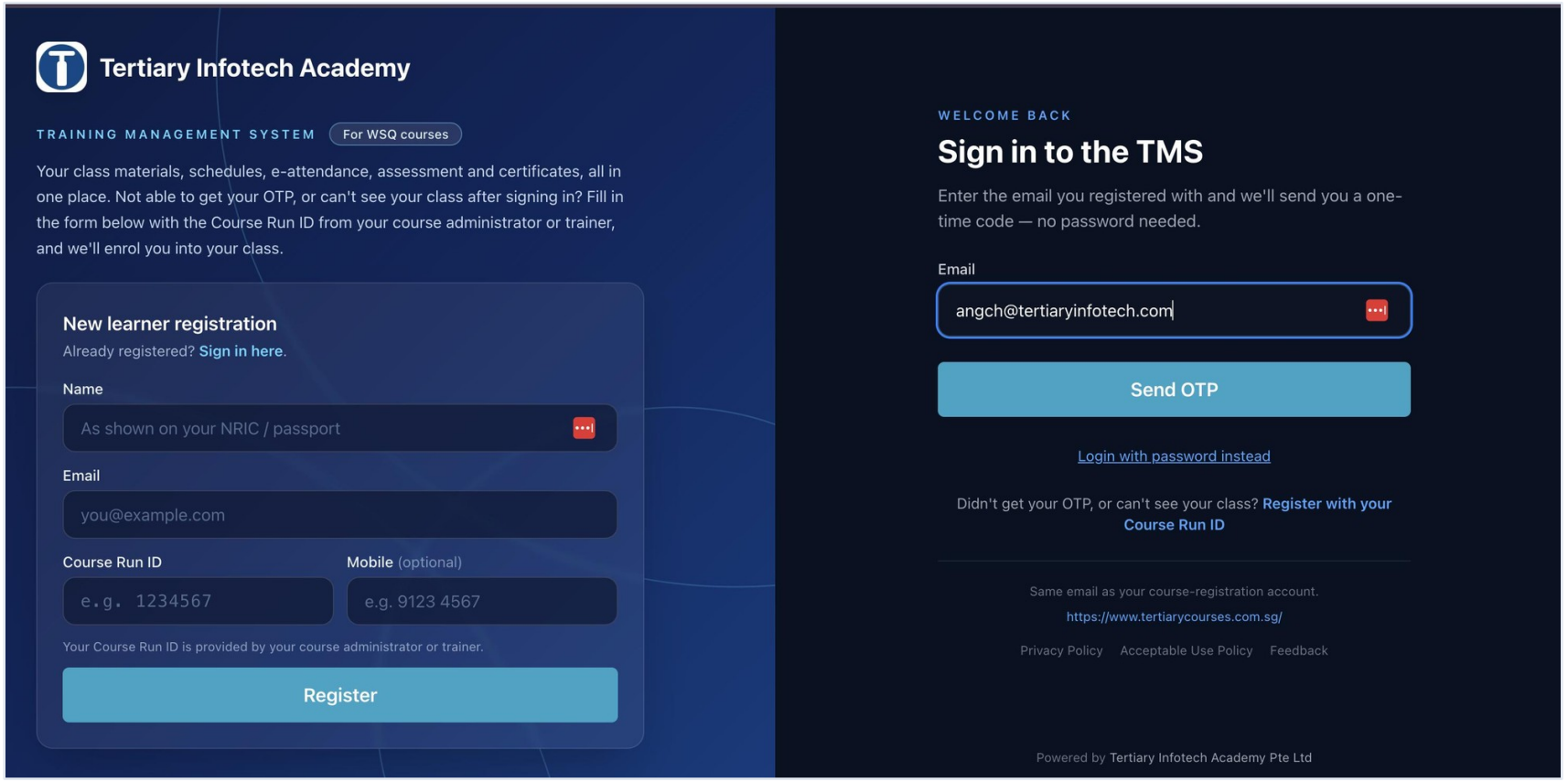
- Killercoda provides the recommended browser-based Kubernetes terminal.
- The public repository supplies 25 individual lab folders.
- Each lab includes realistic mock data and Kubernetes YAML.
- Verification scripts check the expected results.

https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Login to LMS and Download Course Material



Log in at lms-tms.tertiaryinfotech.com with your registered email and the OTP sent to your inbox, then open this course to download the slides and Learner Guide.

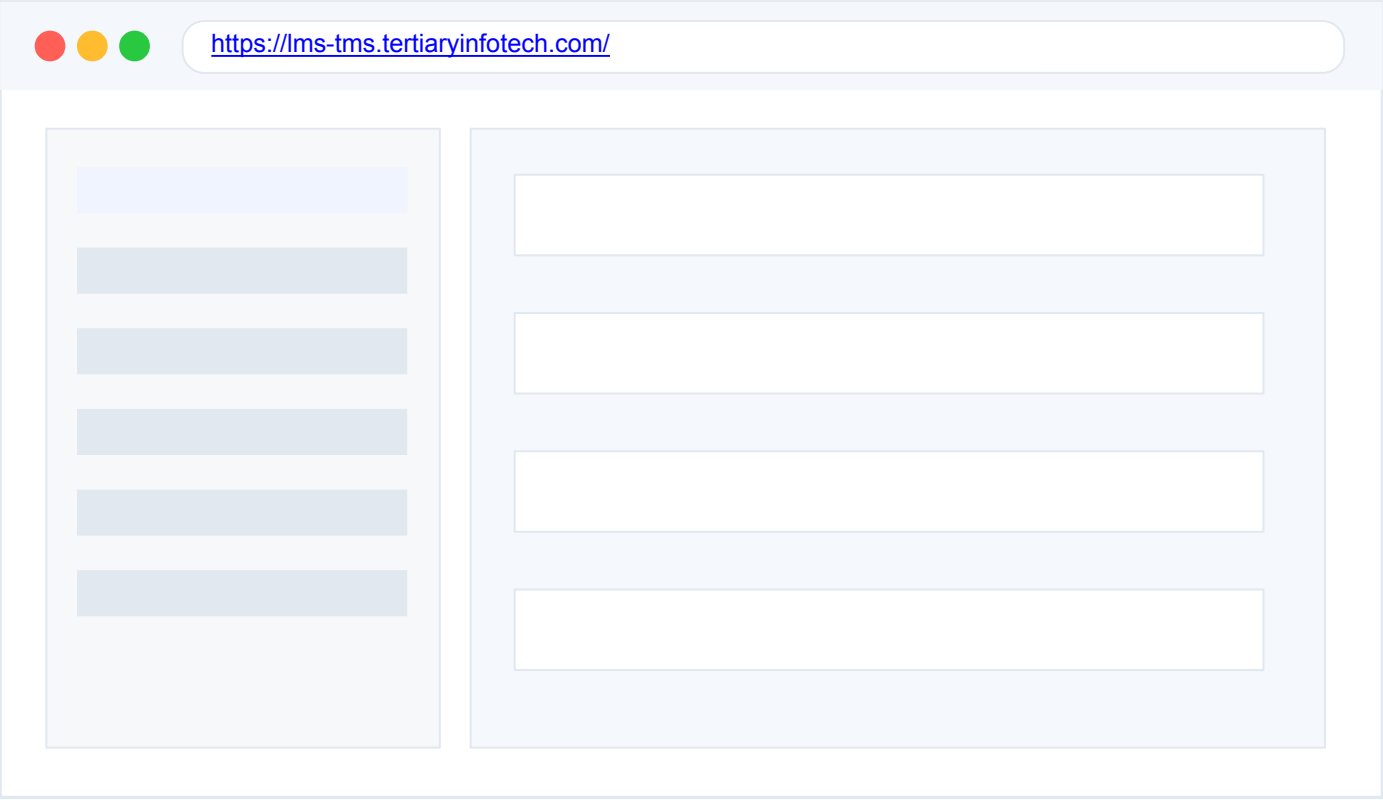
https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

Your Course Resources

Current materials are linked from the enrolled course on the LMS.



RESOURCE OVERVIEW

- PPT and PDF slides explain the mechanisms.
- The Learner Guide provides detailed procedures and reference material.
- Lab folders contain mock data, YAML and checks.
- Candidate papers and submission records support the WSQ assessment.

https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

KCNA Practice Resource

Practice material complements the course; the official curriculum remains authoritative.



RESOURCE OVERVIEW

- Supplementary practice supports concept review.
- This resource is separate from the official KCNA examination.
- The current CNCF curriculum defines the examinable domains.
- Practice results identify topics for further study.

https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 1 ROADMAP

Seven Modules, Five Labs, One Mental Model

Today builds the vocabulary the rest of the week assumes. Nothing after today re-explains a Pod.

1

1 · Cloud native and the CNCF

What the word means, who governs the projects, and why orchestration exists at all.

2

2 · Containerization

Namespaces, cgroups, union filesystems, the OCI specs and the CRI boundary the kubelet talks across.

3

3 · The Kubernetes API

Resources, groups, versions and verbs; the control loop; kubectl as an ordinary HTTPS client. Lab 01.

4

4 · Understanding Pods

Why a Pod is not a container, the Pod manifest field by field, and exposing ports. Lab 02.

5

5 · Commands, args and config

command/args against image ENTRYPOINT/CMD, env, envFrom and the downward API. Lab 03.

6

6 · Multi-container patterns

Sidecar, adapter, ambassador, init containers and native sidecars. Lab 04.

7

7 · Images and registries

Reference anatomy, digests, imagePullPolicy and the pull path through the CRI. Lab 05.

8

Assessment thread

Every module produces evidence you will reuse in the LO1 proof-of-concept task on Day 5.

© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN

Day 1 — Cloud Native Foundations & Kubernetes Core Concepts

8.0 hours of instructional time. Breaks are not counted as instructional time.

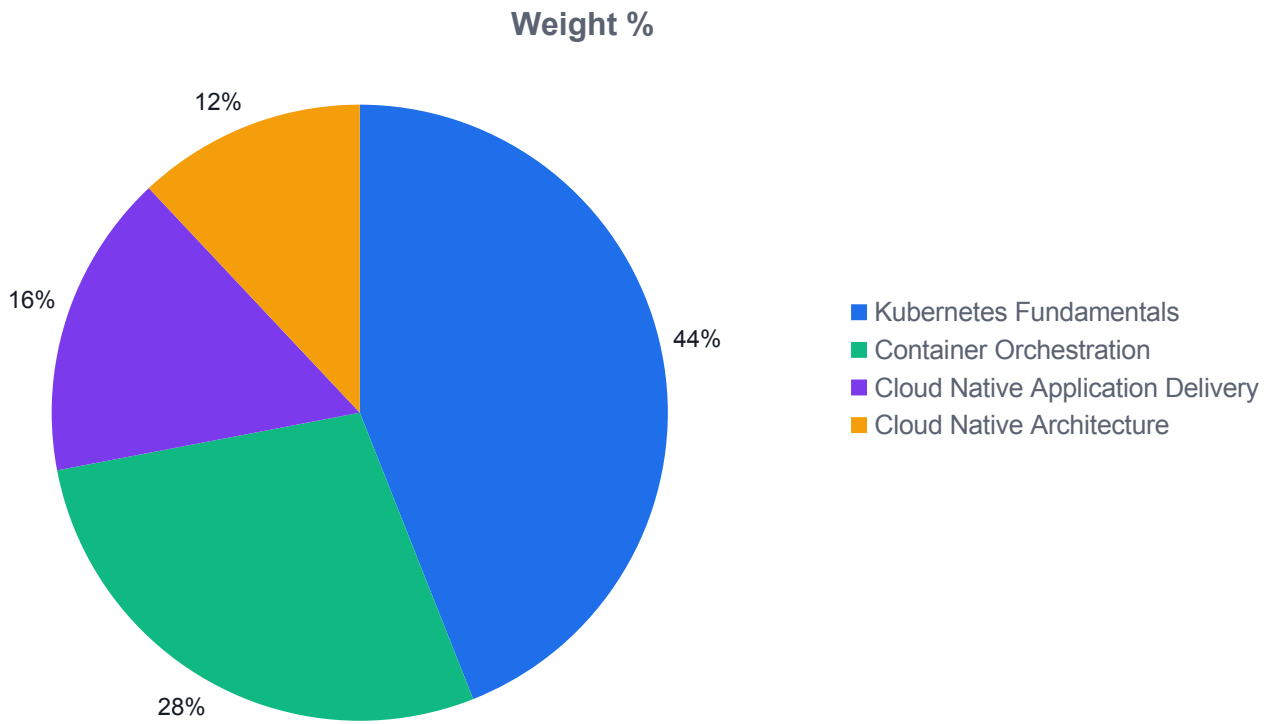
Time	Duration	Session	Method
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

© 2026 Tertiary Infotech Academy Pte Ltd

CERTIFICATION TARGET

The KCNA Exam Blueprint You Are Training Against

Four domains, effective 24 November 2025. This is the current blueprint — not the superseded five-domain split.



WHAT THE DATA SHOWS

44% Kubernetes Fundamentals

Core concepts, administration, scheduling and containerization. Day 1 and Day 2 own this domain.

28% Container Orchestration

Networking, security, troubleshooting and storage — Days 3, 4 and 5.

16% Application Delivery

Packaging, GitOps and debugging; Day 4.

12% Cloud Native Architecture

Observability, ecosystem principles, community. Opens today, closes on Day 5.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

CERTIFICATION TARGET

KCNA Exam Logistics

Book the exam while the course material is fresh; the eligibility window is long but recall is not.

Format	Online, proctored, multiple-choice
Questions	60
Duration	90 minutes
Passing score	75%

Validity	2 years
Prerequisites	None
Retake	One retake included with the exam purchase
Eligibility window	12 months from purchase

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026. Linux Foundation exam policies.

© 2026 Tertiary Infotech Academy Pte Ltd

WSQ OUTCOMES

What This Course Certifies You Can Do

TSC Solution Architecture ICT-DES-4006-1.1. Day 1 carries LO1 — the architectural proof of concept.

LO1

1 Develop a Kubernetes architectural proof of concept.

LO2

2 Identify the technical and practical requirements in a Kubernetes setup.

LO3

3 Develop a solution architecture within Kubernetes.

LO4

4 Prepare a technical blueprint for a Kubernetes-based solution for security and storage.

LO5

5 Demonstrate Kubernetes solution for a specific business problem.

LO6

6 Implement regular monitoring of the Kubernetes system and perform necessary troubleshooting.

Source: Approved TSC mapping for TGS-2023039343 (course_data.LO_MAP).

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 1 CONTRACT

By 6:30 PM You Will Have Produced This

LO1 asks for a proof of concept. A PoC is evidence, not opinion — so every module ends with something you can show.

CONCEPTS YOU WILL BE ABLE TO DEFEND

- Why orchestration exists and what problem each layer solved
- What a container is at the Linux level, and what OCI standardises
- How the API server, etcd, scheduler and kubelet divide the work
- Why the Pod — not the container — is the unit of scheduling
- How a manifest becomes a running process on a node

ARTEFACTS YOU WILL HAVE ON DISK

- A reconnaissance record of an unfamiliar cluster (Lab 01)
- A hand-authored Pod manifest that round-trips (Lab 02)
- An ENTRYPOINT/CMD override matrix with evidence (Lab 03)
- Four multi-container Pods, one per pattern (Lab 04)
- A digest-pinned image reference and its resolved imageID (Lab 05)

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 1 · CLOUD NATIVE ARCHITECTURE

01

What 'Cloud Native' Actually Means

The definition, the foundation that owns it, and the reason orchestration had to be invented.

© 2026 Tertiary Infotech Academy Pte Ltd

DEFINITION

Cloud native names a set of operating properties — not a product, a vendor or a hosting location.

The CNCF definition is written entirely in terms of how a system behaves under change: loosely coupled, resilient, manageable, observable.

© 2026 Tertiary Infotech Academy Pte Ltd

DEFINITION

Reading the CNCF Definition Clause by Clause

Four adjectives do the real work. Learn them: they are the vocabulary the KCNA Cloud Native Architecture domain uses.

1 Scalable apps in dynamic environments

Public, private and hybrid clouds are all in scope. The definition is deliberately not tied to a provider.

2 Exemplar technologies

Containers, service meshes, microservices, immutable infrastructure and declarative APIs — examples, not requirements.

3 Loosely coupled

Components fail and are replaced independently. A restart of one part must not require a coordinated restart of the whole.

4 Resilient

The system is expected to lose instances routinely and to keep serving; recovery is automated, not a runbook.

5 Manageable

Behaviour is changed by changing declared state, not by logging into a machine and editing files.

6 Observable

The system emits enough signal — metrics, logs, traces, events — for an operator to reason about it from outside.

Source: CNCF Cloud Native Definition v1.1 — github.com/cncf/toc, DEFINITION.md.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY ORCHESTRATION EXISTS

Four Deployment Eras and the Problem Each One Left Behind

This is the standard KCNA narrative and it is also the honest engineering history.

STAGE 1

Bare metal

One app per machine to avoid interference. Utilisation is poor and provisioning takes weeks.

STAGE 2

Virtual machines

A hypervisor multiplexes hardware. Utilisation improves, but each guest carries a whole kernel and OS image.

STAGE 3

Containers

Kernel-level isolation with a shared kernel. Start-up is process-speed and images are far smaller — but nothing places them.

STAGE 4

Orchestration

A control plane decides placement, restarts failures, rolls updates and reconciles declared state continuously.

WHAT IT MEANS

Each era solved the previous era's waste and exposed a new gap. Kubernetes exists to close the placement, healing and rollout gap that containers alone left open.

Source: Kubernetes documentation — What is Kubernetes / Why you need Kubernetes: kubernetes.io/docs/concepts/overview/.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

What Actually Changes Between the Four Eras

Note the last row: the unsolved problem is identical in all three columns.

Property	Bare metal	Virtual machine	Container
Isolation boundary	Physical machine	Hypervisor + guest kernel	Kernel namespaces + cgroups
Kernel	One per machine	One per guest	Shared with the host
Unit shipped	OS build + config management	Disk image (GB scale)	Layered OCI image (MB scale, layers shared)
Start-up work	Boot firmware, kernel, init, services	Boot guest kernel and init	Create namespaces, mount rootfs, exec the process
Density on one host	One workload	Tens of guests	Hundreds of processes
What still is not solved	Placement, healing, rollout	Placement, healing, rollout	Placement, healing, rollout — hence Kubernetes

CHOOSE

Containers changed the packaging and the isolation cost. They did not change who decides where a workload runs.

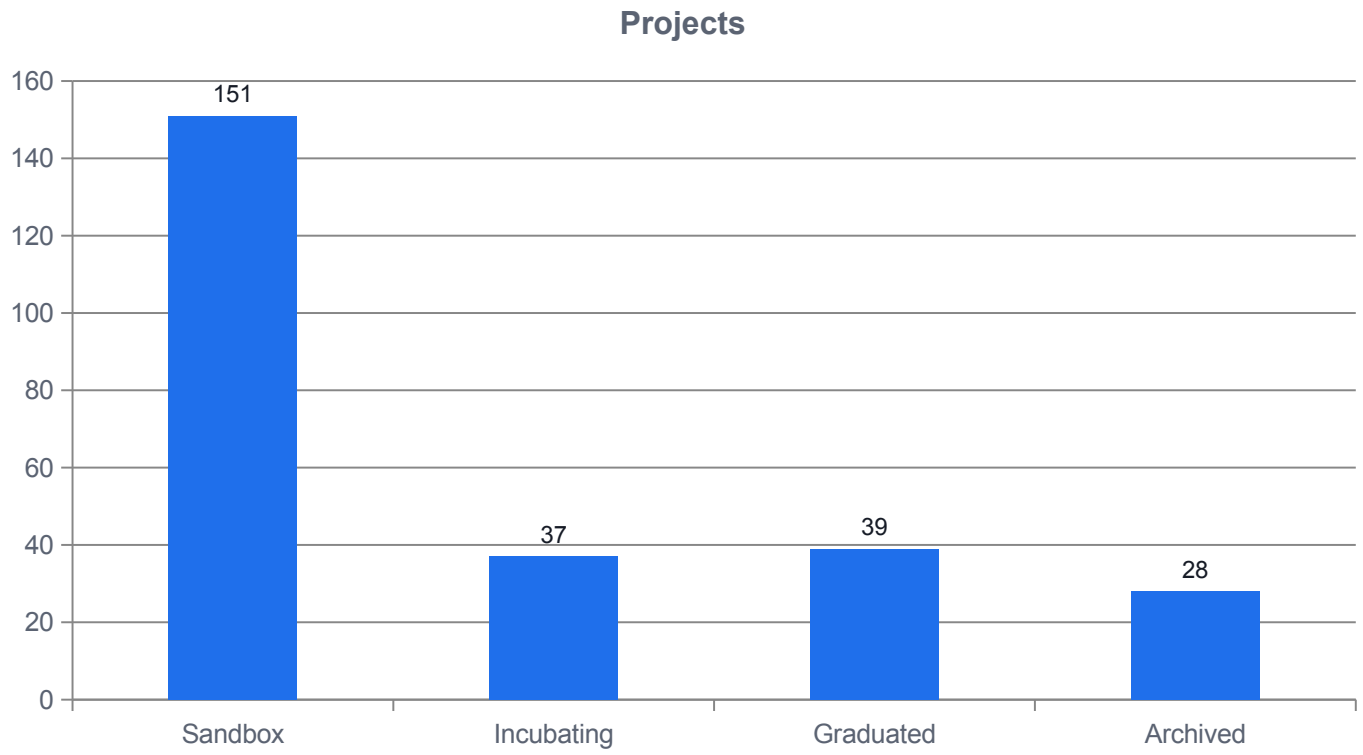
Source: Linux man-pages: namespaces(7), cgroups(7), overlayfs kernel docs. Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

THE ECOSYSTEM

CNCF Hosted Projects by Maturity Level

Counted directly from the landscape source file on 5 September 2026 — 227 currently hosted projects plus 28 archived.



WHAT THE DATA SHOWS

Sandbox is the widest gate

151 projects. Early-stage, experimental, no production expectation — do not read inclusion as endorsement.

Incubating is the proving ground

37 projects with demonstrated production users and a healthy contributor base across organisations.

Graduated is the trust signal

39 projects, including Kubernetes, containerd, etcd, Prometheus, Helm, Envoy, CoreDNS and cert-manager.

Archived is a real outcome

28 projects have been retired. Maturity level can move backwards; check it before you build on a project.

Source: Counted from github.com/cncf/landscape landscape.yml (project maturity field), retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

THE ECOSYSTEM

What Each Maturity Level Actually Requires

These are the criteria a project must satisfy to move up — worth knowing because KCNA asks about them directly.

1 Sandbox

Entry point for early innovation. Accepted by the TOC on a written proposal; no adoption or governance bar to clear yet.

2 Incubating

Must show production use by independent adopters, a substantial ongoing flow of commits and a committer base beyond one employer.

3 Graduated

Adds a completed independent security audit, an explicit governance and code-of-conduct document, and CII/OpenSSF best practices.

4 Archived

The TOC retires a project that is no longer maintained. The code stays; the CNCF marketing, events and services stop.

5 Levels are reviewed, not permanent

Projects move up on application to the TOC and can be moved to archived. Treat the level as current state, not a badge.

6 What the level does not tell you

Fit for your problem. Graduated Vitess is excellent and is still the wrong answer if you are not running MySQL at scale.

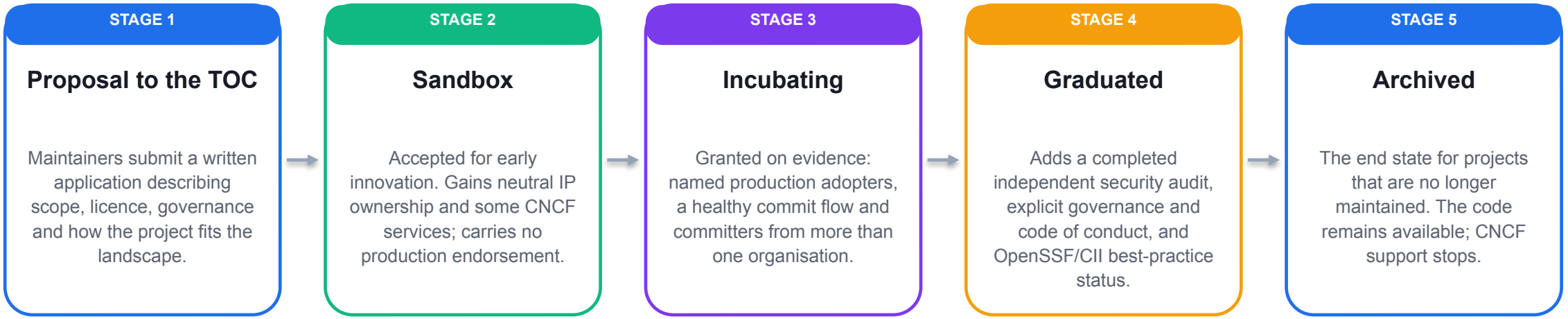
Source: CNCF Technical Oversight Committee graduation criteria — github.com/cncf/toc/blob/main/process/graduation_criteria.md.

© 2026 Tertiary Infotech Academy Pte Ltd

THE ECOSYSTEM

How a Project Moves Through the Maturity Levels

The TOC votes at each transition. Nothing is automatic and nothing is permanent.



WHAT IT MEANS

Level is evidence about the community around a project, not about how good the code is. Read it as a risk signal when you choose a dependency for a platform.

Source: CNCF Technical Oversight Committee graduation criteria — github.com/cncf/toc/blob/main/process/graduation_criteria.md.

© 2026 Tertiary Infotech Academy Pte Ltd

GOVERNANCE

What the CNCF Does — and What It Deliberately Does Not Do

Confusing 'CNCF hosted' with 'CNCF endorsed for your use case' is the most expensive mistake in this domain.

IT DOES

- Hold trademarks and provide neutral IP ownership for hosted projects
- Run the TOC process that accepts, promotes and archives projects
- Fund shared services: security audits, CI credits, documentation and design help
- Run KubeCon + CloudNativeCon and the certification programme (KCNA, CKA, CKAD, CKS)
- Publish the landscape and the conformance programmes

IT DOES NOT

- Write the code — maintainers do, employed by many different organisations
- Set a single technical roadmap across projects
- Guarantee that any hosted project is production-ready for you
- Certify vendors' products; conformance is specific and narrow (for example Certified Kubernetes)
- Prevent a project from being archived when its community stops

Source: CNCF Charter and governance pages — cncf.io/about/charter/; Kubernetes community structure — github.com/kubernetes/community. CNCF Technical Oversight Committee graduation criteria — github.com/cncf/toc/blob/main/process/graduation_criteria.md.

© 2026 Tertiary Infotech Academy Pte Ltd

GOVERNANCE

Who Decides What, in the CNCF and in Kubernetes

KCNA's community domain tests whether you know who owns which decision. Vendors do not own the technical roadmap.

Governing Board	Business oversight: budget, marketing, legal. It does not make technical decisions.
Technical Oversight Committee (TOC)	Accepts, promotes and archives projects; defines the technical vision. Elected, not appointed by vendors.
Technical Advisory Groups (TAGs)	Domain groups the TOC delegates to — for example runtime, network, security, observability, app delivery.
End User community	Adopting organisations feed requirements back to the TOC; the End User Technical Advisory Board coordinates this.
Project maintainers	Each hosted project keeps its own governance document; the CNCF does not run day-to-day project decisions.
Kubernetes SIGs	Kubernetes itself is organised into Special Interest Groups (sig-node, sig-network, sig-storage, sig-auth, …).
Kubernetes Working Groups	Time-boxed, cross-SIG efforts for problems that do not fit one SIG's charter.
KubeCon + CloudNativeCon	The flagship conference; also where maintainer tracks, project meetings and contributor summits happen.

Source: CNCF Charter and governance pages — cncf.io/about/charter/; Kubernetes community structure — github.com/kubernetes/community.

© 2026 Tertiary Infotech Academy Pte Ltd

THE ECOSYSTEM

Graduated Projects You Will Meet This Week

All twelve are CNCF graduated projects. Marks are used under the Linux Foundation trademark usage guidelines.



Kubernetes

Orchestration — the whole week



containerd

CRI runtime — today, Module 2



CRI-O

Alternative CRI runtime — today



etcd

Cluster datastore — today and Day 4



CoreDNS

Cluster DNS — Day 3



Cilium

CNI and network policy — Day 3



Helm

Packaging — Day 4



Argo CD

GitOps delivery — Day 4



Prometheus

Metrics — Day 5



OpenTelemetry

Traces, metrics, logs — Day 5



Fluentd

Log pipeline — Day 5



Envoy

L7 proxy, mesh data plane — Day 3

Source: Counted from github.com/cncf/landscape landscape.yml (project maturity field), retrieved 5 Sep 2026. Artwork: github.com/cncf/artwork.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY ORCHESTRATION

Three Jobs No Container Runtime Does For You

Everything Kubernetes adds over 'docker run' falls into one of these three buckets. Say which one when you justify the PoC.

Placement

- A runtime starts a container where you tell it to.
- A scheduler decides which node can actually host it — CPU, memory, taints, affinity, topology.
- It re-decides every time a Pod is created.

kube-scheduler | spec.nodeName

Healing

- A runtime restarts a process if you configure it to.
- A controller compares observed replicas against declared replicas and creates replacements on surviving nodes.
- Node loss is an ordinary, handled event.

controller-manager | reconciliation

Change

- A runtime has no idea what 'version 2' means.
- A controller sequences a rollout, watches readiness, and stops or reverses on failure.
- The declared state is the record of intent.

Deployment | rollout status

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

COMMUNITY

The Roles Around a Cloud Native Platform

KCNA's community domain expects you to recognise these roles and the boundaries between them.

1 Application developer

Owns the container image and the workload manifest. Should not need to know how the CNI plugin is configured.

2 Platform engineer

Owns the cluster as a product: namespaces, quotas, base policy, golden paths and the tooling other teams consume.

3 Site reliability engineer

Owns service level objectives, on-call, capacity and the observability pipeline that makes those measurable.

4 Security engineer

Owns RBAC, admission policy, supply chain provenance, secrets handling and network isolation.

5 Architect

Owns the trade-off record: what is standardised, what is allowed to vary, and why. That is this course's LO3.

6 Contributor and end user

The CNCF explicitly routes adopter feedback into project direction — participation is part of the model, not charity.

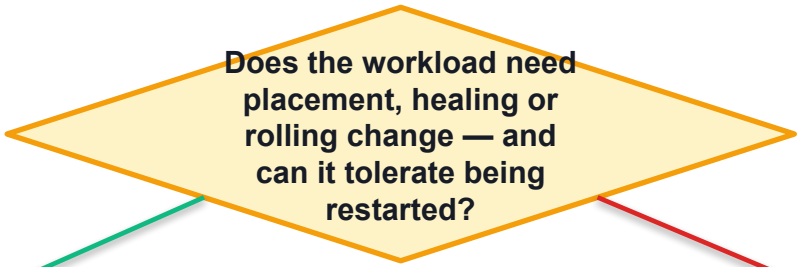
Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026. CNCF Charter and governance pages — cncf.io/about/charter/; Kubernetes community structure — github.com/kubernetes/community.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN JUDGEMENT

Does This Workload Belong on Kubernetes?

A proof of concept that recommends Kubernetes for everything is not an architecture; it is an advertisement.



YES

NO

Yes — orchestrate it

- Stateless or state-externalised services
- Anything horizontally scaled or frequently released
- Batch work that must be retried and tracked
- Workloads that must survive node loss without a human

No — or not yet

- Single-instance appliances with licence pinning to hardware
- Workloads whose vendor forbids containerisation or restarts
- Latency paths needing kernel or device access you cannot grant
- Anything where one VM and a cron job is genuinely sufficient

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

VOCABULARY

Six Terms KCNA Uses Precisely — and Most People Do Not

Exam questions are frequently decided by one of these distinctions.

1 Cloud native ≠ cloud hosted

The definition never mentions where the machines are. An on-premises cluster can be cloud native; a lifted VM is not.

2 Microservices ≠ cloud native

Microservices are one exemplar technology in the definition. A well-run modular monolith on Kubernetes still qualifies.

3 Orchestration ≠ runtime

containerd runs containers. Kubernetes decides which containers should exist and where. Two different jobs, two different APIs.

4 Declarative ≠ configuration file

Declarative means a controller continuously reconciles toward the declared state, not that the state lives in YAML.

5 Immutable infrastructure

You replace instances rather than modify them. This is why in-place patching a running container is an anti-pattern.

6 Serverless

A consumption and scaling model, not the opposite of containers — Knative runs serverless workloads as Pods.

Source: CNCF Cloud Native Definition v1.1 — github.com/cncf/toc, DEFINITION.md.

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 2 · CONTAINERIZATION

02

What a Container Actually Is

Linux namespaces, cgroups and union filesystems; the three OCI specifications; and the CRI boundary.

© 2026 Tertiary Infotech Academy Pte Ltd

LINUX MECHANISM

A Container Is a Process With a Restricted View — Namespaces

Eight kinds of namespace exist in current Linux kernels. 'Container' is not a kernel object — this list is.

1. **mount (mnt)**
   Its own filesystem tree. This is why the container sees the image's rootfs and not the host's /.

2. **process (pid)**
   Its own PID numbering starting at 1. The container's PID 1 is an ordinary host process seen from outside.

3. **network (net)**
   Its own interfaces, routes, ports and iptables/nftables tables. Every Pod gets exactly one of these.

4. **interprocess (ipc)**
   Its own System V IPC and POSIX message queues. Containers in a Pod can be made to share this one.

5. **hostname (uts)**
   Its own hostname and domain name — which is why $HOSTNAME inside a Pod is the Pod name.

6. **user**
   Its own uid/gid mapping, so root inside can be an unprivileged uid outside. Kubernetes support is via user namespaces.

7. **cgroup**
   Hides the host's cgroup hierarchy so the container sees its own limits at the root of /sys/fs/cgroup.

8. **time**
   Isolates the CLOCK_MONOTONIC and boot time offsets. Rarely used, but it completes the list.

Source: Linux man-pages: namespaces(7), cgroups(7), overlayfs kernel docs.

© 2026 Tertiary Infotech Academy Pte Ltd

LINUX MECHANISM

cgroups v2 — Accounting and Limiting, Not Isolating

Namespaces answer 'what can I see'. cgroups answer 'how much can I use'. Both are required to make a container.

1 cpu controller

Weight and bandwidth. Kubernetes maps requests.cpu to a weight and limits.cpu to a bandwidth quota per period.

2 memory controller

High and max watermarks. Exceeding memory.max gets the process OOM-killed — surfacing as exit code 137 / OOMKilled.

3 io controller

Weights and throughput caps per block device. Relevant to noisy-neighbour disk problems.

4 pids controller

Caps the number of processes so one container cannot fork-bomb the node.

5 Unified hierarchy

cgroup v2 replaced v1's separate per-controller trees with one hierarchy; Kubernetes has supported it as the default path since v1.25.

6 Where Kubernetes puts you

The kubelet builds a cgroup path per QoS class and per Pod, so node pressure decisions can be made per Pod, not per process.

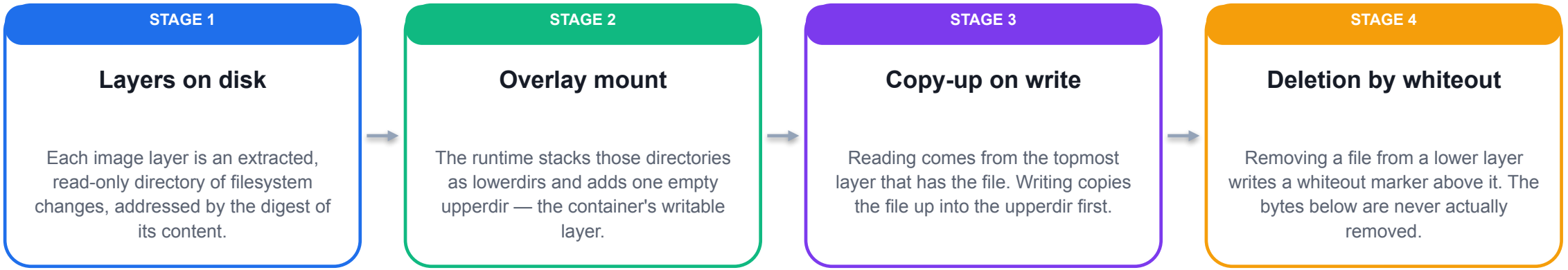
Source: Linux man-pages: namespaces(7), cgroups(7), overlayfs kernel docs. Kubernetes documentation — cgroup v2: kubernetes.io/docs/concepts/architecture/cgroups/.

© 2026 Tertiary Infotech Academy Pte Ltd

LINUX MECHANISM

Union Filesystems: Why Image Layers Are Cheap

This is also why a file 'deleted' in a later layer is still extractable from the image — a real supply-chain trap.



WHAT IT MEANS

Two images sharing a base layer share it once on disk, and the container's writable layer is empty at start. That is why container start-up costs a mount, not a copy.

Source: Linux man-pages: namespaces(7), cgroups(7), overlayfs kernel docs. Open Container Initiative Image Format Specification — github.com/opencontainers/image-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

The Runtime Stack Below the kubelet

Four boundaries, three of them standardised. Know which component sits on which side of each.

KUBERNETES	**kubelet** Node agent. Owns Pod lifecycle; the only Kubernetes component that talks to a runtime	**CRI (gRPC)** RuntimeService + ImageService. The stable contract; not a program you install	**Pod sandbox** RunPodSandbox creates the shared network and IPC namespaces before any app container
CONTAINER RUNTIME	**containerd** CRI-conformant high-level runtime; manages images, snapshots and container lifecycle	**CRI-O** Alternative high-level runtime, built specifically for Kubernetes	**Snapshotter** Assembles the overlay mount for the container rootfs from image layers
OCI LAYER	**OCI bundle** A directory: config.json plus the rootfs the snapshotter mounted	**runc / crun** Low-level runtime. Reads config.json, creates namespaces and cgroups, execs the process	**Registry** Distribution-spec HTTP API the runtime pulls manifests and layers from

MECHANISM

kubelet → CRI → containerd → OCI bundle → runc. Each arrow is a published contract, which is why the runtime is swappable and why Docker Engine's removal in v1.24 changed nothing about your images.

Source: Kubernetes documentation — Container Runtime Interface: kubernetes.io/docs/concepts/architecture/cri/. Open Container Initiative Runtime Specification — github.com/opencontainers/runtime-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

STANDARDS

The Open Container Initiative — Three Specifications

The last tile is the single most commonly missed distinction in this part of the syllabus.

1 image-spec

Defines the image index, the image manifest, the image config and the layer media types. Answers 'what is an image?'

2 runtime-spec

Defines the filesystem bundle and config.json: process, rootfs, mounts, namespaces, cgroups. Answers 'how do I run it?'

3 distribution-spec

Defines the registry HTTP API — /v2/, manifest and blob endpoints, content ranges. Answers 'how do I fetch it?'

4 Why three and not one

Building, running and distributing are done by different programs. Splitting the contracts is what allows substitution.

5 Conformance is testable

Each specification ships a conformance suite, which is how 'OCI-compliant' becomes a checkable claim rather than marketing.

6 CRI is not OCI

CRI is a Kubernetes gRPC API between kubelet and runtime. OCI specifications are vendor-neutral formats. Do not confuse them.

Source: Open Container Initiative Image Format Specification — github.com/opencontainers/image-spec. Open Container Initiative Runtime Specification — github.com/opencontainers/runtime-spec. Open Container Initiative Distribution Specification — github.com/opencontainers/distribution-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Reading an OCI Image Manifest

Digests are abbreviated for the slide; in a real manifest each is a full sha256 of 64 hex characters.

OCI image manifest (structure per image-spec)

```
{
  "schemaVersion": 2,
  "mediaType":
     "application/vnd.oci.image.manifest.v1+json",
  "config": {
    "mediaType":
       "application/vnd.oci.image.config.v1+json",
    "digest": "sha256:b5b2b2c5...", "size": 7023
  },
  "layers": [
    { "mediaType":
        "application/vnd.oci.image.layer.v1.tar+gzip",
      "digest": "sha256:9834876d...",
      "size": 32654 },
    { "digest": "sha256:3c3a4604...",
      "size": 16724 }
  ]
}
```

READING IT

1. **An image is a JSON document**
   Not a file and not a tarball. The manifest is the thing a digest actually names.

2. **config is where ENTRYPOINT lives**
   The referenced config blob carries Entrypoint, Cmd, Env, User, WorkingDir and rootfs.diff_ids — the fields Lab 03 overrides.

3. **layers is an ordered list**
   Applied bottom to top. Each entry is content-addressed, so changing any byte changes the digest all the way up.

4. **An index sits above this**
   For multi-platform images an image index maps linux/amd64 and linux/arm64 to different manifests.

Source: Open Container Initiative Image Format Specification — github.com/opencontainers/image-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Reading an OCI Runtime config.json

Everything you write in a Pod spec ends up here or is discarded. Reading this file demystifies the whole stack.

OCI runtime bundle config.json (abridged)

```
{
  "ociVersion": "1.2.0",
  "process": {
    "user": { "uid": 0, "gid": 0 },
    "args": ["nginx", "-g", "daemon off;"],
    "cwd": "/",
    "capabilities": { "bounding": ["CAP_NET_BIND_SERVICE"] }
  },
  "root": { "path": "rootfs", "readonly": false },
  "linux": {
    "namespaces": [
      { "type": "pid" },  { "type": "ipc" },
      { "type": "uts" },  { "type": "mount" },
      { "type": "network", "path": "/var/run/netns/..." }
    ],
    "resources": { "memory": { "limit": 134217728 } }
  }
}
```

READING IT

1. **This is what runc is handed**
   The bundle is one directory: this file plus the rootfs the snapshotter mounted. Nothing else.

2. **args came from the image config**
   …unless the Pod spec overrode it. This is the exact join point between Kubernetes command/args and the image.

3. **network has a path, the others do not**
   A path means 'join this existing namespace'. That is how the second container in a Pod gets the Pod IP.

4. **resources mirrors the cgroup**
   134217728 bytes is 128Mi — the memory limit written in the Pod spec, translated all the way down.

Source: Open Container Initiative Runtime Specification — github.com/opencontainers/runtime-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Image Versus Container — Two Different Nouns

Lab 05 makes you read both fields on the same Pod and compare them.

Question	Image	Container
What is it	An immutable, content-addressed set of JSON documents and layer blobs	A running (or exited) process with namespaces, cgroups and a writable layer
Where does it live	In a registry and in the node's local content store	On exactly one node, inside one Pod sandbox
Identity	A digest — sha256 of the manifest; a tag is just a mutable pointer at one	A container ID assigned by the runtime; visible as status.containerStatuses[].containerID
Can it change	No. Any change produces a different digest	Yes — the writable layer is thrown away when the container is replaced
Kubernetes field	spec.containers[].image (what you asked for)	status.containerStatuses[].imageID (what actually resolved)

CHOOSE

If you cannot say which of the two a question is about, you cannot answer it. spec is the image; status is the container.

Source: Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/. Open Container Initiative Image Format Specification — github.com/opencontainers/image-spec.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

The Node's Image Store, Read Through the API

Everything you need to reason about images on a node is already in the API.

terminal — training cluster, 5 Sep 2026

```
$ kubectl get nodes -o jsonpath='{range .items[0].status.images[*]}\
    {.sizeBytes}{"\t"}{.names[0]}{"\n"}{end}' | sort -rn

35652389   docker.io/kindest/kindnetd:v20260820-69b56db7
21832241   docker.io/library/nginx@sha256:65645c7bb6a0...
21239459   registry.k8s.io/coredns/coredns:v1.14.6
20455189   registry.k8s.io/etcd:3.7.0-0
14217939   docker.io/kindest/local-path-provisioner:v20260820
 4103199   docker.io/library/alpine@sha256:d9e853e87e55...
 1906887   docker.io/library/busybox@sha256:73aaf090f3d8...
  267933   registry.k8s.io/pause:3.10
```

WHAT TO LOOK AT

1. **You never touched the node**
   The kubelet reports its content store to the API server; the data is in Node.status.images. No SSH, no crictl.

2. **Entries are digest-addressed**
   The store is a content-addressed cache. Tags shown here are names attached to a digest, not the identity itself.

3. **pause is the smallest thing here**
   267 933 bytes. It is the sandbox container that holds every Pod's network namespace open.

4. **sizeBytes is per image, not per node**
   Images sharing a base layer store it once, so these numbers do not sum to disk usage.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

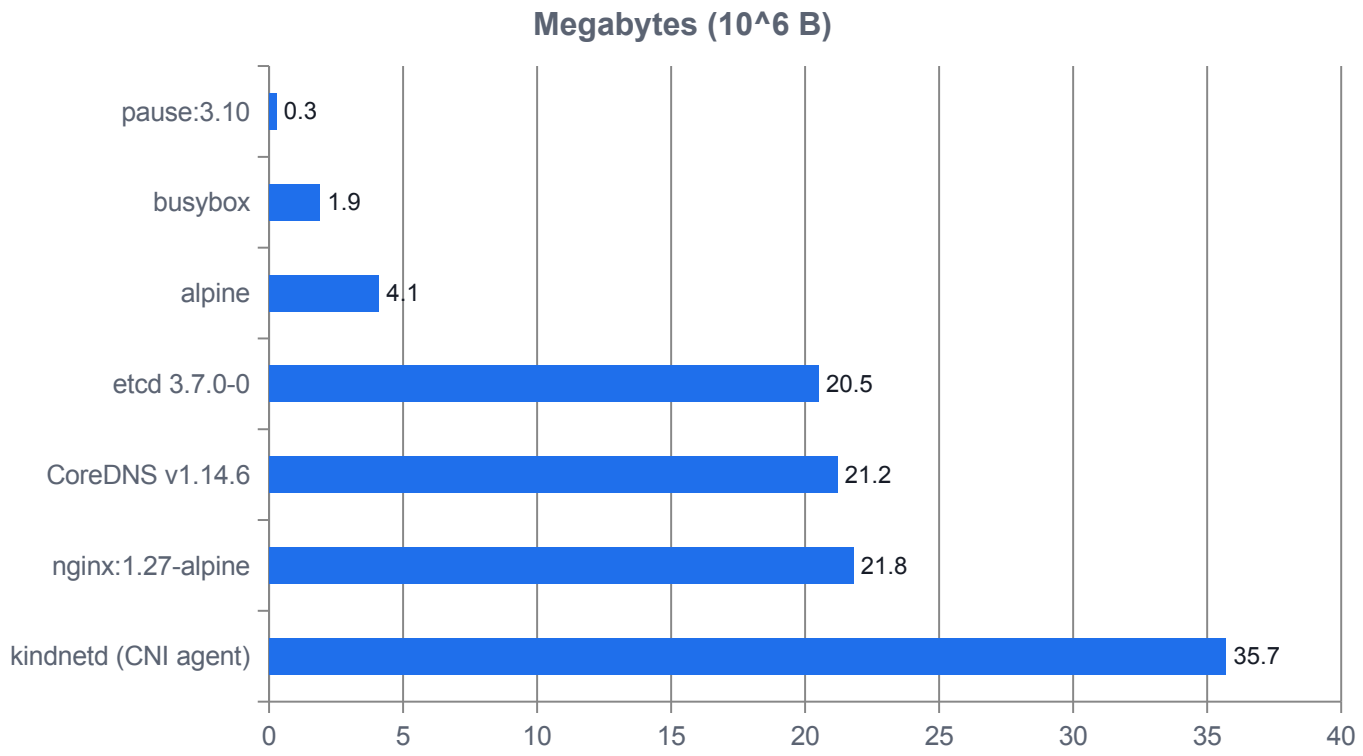
© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

Image Sizes on the Training Node, as containerd Reports Them

Node.status.images[].sizeBytes divided by 10^6, read from the live training cluster. No estimates.

MB (10^6 bytes)



WHAT THE DATA SHOWS

Two orders of magnitude, top to bottom

35.7 MB for the CNI agent against 0.27 MB for the pause sandbox image — both are ordinary containers.

Alpine bases are genuinely small

nginx:1.27-alpine at 21.8 MB carries a full web server; the alpine base underneath it is 4.1 MB.

Small does not mean trivial

busybox at 1.9 MB is the workhorse of every lab today and still gives you a usable shell and coreutils.

Compare with a VM image

These are the sizes shipped and cached per node. A guest OS disk image for the same workload is measured in gigabytes.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Container Runtimes You Will Actually See

'Kubernetes dropped Docker' is a statement about the shim, not about your images.

Runtime	Role	Speaks CRI?	Notes
containerd	High-level runtime: images, snapshots, lifecycle	Yes, via its built-in CRI plugin	CNCF graduated; the default on most managed offerings and on kind
CRI-O	High-level runtime built only for Kubernetes	Yes, natively	CNCF graduated; the default on OpenShift
runc	Low-level runtime: creates namespaces and cgroups, execs	No — it is below the CRI	The OCI reference implementation; invoked by containerd/CRI-O
crun	Low-level runtime, C implementation	No — below the CRI	Drop-in alternative to runc; also used as the base for Kata/gVisor style sandboxes
Docker Engine	Developer tooling and daemon	No — dockershim was removed in v1.24	Images built by Docker are ordinary OCI images and run fine

CHOOSE

Ask two questions of any runtime: does it speak CRI, and does it produce an OCI bundle? Everything else is detail.

Source: Kubernetes documentation — Container Runtime Interface: kubernetes.io/docs/concepts/architecture/cri/.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

CRI Facts Worth Memorising

The CRI is why a cluster can change runtime without a single manifest changing.

Transport	gRPC over a Unix domain socket on the node
Services	RuntimeService (sandboxes, containers, exec) and ImageService (pull, list, remove)
Client	The kubelet — and only the kubelet
Sandbox first	RunPodSandbox is called before any container; it creates the Pod's shared namespaces
Node-level CLI	crictl speaks CRI directly; useful when the API server view and the node disagree
Version skew	The kubelet negotiates the CRI version with the runtime at startup
Where it is configured	kubelet flag --container-runtime-endpoint, pointing at the socket path
Reported back	Node.status.nodeInfo.containerRuntimeVersion — on this cluster, containerd://2.3.4

Source: Kubernetes documentation — Container Runtime Interface: kubernetes.io/docs/concepts/architecture/cri/. Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

Morning Break

11:00 – 11:10 AM

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 3 · KUBERNETES CORE CONCEPTS

03

The API Is the Product

Resources, groups, versions and verbs; the control loop; the control plane and node plane; kubectl as an HTTPS client.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

kubectl is an HTTPS client. Every other Kubernetes behaviour you will learn this week follows from that one fact.

There is no privileged channel, no side door and no local state: the object you get back is the object the API server stored.

© 2026 Tertiary Infotech Academy Pte Ltd

API MODEL

Six Words the API Reference Uses Precisely

Get these six right and kubectl output, RBAC rules and exam questions all become readable.

1 Resource

The plural, lower-case URL segment: pods, services, deployments. It is what appears in the REST path.

2 Kind

The CamelCase type name written in the manifest: Pod, Service, Deployment. One kind, one resource, different spellings.

3 Group

A namespace for kinds: apps, batch, networking.k8s.io. The original core group has an empty name.

4 Version

v1alpha1 → v1beta1 → v1, per group. The group/version pair is what you write as apiVersion.

5 Verb

What you may do: get, list, watch, create, update, patch, delete, deletecollection. RBAC is written in these words.

6 Scope

Namespaced or cluster-scoped. This decides the URL shape and whether a namespace can isolate the object at all.

Source: Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

Proving kubectl Is Just an HTTP Client

Two commands, two URLs. Read the path and you can predict the RBAC rule that would allow or deny it.

terminal — training cluster, 5 Sep 2026

```
$ kubectl -n kube-system get pods --v=6
I0905 16:27:52.389269 loader.go:408] Config loaded from file:
    /tmp/kcna-kind-20260905/kubeconfig
I0905 16:27:52.417996 round_trippers.go:632] "Response"
    verb="GET"
    url="https://127.0.0.1:65108/api/v1/namespaces/kube-system/pods?limit=500"
    status="200 OK" milliseconds=20

$ kubectl -n kube-system get deployments --v=6
I0905 16:27:52.462977 round_trippers.go:632] "Response"
    verb="GET"
    url="https://127.0.0.1:65108/apis/apps/v1/namespaces/kube-system/deployments?lim
it=500"
    status="200 OK" milliseconds=13
```

WHAT TO LOOK AT

1. **--v=6 prints one line per request**
   Verbosity 6 is the level that exposes the round-tripper. Higher levels add headers and bodies.

2. **/api/v1 versus /apis/apps/v1**
   Core group objects live under /api. Every named group lives under /apis. That asymmetry is historical and it is examinable.

3. **limit=500 is server-side pagination**
   kubectl asks for a page and follows a continue token. Large clusters never return one giant list.

4. **The kubeconfig is just credentials**
   Cluster URL, CA bundle, client certificate. Change the file and you change which API server you are addressing.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Captured in courseware/labs/lab-01-kubectl-and-cluster-recon on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Decomposing a Kubernetes REST Path

Once you can write the URL from the manifest, kubectl stops being magic.

anatomy of the request kubectl just made

```
https://127.0.0.1:65108
  /apis             <- named group prefix (/api for core)
  /apps             <- API group
  /v1               <- group version
  /namespaces       <- scope selector
  /kube-system      <- the namespace
  /deployments      <- the resource (plural, lower case)
  /coredns          <- the object name
  /status           <- optional subresource
  ?limit=500        <- pagination

# cluster-scoped objects have no /namespaces segment:
  /api/v1/nodes/kcna-qa-20260905-control-plane

# and the manifest field that names group+version:
  apiVersion: apps/v1     ->  /apis/apps/v1
  apiVersion: v1          ->  /api/v1
```

READING IT

1. **apiVersion is the path**
   apiVersion: apps/v1 in YAML and /apis/apps/v1 in the URL are the same statement written twice.

2. **kind maps to the resource**
   kind: Deployment is served at the resource deployments. The mapping is published by API discovery, not guessed.

3. **Subresources are separate endpoints**
   status, scale, logs, exec and portforward each have their own path — and their own RBAC verbs.

4. **No /namespaces means cluster-scoped**
   Nodes, PersistentVolumes and CRDs cannot be isolated by namespace. That is an architecture constraint, not a preference.

Source: Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

API Discovery: Asking the Server What It Serves

This output is generated from the live OpenAPI discovery document, so it is always right for this cluster's version.

terminal — training cluster, 5 Sep 2026

```
$ kubectl api-resources --api-group=''
NAME                     SHORTNAMES   APIVERSION   NAMESPACED   KIND
bindings                              v1           true         Binding
componentstatuses        cs           v1           false        ComponentStatus
configmaps               cm           v1           true         ConfigMap
endpoints                ep           v1           true         Endpoints
events                   ev           v1           true         Event
limitranges              limits       v1           true         LimitRange
namespaces               ns           v1           false        Namespace
nodes                    no           v1           false        Node
persistentvolumeclaims   pvc          v1           true         PersistentVolumeClaim
persistentvolumes        pv           v1           false        PersistentVolume
pods                     po           v1           true         Pod
resourcequotas           quota        v1           true         ResourceQuota
secrets                               v1           true         Secret
serviceaccounts          sa           v1           true         ServiceAccount
services                 svc          v1           true         Service
```

WHAT TO LOOK AT

1. **Seventeen kinds in the core group**

   Everything with apiVersion: v1. Pods, Services, ConfigMaps and Secrets are all here — the oldest and most used objects.

2. **NAMESPACED is the architectural column**

   Nodes, PersistentVolumes and Namespaces themselves are false. A namespace cannot isolate them for a tenant.

3. **SHORTNAMES are server-published**

   po, svc, cm and pvc come from discovery, not from kubectl. They work for CRDs too if the CRD declares them.

4. **endpoints is still served**

   v1 Endpoints was deprecated in v1.33. Use discovery.k8s.io/v1 EndpointSlice for Service diagnostics from Day 3 onward.

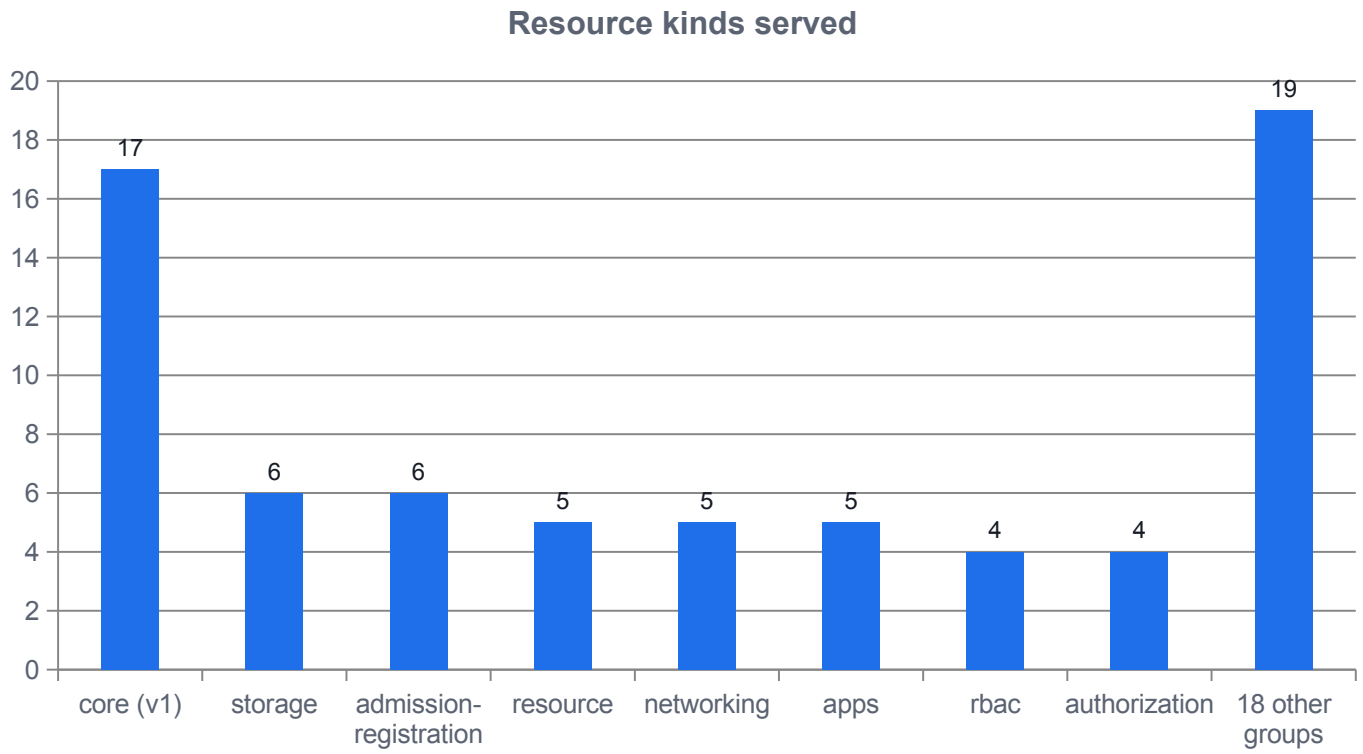
Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. kubernetes.io/blog/2025/04/24/endpoints-deprecation/ — v1 Endpoints deprecated in v1.33; prefer discovery.k8s.io/v1.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

Where the 71 Resource Kinds Live, by API Group

Counted with kubectl api-resources on the training cluster: 71 kinds across 22 API groups and 23 group/version pairs.



WHAT THE DATA SHOWS

The core group is still the biggest

17 of 71 kinds, and it holds the objects you touch daily — Pod, Service, ConfigMap, Secret, Namespace, Node.

apps is smaller than people expect

Five kinds: Deployment, ReplicaSet, StatefulSet, DaemonSet, ControllerRevision. That is the whole of Day 2's workload API.

A long tail of one-kind groups

discovery, coordination, policy, autoscaling, node, events and others each serve one or two kinds.

CRDs extend this list

This cluster runs no operators. Add cert-manager or Argo CD and the count rises without any change to the API server.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

API Groups You Will Use This Week

Ten group/version pairs cover essentially the whole syllabus. Memorising these saves time in the exam.

v1 (core)	Pod, Service, ConfigMap, Secret, Namespace, Node, PersistentVolume — Days 1, 3, 4
apps/v1	Deployment, ReplicaSet, StatefulSet, DaemonSet — Day 2
batch/v1	Job, CronJob — Day 2
autoscaling/v2	HorizontalPodAutoscaler — Day 2
discovery.k8s.io/v1	EndpointSlice — Day 3, replacing v1 Endpoints
networking.k8s.io/v1	Ingress, IngressClass, NetworkPolicy — Day 3
rbac.authorization.k8s.io/v1	Role, RoleBinding, ClusterRole, ClusterRoleBinding — Day 3
storage.k8s.io/v1	StorageClass, CSIDriver, VolumeAttachment — Day 4
policy/v1	PodDisruptionBudget — Day 4
apiextensions.k8s.io/v1	CustomResourceDefinition — the extension point, Day 4

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

kubectl explain Reads the Live OpenAPI Schema

This is the reference you should reach for first — faster than a browser and correct for your cluster.

terminal — training cluster, 5 Sep 2026

```
$ kubectl explain pod.spec.containers.ports
KIND:       Pod
VERSION:    v1

FIELD: ports <[]ContainerPort>

DESCRIPTION:
    List of ports to expose from the container. Not specifying a
    port here DOES NOT prevent that port from being exposed. Any
    port which is listening on the default "0.0.0.0" address
    inside a container will be accessible from the network.
    Cannot be updated.

FIELDS:
  containerPort <integer> -required-
    Number of port to expose on the pod's IP address. This must
    be a valid port number, 0 < x < 65536.
  hostIP <string>   hostPort <integer>   name <string>
```

WHAT TO LOOK AT

1. **The schema comes from the server**
   explain walks the cluster's own OpenAPI v3 document. It cannot drift from the version you are actually running.

2. **<[]ContainerPort> means a list**
   The angle-bracket type tells you the YAML shape before you write a character.

3. **-required- is machine-checked**
   Omit containerPort and admission rejects the object with the exact field path.

4. **Read the DESCRIPTION, not the folklore**
   The API itself states that declaring a port neither opens nor closes anything. Module 4 returns to this.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Three Ways to Change the Cluster — and When Each Is Defensible

The registered topic asks you to generate Pod manifests from the CLI. That is the fourth row, not the first.

Style	Example shape	State of record	Use it when
Imperative command	kubectl run / kubectl create / kubectl expose	Only in the cluster	Throwaway probes and exam speed; never as the source of truth
Imperative object	kubectl create -f / kubectl replace -f	Your file, but applied destructively	Rare; when you must overwrite rather than merge
Declarative	kubectl apply -f / kubectl diff -f	Your files in Git, merged server-side	Everything that outlives the terminal session
Generation, then declaration	kubectl run --dry-run=client -o yaml > file.yaml	Your file, after you finish it	The realistic workflow: generate a skeleton, then own it

CHOOSE

Generate imperatively to save typing; commit declaratively so the cluster can be rebuilt from the repository.

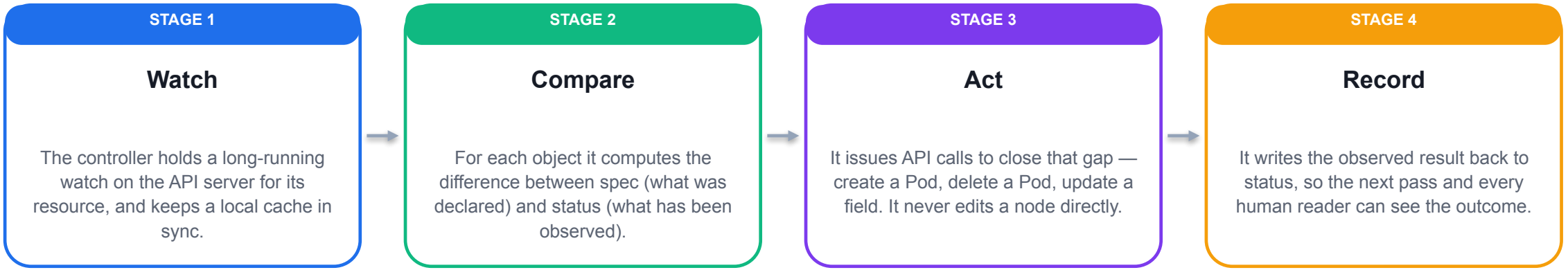
Source: Kubernetes documentation — Managing Kubernetes objects: kubernetes.io/docs/tasks/manage-kubernetes-objects/.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

The Reconciliation Loop Every Controller Runs

One loop, many controllers. Deployment, ReplicaSet, Job, EndpointSlice and node lifecycle all follow this shape.



WHAT IT MEANS

The loop is level-triggered, not edge-triggered: it re-checks current state rather than trusting that it saw every event. That is why Kubernetes recovers after a controller restart.

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

Control Plane and Node Plane

Nine components. If you can say what each one writes, you can troubleshoot most of Day 5.

CONTROL PLANE

kube-apiserver

The only component that reads or writes etcd; authn, authz, admission, validation

etcd

Consistent, watchable key-value store holding all cluster state

kube-scheduler

Filters then scores nodes and binds the Pod by writing spec.nodeName

kube-controller-manager

Runs the built-in reconciliation loops in one process

cloud-controller-manager

Cloud-specific loops: node lifecycle, routes, load balancers

NODE PLANE

kubelet

Registers the node, watches its Pods, drives the runtime, reports status

Container runtime

containerd or CRI-O behind the CRI socket; pulls images and runs containers

kube-proxy

Programs the node dataplane for Service virtual IPs (iptables, IPVS or nftables mode)

CNI plugin

Invoked by the runtime to give the Pod sandbox an interface and an IP

Pods

Your workloads, plus system add-ons such as CoreDNS

MECHANISM

Every write funnels through kube-apiserver. No controller, scheduler or kubelet ever talks to etcd, and none of them talk to each other — they communicate by changing objects.

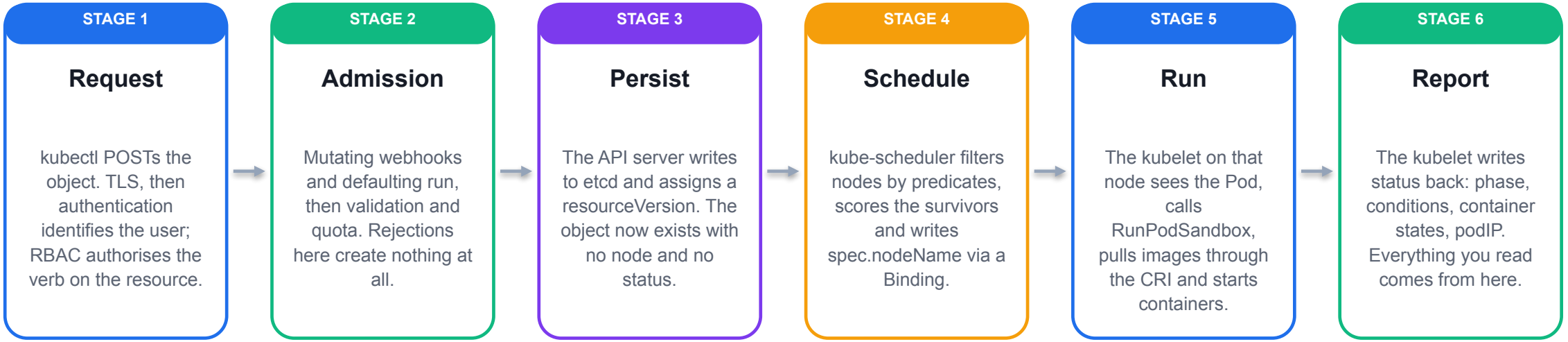
Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

What Happens Between `apply` and a Running Process

This sequence is the single most useful troubleshooting model in Kubernetes, and it is examinable in every domain.



WHAT IT MEANS

Six hops, each one observable. When a Pod misbehaves, decide which hop it stopped at before you touch anything.

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/. Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROL PLANE DETAIL

Some of the Loops Inside kube-controller-manager

One binary, dozens of independent loops. Naming them turns 'Kubernetes did something' into a specific question.

Node controller

1

Watches node heartbeats; marks a node NotReady and eventually evicts its Pods when leases stop arriving.

ReplicaSet controller

2

Counts Pods matching the selector and creates or deletes to reach spec.replicas. The engine behind every Deployment.

Deployment controller

3

Owns ReplicaSets, not Pods. A rollout is one ReplicaSet scaling up while another scales down.

Job / CronJob controllers

4

Track completions and back-off; the CronJob controller creates Job objects on a schedule.

EndpointSlice controller

5

Keeps discovery.k8s.io/v1 EndpointSlices in step with Service selectors and Pod readiness.

Garbage collector

6

Deletes objects whose ownerReferences point at something that no longer exists — the mechanism behind cascading delete.

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

NODE PLANE DETAIL

What the kubelet Is Actually Responsible For

The kubelet is the only component that turns API objects into processes. Everything else just edits objects.

1 Registration

Creates the Node object and keeps its status — capacity, allocatable, conditions, images, addresses.

2 Pod sync loop

Watches Pods bound to its own node and drives the runtime until observed state matches the spec.

3 Volume management

Mounts volumes before containers start; unmounts on teardown. CSI drivers plug in here (Day 4).

4 Probes

Runs liveness, readiness and startup probes and acts on the results locally — no control-plane round trip (Day 5).

5 Resource enforcement

Applies the cgroup limits, and evicts Pods under node memory or disk pressure according to QoS class.

6 What it does not do

It does not schedule, does not know about Deployments and does not talk to other kubelets.

Source: Kubernetes documentation — Cluster Architecture: kubernetes.io/docs/concepts/architecture/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The Five Main Fields of a Pod Object

Real output from the training cluster's CoreDNS Pod, trimmed to the fields that matter.

kubectl -n kube-system get pod coredns-... -o yaml (abridged)

```
apiVersion: v1                # group/version
kind: Pod                     # the type
metadata:                     # identity + bookkeeping
  name: coredns-559f6c778d-cgwt9
  namespace: kube-system
  uid: 954002fb-b38d-4c6f-baed-da666733ceb8
  resourceVersion: "514"
  generation: 1
  creationTimestamp: "2026-09-05T07:03:42Z"
  generateName: coredns-559f6c778d-
  labels:
    k8s-app: kube-dns
    pod-template-hash: 559f6c778d
spec:                         # what YOU declared
  containers:
  - image: registry.k8s.io/coredns/coredns:v1.14.6
status:                       # what the SYSTEM observed
  phase: Running
```

READING IT

1. **You author desired state**
   Set apiVersion, kind, spec and chosen metadata. The API server defaults fields; controllers report status.

2. **uid versus name**
   Names are reusable after deletion; the uid never is. Controllers and ownerReferences track uid, not name.

3. **resourceVersion is opaque**
   It is an etcd revision cursor used for watches and optimistic concurrency — never compare or arithmetic it.

4. **generateName produced this name**
   The ReplicaSet asked for a prefix; the API server appended the random suffix at create time.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Working with Kubernetes objects: kubernetes.io/docs/concepts/overview/working-with-objects/.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Labels, Annotations and Fields — Three Different Jobs

Day 2's ReplicaSet lab and Day 3's Service lab both fail loudly if this distinction is fuzzy.

Property	Labels	Annotations
Purpose	Identifying attributes used to select objects	Arbitrary non-identifying metadata for tools and humans
Queryable	Yes — label selectors, and the API indexes them	No — you cannot select on an annotation
Size	Values limited to 63 characters, restricted charset	Large values allowed; last-applied-configuration lives here
Who reads them	Services, ReplicaSets, NetworkPolicies, scheduling rules, kubectl -l	Ingress controllers, cert managers, deployment tooling, humans
Getting it wrong	A typo silently breaks Service routing — the selector matches nothing	A typo is usually ignored; the tool simply does not act

CHOOSE

If something must be selected on, it is a label. If it is just information, it is an annotation.

Source: Kubernetes documentation — Working with Kubernetes objects: kubernetes.io/docs/concepts/overview/working-with-objects/.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

The metadata Fields Worth Recognising on Sight

Twelve fields. You will meet every one of them before Friday.

name	Unique within namespace and kind; DNS subdomain rules
generateName	Prefix; the server appends a random suffix at create time
namespace	Absent for cluster-scoped kinds; defaults to 'default' if omitted for namespaced kinds
uid	Server-assigned, globally unique, never reused
resourceVersion	Opaque etcd cursor for watch and optimistic concurrency
generation	Incremented when spec changes; compared against status.observedGeneration

creationTimestamp	Server clock at create; the basis of the AGE column
labels	Selectable identifying key/value pairs
annotations	Non-selectable metadata, including kubectl.kubernetes.io/last-applied-configuration
ownerReferences	Who owns this object; drives cascading deletion
finalizers	Named pre-delete hooks; the object stays Terminating until each is removed
deletionTimestamp	Set on delete; presence means 'being deleted', not 'deleted'

Source: Kubernetes documentation — Working with Kubernetes objects: kubernetes.io/docs/concepts/overview/working-with-objects/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

ownerReferences and Cascading Deletion

This single block explains why deleting a Deployment removes its Pods and why a bare Pod survives everything.

real ownerReferences from the training cluster

```
metadata:
  name: coredns-559f6c778d-cgwt9
  ownerReferences:
  - apiVersion: apps/v1
    kind: ReplicaSet
    name: coredns-559f6c778d
    uid: 2cd550e9-ed06-47aa-8462-798b8f99fb8b
    controller: true
    blockOwnerDeletion: true

# the chain on this cluster:
#   Deployment coredns
#     -> ReplicaSet coredns-559f6c778d
#          -> Pod coredns-559f6c778d-cgwt9
#          -> Pod coredns-559f6c778d-n7qbs
```

READING IT

1. **The Pod points up, not down**
   Ownership is recorded on the child. Nothing on the Deployment lists its Pods; the relationship is discovered by uid.

2. **controller: true names the manager**
   Exactly one owner may be the controller. This is what stops two ReplicaSets fighting over the same Pod.

3. **Deleting the parent deletes the children**
   The garbage collector finds objects whose owner uid is gone. That is all 'cascading delete' means.

4. **uid, not name**
   Recreating a ReplicaSet with the same name gives it a new uid, so the orphaned Pods are collected, not adopted.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Working with Kubernetes objects: kubernetes.io/docs/concepts/overview/working-with-objects/.

© 2026 Tertiary Infotech Academy Pte Ltd

SCOPING

Namespaces: What They Do and What They Do Not Do

Every lab this week uses a dedicated namespace so that a mistake in one exercise cannot damage another.

1 A namespace is a name scope

Object names must be unique within a namespace and kind. Two namespaces may both hold a Pod called web.

2 Four exist from the start

default, kube-system, kube-public and kube-node-lease. Do not deploy your workloads into kube-system.

3 It is a policy attachment point

ResourceQuota, LimitRange, RBAC RoleBindings, NetworkPolicy and Pod Security Admission labels all attach to a namespace.

4 It is not a security boundary by itself

Without NetworkPolicy, Pods in different namespaces still reach each other. Isolation is something you add (Day 3).

5 Cluster-scoped objects ignore it

Nodes, PersistentVolumes, StorageClasses, ClusterRoles and CRDs cannot be partitioned this way.

6 It affects DNS, not IPs

A Service is reachable as name.namespace.svc.cluster.local. Pod IPs are flat across the cluster.

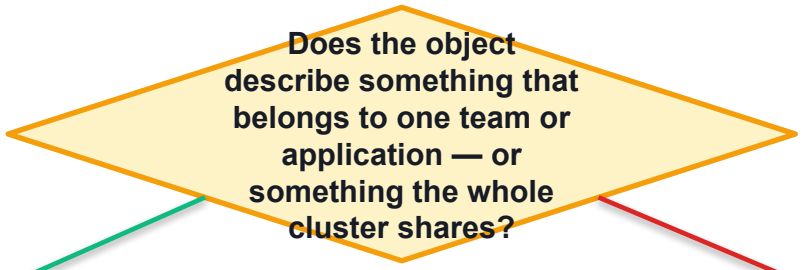
Source: Kubernetes documentation — Namespaces: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN JUDGEMENT

Namespaced or Cluster-Scoped?

Ask kubectl rather than guessing: api-resources --namespaced=false lists every cluster-scoped kind.



YES

NO

Namespaced

- Pod, Service, ConfigMap, Secret, Deployment, Job
- Role, RoleBinding, NetworkPolicy, ResourceQuota
- PersistentVolumeClaim — the claim is tenant-owned
- Can be delegated with a RoleBinding in that namespace

Cluster-scoped

- Node, PersistentVolume, StorageClass, IngressClass
- Namespace itself, ClusterRole, ClusterRoleBinding
- CustomResourceDefinition, admission webhook configurations
- Granting these means granting them everywhere — no partial scope

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

'My Pod Disappeared' — Scope, Not Deletion

This is the most common false alarm in Kubernetes and it costs beginners hours. Diagnose it in seconds instead.

SYMPTOM

kubectl reports the object does not exist, moments after it was created successfully.

EVIDENCE — actual cluster output

```
$ kubectl get pod recon-shell
Error from server (NotFound): pods "recon-shell" not found

$ kubectl config view --minify | grep -A2 context:
  context:
    cluster: kind-kcna-qa
    user: kind-kcna-qa          # <- no namespace: key

$ kubectl -n kcna-lab01 get pod recon-shell
NAME          READY   STATUS    RESTARTS   AGE
recon-shell   1/1     Running   0          64s
```

✓ **The same command with an explicit -n returns the object**

Source: Captured in courseware/labs/lab-01-kubectl-and-cluster-recon on the training cluster, Kubernetes v1.37.0.

ROOT CAUSE

The context declares no default namespace, so every namespaced request went to 'default'. The object was never missing — the request was addressed to the wrong scope.

FIX

- Read the URL the client used: kubectl get pods --v=6
- Confirm scope with kubectl config view --minify
- Pin the namespace in the manifest's metadata, not in your shell
- Or set it deliberately: kubectl config set-context --current --namespace=<ns>

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 01 — Cluster Reconnaissance with kubectl

GOAL

Profile an unfamiliar Kubernetes cluster with kubectl alone — contexts, nodes, API groups, resource schemas and the raw REST calls behind every command — and record the findings as reusable evidence.

YOU WILL PRODUCE

- A namespace kcna-lab01 with a ResourceQuota and LimitRange, discovered and read back as YAML
- A recon-shell Pod that serves the Meridian Freight reconnaissance checklist from a ConfigMap built out of data/recon-checklist.csv
- A verified core-API inventory diff proving every resource in data/core-api-inventory.tsv is served by this cluster

CONCEPTS EXERCISED

- kubectl is an HTTP client: --v=6 exposes the exact GET/POST against /api/v1/...
- API discovery: api-versions, api-resources and namespaced vs cluster-scoped kinds
- kubectl explain reads the live OpenAPI schema, not a static cheat sheet
- Contexts and namespaces scope every request; NotFound usually means wrong scope

LAB	NAMESPACE	MAPS TO	TIME
Lab 01	**kcna-lab01**	**LO1 · A1 · K1**	**45 min**

Full step-by-step procedure: Learner Guide · courseware/labs/lab-01-kubectl-and-cluster-recon/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1:10 – 1:50 PM

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 4 · UNDERSTANDING PODS

04

The Pod Is the Unit of Scheduling

Why a Pod is not a container, the pause sandbox, the manifest field by field, and what exposing a port really means.

© 2026 Tertiary Infotech Academy Pte Ltd

CONCEPT

Six Reasons the Unit Is a Pod and Not a Container

'A Pod is usually one container' is true and unhelpful. The question is why the abstraction exists at all.

1 Some processes must be co-located

A log shipper that tails a file must run on the same disk and the same lifecycle as the writer. Two Pods cannot guarantee that.

2 One IP per unit of scheduling

Service routing, DNS and NetworkPolicy all target a single address. A Pod gives you exactly one, shared by its containers.

3 Atomic placement

The scheduler makes one decision for the whole group. Partial placement — app here, sidecar there — is not expressible.

4 Shared volumes with a defined lifetime

An emptyDir lives and dies with the Pod, so co-operating containers get a scratch space with clear ownership.

5 A single failure and restart domain

restartPolicy, terminationGracePeriodSeconds and eviction apply to the Pod, giving one coherent story for failure.

6 It keeps the container image single-purpose

You add capability by adding a container, not by stuffing a supervisor and three daemons into one image.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Inside a Pod: The Sandbox and Its Containers

registry.k8s.io/pause:3.10 was 267 933 bytes on the training node — the smallest, most important image in the cluster.

POD SANDBOX (PAUSE)

pause container
First thing created; holds the namespaces open so containers can come and go

network namespace
One interface, one Pod IP, one loopback, one port space — created by the CNI plugin

IPC namespace
Shared System V IPC and POSIX queues for the whole Pod

cgroup parent
A per-Pod cgroup under the QoS-class slice; per-container cgroups nest inside it

CONTAINERS IN THE POD

init containers
Run to completion, in order, before any app container starts

app containers
Started together, no ordering guarantee among themselves

each has its own rootfs
Separate mount namespace from the image; nothing is shared implicitly

named volumes
The only shared filesystem, and only where a container explicitly mounts it

MECHANISM

The pause container is why you can restart your application container without losing the Pod IP: the namespace belongs to the sandbox, not to your process.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Kubernetes documentation — Container Runtime Interface: kubernetes.io/docs/concepts/architecture/cri/. Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

CONCEPT

What Containers in a Pod Share — and What They Do Not

Lab 04 proves every line of this by experiment, including the read-only mount that blocks a write from the sidecar.

SHARED · network namespace

1

One Pod IP, one loopback. Container A reaches container B on 127.0.0.1 — and two containers cannot bind the same port.

SHARED · IPC namespace

2

System V IPC and POSIX message queues are common to the Pod.

SHARED · named volumes

3

Only where each container declares its own volumeMounts entry. Mount path and readOnly are per container.

SHARED · lifecycle and node

4

One scheduling decision, one restartPolicy, one termination grace period, one eviction event.

NOT shared · filesystem root

5

Each container gets its own image rootfs. /seed existing in one container says nothing about the other.

NOT shared · PID namespace

6

Off by default; enable deliberately with spec.shareProcessNamespace: true if a container must signal another.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Pod Manifest You Would Defend in Review

Resources requests and limits are also in the real file; Day 2 spends a full module on them.

pod-depot-web.yaml (Lab 02 reference manifest, abridged)

```
apiVersion: v1
kind: Pod
metadata:
  name: depot-web
  namespace: kcna-lab02
  labels:
    app.kubernetes.io/name: depot-web
    app.kubernetes.io/part-of: depot-portal
spec:
  containers:
    - name: web
      image: nginx:1.27-alpine
      imagePullPolicy: IfNotPresent
      ports: [{ name: http, containerPort: 80 }]
      volumeMounts:
        - { name: content, readOnly: true,
            mountPath: /usr/share/nginx/html }
  volumes:
    - { name: content, configMap: { name: depot-content } }
```

READING IT

1. **namespace is in the file**
   A manifest that relies on the ambient namespace is not reproducible. Pin it, or apply it with an explicit -n.

2. **app.kubernetes.io/* labels**
   The recommended label set. Day 2's selectors and Day 3's Services will match on exactly these keys.

3. **A named port**
   name: http lets probes and Services reference the port by name, so renumbering does not break them.

4. **Volume then mount, always two places**
   spec.volumes declares it once for the Pod; each container that wants it repeats the name in volumeMounts.

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0. Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Who Writes Which Part of a Pod Object

If you cannot name the writer of a field, you cannot debug it. Lab 02 has you verify each of these on a live object.

apiVersion, kind	You. They select the schema the request is validated against
metadata.name / labels	You — or a controller, via generateName and template labels
metadata.uid / resourceVersion	kube-apiserver, at create and on every update
spec.containers	You. This is the declared desired state
spec.dnsPolicy, serviceAccountName, schedulerName	kube-apiserver defaulting at admission — you wrote none of them

spec.nodeName	kube-scheduler, by creating a Binding
spec.tolerations (some)	Admission plugins, e.g. the default not-ready/unreachable tolerations
status.phase / conditions	kubelet, reporting what it observed
status.podIP	kubelet, after the CNI plugin assigned it
status.containerStatuses[].imageID	kubelet, after the runtime resolved the reference

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

Server-Side Truth: What You Wrote vs What Was Stored

Reading these three commands is the fastest way to see the boundary between declaration and observation.

terminal — Lab 02 on the training cluster

```
$ kubectl -n kcna-lab02 get pod depot-web -o jsonpath=\

'{.spec.dnsPolicy}{"\t"}{.spec.serviceAccountName}{"\t"}{.spec.schedulerName}{"\n"}'
ClusterFirst   default   default-scheduler

$ kubectl -n kcna-lab02 get pod depot-web -o jsonpath=\
    '{range .status.conditions[*]}{.type}={.status}{"\n"}{end}'
PodReadyToStartContainers=True
Initialized=True
Ready=True
ContainersReady=True
PodScheduled=True

$ kubectl -n kcna-lab02 get pod depot-web -o wide
NAME        READY   STATUS    RESTARTS   AGE   IP           NODE
depot-web   1/1     Running   0          18s   10.244.0.11  kcna-qa-...-control-plane
```

WHAT TO LOOK AT

1. **Three fields you never typed**
   Defaulting happens at admission. The stored object is always larger than the file you applied.

2. **Conditions are ordered by meaning**
   PodScheduled, then Initialized, then ContainersReady, then Ready. The first False one tells you where progress stopped.

3. **READY 1/1 is containers, not conditions**
   Left is ready containers, right is total. A 1/2 means one container in a two-container Pod is failing its readiness probe.

4. **IP and NODE are status**
   Both were written by components after your object was persisted — which is why an unscheduled Pod shows <none>.

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Phase, Conditions and Container State Are Three Different Things

Most 'the Pod is broken' conversations are really disagreements about which of these four fields is being read.

Signal	Where it lives	Values	What it tells you
Pod phase	status.phase	Pending, Running, Succeeded, Failed, Unknown	A coarse life-stage summary; deliberately not a health signal
Pod conditions	status.conditions[]	PodScheduled, Initialized, ContainersReady, Ready (+ PodReadyToStartContainers)	Which stage of admission-to-serving has completed
Container state	status.containerStatuses[].state	waiting{reason}, running{startedAt}, terminated{exitCode}	The precise current state, including the reason string you act on
Last state	status.containerStatuses[].lastState	terminated{exitCode, reason, finishedAt}	Why the previous instance died — indispensable for crash loops
Reason strings	state.waiting.reason	ImagePullBackOff, CrashLoopBackOff, CreateContainerConfigError	The actual diagnosis. STATUS in kubectl get is a rendering of this

CHOOSE

STATUS in `kubectl get pods` is a convenience rendering. Diagnose from containerStatuses, not from the column.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

LIFECYCLE

The Five Pod Phases, Precisely

Phase is a five-valued summary of a much richer state. Know the five, then stop relying on them.

Pending

1

Accepted by the cluster but not all containers are running. Covers 'not scheduled yet' and 'still pulling images'.

Running

2

Bound to a node, all containers created, and at least one is running or starting or restarting. Not the same as Ready.

Succeeded

3

All containers terminated with success and will not be restarted. Only reachable with restartPolicy Never or OnFailure.

Failed

4

All containers terminated and at least one failed — non-zero exit or killed by the system.

Unknown

5

The control plane cannot reach the node that owns the Pod. Almost always a node or network problem, not an application one.

Why phase is not enough

6

A crash-looping container keeps the Pod in Running. Ready and the container state carry the truth.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

From Bound Pod to Ready Endpoint

Each stage emits an event, which is why kubectl describe reads like a transcript of this diagram.



WHAT IT MEANS

Readiness — not Running — is what puts a Pod into service. Day 3 and Day 5 both hang off that last stage.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. kubernetes.io/blog/2025/04/24/endpoints-deprecation/ — v1 Endpoints deprecated in v1.33; prefer discovery.k8s.io/v1.

© 2026 Tertiary Infotech Academy Pte Ltd

LIFECYCLE

spec.restartPolicy — Three Values, One Pod-Wide Decision

It applies to the whole Pod, not per container — the one exception being an init container marked restartPolicy: Always, which is a native sidecar.

Always

- The default, and the only value a Deployment's Pod template may use.
- Every container is restarted when it exits, whatever the exit code.
- The Pod can never reach Succeeded — long-running services live here.

default services

OnFailure

- Restart only on a non-zero exit.
- A clean exit lets the Pod reach Succeeded and stop.
- The natural choice for retryable batch work — Jobs use it (Day 2).

Job retry

Never

- No restarts at all. The Pod reaches Succeeded or Failed on the first exit.
- Used for one-shot diagnostics and for Jobs that must not re-run automatically.
- It is what makes the Lab 03 override Pods show Completed.

one-shot Completed

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Kubernetes documentation — Sidecar Containers: kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Generating a Manifest From the CLI, Then Finishing It

This is the fastest legitimate way to start a manifest, and it is explicitly in the registered outline for Topic 1.

terminal — the registered topic 'generating Pod manifests via CLI'

```
$ kubectl run depot-web --image=nginx:1.27-alpine \
    --port=80 --dry-run=client -o yaml

apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: depot-web        # generator label, not a real one
  name: depot-web         # and no namespace at all
spec:
  containers:
  - image: nginx:1.27-alpine
    name: depot-web
    ports:
    - containerPort: 80
    resources: {}         # no requests, no limits
status: {}
```

READING IT

1. **--dry-run=client contacts nothing**
   The generator renders the object locally. Nothing is created and no credentials are needed.
2. **Three defects to fix by hand**
   Empty resources, a meaningless run: label, and no namespace. This is a skeleton, not a deliverable.
3. **Delete creationTimestamp and status**
   Both are server-owned. Leaving them is harmless but signals a manifest nobody read.
4. **The workflow the exam rewards**
   Generate, redirect to a file, edit, then apply. Typing YAML from memory under time pressure is the slow path.

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Three Dry Runs, Three Different Guarantees

Lab 02 shows a Pod that passes client validation and is rejected server-side, and one that passes both and still fails on the node.

Mode	Where it runs	What it catches	What it misses
--dry-run=client	Entirely in kubectl	YAML syntax and basic schema shape	Defaulting, admission webhooks, quota, Pod Security Admission
--dry-run=server	Full admission chain on the API server, nothing persisted	Validation, defaulting, mutating and validating webhooks, quota	Anything that only fails on a node — image pulls, scheduling, crashes
kubectl diff -f	Server-side, against the stored object	Exactly what would change if you applied	Same node-level failures; also needs read access to the object
kubectl apply	For real	Everything the server checks	Nothing — but now you own the consequences

CHOOSE

Put --dry-run=server in CI. It is the strongest check available without creating anything.

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0. Kubernetes documentation — Kubernetes API concepts: kubernetes.io/docs/reference/using-api/api-concepts/.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Validation Rejects the Object — Read the Field Path

Admission-time errors are the cheapest class of failure. Learn to recognise them by the 'is invalid:' prefix.

SYMPTOM

A one-character edit to containerPort makes the API server refuse the Pod outright.

EVIDENCE — actual cluster output

```
$ sed 's/containerPort: 80$/containerPort: 80000/' \
    /tmp/my-depot-web.yaml | kubectl apply -f - --dry-run=server

The Pod "depot-web" is invalid:
  spec.containers[0].ports[0].containerPort:
    Invalid value: 80000: must be between 1 and 65535, inclusive

$ kubectl -n kcna-lab02 get pod depot-web-typo
Error from server (NotFound): pods "depot-web-typo" not found
```

ROOT CAUSE

The value is outside the range published in the OpenAPI schema. Validation runs before persistence, so no object was created and no node was ever involved.

FIX

- Read the field path — it is exact: containers[0].ports[0]
- Confirm the legal range with kubectl explain on that field
- Nothing to clean up; admission failures leave no state
- Run --dry-run=server in CI so this never reaches a cluster

✓ **kubectl apply --dry-run=server returns 'created (server dry run)'**

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Exposing Ports for a Pod — Every Field in the Block

Everything here is metadata plus one genuine node-level reservation. Know which is which.

spec.containers[].ports — the registered topic 'exposing ports for Pods'

```
ports:
  - name: http           # DNS_LABEL, unique in the Pod
    containerPort: 80    # required, 1-65535
    protocol: TCP        # TCP | UDP | SCTP, defaults to TCP

  - name: metrics
    containerPort: 9090

# fields you should almost never use:
  - name: legacy
    containerPort: 8080
    hostPort: 8080       # binds a port on the NODE itself
    hostIP: 0.0.0.0      # which node address to bind
```

READING IT

1. **name is the useful part**
   Services (targetPort: http) and probes (port: http) can reference it, so renumbering the container does not break them.

2. **protocol defaults to TCP**
   Write it anyway when you mean UDP or SCTP — a silent default is a review comment waiting to happen.

3. **hostPort punches through the Pod network**
   It reserves that port on every node the Pod can land on, which caps replicas at one per node. Use a Service instead.

4. **Two containers, one port space**
   Because the Pod shares one network namespace, two containers cannot both bind 8080. Lab 04 demonstrates this.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

CORRECTION

What containerPort Does — and What It Absolutely Does Not

This is the single most misunderstood field in the Pod spec, and the API reference is unusually blunt about it.

1 It does NOT open a port

The process opens the port. nginx listens on 80 because of its own configuration, not because your YAML said so.

2 It does NOT close a port

The API documentation says so explicitly: any port listening on 0.0.0.0 in the container is reachable on the Pod IP regardless.

3 It does NOT publish to the node

Only hostPort does that, and Services are the supported way to expose a workload (Day 3).

4 It DOES give the port a name

Which is the only reason to write it: probes and Service targetPort can then use a symbol instead of a number.

5 It DOES feed tooling

kubectl describe, dashboards, service meshes and some scrape auto-discovery read the declared ports.

6 Proved in Lab 02

A wget from a second Pod reaches the Pod IP on an undeclared port. Declaring nothing changes nothing.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Every 'Port' You Will Meet This Week, Disambiguated

Eight fields, one word. Naming which one you mean removes most networking confusion before it starts.

containerPort	Pod spec. Declares and names a port the container is expected to listen on
hostPort	Pod spec. Actually reserves that port on the node — avoid unless you are writing a DaemonSet
Service port	The port the Service itself listens on, at the cluster IP (Day 3)
Service targetPort	Where traffic is sent inside the Pod — a number, or the containerPort name

nodePort	30000-32767 by default; opened on every node for a type: NodePort Service (Day 3)
probe port	httpGet.port / tcpSocket.port; accepts the same port name (Day 5)
port-forward local:remote	A debugging tunnel through the API server; single user, dies with your terminal
EndpointSlice port	What the endpoint actually resolved to, per backend — the field to check when routing misses

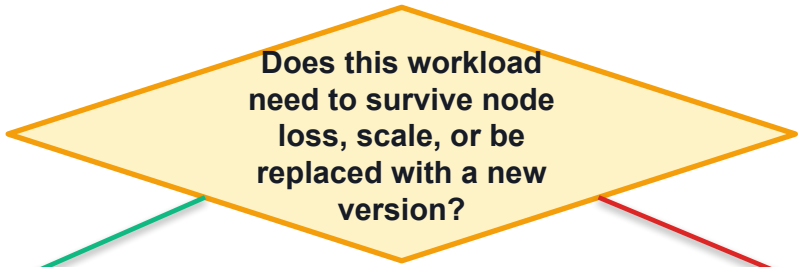
Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Kubernetes documentation — Service: kubernetes.io/docs/concepts/services-networking/service/.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN JUDGEMENT

Should Anything Ever Be a Bare Pod?

Today you write Pods directly so that nothing is hidden. From Day 2 onward you will almost never do it again.



YES

No — a bare Pod is honest

- Learning and lab exercises, exactly as in today's five labs
- Short-lived debugging shells created with --rm
- One-shot diagnostics you will delete in minutes
- Anything where you genuinely want it gone when the node goes

NO

Yes — put a controller in front

- A bare Pod is never rescheduled: lose the node, lose the Pod
- Its spec is almost entirely immutable, so 'update' means delete and recreate by hand
- Nothing counts replicas, so there is no scaling and no self-healing
- Deployment, StatefulSet, DaemonSet, Job or CronJob — Day 2

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 02 — Authoring Your First Pod Manifest

GOAL

Hand-author a valid Pod manifest field by field — using kubectl explain and --dry-run=client as the authoring tools — then run it, serve real content from a ConfigMap, and diagnose an ImagePullBackOff caused by a single mistyped tag.

YOU WILL PRODUCE

- A depot-web Pod in kcna-lab02 serving data/depot-status.html and data/depots.csv from the ConfigMap depot-content
- A learner-authored manifest in /tmp that round-trips cleanly against manifests/20-depot-web-pod.yaml
- A documented ImagePullBackOff diagnosis with the exact kubelet Failed event text

CONCEPTS EXERCISED

- The five top-level fields every object has: apiVersion, kind, metadata, spec, status
- kubectl explain and --dry-run=client -o yaml as manifest authoring aids, not toys
- containerPort is documentation, not a firewall or a publish rule
- Waiting-state reasons: ImagePullBackOff, ErrImagePull and the backoff loop behind them

LAB	NAMESPACE	MAPS TO	TIME
Lab 02	**kcna-lab02**	**LO1 · A1 · K1**	**45 min**

Full step-by-step procedure: Learner Guide · courseware/labs/lab-02-first-pod-manifest/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

ImagePullBackOff: One Missing Character

ErrImagePull is one failed attempt; ImagePullBackOff is the kubelet pausing between retries. Same cause, two stages.

SYMPTOM

The Pod is accepted and scheduled, then never starts. RESTARTS stays at 0 throughout.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab02 get pod depot-web-badtag
NAME               READY   STATUS             RESTARTS   AGE
depot-web-badtag   0/1     ImagePullBackOff   0          45s

$ kubectl -n kcna-lab02 describe pod depot-web-badtag | tail -6
  Normal   Scheduled  62s       default-scheduler  Successfully assigned
  Normal   Pulling    22s (x3)  kubelet  Pulling image "nginx:1.27-alpin"
  Warning  Failed     21s (x3)  kubelet  Failed to pull image:
     failed to resolve reference
     "docker.io/library/nginx:1.27-alpin": not found
  Warning  Failed     21s (x3)  kubelet  Error: ErrImagePull
  Warning  Failed      8s (x4)  kubelet  Error: ImagePullBackOff
```

✓ **STATUS moves ImagePullBackOff → ContainerCreating → Running**

Source: Captured in courseware/labs/lab-02-first-pod-manifest on the training cluster, Kubernetes v1.37.0.

ROOT CAUSE

The image tag is misspelled — the trailing 'e' of 'alpine' is missing. Schema validation cannot know which tags exist in a registry, so the object was accepted and only the kubelet failed.

FIX

- kubectl logs is useless here — no container was ever created
- Read the Pulling event: the kubelet echoes the reference you asked for
- 'not found' after the tag means the tag is wrong; 401 would mean credentials; x509 would mean TLS
- image is one of the few mutable Pod fields, so kubectl set image repairs it in place

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A Pod Stuck Pending Is a Scheduling Verdict

Day 2 tunes requests and Day 5 troubleshoots this properly. Today, learn to read the message.

SYMPTOM

The Pod exists in etcd, has no NODE, and never leaves Pending.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab25 get pod manifest-indexer-8b7d5c9f47-lm3zq
NAME                              READY   STATUS    RESTARTS   AGE
manifest-indexer-8b7d5c9f47-lm3zq 0/1     Pending   0          4m

$ kubectl -n kcna-lab25 describe pod manifest-indexer-... | tail -3
Events:
  Warning  FailedScheduling  4m5s (x2 over 4m20s)  default-scheduler
    0/1 nodes are available: 1 Insufficient cpu, 1 Insufficient memory.
    preemption: 0/1 nodes are available: 1 No preemption victims found for
incoming pod.
```

✓ **kubectl get pod -o wide shows a NODE and the phase leaves Pending**

ROOT CAUSE

The Pod's resource requests exceed what any node still has allocatable. The scheduler ran, evaluated every node, and rejected all of them — this is a verdict, not a hang.

FIX

- Read the node tally first: 0/1 nodes are available
- Then the predicate: Insufficient cpu is capacity; 'didn't match node affinity' is placement
- 'No preemption victims' means evicting lower-priority Pods would not help either
- A Pending Pod with NO FailedScheduling event at all means the scheduler itself is not running

Source: Captured in courseware/labs/lab-25-troubleshooting-triage on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 5 · COMMANDS, ARGUMENTS & CONFIGURATION

05

Configuring a Container From Outside Its Image

command and args against ENTRYPOINT and CMD; env, envFrom, configMapKeyRef and the downward API.

© 2026 Tertiary Infotech Academy Pte Ltd

THE RULE

command and args Against the Image's ENTRYPOINT and CMD

Kubernetes and OCI use different names for the same two things. That mismatch is the whole difficulty.

What you set in the Pod spec	What the container runs	Image ENTRYPOINT	Image CMD
neither command nor args	ENTRYPOINT CMD	used	used
args only	ENTRYPOINT args	used	replaced
command only	command	replaced	discarded
both command and args	command args	replaced	replaced

CHOOSE

command → ENTRYPOINT, args → CMD. The dangerous row is the third: setting command alone silently throws CMD away.

Source: Kubernetes documentation — Define a command and arguments / Downward API: kubernetes.io/docs/tasks/inject-data-application/. Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

The API's Own Words on command

Four behaviours, all documented in the schema, all examinable. Read the field description, not a blog post.

terminal — training cluster, 5 Sep 2026

```
$ kubectl explain pod.spec.containers.command
KIND:       Pod
VERSION:    v1

FIELD: command <[]string>

DESCRIPTION:
    Entrypoint array. Not executed within a shell. The container
    image's ENTRYPOINT is used if this is not provided. Variable
    references $(VAR_NAME) are expanded using the container's
    environment. If a variable cannot be resolved, the reference
    in the input string will be unchanged. Double $$ are reduced
    to a single $, which allows for escaping the $(VAR_NAME)
    syntax: i.e. "$$(VAR_NAME)" will produce the string literal
    "$(VAR_NAME)". Cannot be updated.
```

WHAT TO LOOK AT

1. **'Not executed within a shell'**
   command: ["echo a && echo b"] cannot work — there is no shell to interpret &&. Ask for one: ["/bin/sh", "-c"].

2. **$(VAR_NAME), not ${VAR}**
   Expansion is done by the kubelet before the process starts, and only in this one syntax.

3. **Unresolvable references are left alone**
   You get the literal text $(MISSING) in your argument list rather than an empty string or an error.

4. **'Cannot be updated'**
   command and args are immutable on a running Pod. Only image, and a short list of other fields, can be patched.

Source: Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026. Kubernetes documentation — Define a command and arguments / Downward API: kubernetes.io/docs/tasks/inject-data-application/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The Override Matrix as Three Real Pods

Overriding command on an unfamiliar image means taking responsibility for the initialisation you just skipped.

manifests/10-entrypoint-matrix.yaml (Lab 03, abridged)

```
# A — neither field: image defaults win
  containers:
    - name: web
      image: nginx:1.27-alpine
  # runs /docker-entrypoint.sh  nginx -g 'daemon off;'

# B — args only: ENTRYPOINT kept, CMD replaced
      args: ["nginx", "-v"]
  # banner still prints, then: nginx version: 1.27.5

# C — command only: ENTRYPOINT replaced, CMD discarded
      command: ["/bin/sh", "-c"]
      args: ["echo case C; ..."]
  # NO entrypoint banner at all — that absence is the evidence
```

READING IT

1. **Case A leaves the Pod Running**
   nginx serves. The image's own entrypoint script did its configuration work first.

2. **Case B still runs the entrypoint**
   The banner appears, then your arguments run instead of the default CMD. Container exits 0 → Completed.

3. **Case C is the risky one**
   No banner means /docker-entrypoint.sh never ran, so the IPv6 listener, template substitution and worker tuning silently did not happen.

4. **The API stores <none> for unset fields**
   Kubernetes never copies the image's ENTRYPOINT into the spec. The runtime resolves it at start-up.

Source: Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Five Ways to Get Configuration Into a Container

Lab 03 exercises all five against the same booking service so the differences are visible side by side.

Mechanism	Shape	Best for	Trap
env	name/value pairs written inline	A handful of literal settings	Duplicated across manifests; no single source of truth
env + configMapKeyRef	One key from a ConfigMap, renamed on the way in	Selecting and renaming individual settings	Missing key blocks start unless optional: true
envFrom + configMapRef	Every key becomes an environment variable	Bulk application defaults from --from-env-file	Key collisions; add prefix: to keep sources apart
Downward API	fieldRef and resourceFieldRef	Telling a Pod its own name, node, namespace, IP or memory limit	Only a defined subset of fields is exposed
Volume mount	ConfigMap or Secret projected as files	Large payloads, config files, anything hot-reloadable	Sub-path mounts do not receive updates

CHOOSE

Start-up-only settings go in env; anything the process should re-read at runtime goes in a volume.

Source: Kubernetes documentation — Define a command and arguments / Downward API: kubernetes.io/docs/tasks/inject-data-application/. Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

One Container, Four Injection Mechanisms

Every line here is verified in Lab 03 by comparing the stored spec against `kubectl exec ... -- env`.

manifests/20-booking-config.yaml (Lab 03, abridged)

```
envFrom:
  - configMapRef: { name: route-defaults }
  - prefix: MF_
    configMapRef: { name: route-defaults }
  - configMapRef: { name: route-overrides }
    optional: true      # absent ConfigMap is tolerated
env:
  - { name: SERVICE_NAME, value: booking }
  - name: TRAILERS_PER_BAY
    valueFrom:
      configMapKeyRef:
        name: route-defaults
        key: MAX_TRAILERS_PER_BAY
  - name: NODE_NAME
    valueFrom:
      fieldRef: { fieldPath: spec.nodeName }
  - name: BOOKING_ENDPOINT
    value: http://$(SERVICE_NAME).kcna-lab03.svc:8080
```

READING IT

1. **prefix keeps two sources apart**
   The same nine keys appear twice — once bare, once as MF_*. This is how you merge config without collisions.

2. **optional: true is load-bearing**
   Without it, a missing ConfigMap leaves the Pod in CreateContainerConfigError forever.

3. **fieldRef is the downward API**
   The Pod learns its own node name without holding any RBAC permission to read the Node object.

4. **$(VAR) resolves at start, in the kubelet**
   The stored object still contains the literal $(SERVICE_NAME); only the process sees the expansion. And it can only reference earlier entries in this same env list.

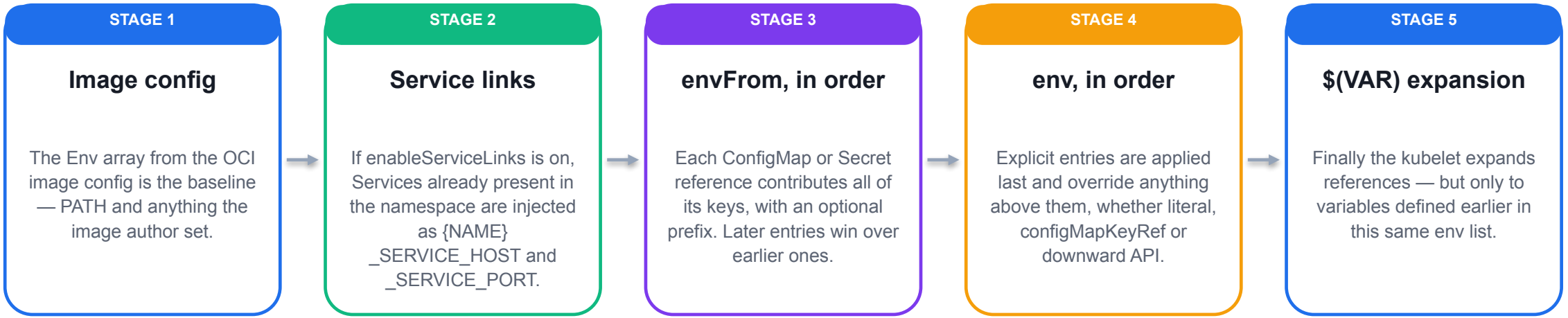
Source: Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

How the kubelet Assembles a Container's Environment

Five contributions merged in a fixed order, all before the process is executed.



WHAT IT MEANS

This ordering is why a $(VAR) reference cannot resolve a key that arrived through envFrom, and why an explicit env entry always beats a bulk import.

Source: Kubernetes documentation — Define a command and arguments / Downward API: kubernetes.io/docs/tasks/inject-data-application/. Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Environment Variables Are a Snapshot; Mounted Files Are Not

Lab 03 patches a ConfigMap and shows the env var stubbornly unchanged while a new file appears in the mounted directory.

Behaviour	env / envFrom	ConfigMap or Secret volume
When it is read	Once, when the container process is created	Continuously — the kubelet refreshes the projected directory
Effect of editing the source	None until the container is recreated	The file changes in place, typically within about a minute
How the update is made atomic	n/a	The kubelet writes a timestamped directory and flips the ..data symlink
Visible to a process that re-reads	No — the process would have to be restarted	Yes, if the application watches or re-reads the file
Known exception	—	subPath mounts are resolved once and never updated

CHOOSE

Secrets and anything hot-reloadable belong in a volume. Start-up-only settings are fine as environment variables.

Source: Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0. Kubernetes documentation — ConfigMaps: kubernetes.io/docs/concepts/configuration/configmap/.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

CrashLoopBackOff Is a Waiting Reason, Not an Error

The back-off doubles — roughly 10s, 20s, 40s, 80s — and is capped at five minutes. Slow status flips are expected.

SYMPTOM

The container starts, exits immediately, and the restart counter climbs while the intervals get longer.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab03 get pod booking-missing-env
NAME                  READY   STATUS             RESTARTS   AGE
booking-missing-env   0/1     CrashLoopBackOff   3 (28s)    72s

$ kubectl -n kcna-lab03 logs booking-missing-env
Error from server (BadRequest): container "app" in pod
  "booking-missing-env" is waiting to start: CrashLoopBackOff

$ kubectl -n kcna-lab03 logs booking-missing-env --previous
[preflight] booking-api starting
[preflight] FATAL: TARIFF_TABLE is not set - refusing to start
```

ROOT CAUSE

The application requires TARIFF_TABLE and exits 1 when it is missing. Nothing upstream complained: the API server accepted the Pod, the scheduler placed it and the image pulled cleanly.

FIX

- --previous is mandatory: the current container does not exist yet
- Read lastState.terminated.exitCode — 1 is the app deciding to stop
- Exit 137 is SIGKILL (usually OOMKilled); 143 is SIGTERM
- Started and Finished in the same second means start-up, not load

✓ **A rebuilt Pod reaches Ready and the restart counter stops moving**

Source: Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

What You May Change on a Running Pod

This list is not trivia — it is the argument for never managing bare Pods in production, delivered by the API itself.

spec.containers[*].image	Mutable — this is why kubectl set image repaired the bad tag in Lab 02
spec.initContainers[*].image	Mutable, same reason
spec.activeDeadlineSeconds	Mutable
spec.tolerations	Additions to existing tolerations only

spec.terminationGracePeriodSeconds	May be set to 1 if it was previously negative
Everything else	Forbidden. The API server returns 'pod updates may not change fields other than …'
So how do you change env?	You replace the object. A Deployment does that for you (Day 2)
Why the restriction exists	A Pod is bound to a node and already running; most spec changes cannot be applied in place

Source: Captured in courseware/labs/lab-03-commands-args-env on the training cluster, Kubernetes v1.37.0. Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 03 — Commands, Arguments and Environment

GOAL

Configure a container entirely from outside its image — override ENTRYPOINT and CMD with command/args, inject settings with env, envFrom and the downward API, and mount a JSON dataset — then diagnose a CrashLoopBackOff caused by one missing variable.

YOU WILL PRODUCE

- An entrypoint-override matrix of three Pods proving how command and args map onto image ENTRYPOINT and CMD
- A booking-config Pod driven by ConfigMaps built from data/route-defaults.env and data/tariff-rates.json
- A CrashLoopBackOff diagnosis read from kubectl logs --previous and lastState.terminated.exitCode

CONCEPTS EXERCISED

- command overrides ENTRYPOINT, args overrides CMD — and command alone discards CMD
- env, envFrom (with prefix and optional), configMapKeyRef and the downward API fieldRef/resourceFieldRef
- $(VAR) expansion inside env values and args happens in the kubelet, before the shell runs
- CrashLoopBackOff is a back-off state, not an error: read exitCode and logs --previous

LAB	NAMESPACE	MAPS TO	TIME
Lab 03	kcna-lab03	LO1 · A1 · K1	40 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-03-commands-args-env/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Afternoon Break

3:50 – 4:00 PM

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 6 · MULTI-CONTAINER POD PATTERNS

06

Adding Capability Without Touching the Application

Sidecar, adapter and ambassador; init containers; and native sidecars from Kubernetes v1.29.

© 2026 Tertiary Infotech Academy Pte Ltd

THE THREE PATTERNS

Same Mechanics, Three Different Intents

The mechanics are identical — an extra container in the Pod. The intent is what you write down in the design record.

Sidecar — adds a capability

- The application does something useful but incompletely.
- A second container extends it: ships logs, reloads config, rotates certificates, refreshes a git checkout.
- Communication is usually a shared volume.
- Real example: Fluent Bit tailing an application's log file.

shared emptyDir | logs

Adapter — reshapes an output

- The application already produces the data, in the wrong format.
- The adapter reads it and re-emits it in a standard shape.
- The application is unchanged and unaware.
- Real example: an exporter turning a private log into Prometheus exposition format.

read-only mount | metrics

Ambassador — owns outbound traffic

- The application makes the simplest possible call: localhost.
- The ambassador handles discovery, TLS, retries, sharding and topology on its behalf.
- Communication is the shared loopback interface.
- Real example: an Envoy sidecar in a service mesh.

127.0.0.1 | proxy

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/. Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Sidecar — Two Containers, One Volume, Two Mount Modes

Note READY 2/2 on this Pod: the left number counts ready containers, the right counts total containers.

manifests/20-sidecar-logship.yaml (Lab 04, abridged)

```
spec:
  containers:
    - name: app
      image: busybox:1.36
      volumeMounts:
        - { name: varlog, mountPath: /var/log/depot }
        - { name: seed,   mountPath: /seed }
    - name: shipper
      image: busybox:1.36
      volumeMounts:
        - name: varlog
          mountPath: /var/log/depot
          readOnly: true          # same volume, RO
  volumes:
    - name: varlog
      emptyDir: { sizeLimit: 16Mi }
    - name: seed
      configMap: { name: access-seed }
```

READING IT

1. **Mount options are per container**
   One volume, mounted read-write in app and read-only in shipper. A write from the shipper fails with 'Read-only file system'.
2. **Only declared mounts exist**
   The shipper never mounts seed, so /seed does not exist in it. Same Pod, different filesystems.
3. **emptyDir needs a sizeLimit**
   An unbounded scratch volume can fill the node's disk and get the Pod evicted.
4. **The result is an ordinary container log**
   Anything that already collects container logs picks the shipped output up for free. That is the sidecar's whole value.

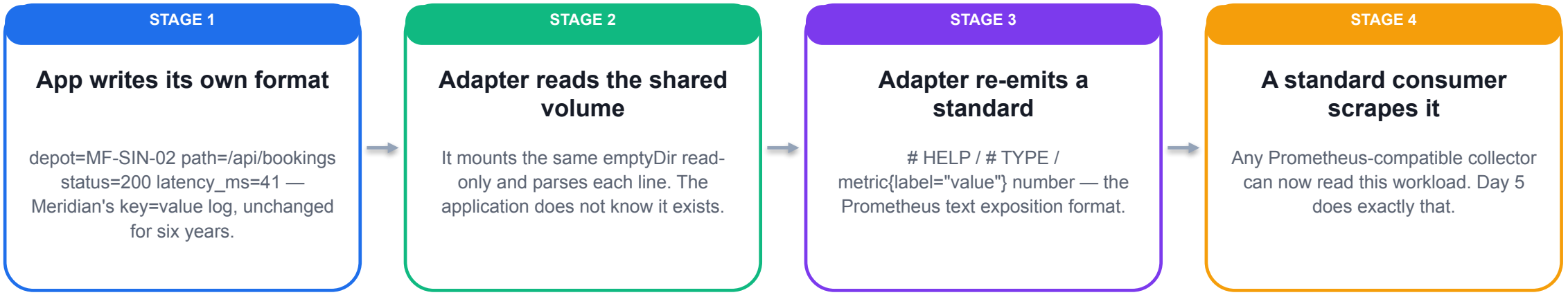
Source: Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Adapter: Turning a Private Format Into a Standard One

Lab 04 produces real counters — 200, 201, 404 and 503 series — from a log the adapter did not write.



WHAT IT MEANS

Nothing in the application changed, no rebuild was scheduled, and the workload became observable. That is the strongest single argument for composite Pods in a PoC.

Source: Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0. Prometheus exposition format: prometheus.io/docs/instrumenting/exposition_formats/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Ambassador — the Proxy Config Is the Contract

An ambassador represents the outside world to the application on localhost.

ambassador-conf ConfigMap + proxy access log (Lab 04)

```
# what the application knows:
#     http://127.0.0.1:9000

server {
    listen       9000;
    location / {
        proxy_pass
          http://tariff-backend.kcna-lab04.svc.cluster.local:80;
        proxy_set_header  X-Meridian-Via  ambassador;
        proxy_connect_timeout 2s;
        proxy_read_timeout    5s;
    }
}

# the proxy's access log, from inside the Pod:
127.0.0.1 - - [05/Sep/2026:16:16:02]
  "GET /tariff-quote.json HTTP/1.1" 200 341 "-" "Wget"
```

READING IT

1. **The client address is 127.0.0.1**
   Two containers, one loopback interface — direct evidence that they share a network namespace.
2. **The app never learns the Service name**
   Discovery, timeouts and headers are the ambassador's problem. Adding mTLS later edits this ConfigMap, not the application.
3. **One port space per Pod**
   The ambassador owns 9000. No other container in this Pod may bind it.
4. **This is how a service mesh works**
   Istio and Linkerd inject exactly this shape automatically instead of you writing it by hand.

Source: Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Choosing Between the Three Patterns

If two containers must scale independently or fail independently, they belong in two Pods — not in one.

Question	Sidecar	Adapter	Ambassador
Direction of concern	Augments the app	Reshapes the app's output	Owns the app's outbound traffic
App awareness	None	None	None — it just calls localhost
Usual channel	Shared volume	Shared volume, read-only	Shared loopback interface
Typical job	Log shipping, config reload, cert rotation	Metrics exporter, log reformatter, protocol translation	Proxy, TLS origination, retries, sharding
Real-world example	Fluent Bit tailing a log file	A *_exporter translating to Prometheus	Envoy in Istio or Linkerd

CHOOSE

Name the intent in the design record. The reviewer cannot tell the three apart from the YAML alone.

Source: Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0. Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

ORDERING

Init Containers: Sequencing Before Anything Serves

Init containers solve ordering. They do not solve 'I need this helper to keep running' — which is the next slide.

1 They run first, in list order

Each must exit 0 before the next starts. No app container is created until all of them have completed.

2 They are ordinary containers

Own image, own command, own volumeMounts, own resources. Often a smaller, sharper image than the app.

3 Typical work

Wait for a dependency, fetch a schema, render a config file into a shared emptyDir, run a database migration.

4 Failure restarts the sequence

Subject to the Pod's restartPolicy. The Pod stays in Pending with reason Init:CrashLoopBackOff.

5 They are not in READY n/m

The ratio counts spec.containers only. Their state lives in status.initContainerStatuses.

6 Their resources still count

The scheduler uses the maximum of any single init container and the sum of the app containers.

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Native Sidecars: initContainers With restartPolicy: Always

Available by default from Kubernetes v1.29 and GA in v1.33. Prefer it for anything the application needs at start-up.

manifests/50-native-sidecar.yaml (Lab 04, abridged)

```
spec:
  initContainers:
    - name: prepare
      image: busybox:1.36
      command: ["/bin/sh", "-c", "..."]
      # no restartPolicy -> classic init container
    - name: shipper
      image: busybox:1.36
      restartPolicy: Always      # <- native sidecar
      volumeMounts:
        - { name: varlog, mountPath: /var/log/depot }
  containers:
    - name: app
      image: busybox:1.36

# status.initContainerStatuses:
# prepare  {"terminated":{"exitCode":0,...}}
# shipper  {"running":{"startedAt":"..."}}
```

READING IT

1. **One field changes everything**
   restartPolicy: Always on an init container means 'start it before the app containers and keep it running'.
2. **Ordering is finally guaranteed**
   The shipper is up before the app writes its first line, so no early output is lost.
3. **READY shows 1/1, not 2/2**
   Native sidecars are not counted in the ratio — the Pod above has two running processes and reports one container.
4. **It does not block Job completion**
   An old-style sidecar keeps a Job's Pod running forever. This is the reason native sidecars exist.

Source: Kubernetes documentation — Sidecar Containers: kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/. Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Three Kinds of Container in One Pod Spec

Six rows. If you can reproduce this table you have the whole multi-container section of the syllabus.

Property	Classic init container	Native sidecar	Regular container
Field	initContainers[]	initContainers[] + restartPolicy: Always	containers[]
Must exit	Yes, with 0	No	No
Start order	Before everything, in list order	Before regular containers, in list order	No guarantee among peers
Counted in READY n/m	No	No	Yes
Blocks Job completion	No	No	Yes
Available since	v1.0	v1.29, GA v1.33	v1.0

CHOOSE

Use a native sidecar whenever the helper must be up before the app, must outlive start-up, or runs inside a Job.

Source: Kubernetes documentation — Sidecar Containers: kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/. Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

'But They're in the Same Pod!' — Mounts Are Per Container

Kubernetes does referential integrity inside an object and not between objects. That single rule predicts both behaviours.

SYMPTOM

A sidecar manifest is rejected outright at admission; no Pod is created and there is nothing to describe.

EVIDENCE — actual cluster output

```
$ kubectl apply -f manifests/90-sidecar-broken-volume.yaml
The Pod "depot-sidecar-broken" is invalid:
  spec.containers[1].volumeMounts[0].name:
    Not found: "applogs"

$ kubectl -n kcna-lab04 get pod depot-sidecar-broken
Error from server (NotFound): pods "depot-sidecar-broken" not found
```

✓ **kubectl apply --dry-run=server accepts the corrected manifest**

ROOT CAUSE

The app mounts a volume named varlog; the shipper mounts applogs, which is never declared in spec.volumes. A volumeMounts name is an intra-object reference, so the API server can check it immediately.

FIX

- Read the index: containers[1] is the second container, the shipper
- Contrast with a missing ConfigMap — an inter-object reference the API server does not verify, which fails later at the kubelet
- Nothing was created, so there is no state to clean up
- The silent version is worse: the right volume at the wrong mountPath is perfectly valid and simply tails nothing

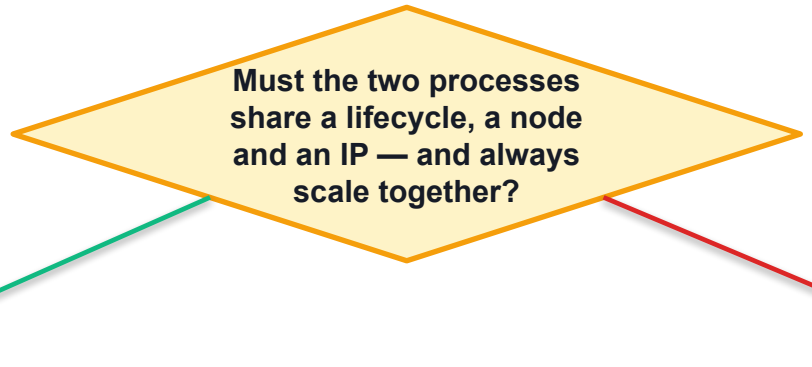
Source: Captured in courseware/labs/lab-04-multi-container-patterns on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN JUDGEMENT

One Pod With Two Containers, or Two Pods?

The wrong answer here shows up months later as a deployment that cannot be scaled without collateral damage.



One Pod, two containers

- A helper that is useless without this specific instance
- Communication over localhost or a shared scratch volume
- Restarting one without the other makes no sense
- Sidecar, adapter and ambassador all live here

Two Pods, joined by a Service

- Each side scales on its own demand curve
- Different teams release them on different cadences
- One is a shared dependency for many clients
- Failure of one should not take out the other

Source: Kubernetes documentation — Pods and Pod lifecycle: kubernetes.io/docs/concepts/workloads/pods/.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 04 — Multi-Container Pod Patterns: Sidecar, Adapter, Ambassador

GOAL

Build and read the three classic multi-container Pod patterns — sidecar, adapter and ambassador — plus a v1.29+ native sidecar, and prove what containers in a Pod really share: one network namespace and named volumes, nothing else.

YOU WILL PRODUCE

- Four running Pods (sidecar, adapter, ambassador, native sidecar) that all consume data/access-seed.log and data/tariff-quote.json
- Prometheus-format metrics produced by an adapter container from a proprietary key=value log it did not write
- An admission-time validation error proving that each container has its own independent volumeMounts list

CONCEPTS EXERCISED

- Containers in a Pod share the network namespace and volumes, not the filesystem root or PID namespace
- Sidecar adds a capability, adapter reshapes an output, ambassador owns the outbound connection
- kubectl logs -c <container> and the error you get when you forget it
- Native sidecars: initContainers with restartPolicy: Always (v1.29+, GA v1.33) give ordering guarantees

LAB	NAMESPACE	MAPS TO	TIME
Lab 04	kcna-lab04	LO1 · A1 · K1	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-04-multi-container-patterns/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

MODULE 7 · IMAGES, REGISTRIES AND THE PULL PATH

07

Naming Bytes You Can Prove

Reference anatomy, silent defaults, digests versus tags, imagePullPolicy and what the kubelet actually asks for.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Anatomy of an Image Reference

Every image reference you have ever written was rewritten before it left the node. Know what was added.

reference normalisation (Lab 05 data/reference-forms.tsv)

```
[REGISTRY[:PORT]/]REPOSITORY[:TAG][@DIGEST]

 docker.io / library/nginx : 1.27-alpine @ sha256:3d69...
 |           |              |            |
 registry    repository     tag          digest

# what you write          # what the runtime resolves
nginx                     docker.io/library/nginx:latest
nginx:1.27-alpine         docker.io/library/nginx:1.27-alpine
library/nginx:1.27        docker.io/library/nginx:1.27
registry.k8s.io/pause:3.10   (unchanged - no library/)
ghcr.io/meridian/api:2.4.1   (unchanged)
localhost:5000/api:dev  (host:port means REGISTRY, not repo)
```

READING IT

1. **Two defaults are applied silently**
   A missing registry becomes docker.io — and a single-segment repository also gains library/. A missing tag becomes :latest.

2. **library/ is a Docker Hub convention**
   registry.k8s.io/pause gets no library/ prefix. The rule is not universal, which is exactly why it surprises people.

3. **:latest is not 'the newest'**
   It is just the default tag name. It may be years old, or may not exist in the repository at all.

4. **A colon in the first segment means a port**
   localhost:5000/api:dev is registry localhost:5000, repository api, tag dev — not a repository called localhost.

Source: Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/. Captured in courseware/labs/lab-05-container-images-oci on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

What Happens When the kubelet Needs an Image

Five stages between 'image: nginx' and a process. Each one produces a distinguishable error message.



WHAT IT MEANS

Authentication failures appear at Resolve, network and corruption failures at Fetch, and disk-pressure failures at Unpack. The stage tells you where to look.

Source: Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/. Open Container Initiative Distribution Specification — github.com/opencontainers/distribution-spec. Kubernetes documentation — Container Runtime Interface: kubernetes.io/docs/concepts/architecture/cri/.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Tag Versus Digest — the Reproducibility Trade

Lab 05's scenario is a real one: the same tag pulled six weeks apart gave two environments two different images.

Property	Tag (nginx:1.27-alpine)	Digest (nginx@sha256:…)
What it names	A mutable pointer the publisher can re-aim	The content itself — the sha256 of the manifest
Can the bytes change	Yes, silently, at any time	No. Different bytes produce a different digest
Failure mode	Succeeds quietly with different content	Fails loudly: 'failed to resolve reference … not found'
Multi-platform	The index picks the right manifest per node architecture	Pin the index digest, not a platform manifest, or arm64 nodes will fail
Readability	Human-meaningful; good in a changelog	Opaque; needs tooling or an annotation to explain it
Where each belongs	Development and local iteration	Anything you must be able to reproduce or attest

CHOOSE

Tag for humans, digest for machines. Record the resolved imageID at every release even if you deploy by tag.

Source: Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/. Open Container Initiative Image Format Specification — github.com/opencontainers/image-spec. Captured in courseware/labs/lab-05-container-images-oci on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

EVIDENCE

Three Fields, Three Different Answers

If you can only remember one image fact from today, make it this: spec is the request, status.imageID is the answer.

terminal — Lab 05 on the training cluster

```
$ kubectl -n kcna-lab05 get pod image-catalog -o jsonpath=\
  '{.spec.containers[0].image}{"\n"}\
   {.status.containerStatuses[0].image}{"\n"}\
   {.status.containerStatuses[0].imageID}{"\n"}'

nginx:1.27-alpine
docker.io/library/nginx:1.27-alpine
docker.io/library/nginx@sha256:3d696e8357051647b844d8c7cf4a...

# and the runtime that resolved it:
$ kubectl get nodes -o jsonpath=\
  '{.items[0].status.nodeInfo.containerRuntimeVersion}'
containerd://2.3.4
```

WHAT TO LOOK AT

1. **spec.image is what you asked for**
   Kubernetes never rewrites it. It is the record of your intent, unchanged.

2. **status…image is the normalised form**
   The runtime's expansion of your short name — this is where the silent docker.io/library/ default becomes visible.

3. **status…imageID is what ran**
   The digest of the bytes on this node. This is the field you record at every release.

4. **The runtime is reported too**
   containerd://2.3.4 on this cluster. Everything before :// is the CRI implementation.

Source: Captured in courseware/labs/lab-05-container-images-oci on the training cluster, Kubernetes v1.37.0. Measured on the course training cluster: kind, Kubernetes v1.37.0, containerd 2.3.4, linux/arm64, 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

imagePullPolicy — Three Values and Their Defaults

The fourth row is the one that catches people: changing the tag also changes the pull behaviour.

Policy	Behaviour	Use when	Failure it produces
Always	Contact the registry on every container start and re-resolve the reference	Mutable tags, or when freshness must be guaranteed	ErrImagePull if the registry is unreachable — even though the node has a copy
IfNotPresent	Use the node's copy if that reference is already present	Fixed tags and digests — the normal production choice	Only fails if the image is absent and cannot be pulled
Never	Never contact a registry under any circumstances	Air-gapped nodes and pre-loaded images	ErrImageNeverPull if the image is not already on the node
Omitted (default)	:latest or no tag → Always; any other tag or a digest → IfNotPresent	Never rely on this — state the policy explicitly	The default silently changes when someone edits the tag

CHOOSE

Write the policy down. A reviewer should not have to recall a defaulting table to know what your manifest does.

Source: Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/. Captured in courseware/labs/lab-05-container-images-oci on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

ErrImageNeverPull Is Not ErrImagePull

Three similar-looking reasons, three completely different root causes. Read the whole string.

SYMPTOM

A Pod sits in Pending. No registry was contacted, so there is no network problem to investigate.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab05 get pod pp-never
NAME       READY   STATUS             RESTARTS   AGE
pp-never   0/1     ErrImageNeverPull   0          19s

$ kubectl -n kcna-lab05 describe pod pp-never | tail -4
Events:
  Type     Reason             Age               From     Message
  Warning  ErrImageNeverPull  6s (x3 over 21s)  kubelet
    Container image "meridianfreight.example.com/depot-portal/
    depot-api:0.1.0-poc" is not present with pull policy of Never
```

ROOT CAUSE

imagePullPolicy: Never and the image was never loaded onto this node. The kubelet is doing exactly what it was told; the mistake is in the assumption that the image was pre-staged.

FIX

- Distinguish the three: ErrImagePull (registry said no), ImagePullBackOff (retrying), ErrImageNeverPull (never asked)
- Check the node's content store through Node.status.images
- Pre-load the image, or change the policy to IfNotPresent
- Never is legitimate — for air-gapped clusters and kind load

✓ **The Pod leaves Pending once the image is present on the node**

Source: Captured in courseware/labs/lab-05-container-images-oci on the training cluster, Kubernetes v1.37.0.

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTRIES

What a Registry Is, and What It Owes You

Almost every image failure is a registry conversation you cannot see. The event text tells you which half failed.

1 It is an HTTP API

The distribution-spec defines /v2/ endpoints for manifests and blobs. Any conformant implementation will do.

2 Content-addressed storage

Blobs are stored and fetched by digest, so a corrupted or substituted layer cannot be silently accepted.

3 Tags are a mutable index

The registry maps a tag to a manifest digest, and the publisher may re-point it whenever they like.

4 Authentication is per registry

Kubernetes supplies credentials with imagePullSecrets on the Pod or the ServiceAccount; 401 in the event text means this.

5 registry.k8s.io is a redirector

Kubernetes' own images are served through it; k8s.gcr.io is frozen and should not be used for new work.

6 Rate limits are real

Anonymous Docker Hub pulls are throttled. A cluster that pulls on every start will eventually be told to wait.

Source: Open Container Initiative Distribution Specification — github.com/opencontainers/distribution-spec. Kubernetes documentation — Images: kubernetes.io/docs/concepts/containers/images/.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 05 — Container Images, OCI and the Runtime Interface

GOAL

Take apart what an image reference really means — registry, repository, tag, digest and OCI manifest — see how the kubelet hands it to a CRI runtime, and prove why a moved tag breaks reproducibility while a digest cannot.

YOU WILL PRODUCE

- An image-catalog Pod serving data/image-inventory.csv and data/reference-forms.tsv, with its resolved status.imageID captured
- A digest-pinned Pod repaired from a stale sha256 reference to the digest this cluster actually resolved
- An imagePullPolicy matrix showing Always, IfNotPresent and an ErrImageNeverPull for a Never-policy image

CONCEPTS EXERCISED

- Image reference anatomy and the implicit docker.io/library/ + :latest defaults
- OCI image spec: index -> manifest -> config + content-addressed layers
- spec.image is what you asked for; status.imageID is what the runtime resolved
- imagePullPolicy semantics and defaults; CRI is the kubelet-to-containerd boundary

LAB	NAMESPACE	MAPS TO	TIME
Lab 05	kcna-lab05	LO1 · A1 · K1	40 min

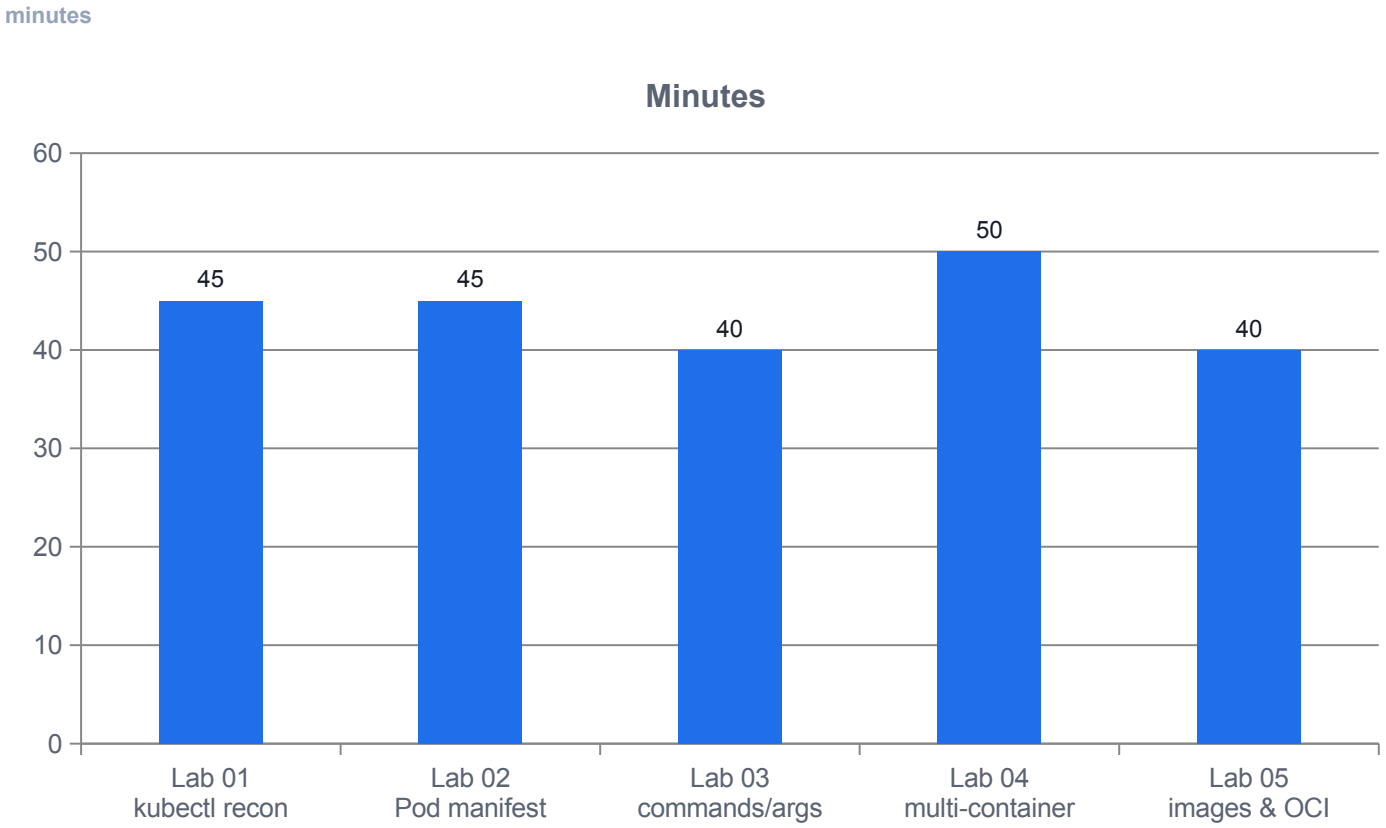
Full step-by-step procedure: Learner Guide · courseware/labs/lab-05-container-images-oci/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

HOW TODAY WAS SPENT

Day 1 Hands-On Time, Lab by Lab

220 of today's 480 instructional minutes — 46% — are hands-on lab time. Breaks are excluded from both figures.



WHAT THE DATA SHOWS

Five labs, 220 minutes

Every registered sub-topic in Topic 1 has a lab that produces evidence for it.

Lab 04 is the longest at 50 minutes

Four Pods, four patterns, plus an admission-time failure injection to diagnose.

Labs 01 and 02 are the foundation

45 minutes each. Everything from Day 2 onward assumes both are complete.

The remaining 260 minutes

Lecture, demonstration and didactic questioning across the four sessions in today's schedule.

Source: course_data.LABS and the Day 1 Lesson Plan schedule (8.0 h instructional, breaks excluded).

© 2026 Tertiary Infotech Academy Pte Ltd

RECAP

What You Can Now Do, and What You Can Now Show

LO1 asks for an architectural proof of concept. Column two is the start of that evidence file.

YOU CAN EXPLAIN

- Why orchestration exists, and what containers did and did not solve
- What a container is at the Linux level: namespaces, cgroups, overlay layers
- The three OCI specifications and how CRI differs from all of them
- How a request travels: admission → etcd → scheduler → kubelet
- Why the Pod is the unit of scheduling and what its containers share
- The difference between spec, status, phase, conditions and container state

YOU CAN SHOW

- A reconnaissance record of a cluster you had never seen (Lab 01)
- A Pod manifest you wrote by hand that survives --dry-run=server (Lab 02)
- Evidence for all four rows of the command/args matrix (Lab 03)
- Sidecar, adapter, ambassador and a native sidecar, running (Lab 04)
- A digest-pinned Pod and the imageID this cluster resolved (Lab 05)
- Six diagnosed failures with the exact event text for each

Source: course_data.LABS and the Day 1 Lesson Plan schedule (8.0 h instructional, breaks excluded).

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM FOCUS

Day 1 Facts KCNA Asks About Directly

Ten statements. If any of them is unfamiliar, revisit that module's lab before tomorrow.

Core group path	/api/v1/... — named groups use /apis/<group>/<version>/...
Five universal fields	apiVersion, kind, metadata, spec, status
Namespaced or not	Nodes, PersistentVolumes, StorageClasses and CRDs are cluster-scoped
Pause container	Holds the Pod's network and IPC namespaces open; created by RunPodSandbox
command / args	command → ENTRYPOINT, args → CMD; command alone discards CMD

Native sidecar	initContainers entry with restartPolicy: Always; v1.29, GA v1.33
imagePullPolicy default	:latest or no tag → Always; any other tag or a digest → IfNotPresent
CRI vs OCI	CRI is kubelet↔runtime gRPC; OCI is image, runtime and distribution formats
Maturity levels	Sandbox → Incubating → Graduated; Archived is retirement
Endpoints	v1 Endpoints deprecated in v1.33 — use discovery.k8s.io/v1 EndpointSlice

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026. kubernetes.io/blog/2025/04/24/endpoints-deprecation/ — v1 Endpoints deprecated in v1.33; prefer discovery.k8s.io/v1.

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM FOCUS

Six Distinctions That Decide Exam Questions

Each of these has appeared today with real evidence attached. Recall the evidence, not the sentence.

1 Running is not Ready

Running means containers were created. Ready means the readiness probe passed and the Pod is a Service backend.

2 ErrImagePull is not ImagePullBackOff

The first is one failed attempt; the second is the kubelet pausing between retries. ErrImageNeverPull is neither.

3 CrashLoopBackOff is a waiting reason

The diagnosis is in lastState.terminated.exitCode and in logs --previous, never in the status column.

4 Admission failure is not a runtime failure

'is invalid:' means nothing was created. Read the field path and stop looking at nodes.

5 containerPort declares nothing

It names a port. Reachability comes from the process listening and from the Pod network.

6 Namespace is not isolation

It scopes names and attaches policy. Traffic isolation requires NetworkPolicy — Day 3.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

TOMORROW

Day 2 — Workloads, Scheduling & Container Orchestration

Day 2 is 44%-domain territory too: Kubernetes Fundamentals covers scheduling and administration as well as core concepts.

WHAT WE BUILD ON TODAY

- Labels and selectors become the binding mechanism for ReplicaSets
- The reconciliation loop becomes rollout, rollback and revision history
- 'Never manage bare Pods' becomes Deployment, DaemonSet, Job and CronJob
- Node allocatable becomes requests, limits, QoS class and eviction

WHAT IS NEW

- apps/v1 controllers and how ownership chains actually work
- The scheduler's filter-and-score cycle, nodeSelector and affinity
- Taints, tolerations and why a Pod can be rejected by every node
- Autoscaling: HorizontalPodAutoscaler against real metrics

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA DOMAIN · CONTAINER ORCHESTRATION (28%)

07

Container Orchestration Fundamentals

Desired state, controllers and the reconciliation loop — the one idea every workload object on Day 2 is built from.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 2 ROADMAP

Where Today Goes

Six modules, five labs, one continuous argument: declare the shape, let a controller close the gap.

1 Orchestration fundamentals

Desired state, the reconciliation loop, owner references and cascading deletion. The mechanism under every controller.

2 Labels, selectors, ReplicaSets

The only binding between a controller and the Pods it owns. Lab 06 proves adoption and orphaning by relabelling.

3 Deployments and rollouts

RollingUpdate arithmetic, revision history, rollback, progressDeadlineSeconds. Lab 07 wedges a rollout on purpose.

4 The workload family

DaemonSet, Job, CronJob, StatefulSet preview — and the adapter pattern as a Pod-level composition. Lab 08.

5 The scheduler

Filter, score, bind. nodeSelector, node affinity, taints and tolerations, static Pods and nodeName. Lab 09.

6 Resources and autoscaling

requests vs limits, the three QoS classes, LimitRange, ResourceQuota, HPA/VPA/Cluster Autoscaler/KEDA. Lab 10.

© 2026 Tertiary Infotech Academy Pte Ltd

BRIDGING DAY 1

What Day 1 Gave You, What Day 2 Adds

Day 1 was one Pod on one node. Day 2 is many Pods across many nodes, with something deciding where and how many.

DAY 1 ESTABLISHED

- A Pod is the smallest schedulable unit and it is mortal — it is never rescheduled, only replaced.
- The API server is the only writer to etcd; every component reads and writes desired state through it.
- kubelet runs containers on one node via the CRI; it does not decide which node.
- Multi-container patterns (sidecar, adapter, ambassador) compose behaviour inside a single Pod.
- Manifests are declarations, not commands.

DAY 2 ADDS

- Controllers that keep a *set* of Pods at a declared count, shape and version.
- The scheduler as a separate, replaceable decision engine that only writes .spec.nodeName.
- Rollout mechanics: two ReplicaSets, an availability floor and a capacity ceiling.
- Resource accounting — the contract between what you asked for and what the node can give.
- Autoscaling as a second-order controller over the first.

© 2026 Tertiary Infotech Academy Pte Ltd

THE CENTRAL IDEA

Kubernetes does not run your application. It runs a loop that keeps reality equal to your declaration.

Every object on Day 2 — ReplicaSet, Deployment, DaemonSet, Job, HPA — is the same loop with a different definition of "equal". Learn the loop once.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

The Reconciliation Loop, Stage by Stage

No step in this loop is a one-shot command. Each pass recomputes the whole difference from scratch.



WHAT IT MEANS

The controller is not told what changed. It is told what should be true, and it recomputes the difference every time. That is why it is safe to run it repeatedly.

Source: kubernetes.io/docs/concepts/architecture/controller/

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

Inside kube-controller-manager

SHARED MACHINERY

Shared informer
One watch per resource type, cached and fanned out to every controller that needs it

Work queue
Rate-limited, de-duplicating queue of object keys awaiting reconciliation

Reflector / lister
Keeps the local store in sync; listers answer reads without hitting the API server

WORKLOAD CONTROLLERS

Deployment ctrl
Owns ReplicaSets; drives rollouts and revision history

ReplicaSet ctrl
Owns Pods; creates and deletes to reach .spec.replicas

Job / CronJob ctrl
Runs Pods to completion; CronJob creates Jobs on schedule

DaemonSet ctrl
One Pod per matching node; reacts to node add and remove

CLUSTER CONTROLLERS

Node lifecycle
Marks NotReady nodes, applies NoExecute taints, triggers eviction

Endpoint / EndpointSlice
Keeps Service backends in step with Pod readiness

Garbage collector
Deletes dependents whose owner is gone, per propagation policy

Source: kubernetes.io/docs/concepts/architecture/controller/

© 2026 Tertiary Infotech Academy Pte Ltd

A DISTINCTION THAT MATTERS

Level-Triggered vs Edge-Triggered Control

Kubernetes controllers are level-triggered. This single design choice is why the system self-heals rather than drifts.

Property	Edge-triggered (event-driven)	Level-triggered (Kubernetes)
Input	The change event itself	The current value of desired and observed state
Missed event	Work is lost; the system is now permanently wrong	Next resync recomputes the same difference and fixes it
Restart of controller	History is gone; needs replay or a rebuild	Relists everything and carries on — no replay needed
Duplicate delivery	Risks double-applying the action	Second pass finds no difference and does nothing
Failure mode	Silent divergence you only find by inspection	Slow convergence, visible in .status and Events

CHOOSE

Level-triggered with edge-triggered hints: the watch makes it fast, the periodic resync makes it correct.

Source: kubernetes.io/docs/concepts/architecture/controller/

© 2026 Tertiary Infotech Academy Pte Ltd

PROPERTIES YOU CAN RELY ON

What the Loop Buys You — and What It Costs

Self-healing is a consequence of the loop, not a separate feature that can be switched on.

1 Idempotence

Applying the same manifest twice changes nothing the second time. Re-running a GitOps sync is safe by construction.

2 Eventual consistency

There is no instant in which the cluster is guaranteed correct — only a guarantee that it is converging.

3 Self-healing without supervision

Delete a Pod owned by a ReplicaSet and a replacement appears with no human, script or alarm involved.

4 Explainability through .status

Conditions such as Available, Progressing and PodScheduled are the controller telling you what it currently believes.

5 Fight-with-yourself risk

Two controllers owning the same field oscillate. This is why an HPA and a hard-coded .spec.replicas conflict.

6 Convergence is not instant

minReadySeconds, backoff timers and stabilisation windows all deliberately slow the loop to keep it stable.

Source: kubernetes.io/docs/concepts/architecture/controller/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL API OUTPUT

Owner References: How the Graph Is Wired

Ownership is the data structure behind adoption, cascading deletion and every 'who created this?' question.

kubectl get pod tracking-api-4m2ql -o yaml (extract)

```
metadata:
  name: tracking-api-4m2ql
  namespace: kcna-lab06
  labels:
    app: tracking-api
    tier: edge
  ownerReferences:
  - apiVersion: apps/v1
    kind: ReplicaSet
    name: tracking-api
    uid: 6f1c9e5a-...
    controller: true
    blockOwnerDeletion: true
```

READING IT

1. **ownerReferences is on the dependent**
   The Pod points up at its owner. The ReplicaSet does not keep a list of Pod names anywhere.
2. **controller: true names the manager**
   Exactly one owner may be the controller. That is the loop allowed to create and delete this Pod.
3. **uid, not just name**
   A ReplicaSet deleted and recreated with the same name is a different uid, so old Pods are not re-adopted.
4. **blockOwnerDeletion: true**
   Foreground deletion of the owner waits in the API server until this dependent is gone.

Source: kubernetes.io/docs/concepts/architecture/garbage-collection/

© 2026 Tertiary Infotech Academy Pte Ltd

CASCADING DELETION

Three Propagation Policies

Deleting a Deployment does not delete Pods directly — it deletes an owner and lets the garbage collector follow the ownerReferences graph.

Policy	What the API server does	When you want it
Background (default for most kubectl deletes)	Owner is deleted immediately; the garbage collector removes dependents afterwards, asynchronously	Normal cleanup where you do not need to wait
Foreground	Owner gets a foregroundDeletion finalizer and stays visible until every blocking dependent is deleted	Ordered teardown; you must know children are really gone
Orphan	ownerReferences are stripped from the dependents; the children survive the parent	Migrating Pods between controllers, or keeping a ReplicaSet's Pods while replacing the Deployment

CHOOSE

Foreground when order matters, orphan when you are re-parenting, background otherwise.

Source: kubernetes.io/docs/concepts/architecture/garbage-collection/

© 2026 Tertiary Infotech Academy Pte Ltd

VOCABULARY YOU WILL BE EXAMINED ON

Orchestration Terms, Precisely

Desired state	.spec — what you declared through the API server
Observed state	.status — what a controller reports it currently sees
Reconciliation	One pass of diffing spec against status and acting
Controller	A loop owning one resource kind and its dependents
Operator	A controller for a CustomResource, encoding domain operational knowledge
Informer	Cached watch + local index shared across controllers
Work queue	Rate-limited, de-duplicated queue of keys to reconcile
Owner reference	Pointer from dependent to owner, used by the GC
Finalizer	String that blocks deletion until a controller clears it
Self-healing	Convergence after an unplanned change — not a feature flag
Declarative	You state the end shape; the system computes the steps
Imperative	You state the steps; nothing re-applies them later

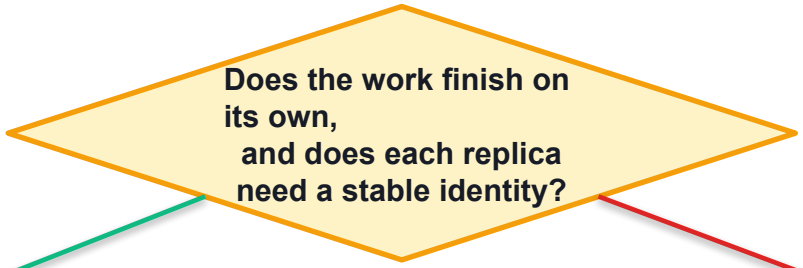
Source: kubernetes.io/docs/concepts/architecture/controller/

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN DECISION

Which Workload Controller Is Correct?

Two questions decide it. Everything else is a detail of the manifest you generate afterwards.



YES

NO

RUN-TO-COMPLETION WORK

- Job — a fixed number of successful completions, then done. Use completions and parallelism.
- CronJob — a Job factory on a cron schedule, with concurrencyPolicy to control overlap.
- restartPolicy must be Never or OnFailure; Always is rejected by the API server for Jobs.
- Never model a long-running server as a Job to 'restart it'.

LONG-RUNNING SERVICE

- Deployment — interchangeable, stateless replicas; the default answer for a web or API tier.
- StatefulSet — ordered, stable network identity and stable per-replica storage (Day 4).
- DaemonSet — exactly one Pod per matching node; scale is a property of the cluster, not of you.
- Bare Pod — only for a one-off diagnostic; nothing will replace it.

Source: kubernetes.io/docs/concepts/workloads/controllers/job/

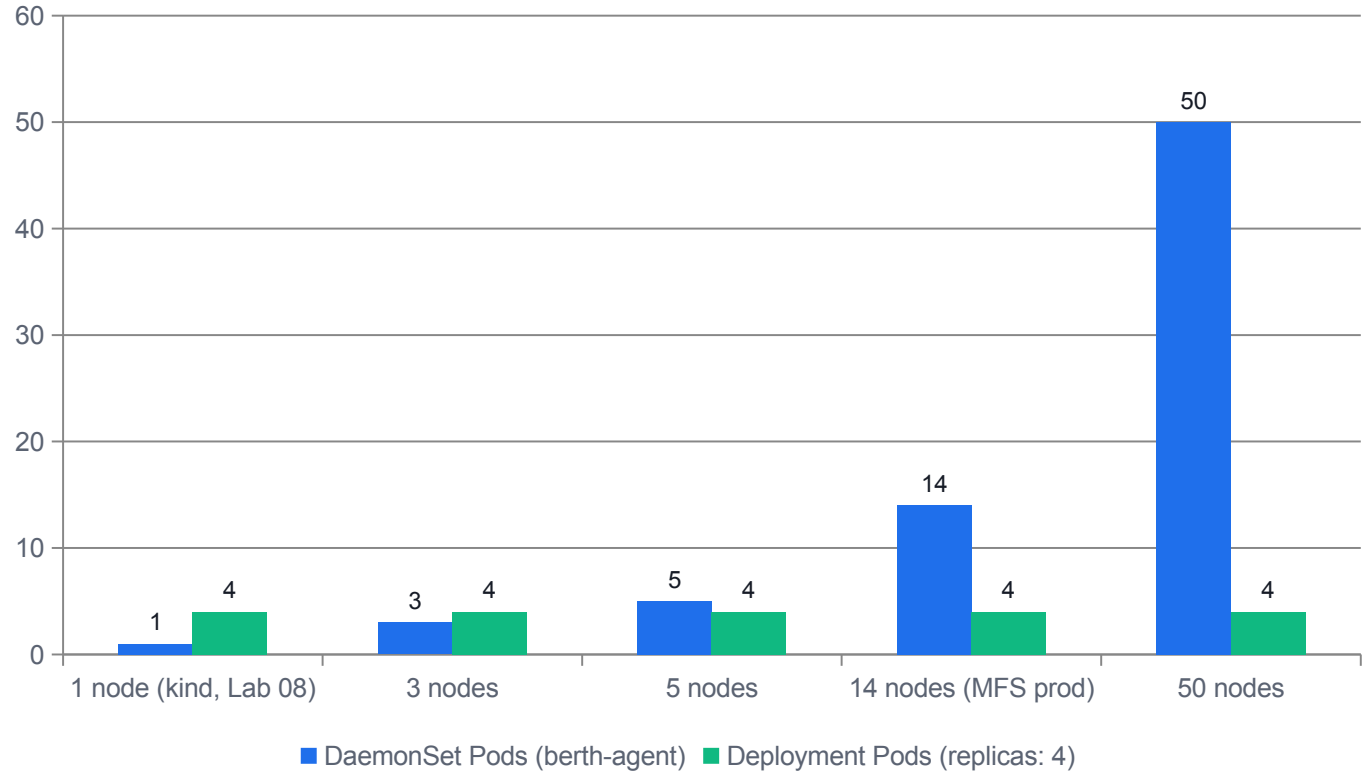
© 2026 Tertiary Infotech Academy Pte Ltd

WORKED MODEL

Why DaemonSet Scale Is Not Your Decision

Worked arithmetic model, not measured data — Pod count is derived from node count. Only the 1-node column is a measured lab observation.

Pods created by one manifest



Source: kubernetes.io/docs/concepts/workloads/controllers/daemonset/

WHAT THE DATA SHOWS

The model is definitional

A DaemonSet has no .spec.replicas field at all; the count equals the number of nodes that pass its node selection.

Lab 08 measures the leftmost bar

On the single-node kind cluster the DaemonSet reports DESIRED 1 / READY 1 — the same manifest, a different cluster.

A Deployment is flat

replicas: 4 stays 4 on 50 nodes. Adding capacity does not add copies unless an autoscaler asks for them.

Budget the difference

A 100 MiB agent costs 100 MiB on a laptop cluster and 5 GiB across 50 nodes. Node-scaled cost is easy to miss.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · WORKLOADS & SCHEDULING

08

Labels, Selectors and ReplicaSets

Labels are not documentation. They are the only wire between a controller and the Pods it manages.

© 2026 Tertiary Infotech Academy Pte Ltd

THE PRIMITIVE

What a Label Actually Is

Every controller-to-Pod relationship in this deck is a label query evaluated continuously.

1 A key/value pair on metadata

Attached to any object, not just Pods. Indexed by the API server so selector queries are cheap at cluster scale.

2 Key = optional prefix + name

Name is up to 63 characters of alphanumerics, '-', '_', '.'. The optional prefix is a DNS subdomain plus '/'.

3 Prefixes are namespaced by convention

kubernetes.io/ and k8s.io/ are reserved for core components. Use your own domain: mfs.io/zone, app.kubernetes.io/name.

4 Values are constrained too

At most 63 characters, may be empty, must start and end alphanumeric. A label value cannot hold a JSON blob.

5 Labels are for selection, annotations are not

If you will never query on it, it is an annotation. Annotations have no length limit and no selector support.

6 Mutable at any time

kubectl label --overwrite changes ownership consequences immediately — Lab 06 orphans a Pod exactly this way.

Source: kubernetes.io/docs/concepts/overview/working-with-objects/labels/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A ReplicaSet Selector, Field by Field

This is the exact ReplicaSet from Lab 06. Read the selector first, the template second — the selector is the contract.

labs/lab-06/manifests/30-replicaset-berth-status.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: berth-status
spec:
  replicas: 2
  selector:
    matchLabels:
      app: berth-status
    matchExpressions:
    - {key: tier, operator: NotIn,
       values: [experimental]}
    - {key: release, operator: Exists}
  template:
    metadata:
      labels:
        app: berth-status
        tier: edge
        release: r24
```

READING IT

1. **matchLabels and matchExpressions are ANDed**
   All of them must hold. There is no OR at this level — you get OR only inside an In operator's value list.

2. **Exists takes no values**
   The release key must be present; its content is irrelevant. DoesNotExist is the complement and also takes no values.

3. **The template labels must satisfy the selector**
   Not the reverse. The selector may be a strict subset; a template missing 'release' would be rejected.

4. **.spec.selector is immutable on Deployments**
   It is settable once. Getting the selector wrong is a delete-and-recreate, not an edit.

Source: kubernetes.io/docs/concepts/workloads/controllers/replicaset/

© 2026 Tertiary Infotech Academy Pte Ltd

TWO SELECTOR GRAMMARS

Equality-Based vs Set-Based Selection

Aspect	Equality-based	Set-based
Syntax on the CLI	-l app=tracking-api, -l env!=prod	-l 'app in (a,b)', -l 'tier notin (experimental)', -l 'release', -l '!provenance'
Operators	= , == , !=	In, NotIn, Exists, DoesNotExist
In API objects	matchLabels — a plain map	matchExpressions — a list of {key, operator, values}
Supported by	Everything, including legacy .spec.selector maps on Service and ReplicationController	Job, Deployment, ReplicaSet, DaemonSet, NetworkPolicy — not by Service selectors
Shell trap	Usually safe unquoted	Must be single-quoted: the shell eats ( ) and !

CHOOSE

Set-based when you need absence, ranges or exclusion; equality-based for the common single-key case.

Source: kubernetes.io/docs/concepts/overview/working-with-objects/labels/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Adoption Made Visible

Nothing here was configured to be adopted. The label query alone decided it.

Lab 06 — captured on the isolated kind cluster

```
$ kubectl get pods -n kcna-lab06 -l app=tracking-api \
    -L provenance,tier,release
NAME                  READY   STATUS    AGE   PROVENANCE     TIER   RELEASE
tracking-api-4m2ql    1/1     Running   41s   replicaset     edge   r24
tracking-api-legacy   1/1     Running   3m    hand-created   edge   r24
tracking-api-x7ntb    1/1     Running   41s   replicaset     edge   r24

$ kubectl get pod tracking-api-legacy -n kcna-lab06 \
    -o jsonpath='{.metadata.ownerReferences[0].kind}/'\
'{.metadata.ownerReferences[0].name} '\
'controller={.metadata.ownerReferences[0].controller}'
ReplicaSet/tracking-api controller=true
```

WHAT TO LOOK AT

1. **replicas: 3 created only two Pods**
   A hand-made Pod already matched the selector, so the controller counted it and made up the difference.
2. **-L prints a label as a column**
   provenance was set by hand on the legacy Pod and by the template on the other two. Same selector, different origin.
3. **Adoption rewrites ownerReferences**
   The pre-existing Pod now has controller=true pointing at the ReplicaSet. Nothing about the Pod's spec changed.
4. **Consequence**
   Any Pod that matches an active selector will be claimed. A careless label is a production incident.

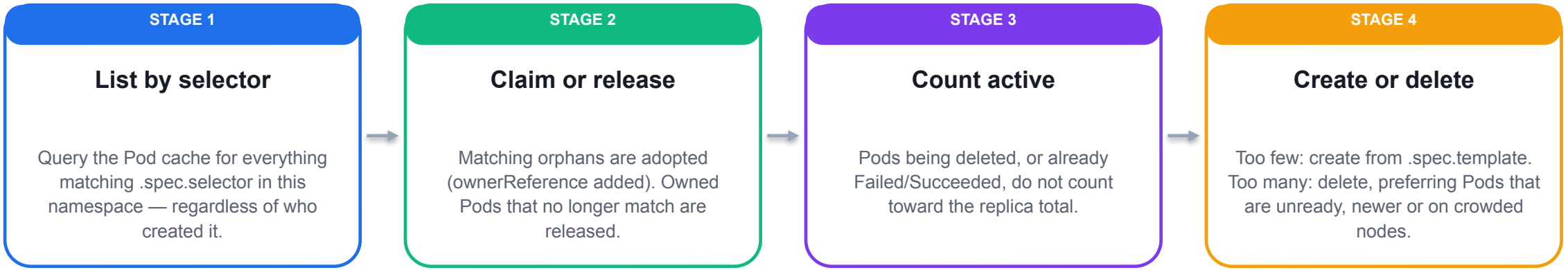
Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

How a ReplicaSet Closes the Gap

Four steps, repeated forever. Step two is the one people forget exists.



WHAT IT MEANS

A ReplicaSet never tracks Pods by name. It re-runs the label query every pass, which is precisely why relabelling a Pod out of the selector triggers a replacement.

Source: kubernetes.io/docs/concepts/workloads/controllers/replicaset/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Orphaning by Relabelling

One kubectl label command, two consequences: a Pod leaves the controller's world and a new Pod enters it.

Lab 06 — captured on the isolated kind cluster

```
$ kubectl label pod tracking-api-legacy -n kcna-lab06 \
    tier=quarantine --overwrite
pod/tracking-api-legacy labeled

$ kubectl get pod tracking-api-legacy -n kcna-lab06 \
    -o jsonpath='{.metadata.ownerReferences}'
[]

$ kubectl get pods -n kcna-lab06 -l app=tracking-api
NAME                READY   STATUS    RESTARTS   AGE
tracking-api-4m2ql  1/1     Running   0          4m
tracking-api-9kdd8  1/1     Running   0          2s
tracking-api-x7ntb  1/1     Running   0          4m
```

WHAT TO LOOK AT

1. **ownerReferences is now []**
   The Pod stopped matching {app, tier=edge}, so the controller released it rather than deleting it.

2. **The released Pod keeps running**
   Release is not eviction. tracking-api-legacy is still Running — it is simply unmanaged now.

3. **A replacement appeared in 2 seconds**
   The count of matching Pods fell to 2 against replicas: 3, so the loop created tracking-api-9kdd8.

4. **Both effects come from one edit**
   One label change caused both an orphan and a creation. This is the whole labels-are-load-bearing lesson.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Selector and Template Labels That Disagree

The API server refuses to create a controller that could never converge. This is a guard rail, not a bug.

SYMPTOM

kubectl apply is rejected outright — no ReplicaSet is created and no Pod is ever attempted.

EVIDENCE — actual cluster output

```
$ kubectl apply -f 50-replicaset-broken-selector.yaml
The ReplicaSet "cargo-manifest-broken" is invalid:
spec.template.metadata.labels: Invalid value:
map[string]string{"app":"cargo-manifest",
"env":"staging", "release":"r24",
"tier":"internal"}: `selector` does not match
template `labels`
```

ROOT CAUSE

- The selector required tier=edge but the template stamps tier=internal onto every Pod it makes.
- Without the guard the ReplicaSet would create a Pod, fail to see it, create another — an infinite Pod loop.
- Validation is synchronous in the API server, so nothing is partially created.

FIX

- Make the selector a subset of the template labels — every selector key must appear with the same value in the template.
- Read the message backwards: it prints the *template* labels and tells you the selector does not match them.
- Generate the pair with kubectl create ... --dry-run=client -o yaml so they cannot drift by hand.

✓ **kubectl apply --dry-run=server catches this before it reaches a real cluster**

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

CONVENTION

The Recommended Labels — and Why They Exist

app.kubernetes.io/name	The application itself, e.g. freight-portal
app.kubernetes.io/instance	A unique name for this deployment of it
app.kubernetes.io/version	The current app version or image tag
app.kubernetes.io/component	Role within the architecture, e.g. database, cache
app.kubernetes.io/part-of	The higher-level product this belongs to
app.kubernetes.io/managed-by	Tool driving the change, e.g. Helm, Flux, Argo CD

pod-template-hash	Added by the Deployment controller — do not set it yourself
kubernetes.io/metadata.name	Auto-added to every Namespace; used by NetworkPolicy on Day 3
controller-revision-hash	Added by DaemonSet and StatefulSet for revision tracking
Why standardise	Dashboards, cost tooling and policy engines query these keys across teams
What not to encode	Anything over 63 characters, or that changes per request
Sanity rule	If a human will never select on it, make it an annotation

Source: kubernetes.io/docs/concepts/overview/working-with-objects/labels/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 06 — Labels, Selectors and ReplicaSets

GOAL

Use labels and selectors as the binding mechanism between controllers and Pods, and prove it by watching a ReplicaSet adopt a hand-created Pod and replace one you relabel out of its selector.

YOU WILL PRODUCE

- A tracking-api ReplicaSet (matchLabels) that adopts an existing Pod instead of creating a third replica
- A berth-status ReplicaSet driven by matchExpressions using In, NotIn and Exists
- A label-audit Pod that reads the MFS service inventory and prints the standard label set

CONCEPTS EXERCISED

- Labels are the only binding between a controller and its Pods - there are no object references
- matchLabels is equality-based AND; matchExpressions adds In, NotIn, Exists and DoesNotExist
- Adoption and orphaning: ownerReferences are added or released purely on label match
- A ReplicaSet selector must be a subset of the template labels or the API server rejects it

LAB	NAMESPACE	MAPS TO	TIME
Lab 06	kcna-lab06	LO2 · A3 · K3	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-06-labels-selectors-replicasets/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Morning Break

11:00 – 11:10 AM

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · WORKLOADS & SCHEDULING

09

Deployments, Rollout Strategy and Rollback

A Deployment does not manage Pods. It manages ReplicaSets, and a rollout is arithmetic on two of them at once.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

The Three-Level Ownership Chain

YOU DECLARE

Deployment
apps/v1 · holds the Pod template, the strategy and the revision history

kubectl apply
Writes .spec; every later change is a diff against what is stored

CONTROLLER CREATES

ReplicaSet (new)
One per distinct Pod template hash; scaled up during a rollout

ReplicaSet (old)
Kept at 0 replicas as the revision record, up to revisionHistoryLimit

pod-template-hash
Label the Deployment controller adds so the two ReplicaSets never fight over Pods

CLUSTER RUNS

Pods (new template)
Counted as UPDATED; must pass readiness plus minReadySeconds to be AVAILABLE

Pods (old template)
Deleted only as the availability floor allows

kubelet
Never sees the Deployment at all — it only ever sees Pods

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

The Strategy Block, Field by Field

Every number in this block is a promise about capacity during the change, not about the steady state.

labs/lab-07/manifests/20-deployment-r24-2.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: freight-portal
  annotations:
    kubernetes.io/change-cause: >-
      r24.2 - customs banner rollout
spec:
  replicas: 4
  revisionHistoryLimit: 3
  minReadySeconds: 2
  progressDeadlineSeconds: 120
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

READING IT

1. **maxSurge sets the ceiling**
   Total Pods may reach replicas + maxSurge = 5. This is the extra capacity the rollout is allowed to borrow.

2. **maxUnavailable sets the floor**
   Available Pods may not drop below replicas - maxUnavailable = 4. Zero here means no capacity loss at all.

3. **They may not both be zero**
   The API server rejects it — the rollout would have no room to move in either direction.

4. **minReadySeconds delays 'Available'**
   A Pod must stay Ready for 2 seconds before it counts, so a crash-on-start image cannot fake progress.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

TWO STRATEGIES

RollingUpdate vs Recreate

Aspect	RollingUpdate (default)	Recreate
Order of operations	Add new Pods and remove old ones incrementally, respecting the ceiling and floor	Terminate every old Pod first, then start the new ones
Downtime	None, if maxUnavailable is 0 and readiness is honest	A full outage for the duration of the change
Versions running together	Two versions serve traffic simultaneously — your API and schema must tolerate it	Never more than one version at a time
Correct when	Stateless, backward-compatible services — almost everything	Exclusive locks, singleton writers, or a schema migration that cannot run mixed
Tuning fields	maxSurge, maxUnavailable, minReadySeconds	None — the strategy has no sub-fields

CHOOSE

RollingUpdate unless two versions of your code cannot safely coexist for sixty seconds.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

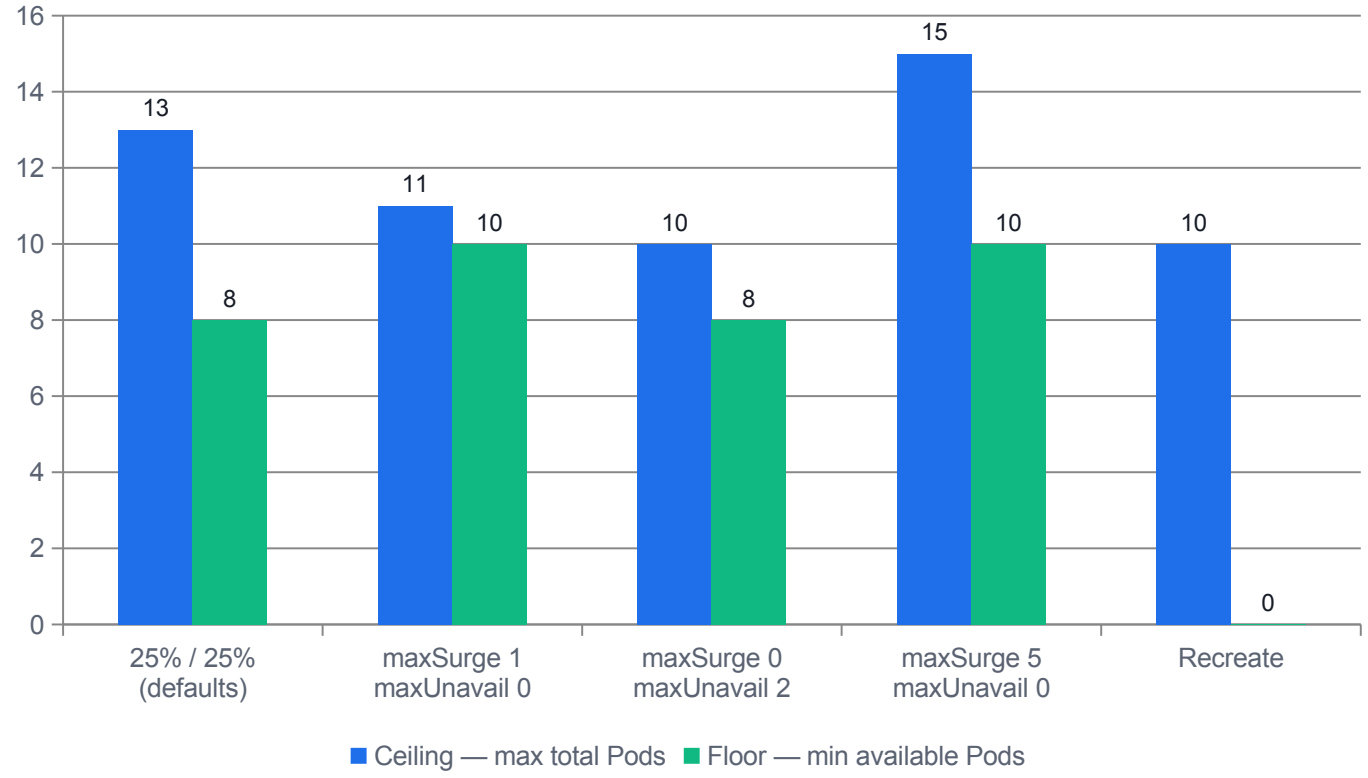
© 2026 Tertiary Infotech Academy Pte Ltd

WORKED CAPACITY MODEL

maxSurge and maxUnavailable at replicas = 10

Worked arithmetic model — computed from the documented rounding rules, not measured. Verify it yourself against any Deployment you own.

Pods (worked model, replicas = 10)



WHAT THE DATA SHOWS

The arithmetic is fixed

Ceiling = replicas + maxSurge. Floor = replicas - maxUnavailable. Nothing else moves these two numbers.

Percentages round in opposite directions

maxSurge rounds up, maxUnavailable rounds down. At 10 replicas the 25% defaults give surge 3, unavailable 2.

Zero unavailable costs capacity

You must have headroom for the extra Pods, and quota to admit them, or the rollout simply stalls.

Recreate has no floor

It is plotted here to show the contrast: available Pods reach 0 by design.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

THE FINE PRINT

Percentages, Rounding and Edge Cases

These rules are why two Deployments with the same percentages behave differently at different replica counts.

1 Both fields accept an int or a percentage string

maxSurge: 1 and maxSurge: "25%" are both valid. The string form is resolved against .spec.replicas at rollout time.

2 maxSurge rounds up

25% of 10 is 2.5, which becomes 3. The ceiling is generous so a rollout can always start.

3 maxUnavailable rounds down

25% of 10 is 2.5, which becomes 2. The floor is conservative so availability is protected.

4 Both zero is rejected

spec.strategy.rollingUpdate.maxUnavailable: Invalid value: may not be 0 when maxSurge is 0.

5 Percentages plus scaling interact

Change replicas mid-rollout and the effective integers change with it. Prefer explicit integers for predictable behaviour.

6 maxUnavailable can exceed the deficit

maxUnavailable greater than or equal to replicas makes the rollout behave close to Recreate while still using two ReplicaSets.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

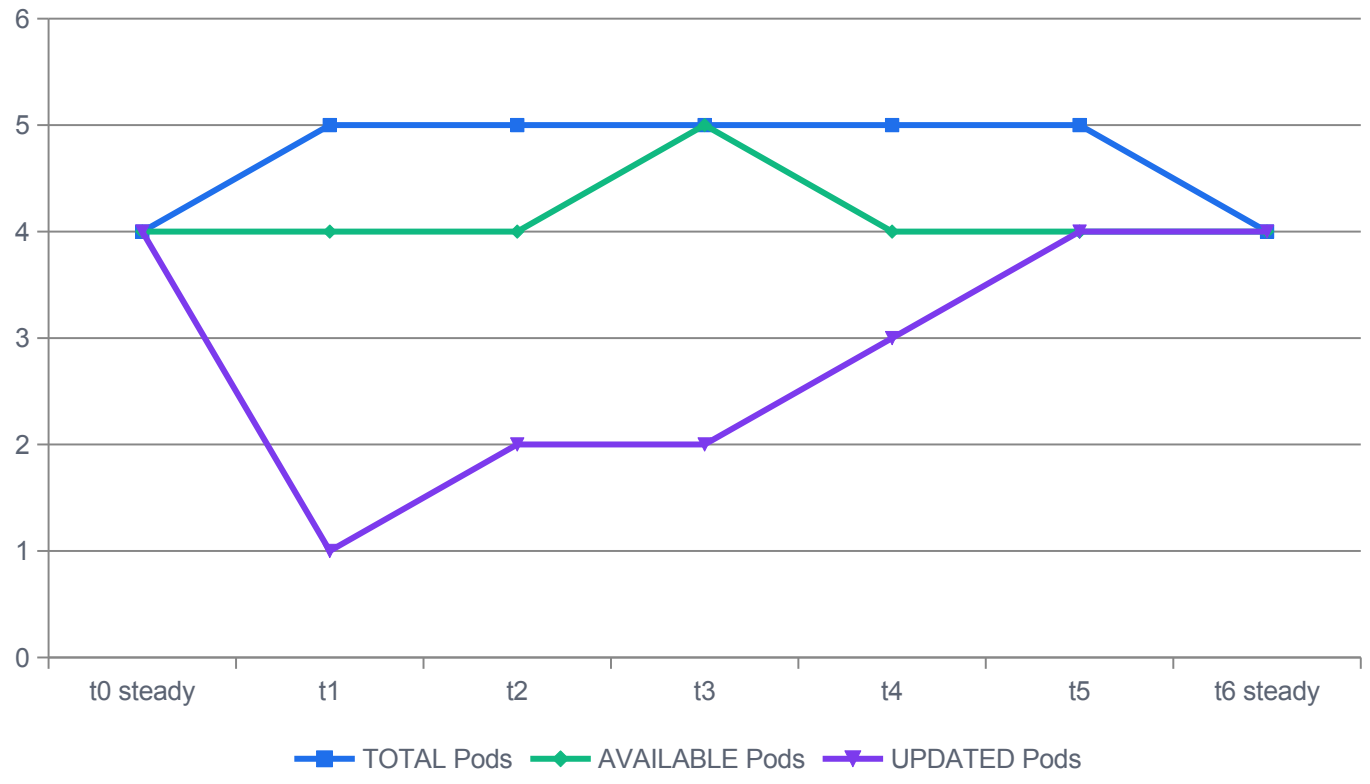
© 2026 Tertiary Infotech Academy Pte Ltd

MEASURED LAB OBSERVATION

A Real Rollout at replicas 4, maxSurge 1, maxUnavailable 0

These seven rows are captured cluster output, not a simulation. Compare them against the previous slide's model.

Pod count (captured during rollout)



WHAT THE DATA SHOWS

Measured, not modelled

Successive samples of kubectl get deploy -w during Lab 07 on the isolated kind cluster.

TOTAL never exceeds 5

That is the ceiling, 4 + maxSurge 1. The controller refuses to create a sixth Pod even to go faster.

AVAILABLE never drops below 4

That is the floor, 4 - maxUnavailable 0. Serving capacity is preserved for the whole change.

UPDATED is the progress bar

It climbs 1, 2, 3, 4 as new Pods pass readiness and minReadySeconds; TOTAL falls back to 4 at the end.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

REVISIONS

What Creates a Revision — and What Does Not

Rollback is only possible because the old ReplicaSet object still exists. Understand that and rollback stops being magic.

1 Any change to .spec.template creates one

Image, command, env, resources, labels, annotations inside the template — all of it hashes into a new ReplicaSet.

2 Changing .spec.replicas does NOT

Scaling is not a new Pod template, so no revision is cut and no new ReplicaSet appears.

3 Revision numbers live on the ReplicaSet

deployment.kubernetes.io/revision is an annotation on each ReplicaSet, not a field on the Deployment.

4 CHANGE-CAUSE comes from an annotation you set

kubernetes.io/change-cause on the Deployment. If you never set it, the history column reads <none>.

5 revisionHistoryLimit prunes old ReplicaSets

Default is 10. Old ReplicaSets are kept at 0 replicas purely so you can roll back to them.

6 There is no snapshot store

History is the surviving ReplicaSet objects. Prune them and the corresponding rollback target is gone forever.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

kubectl rollout history

Revision history is not a log. It is a set of live objects you can inspect and scale.

Lab 07 — captured on the isolated kind cluster

```
$ kubectl rollout history deployment/freight-portal \
    -n kcna-lab07
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1         r24.1 - initial portal release
2         r24.2 - customs banner, zero-unavailable rollout

$ kubectl get rs -n kcna-lab07 -o custom-columns=\
NAME:.metadata.name,\
REVISION:.metadata.annotations.'deployment\.kubernetes\.io/revision',\
DESIRED:.spec.replicas
NAME                         REVISION   DESIRED
freight-portal-6c9f4b8d57    1          0
freight-portal-7d84c65f9b    2          4
```

WHAT TO LOOK AT

1. **Two revisions, two ReplicaSets**
   The history command is a rendering of the ReplicaSet objects that still exist. It reads nothing else.

2. **Revision 1 sits at DESIRED 0**
   The old ReplicaSet is retained but scaled to zero. It costs one API object and no compute.

3. **CHANGE-CAUSE is your text**
   It is copied from the kubernetes.io/change-cause annotation. Discipline here is the difference between a usable history and a list of numbers.

4. **The hash suffix is pod-template-hash**
   6c9f4b8d57 identifies the template, so an identical template always maps back to the same ReplicaSet.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Inspecting One Revision's Pod Template

Before you roll back, read what you are rolling back to. The revision carries far more than an image tag.

Lab 07 — captured on the isolated kind cluster

```
$ kubectl rollout history deployment/freight-portal \
    --revision=1 -n kcna-lab07
deployment.apps/freight-portal with revision #1
Pod Template:
  Labels:  app=freight-portal
           pod-template-hash=6c9f4b8d57
           release=r24.1
           tier=edge
  Annotations:  kubernetes.io/change-cause:
                r24.1 - initial portal release
  Containers:
   web:
    Image:  nginx:1.27-alpine
    Limits:    cpu: 200m   memory: 128Mi
    Requests:  cpu: 50m    memory: 64Mi
    Readiness: http-get http://:http/index.html
               delay=2s timeout=1s period=3s
               #success=1 #failure=3
```

WHAT TO LOOK AT

1. **This is the whole template, not a diff**
   --revision renders the stored Pod template of that ReplicaSet. Diff two revisions by rendering both.

2. **pod-template-hash is inside the labels**
   The controller injected it. You never write this label, and editing it breaks ownership.

3. **Resources travel with the revision**
   A rollback restores the old requests and limits too, which can change the Pod's QoS class.

4. **Readiness settings are part of the contract**
   Roll back and you also roll back the probe timings that define what AVAILABLE means.

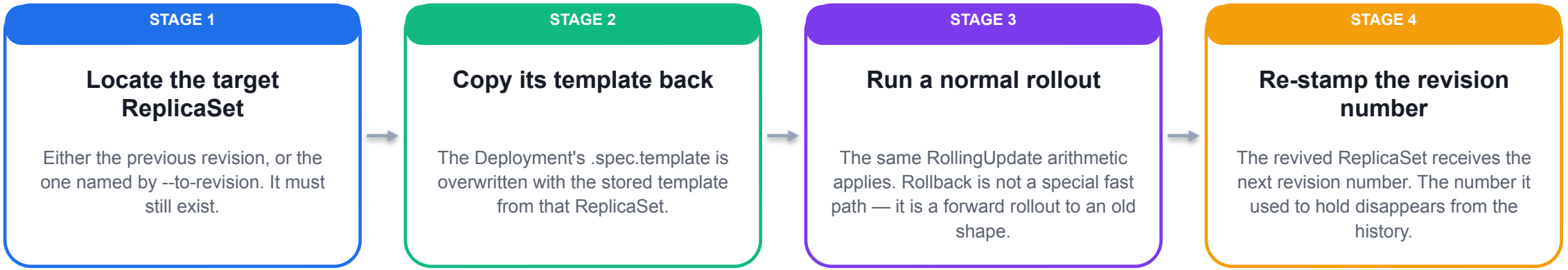
Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

What `kubectl rollout undo` Actually Does

There is no restore-from-backup step anywhere in this sequence.



WHAT IT MEANS

Rollback is a forward change that happens to point backwards. That is why it obeys maxSurge, maxUnavailable and progressDeadlineSeconds exactly like any other rollout.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

History After a Rollback

The revision numbering after an undo surprises almost everyone the first time. Read it carefully.

Lab 07 — captured on the isolated kind cluster

```
$ kubectl rollout undo deployment/freight-portal \
    -n kcna-lab07
deployment.apps/freight-portal rolled back

$ kubectl rollout history deployment/freight-portal \
    -n kcna-lab07
deployment.apps/freight-portal
REVISION  CHANGE-CAUSE
1         r24.1 - initial portal release
3         r24.3 - BROKEN image tag (deliberate)
4         r24.2 - customs banner, zero-unavailable rollout

NAME                          REVISION   DESIRED   IMAGE
freight-portal-5f7b9c4d68     3          0         nginx:1.27-alpine-mfs-hotfix
freight-portal-6c9f4b8d57     1          0         nginx:1.27-alpine
freight-portal-7d84c65f9b     4          4         nginx:1.27-alpine
```

WHAT TO LOOK AT

1. **Revision 2 has vanished**
   Its ReplicaSet was re-stamped as revision 4. The object is the same; only the annotation changed.

2. **The broken revision 3 is still listed**
   Rollback does not delete the bad revision. It stays at DESIRED 0 as a record until history is pruned.

3. **Only one ReplicaSet has DESIRED 4**
   Exactly one revision is live at a time. The others exist purely as rollback targets.

4. **Numbers are not stable identities**
   Never script against a revision number. Script against the image or the change-cause you control.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Rollout Wedged by a Bad Image — ProgressDeadlineExceeded

This is the single most valuable failure on Day 2: a safe stall, not an outage. The strategy did its job.

SYMPTOM

The rollout stops at one updated Pod and never finishes; the old version keeps serving traffic the whole time.

EVIDENCE — actual cluster output

```
$ kubectl rollout status deployment/freight-portal
Waiting for deployment "freight-portal" rollout to
finish: 1 out of 4 new replicas have been updated...
error: deployment "freight-portal" exceeded its
progress deadline

Warning  Failed   41s (x4)  kubelet  Failed to pull image
  "nginx:1.27-alpine-mfs-hotfix": ... not found
Warning  Failed   41s (x4)  kubelet  Error: ErrImagePull
Warning  Failed   5s  (x7)  kubelet  Error: ImagePullBackOff

Available=True    reason=MinimumReplicasAvailable
Progressing=False reason=ProgressDeadlineExceeded
```

ROOT CAUSE

- The new image tag does not exist in the registry, so the new Pod never becomes Ready.
- maxUnavailable: 0 forbids deleting any old Pod until a new one is Available — so the rollout correctly refuses to proceed.
- progressDeadlineSeconds: 120 elapsed with no progress, so the controller set Progressing=False.

FIX

- kubectl rollout undo to return to the last good revision — the wedge is safe to sit in while you decide.
- Read both conditions: Available=True means users are fine; Progressing=False means the change is not.
- Pin real digests or verified tags in CI so a typo cannot reach the cluster.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Deployment Status Fields and Conditions

.status.replicas	Total Pods owned across all its ReplicaSets
.status.updatedReplicas	Pods already running the current template
.status.readyReplicas	Pods whose readiness probes currently pass
.status.availableReplicas	Ready for at least minReadySeconds — the number the floor is measured against
.status.unavailableReplicas	Desired minus available; drives the maxUnavailable check
.status.observedGeneration	Which .metadata.generation the controller has processed

Available	True when availableReplicas meets the minimum — this is about users
Progressing	True while the rollout is advancing — this is about the change
ReplicaFailure	True when Pod creation is being refused, e.g. by quota
reason=NewReplicaSetAvailable	The healthy terminal reason on Progressing
reason=ProgressDeadlineExceeded	No progress within progressDeadlineSeconds
Rule of thumb	Available answers 'is the service up?'; Progressing answers 'is my change landing?'

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Editing an Immutable Selector

Immutability here is protecting you from an orphaned fleet, but it means selector design is a one-way door.

SYMPTOM

A label-scheme cleanup is rejected on apply; the Deployment cannot be edited into the new shape at all.

EVIDENCE — actual cluster output

```
$ kubectl apply -f 40-deployment-selector-change.yaml
The Deployment "freight-portal" is invalid:
spec.selector: Invalid value: v1.LabelSelector{
MatchLabels:map[string]string{
"app":"freight-portal", "tier":"frontend"},
MatchExpressions:[]v1.LabelSelectorRequirement(nil)}:
field is immutable
```

✓ **kubectl apply --dry-run=server surfaces this without touching the running Deployment**

ROOT CAUSE

- A Deployment's .spec.selector is settable once, at creation.
- Changing it would strand every existing ReplicaSet and Pod outside the new selector with no owner.
- The template labels may change freely — only the selector is frozen.

FIX

- Create a new Deployment with the new selector, shift traffic at the Service, then delete the old one.
- Or delete the Deployment with --cascade=orphan, then adopt the surviving Pods with the replacement.
- Design the label scheme before the first apply; treat the selector as part of the object's identity.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

DELIVERY PATTERNS

Blue/Green and Canary Are Built On These Primitives

Kubernetes ships one strategy field. Everything else in the release-engineering vocabulary is composed from Deployments, Services and labels.

Rolling update

- Native to the Deployment object — one strategy field.
- Two ReplicaSets, gradual replacement, automatic.
- Traffic split follows Pod counts, not policy.
- Rollback is kubectl rollout undo.
- Cost: two versions live simultaneously.

strategy.type | maxSurge | built-in

Blue/green

- Two complete Deployments, blue and green, both running.
- A Service selector edit cuts all traffic at once.
- Rollback is instantaneous — flip the selector back.
- Cost: double the capacity for the overlap window.
- Not a Kubernetes object; you assemble it.

2 Deployments | Service selector | assembled

Canary

- Small second Deployment sharing the Service selector.
- Split is by replica ratio, e.g. 1 canary to 9 stable.
- Promote by scaling up; abort by scaling to zero.
- Finer, weighted splits need a mesh or Gateway API.
- Argo Rollouts and Flagger automate the judgement.

replica ratio | Argo Rollouts | Flagger

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

CHOOSING A RELEASE PATTERN

Cost, Blast Radius and Rollback Speed

Pattern	Extra capacity	Rollback	Blast radius
RollingUpdate	maxSurge only (e.g. +25%)	Another full rollout, minutes	Grows gradually as Pods are replaced
Recreate	None	Another full rollout, plus a second outage	Total for the duration
Blue/green	100% — both stacks run	Seconds: revert one Service selector	All users at once, at the moment of the flip
Canary (replica ratio)	One or two extra Pods	Seconds: scale the canary to 0	Roughly the canary's share of replicas
Canary (mesh-weighted)	One or two extra Pods	Seconds: set the route weight to 0	Exactly the configured traffic percentage

CHOOSE

Rolling for routine change; canary when the risk is unknown; blue/green when rollback must be instant.

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

What the CLI Writes For You

Hand-writing apiVersion, kind and a matching selector is wasted keystrokes and a common source of error.

kubectl create deployment ... --dry-run=client -o yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: freight-portal
  name: freight-portal
spec:
  replicas: 4
  selector:
    matchLabels:
      app: freight-portal
  template:
    metadata:
      labels:
        app: freight-portal
    spec:
      containers:
      - image: nginx:1.27-alpine
        name: nginx
```

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

READING IT

1. **--dry-run=client renders without sending**
   Nothing reaches the API server. --dry-run=server validates against admission but still creates nothing.
2. **The selector and template labels agree by construction**
   This is the reliable cure for the 'selector does not match template labels' rejection two modules ago.
3. **What it will not generate**
   No strategy block, no resources, no probes, no change-cause. Those are yours to add.
4. **Use it as a skeleton, not an answer**
   Generate, then edit in the fields that make it production-worthy, then commit the file.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 07 — Deployments, Rollout Strategy and Rollback

GOAL

Drive a Deployment through three revisions of the MFS Freight Portal, make maxSurge and maxUnavailable arithmetic visible during the update, then wedge the rollout with a bad image and recover with rollout undo.

YOU WILL PRODUCE

- A freight-portal Deployment at revision 2 serving the r24.2 page, with a readable rollout history and change-cause column
- A wedged revision 3 stuck in ImagePullBackOff while the old ReplicaSet keeps 4 Pods Available
- A verified rollback to revision 2 confirmed by curl through the ClusterIP Service

CONCEPTS EXERCISED

- A Deployment owns ReplicaSets; a ReplicaSet owns Pods - rollouts are ReplicaSet scaling, not Pod mutation
- maxSurge sets the ceiling (replicas + surge) and maxUnavailable sets the floor (replicas - unavailable)
- rollout status / history / undo, kubernetes.io/change-cause and revisionHistoryLimit
- .spec.selector is immutable on an existing Deployment

LAB	NAMESPACE	MAPS TO	TIME
Lab 07	kcna-lab07	LO2 · A3 · K3	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-07-deployments-rollout-rollback/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1:10 – 1:50 PM

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · WORKLOADS & SCHEDULING

10

DaemonSets, Jobs, CronJobs and the Adapter Pattern

Five controllers, five different definitions of "correct". Choosing the wrong one is the most expensive mistake here.

© 2026 Tertiary Infotech Academy Pte Ltd

THE WHOLE FAMILY

Five Controllers, Compared

Read the third column first. It is the only column that actually decides which object you should be writing.

Controller	API group	Defines correct as	Scale is set by
Deployment	apps/v1	N interchangeable Pods on the current template	.spec.replicas, or an HPA
ReplicaSet	apps/v1	N Pods matching a selector	.spec.replicas — usually owned by a Deployment
StatefulSet	apps/v1	N ordered Pods with stable names and stable storage	.spec.replicas, applied in ordinal order
DaemonSet	apps/v1	One Pod on every node that passes node selection	The cluster — there is no replicas field
Job	batch/v1	.spec.completions successful Pod terminations	completions and parallelism
CronJob	batch/v1	A Job created at each scheduled time	The schedule, bounded by concurrencyPolicy

Source: kubernetes.io/docs/concepts/workloads/controllers/job/

© 2026 Tertiary Infotech Academy Pte Ltd

NODE-SCOPED WORKLOADS

DaemonSet Mechanics

If the right number of copies is 'however many nodes there are', a Deployment is the wrong object.

1 One Pod per eligible node

The controller watches Node objects. A node joining the cluster gets a Pod; a node leaving takes its Pod with it.

2 Eligibility is normal scheduling

nodeSelector and node affinity in the template narrow the set. The default scheduler places DaemonSet Pods like any other Pod.

3 Tolerations are usually required

To land on control-plane nodes a DaemonSet must tolerate node-role.kubernetes.io/control-plane:NoSchedule.

4 Default tolerations are added automatically

The DaemonSet controller adds tolerations for node pressure and unschedulable taints so agents survive node trouble.

5 Its own rolling update strategy

updateStrategy RollingUpdate with maxUnavailable, or OnDelete where Pods are only replaced when you delete them.

6 Classic uses

CNI agents, kube-proxy, log shippers, node exporters, storage plugins, security agents — infrastructure, not apps.

Source: kubernetes.io/docs/concepts/workloads/controllers/daemonset/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A DaemonSet Agent, Field by Field

A node agent needs two things the average Deployment does not: a toleration and knowledge of its own node.

labs/lab-08/manifests/10-daemonset-berth-agent.yaml

```yaml
kind: DaemonSet
spec:
  selector:
    matchLabels: {app: berth-agent}
  template:
    spec:
      tolerations:
      - key: node-role.kubernetes.io/control-plane
        operator: Exists
        effect: NoSchedule
      containers:
      - name: agent
        image: busybox:1.36
        env:
        - name: NODE_NAME
          valueFrom:
            fieldRef:
              fieldPath: spec.nodeName
```

READING IT

1. **No replicas field exists**
   jsonpath for .spec.replicas returns nothing. The desired count is derived from the node set, every pass.
2. **The toleration uses Exists, not Equal**
   It tolerates the control-plane taint whatever its value. This is the standard idiom for role taints.
3. **fieldRef spec.nodeName is the downward API**
   The agent learns which node it is on without an API call and without credentials.
4. **kind removes the control-plane taint on single-node clusters**
   So Lab 08 sees DESIRED 1 either way — check the node's taints rather than assuming.

Source: kubernetes.io/docs/concepts/workloads/controllers/daemonset/

© 2026 Tertiary Infotech Academy Pte Ltd

RUN-TO-COMPLETION

Job Semantics You Must Know

A Job is the only workload where 'the Pod exited' is the success condition rather than the failure condition.

1 completions and parallelism

completions is how many successes are needed in total; parallelism is how many Pods may run at once. They are independent numbers.

2 completionMode: Indexed

Each Pod gets a unique index 0..completions-1 in an annotation, an env var and its hostname — for sharded work.

3 backoffLimit counts Pod failures

backoffLimit: 2 means one initial attempt plus two retries. Exceeding it sets Failed with BackoffLimitExceeded.

4 Retry backoff is exponential

Roughly 10s, 20s, 40s, doubling and capped at six minutes — so a fast-failing Job still slows down.

5 restartPolicy must be Never or OnFailure

Always is rejected. OnFailure restarts the container in place; Never creates a fresh Pod for each attempt.

6 ttlSecondsAfterFinished cleans up

The TTL controller deletes the finished Job and its Pods after the given time. Without it, Jobs accumulate forever.

Source: kubernetes.io/docs/concepts/workloads/controllers/job/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

An Indexed Job, Field by Field

Read completions and parallelism together: they describe the shape of the work, not the size of the cluster.

labs/lab-08/manifests/20-job-manifest-reconcile.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: manifest-reconcile
spec:
  completions: 6
  parallelism: 2
  completionMode: Indexed
  backoffLimit: 2
  ttlSecondsAfterFinished: 1800
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: reconcile
        image: busybox:1.36
```

READING IT

1. **Six successes, two at a time**
   The controller keeps two Pods in flight until six have succeeded — three waves on this configuration.

2. **Indexed gives each Pod a shard**
   Index 0-5 is exposed as JOB_COMPLETION_INDEX and appears in the Pod name, so work can be partitioned without a queue.

3. **backoffLimit: 2 permits three Pods per failure path**
   One attempt plus two retries. Lab 08 observes exactly three Pods created before BackoffLimitExceeded.

4. **The template is immutable once created**
   You cannot edit a running Job's container. Delete and recreate — which is why the TTL matters.

Source: kubernetes.io/docs/concepts/workloads/controllers/job/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

A Completed Indexed Job

Three signals agree here: the Job status, the index range and the Pod names.

Lab 08 — captured on the isolated kind cluster

```
$ kubectl get job manifest-reconcile -n kcna-lab08
NAME                 STATUS     COMPLETIONS   DURATION   AGE
manifest-reconcile   Complete   6/6           26s        30s

$ kubectl get job manifest-reconcile -n kcna-lab08 \
    -o jsonpath='completed={.status.completedIndexes} '\
'succeeded={.status.succeeded}'
completed=0-5 succeeded=6

$ kubectl get pods -n kcna-lab08 -l job-name=manifest-reconcile
NAME                           READY   STATUS      AGE
manifest-reconcile-0-4k9tp     0/1     Completed   30s
manifest-reconcile-1-lq7d2     0/1     Completed   30s
manifest-reconcile-5-t8vzn     0/1     Completed   12s
```

WHAT TO LOOK AT

1. **STATUS Complete, COMPLETIONS 6/6**
   The Job object reports its own terminal state; you do not have to count Pods yourself.

2. **completedIndexes is a compressed range**
   0-5 means every index finished. A gap such as 0-2,4 tells you exactly which shard is missing.

3. **The index is in the Pod name**
   manifest-reconcile-0-4k9tp is index 0. Under Indexed mode the shard is visible without reading logs.

4. **READY 0/1 with STATUS Completed is correct**
   The container has exited. A Job Pod that stays 1/1 Ready forever is a Job that will never finish.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A Job That Exhausts Its backoffLimit

Three Pods, one root cause. The retry count is a policy you chose, not a diagnosis.

SYMPTOM

The Job never completes; three Pods are created in turn and each exits non-zero for the same reason.

EVIDENCE — actual cluster output

```
$ kubectl get job customs-export -n kcna-lab08
NAME             STATUS   COMPLETIONS   DURATION   AGE
customs-export   Failed   0/1           41s        2m

Normal   SuccessfulCreate  2m10s  job-controller
  Created pod: customs-export-9wq4k
Normal   SuccessfulCreate  2m     job-controller
  Created pod: customs-export-cn2xr
Normal   SuccessfulCreate  95s    job-controller
  Created pod: customs-export-4tbvz
Warning  BackoffLimitExceeded  89s  job-controller
  Job has reached the specified backoff limit
```

✓ **STATUS returns to Complete with COMPLETIONS 1/1 once the source path is corrected**

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

ROOT CAUSE

- The container cannot find its input file and exits non-zero: "wc: /data/customs-export.csv: No such file or directory".
- backoffLimit: 2 permits one attempt plus two retries, so exactly three Pods were created.
- restartPolicy: Never means each attempt is a new Pod, which is why the failures are visible as separate objects.

FIX

- Read the last Pod's logs, not the Job's events, for the actual application error.
- Fix the input path or mount, then delete and re-create the Job — its template is immutable.
- Set a realistic backoffLimit: retrying a missing file twice buys nothing but noise.

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A CronJob, Field by Field

A CronJob is a Job factory. Everything about the work itself lives one level down, inside jobTemplate.

labs/lab-08/manifests/40-cronjob-tariff-sync.yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: tariff-sync
spec:
  schedule: "*/2 * * * *"
  timeZone: "Asia/Singapore"
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 60
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          restartPolicy: OnFailure
```

READING IT

1. **schedule is standard five-field cron**
   minute, hour, day-of-month, month, day-of-week. "*/2" in the first field means every second minute.
2. **timeZone is a real field, not a comment**
   Stable since Kubernetes v1.27. Without it the schedule is interpreted in the controller manager's own time zone.
3. **concurrencyPolicy governs overlap**
   Forbid skips the run if the previous Job is still going; Replace kills it; Allow (the default) lets both run.
4. **Two history limits, not one**
   Three successful and one failed Job are retained. This is how you keep evidence without unbounded growth.

Source: kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Cron Fields and CronJob Behaviour

Field order	minute (0-59) hour (0-23) day-of-month (1-31) month (1-12) day-of-week (0-6)
***/n**	Every n units of that field
a-b	Inclusive range; a,b is an explicit list
@hourly / @daily	Shorthand macros, also accepted
concurrencyPolicy: Allow	Default. Overlapping Jobs run in parallel
concurrencyPolicy: Forbid	Skip this occurrence if the previous Job is still active

concurrencyPolicy: Replace	Delete the running Job and start the new one
startingDeadlineSeconds	Give up on a missed occurrence after this many seconds
suspend: true	Stop creating new Jobs; running Jobs are unaffected
Missed-schedule limit	More than 100 missed start times and the controller stops and reports an error
Guarantee level	At-least-once in practice — a Job may occasionally be created twice, so make the work idempotent
Manual fire	kubectl create job <name> --from=cronjob/<name> for an out-of-band run

Source: kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/

© 2026 Tertiary Infotech Academy Pte Ltd

PREVIEW OF DAY 4

Why StatefulSet Is a Different Animal

You will build one in Lab 19. Today you only need to recognise the three guarantees that make it the right choice.

Stable identity

- Pods are named <sts>-0, <sts>-1, ... not random.
- The name survives rescheduling to another node.
- A headless Service gives each Pod its own DNS name.
- Peers can therefore find each other by name.
- A Deployment can promise none of this.

ordinal names | headless Service

Stable storage

- volumeClaimTemplates create one PVC per ordinal.
- Pod 2 always reattaches to the same PVC.
- PVCs deliberately outlive the Pods that used them.
- Deleting the StatefulSet does not delete the claims.
- Covered properly in Day 4 with PV/PVC and CSI.

volumeClaimTemplates | per-ordinal PVC

Ordered operations

- Scale-up proceeds 0, 1, 2 and waits for Ready.
- Scale-down and rolling updates go in reverse order.
- podManagementPolicy: Parallel opts out of the wait.
- partition in updateStrategy gates a staged rollout.
- The cost is that rollouts are slower by design.

ordered ready | partition

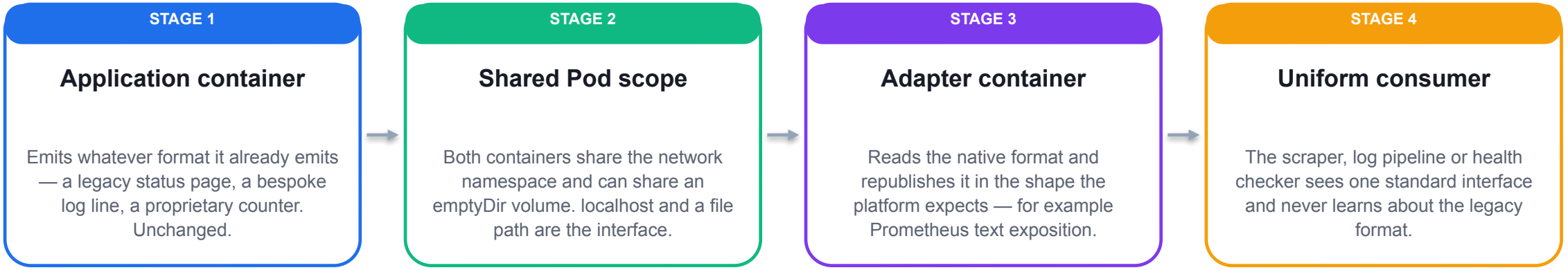
Source: kubernetes.io/docs/concepts/workloads/controllers/statefulset/

© 2026 Tertiary Infotech Academy Pte Ltd

COMPOSITION PATTERN

The Adapter Pattern Inside One Pod

Registered Topic 2 names the adapter pattern explicitly — it is the workload-composition counterpart to the controller patterns in this module.



WHAT IT MEANS

The adapter normalises output for the outside world. Contrast: a sidecar adds capability to the app, and an ambassador normalises the app's outbound connections.

Source: kubernetes.io/blog/2015/06/the-distributed-system-toolkit-patterns/

© 2026 Tertiary Infotech Academy Pte Ltd

TELLING THEM APART

Sidecar, Adapter and Ambassador

All three are just extra containers in one Pod. The distinction is intent, and intent is what an exam question tests.

Pattern	Direction	What it normalises	Concrete example
Sidecar	Alongside the app	Nothing — it adds a capability the app lacks	Log shipper, config reloader, certificate refresher
Adapter	Outbound, app to platform	The app's output format	Exporter turning a bespoke status page into Prometheus metrics
Ambassador	Outbound, app to network	The app's outbound connections	Local proxy so the app dials localhost and the proxy handles sharding or TLS
Init container	Before the app starts	Nothing — it establishes preconditions	Schema migration, waiting for a dependency, fetching config

CHOOSE

Ask which direction data flows and who is being spared a change — that names the pattern.

Source: kubernetes.io/blog/2015/06/the-distributed-system-toolkit-patterns/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 08 — DaemonSets, Jobs and CronJobs

GOAL

Run the three non-Deployment workload controllers against real MFS port data: a per-node DaemonSet agent, an Indexed Job that splits a work queue across parallel Pods, and a CronJob on a schedule - then fail a Job past its backoffLimit and read the reason.

YOU WILL PRODUCE

- A berth-agent DaemonSet reporting DESIRED=CURRENT=1 on the single-node kind cluster, reading berth-readings.csv
- A completed Indexed Job (completions 6, parallelism 2) whose six Pods each processed a distinct queue line
- A tariff-sync CronJob with concurrencyPolicy Forbid and a retained Job history

CONCEPTS EXERCISED

- DaemonSet replica count is derived from the node set, never from a replicas field
- completions vs parallelism vs backoffLimit, and Indexed completion mode with JOB_COMPLETION_INDEX
- CronJob schedule, timeZone, concurrencyPolicy and the successful/failed Job history limits
- A Job that exhausts backoffLimit is marked Failed with reason BackoffLimitExceeded and stops retrying

LAB	NAMESPACE	MAPS TO	TIME
Lab 08	kcna-lab08	LO2 · A3 · K3	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-08-daemonsets-jobs-cronjobs/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA · KUBERNETES FUNDAMENTALS — SCHEDULING

11

The Scheduler: Filter, Score, Bind

kube-scheduler writes exactly one field. Everything else in this module is about how it decides what to write there.

© 2026 Tertiary Infotech Academy Pte Ltd

THE SCHEDULER'S JOB

A Pod with an empty .spec.nodeName is the scheduler's entire input queue.

Binding is one API write. Placement policy, affinity, taints, priority and spread constraints all exist only to influence that single field.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

The Scheduling Framework

Two cycles. The scheduling cycle is serialised for correctness; the binding cycle is asynchronous for speed.

SCHEDULING CYCLE — ONE POD AT A TIME

PreFilter
Cheap precomputation and early rejection before any node is examined

Filter (predicates)
Each node is kept or discarded: fit, taints, affinity, volume topology, node conditions

Score (priorities)
Every surviving node scored 0-100 per plugin, then combined using plugin weights

Reserve / Permit
Provisionally claim resources; extension point where plugins may delay or reject

BINDING CYCLE — MAY RUN IN PARALLEL

PreBind
Work that must complete first, typically volume attachment and provisioning

Bind
POST to the Pod's binding subresource, which sets .spec.nodeName

PostBind
Cleanup and notification; the kubelet on that node now sees the Pod on its watch

MECHANISM

If no node survives Filter the Pod stays Pending and a FailedScheduling Event is emitted — the scheduler retries with backoff, forever, until something changes.

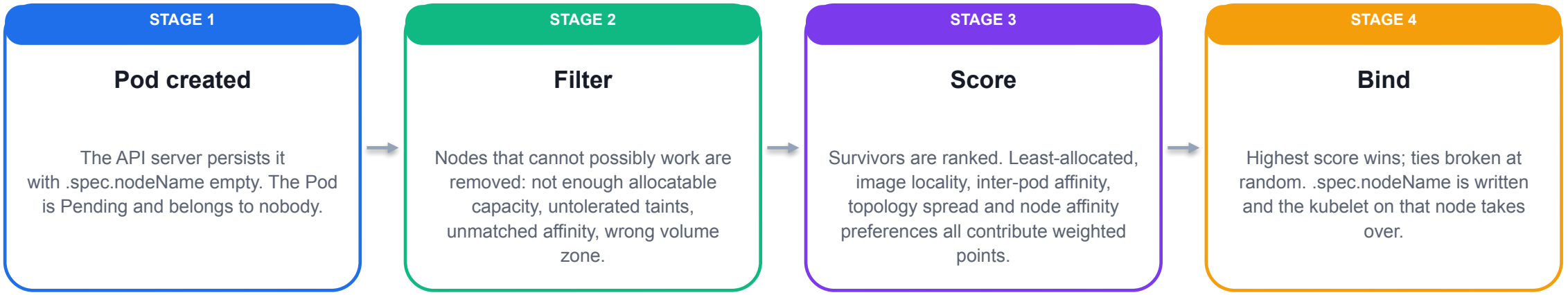
Source: kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

From Pending to Running

Learn this order. Almost every scheduling question resolves to 'was that a filter or a score?'.



WHAT IT MEANS

Filter answers 'can it run here?'. Score answers 'where is it best?'. Only Filter can leave a Pod Pending — a low score never blocks scheduling.

Source: kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/

© 2026 Tertiary Infotech Academy Pte Ltd

FILTERING

What Can Eliminate a Node Outright

Every one of these produces a distinct clause in the FailedScheduling message — which is why that message is worth reading word by word.

1 Insufficient allocatable resources

Sum of requests of existing Pods plus this Pod's requests must fit within the node's Allocatable. Limits are ignored here.

2 Untolerated NoSchedule or NoExecute taints

The node repels the Pod unless the Pod carries a matching toleration. PreferNoSchedule does not filter, it only scores.

3 Unmatched required node affinity or nodeSelector

requiredDuringSchedulingIgnoredDuringExecution and nodeSelector are both hard filters.

4 Node conditions and unschedulability

A cordoned node carries node.kubernetes.io/unschedulable; memory or disk pressure taints filter too.

5 Volume constraints

Node volume limits, zone of an already-bound PersistentVolume, and CSI topology all restrict the node set.

6 Ports and hostname

hostPort collisions and required hostname or OS/architecture labels remove nodes before scoring begins.

Source: kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Scoring Plugins and What They Reward

NodeResourcesFit	By default LeastAllocated — spread load across nodes
MostAllocated (opt-in)	Bin-packing instead; better for autoscaler-driven cost reduction
ImageLocality	Nodes that already hold the image start the Pod faster
InterPodAffinity	Rewards or penalises co-location with matching Pods
NodeAffinity	Applies the weights from preferredDuringScheduling terms
PodTopologySpread	Rewards nodes that reduce skew across a topology domain

TaintToleration	Penalises nodes with tolerated PreferNoSchedule taints
NodeResourcesBalancedAllocation	Prefers nodes where CPU and memory usage stay proportional
Weights are configurable	KubeSchedulerConfiguration profiles set per-plugin weights
Multiple profiles	One scheduler binary can expose several named profiles
schedulerName	A Pod selects a profile or a wholly custom scheduler by name
Score never blocks	A score of zero still schedules if the node passed Filter

Source: kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/

© 2026 Tertiary Infotech Academy Pte Ltd

NODE SELECTION

nodeSelector vs Node Affinity

Both are evaluated in the same Filter stage. Affinity is strictly more expressive; nodeSelector is strictly shorter.

Aspect	nodeSelector	nodeAffinity
Location	.spec.nodeSelector — a flat map of labels	.spec.affinity.nodeAffinity — nested terms
Match logic	Every key/value must match exactly; equality only	In, NotIn, Exists, DoesNotExist, Gt, Lt
Soft option	None — it is always a hard requirement	preferredDuringSchedulingIgnoredDuringExecution with weights 1-100
Multiple alternatives	Impossible — you cannot express 'ssd OR nvme'	nodeSelectorTerms are ORed; matchExpressions within a term are ANDed
When to use	One obvious label, no fallback wanted	Anything with alternatives, negation or a preference

CHOOSE

nodeSelector for the trivial case; nodeAffinity the moment you need OR, NOT or a soft preference.

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

requiredDuringSchedulingIgnoredDuringExecution

The field name is long because it encodes two independent promises. Learners who read it as one word get the semantics wrong.

labs/lab-09/manifests/20-pod-affinity-required.yaml

```
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnored\
DuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: mfs.io/disk
            operator: In
            values:
            - ssd
            - nvme
          - key: mfs.io/decommission
            operator: DoesNotExist
```

READING IT

1. **Read the name as two halves**
   requiredDuringScheduling: a hard filter now. IgnoredDuringExecution: if the node's labels later change, the running Pod is not evicted.

2. **nodeSelectorTerms is a list — OR**
   A node matching any one term passes. Add a second term to express a genuine alternative.

3. **matchExpressions within a term is AND**
   This node must have disk in {ssd, nvme} AND must not carry the decommission key at all.

4. **DoesNotExist takes no values**
   It is the clean way to express 'not being drained'. Supplying values with it is a validation error.

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

Preferred Affinity and Its Weights

Lab 09 applies this Pod without ever creating the sg-tuas label — and it still reaches Running. That is the whole point of 'preferred'.

labs/lab-09/manifests/30-pod-affinity-preferred.yaml

```
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnored\
DuringExecution:
      - weight: 80
        preference:
          matchExpressions:
          - key: mfs.io/zone
            operator: In
            values: [sg-tuas]
      - weight: 20
        preference:
          matchExpressions:
          - key: mfs.io/disk
            operator: In
            values: [ssd, nvme]
```

READING IT

1. **This is a list of weighted preferences**
   Each entry pairs a weight of 1-100 with a preference. The structure differs from the required form — note 'preference', not 'nodeSelectorTerms'.

2. **Weights are summed into the node's score**
   A node in sg-tuas with an SSD earns 100 from this plugin; one with only the SSD earns 20.

3. **Nothing here can make a Pod Pending**
   If no node matches any preference the Pod still schedules — it simply lands wherever else scored highest.

4. **Weight is relative, not absolute**
   It competes with every other scoring plugin, so 80 does not guarantee the outcome.

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

DECODING THE NAMES

Four Affinity Suffixes, Two Questions

Two independent axes — hard versus soft, and scheduling time versus run time — produce the four long names.

requiredDuringScheduling...

1

Hard. Evaluated in Filter. No matching node means the Pod stays Pending indefinitely.

preferredDuringScheduling...

2

Soft. Evaluated in Score. Influences ranking only; never blocks placement.

...IgnoredDuringExecution

3

The only execution behaviour that exists today for node affinity: label changes do not disturb running Pods.

...RequiredDuringExecution

4

Would evict a running Pod when the node stops matching. Still not implemented — do not claim it in an exam answer.

Node affinity vs pod affinity

5

Node affinity matches node labels. Pod affinity matches labels on other Pods, then generalises via topologyKey.

Taints are the mirror image

6

Affinity is the Pod choosing nodes. Taints are the node rejecting Pods. Both are enforced in Filter.

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Pod Pending: Unsatisfiable Node Affinity

Read the two halves separately: the filter clause tells you why, the preemption clause tells you it is not a capacity problem.

SYMPTOM

draft-optimiser never leaves Pending; no container is ever created and no image is ever pulled.

EVIDENCE — actual cluster output

```
$ kubectl describe pod draft-optimiser -n kcna-lab09
Events:
  Type     Reason            Age   From
  Warning  FailedScheduling  63s   default-scheduler
    0/1 nodes are available: 1 node(s) didn't match
    Pod's node affinity/selector. preemption: 0/1
    nodes are available: 1 Preemption is not helpful
    for scheduling.

PodScheduled=False  reason=Unschedulable
spec.nodeName: []
```

✓ PodScheduled flips to True within seconds of the correct node label being applied — no restart needed

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

ROOT CAUSE

- The Pod requires mfs.io/hardware=gpu-a100 as a requiredDuringScheduling term and no node carries that label.
- Every node was removed in the Filter stage, so scoring never ran and nothing was bound.
- "Preemption is not helpful" means evicting lower-priority Pods would not help — the node is wrong, not full.

FIX

- Confirm the label really is absent: kubectl get nodes -l mfs.io/hardware=gpu-a100 returns No resources found.
- Either label a node that genuinely has the hardware, or downgrade the term to preferred if it was only a wish.
- Never 'fix' this by deleting the Pod — it will recur on the next apply.

© 2026 Tertiary Infotech Academy Pte Ltd

REPULSION

Taints and Their Three Effects

Taints are how a cluster reserves nodes for a purpose without needing every workload author to cooperate.

1 A taint lives on the Node

key=value:effect. It repels every Pod that does not carry a matching toleration. It is cluster-scoped state, not namespaced.

2 NoSchedule

New Pods without a toleration are filtered out. Pods already running on the node are untouched.

3 PreferNoSchedule

A scoring penalty only. The Pod will still land there if nothing better exists.

4 NoExecute

Filters new Pods and evicts running ones that do not tolerate it. tolerationSeconds gives a grace period before eviction.

5 Kubernetes taints nodes itself

node.kubernetes.io/not-ready, unreachable, memory-pressure, disk-pressure, pid-pressure and unschedulable are applied automatically.

6 Node labels and taints survive namespaces

Deleting the lab namespace does not remove them — Lab 09 explicitly reverts the taint and both labels at cleanup.

Source: kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A Toleration, Field by Field

Pair this with the previous slide: the node repels, the Pod carries the exemption. Neither side alone decides placement.

labs/lab-09/manifests/60-pod-toleration.yaml

```yaml
# node side, applied by the administrator
# taint: mfs.io/maintenance=window:NoSchedule

spec:
  tolerations:
  - key: mfs.io/maintenance
    operator: Equal
    value: window
    effect: NoSchedule
  # eviction grace period, NoExecute only:
  # - key: node.kubernetes.io/not-ready
  #   operator: Exists
  #   effect: NoExecute
  #   tolerationSeconds: 300
```

READING IT

1. **A toleration permits, it does not attract**

   Tolerating the maintenance taint lets this Pod land on that node. It does not make the scheduler prefer it.

2. **Equal needs a value; Exists must not have one**

   operator: Exists with no key at all tolerates every taint — powerful and easy to abuse.

3. **Omitting effect widens the toleration**

   With no effect field the toleration matches all three effects for that key.

4. **tolerationSeconds only means anything for NoExecute**

   The DefaultTolerationSeconds admission controller adds not-ready and unreachable tolerations of 300 seconds, which is why a Pod is not evicted the instant a node blips.

Source: kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Pod Pending: Untolerated Taint

Two Pods, one node, one difference. This is the cleanest demonstration of Filter-stage rejection in the course.

SYMPTOM

yard-report sits Pending on a cluster that has ample free CPU and memory.

EVIDENCE — actual cluster output

```
$ kubectl describe pod yard-report -n kcna-lab09
Events:
  Type     Reason            Age   From
  Warning  FailedScheduling  12s   default-scheduler
    0/1 nodes are available: 1 node(s) had untolerated
    taint {mfs.io/maintenance: window}. preemption:
    0/1 nodes are available: 1 Preemption is not
    helpful for scheduling.

$ kubectl get pods -n kcna-lab09 -o wide
crane-controller   Running   kind-control-plane
yard-report        Pending   <none>
```

✓ **The same Pod schedules immediately once the taint is removed or the toleration is added**

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

ROOT CAUSE

- The only node carries mfs.io/maintenance=window:NoSchedule and this Pod declares no toleration.
- crane-controller, an otherwise identical Pod, is Running purely because it carries the matching toleration.
- Free capacity is irrelevant — TaintToleration runs in Filter, before any capacity scoring.

FIX

- Add the matching toleration if the Pod genuinely belongs on maintenance-marked nodes.
- Or remove the taint with a trailing dash: kubectl taint node <node> mfs.io/maintenance=window:NoSchedule-
- Distinguish the messages: 'untolerated taint' is policy, 'Insufficient cpu' is capacity.

© 2026 Tertiary Infotech Academy Pte Ltd

DIRECTION OF CONTROL

Affinity, nodeSelector, Taints and Spread

Placement is a negotiation between two parties. Knowing which party owns each field tells you who to talk to when a Pod will not schedule.

Mechanism	Who declares it	Effect	Stage
nodeSelector	Pod author	Pod insists on nodes with these labels	Filter
required nodeAffinity	Pod author	Same, with richer operators and OR	Filter
preferred nodeAffinity	Pod author	Pod nudges the ranking	Score
Taint (NoSchedule)	Cluster administrator	Node rejects Pods that lack a toleration	Filter
Toleration	Pod author	Pod is exempted from a taint — permission, not attraction	Filter
Topology spread	Pod author	Limits skew across zones or nodes; hard or soft by whenUnsatisfiable	Filter or Score

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

POD-RELATIVE PLACEMENT

Pod Affinity, Anti-Affinity and topologyKey

Node affinity asks about the machine. Pod affinity asks about the neighbours — and topologyKey defines what 'neighbour' means.

1 Match Pods, then generalise to a domain

labelSelector picks Pods; topologyKey names the node label whose value defines the domain to co-locate within or spread across.

2 topologyKey is the whole idea

kubernetes.io/hostname means per-node. topology.kubernetes.io/zone means per-zone. Same rule, different blast radius.

3 podAffinity co-locates

Put the cache in the same zone as the app to cut cross-zone latency and egress cost.

4 podAntiAffinity separates

Required anti-affinity on hostname is the classic 'never two replicas on one node' rule.

5 It is expensive at scale

Evaluating these rules is O(Pods x nodes); the documentation warns against them on clusters of many hundreds of nodes.

6 namespaceSelector widens the scope

By default only Pods in the same namespace are considered; namespaces or namespaceSelector extends the match.

Source: kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

Pod Topology Spread Constraints

This is the modern answer to 'spread my replicas out'. It is quantitative where podAntiAffinity is binary.

topologySpreadConstraints — HA across zones and nodes

```
spec:
  topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: DoNotSchedule
    labelSelector:
      matchLabels:
        app: freight-portal
  - maxSkew: 2
    topologyKey: kubernetes.io/hostname
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        app: freight-portal
```

READING IT

1. **maxSkew bounds the difference**
   The gap between the most and least populated domain may not exceed this number for the selected Pods.

2. **whenUnsatisfiable picks the stage**
   DoNotSchedule makes it a Filter rule — the Pod can go Pending. ScheduleAnyway demotes it to a Score preference.

3. **Two constraints are ANDed**
   Spread evenly across zones and loosely across nodes at the same time. This is the standard HA shape.

4. **More precise than anti-affinity**
   Anti-affinity is all-or-nothing per domain; spread lets you say 'balanced to within one'.

Source: kubernetes.io/docs/concepts/scheduling-eviction/topology-spread-constraints/

© 2026 Tertiary Infotech Academy Pte Ltd

WHEN THE CLUSTER IS FULL

PriorityClass, Preemption and Scheduling Gates

Priority changes who wins a contested node. Scheduling gates change when the contest happens at all.

1 PriorityClass is a cluster-scoped object

It maps a name to an integer value. Pods reference it by .spec.priorityClassName; admission stamps .spec.priority.

2 globalDefault applies to Pods that name nothing

Only one PriorityClass may set it. Without one, unlabelled Pods get priority 0.

3 Preemption evicts lower-priority Pods

If a pending high-priority Pod would fit after evicting lower-priority victims, the scheduler removes them and reserves the space.

4 preemptionPolicy: Never opts out

The Pod still enjoys higher scheduling priority in the queue but never causes an eviction.

5 System classes are reserved

system-cluster-critical and system-node-critical protect control-plane and node add-ons from eviction.

6 Scheduling gates hold a Pod before Filter

.spec.schedulingGates keeps the Pod SchedulingGated; an external controller clears the list when its precondition is met.

Source: kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTERED TOPIC ITEM

Scheduling a Pod Without the Scheduler

Three ways to get a Pod onto a node without the scheduler choosing — and one that only looks like it.

Route	How placement happens	Who owns the Pod object	Correct use
spec.nodeName set by you	The scheduler is bypassed entirely; the kubelet on that node picks the Pod up from its watch	You — nothing recreates it	Debugging a specific node; almost never in production
Static Pod	kubelet reads a manifest from its configured staticPodPath directory and starts it directly	The kubelet; the API server only holds a mirror Pod	Bootstrapping the control plane itself, before any API server exists
Custom schedulerName	A different scheduler binary watches for that name and performs the binding	Your controller	Batch, gang or hardware-aware scheduling
DaemonSet	Normal scheduling, but the controller creates one Pod per node so placement looks automatic	The DaemonSet controller	The supported way to get a Pod on every node

Source: kubernetes.io/docs/tasks/configure-pod-container/static-pod/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Node Taints and Labels Before Placement

Diagnose placement from the node side first. Two commands here explain both Pending Pods in this module.

Lab 09 — captured on the isolated kind cluster

```
$ kubectl describe node kind-control-plane | head
Labels:  kubernetes.io/arch=arm64
         kubernetes.io/hostname=kind-control-plane
         kubernetes.io/os=linux
         mfs.io/disk=ssd
         mfs.io/zone=sg-harbourfront
Taints:  mfs.io/maintenance=window:NoSchedule
Unschedulable:  false

$ kubectl get nodes -l mfs.io/hardware=gpu-a100
No resources found

$ kubectl get pod crane-controller -n kcna-lab09 \
    -o jsonpath='{.spec.nodeName}'
kind-control-plane
```

WHAT TO LOOK AT

1. **Labels are the affinity surface**
   mfs.io/disk and mfs.io/zone were applied by hand in Lab 09; the kubernetes.io/* ones are set by the kubelet.

2. **Taints is a separate field from Labels**
   A taint is not a label. Selectors cannot see taints and tolerations cannot see labels.

3. **No resources found is a real answer**
   It proves the gpu-a100 requirement is unsatisfiable rather than merely slow.

4. **Unschedulable: false**
   This is what kubectl cordon flips. A cordoned node also gains the unschedulable taint automatically.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 09 — Scheduling: nodeSelector, Affinity, Taints and Tolerations

GOAL

Create the placement conditions yourself on a single-node kind cluster - label the node, taint it, untaint it - and prove how nodeSelector, required vs preferred nodeAffinity and tolerations decide where a Pod lands, including one Pod left Pending forever.

YOU WILL PRODUCE

- A node labelled from data/node-placement-policy.csv and temporarily tainted mfs.io/maintenance=window:NoSchedule
- Four Pods placed by nodeSelector, required affinity, preferred affinity and a toleration
- Two Pods stuck Pending with readable FailedScheduling events: one unsatisfiable affinity, one missing toleration

CONCEPTS EXERCISED

- Scheduling is filter-then-score: hard constraints eliminate nodes, soft preferences only rank them
- requiredDuringScheduling blocks placement; preferredDuringScheduling only lowers a node's score
- Taints repel, tolerations permit - a toleration never attracts a Pod to a node
- FailedScheduling events name the exact predicate that rejected each node

LAB	NAMESPACE	MAPS TO	TIME
Lab 09	kcna-lab09	LO2 · A3 · K3	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-09-scheduling-affinity-taints/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Afternoon Break

3:50 – 4:00 PM

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2 · WORKLOADS & SCHEDULING

12

Resource Limits, QoS and Autoscaling

Requests are a scheduling promise. Limits are a runtime ceiling. They are enforced by different components, at different times, with different consequences.

© 2026 Tertiary Infotech Academy Pte Ltd

THE TWO NUMBERS

What requests and limits Actually Do

Two fields, two enforcement points: the scheduler at admission time, the kubelet for the container's whole life.

1 requests is the scheduler's input

The Filter stage sums the requests of all Pods on a node and checks the new Pod still fits inside Allocatable. Nothing else uses it for placement.

2 requests is also a floor at runtime

The CPU request becomes the container's cgroup share weight, so under contention it receives at least its proportion.

3 limits is the kubelet's runtime ceiling

Enforced by the container runtime through cgroups. The scheduler never looks at limits when placing a Pod.

4 Set on the container, not the Pod

The Pod's effective request is the sum across its regular containers, compared against the largest single init container — not a plain total of everything.

5 Units are exact and easy to get wrong

CPU: 1 = 1000m = one core-second per second. Memory: Mi is 2^20 bytes, M is 10^6 bytes. They are not the same number.

6 Omitting requests is a decision

A container with no request may be scheduled anywhere and is the first candidate for eviction under node pressure.

Source: kubernetes.io/docs/concepts/configuration/manage-resources-containers/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A Resources Block, Field by Field

Read the pair together. The request buys you a place; the limit is where the kernel stops you.

labs/lab-10/manifests/11-pod-qos-burstable.yaml

```
spec:
  containers:
  - name: rate-quoter
    image: nginx:1.27-alpine
    resources:
      requests:
        cpu: 50m
        memory: 64Mi
      limits:
        cpu: 200m
        memory: 192Mi
# QoS class computed by the kubelet: Burstable
# scheduler reserves 50m / 64Mi on the node
# cgroup ceiling is 200m CPU / 192Mi memory
```

READING IT

1. **50m reserves one twentieth of a core**
   That is what the scheduler subtracts from the node's Allocatable. The other 150m of headroom is not reserved for anybody.
2. **The gap between request and limit is the burst**
   It is best-effort capacity: available if the node is quiet, throttled away if it is busy.
3. **Requests below limits means Burstable**
   Any container where the two differ, or where only one is set, puts the whole Pod in the Burstable class.
4. **Overcommitment is the point**
   A cluster where every Pod could simultaneously hit its limits is a cluster you have hugely overpaid for.

Source: kubernetes.io/docs/concepts/configuration/manage-resources-containers/

© 2026 Tertiary Infotech Academy Pte Ltd

THE ASYMMETRY THAT CATCHES EVERYONE

CPU Throttling vs Memory OOMKill

This single distinction explains most production resource incidents, and it is a reliable exam topic.

Aspect	CPU — compressible	Memory — incompressible
What the limit does	Caps the cgroup CPU quota per 100 ms period	Caps the cgroup memory ceiling in bytes
On exceeding it	The container is throttled: it waits and runs slower	The kernel OOM killer terminates the process
Visibility	container_cpu_cfs_throttled_seconds_total — a metric you must go looking for	State Terminated, Reason OOMKilled, Exit Code 137 — impossible to miss
User experience	Latency rises; requests time out; nothing crashes	Abrupt restart, in-flight work lost
Practical guidance	A tight CPU limit is a silent latency bug — many teams set requests and omit CPU limits	Always set a memory limit; without one a leak takes the whole node down

CHOOSE

Reclaim CPU by slowing a process; the only way to reclaim memory is to kill something.

Source: kubernetes.io/docs/concepts/configuration/manage-resources-containers/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Node Allocatable and Allocated

This is the single most useful capacity command in the course. Read the requests column, not the limits column.

Lab 10 — captured on the isolated kind cluster

```
$ kubectl get nodes -o custom-columns=NODE:.metadata.name,\
CPU_ALLOC:.status.allocatable.cpu,\
MEM_ALLOC:.status.allocatable.memory,PODS:.status.allocatable.pods
NODE                 CPU_ALLOC   MEM_ALLOC   PODS
kind-control-plane   8           8039488Ki   110

$ kubectl describe node kind-control-plane
Non-terminated Pods:
  NAMESPACE   NAME           CPU Requests  CPU Limits  Memory Requests
  kcna-lab10  berth-ledger   100m (1%)     100m (1%)   128Mi (1%)
  kcna-lab10  notice-board   0 (0%)        0 (0%)      0 (0%)
  kcna-lab10  rate-quoter    50m (0%)      200m (2%)   64Mi (0%)
Allocated resources:
  (Total limits may be over 100 percent, i.e. overcommitted.)
```

WHAT TO LOOK AT

1. **Allocatable, not Capacity**
   Allocatable is Capacity minus kube-reserved, system-reserved and the eviction threshold. Only Allocatable is schedulable.
2. **The percentages are of Allocatable**
   100m of 8 cores is 1%. This table is the scheduler's arithmetic made visible.
3. **notice-board contributes 0 (0%)**
   A BestEffort Pod reserves nothing, so the scheduler will keep placing Pods as if it were not there.
4. **Limits may exceed 100%**
   kubectl says so itself. Overcommitment is normal; it is requests, not limits, that must fit.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

NODE ACCOUNTING

Capacity, Allocatable, Requested, Used

Four different numbers that people routinely conflate. Only one of them is what the scheduler uses.

Capacity

1

Everything the machine physically has, as reported by the kubelet at registration.

Allocatable

2

Capacity minus kube-reserved, system-reserved and eviction-hard thresholds. This is the number the scheduler divides up.

Requested (Allocated)

3

The sum of container requests for non-terminated Pods. The scheduler will not let this exceed Allocatable.

Used

4

Actual consumption right now, from the metrics pipeline. It is unrelated to the scheduler's arithmetic.

The common misdiagnosis

5

A node at 12% actual CPU can still reject Pods, because Requested is already at Allocatable. Nothing is wrong.

The opposite trap

6

A node whose requests are modest can still be in memory pressure if the Pods are all bursting past their requests.

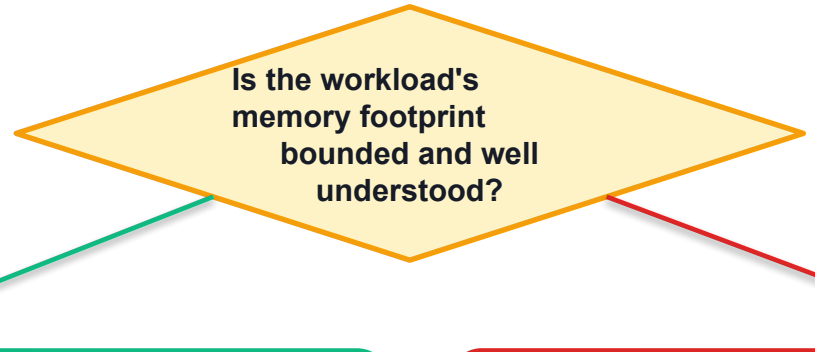
Source: kubernetes.io/docs/concepts/configuration/manage-resources-containers/

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN DECISION

How Should I Set Requests and Limits?

The question is about memory, because memory is the resource where being wrong kills the process.



YES — SIZE IT DELIBERATELY

- Set memory request = limit at the observed steady-state peak plus headroom. Predictable, and Guaranteed if CPU matches too.
- Set the CPU request at the p50-p90 of observed usage so the scheduler reserves real work, not a wish.
- Consider omitting the CPU limit for latency-sensitive services; keep the memory limit always.
- Re-derive from metrics after every significant release, or let VPA recommend in off mode.

NO — MEASURE BEFORE YOU COMMIT

- Do not guess a number and enshrine it; an invented limit becomes a permanent OOMKill generator.
- Start with a generous memory limit and a modest CPU request, then tighten using real percentiles.
- Use a LimitRange so the namespace has sane defaults while you learn the workload.
- Never leave production BestEffort — it is first to be evicted and reserves nothing.

Source: kubernetes.io/docs/concepts/configuration/manage-resources-containers/

© 2026 Tertiary Infotech Academy Pte Ltd

CLASSIFICATION

The Three QoS Classes

Three classes, derived automatically, consequential only when the node runs out of something.

1 Guaranteed

Every container has requests equal to limits, for both CPU and memory. Highest protection from node-pressure eviction.

2 Burstable

At least one container has a request, but the class test for Guaranteed fails. The large middle ground.

3 BestEffort

No requests and no limits anywhere in the Pod. Reserves nothing and is evicted first.

4 You do not set it

.status.qosClass is computed by the kubelet from the resources you declared. There is no field to choose it.

5 It drives eviction order, not scheduling

The scheduler does not read qosClass. It matters when a node is under memory or disk pressure.

6 Guaranteed is not immunity

A Guaranteed Pod that exceeds its own memory limit is still OOMKilled — Lab 10 demonstrates exactly this.

Source: kubernetes.io/docs/concepts/workloads/pods/pod-qos/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

QoS Class Computed, Not Declared

Three Pods, three classes, one rule. The fourth line is the punchline for the next slide.

Lab 10 — captured on the isolated kind cluster

```
$ kubectl get pods -n kcna-lab10 -o custom-columns=\
POD:.metadata.name,QOS:.status.qosClass,\
CPU_REQ:.spec.containers[0].resources.requests.cpu,\
MEM_LIM:.spec.containers[0].resources.limits.memory
POD            QOS          CPU_REQ   MEM_LIM
berth-ledger   Guaranteed   100m      128Mi
notice-board   BestEffort   <none>    <none>
rate-quoter    Burstable    50m       192Mi

$ kubectl get pod ledger-compactor -n kcna-lab10 \
    -o jsonpath='{.status.qosClass} / {.status.phase}'
Guaranteed / Failed
```

WHAT TO LOOK AT

1. **qosClass is under .status**
   It is observed state written by the kubelet, which is why there is no way to request a class directly.

2. **berth-ledger is Guaranteed**
   Its requests equal its limits for both CPU and memory: 100m/100m and 128Mi/128Mi.

3. **notice-board declares nothing**
   <none> in both columns is what BestEffort looks like. It was applied before the LimitRange existed.

4. **Guaranteed / Failed together**
   The last line is the OOMKilled Pod. Its class protected it from eviction, not from its own limit.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

OOMKilled with Exit Code 137

An exit code worth memorising: 137 always means SIGKILL, and in a container it almost always means memory.

SYMPTOM

The container terminates two seconds after start and its final log line never prints, yet the manifest is valid and the Pod was scheduled normally.

EVIDENCE — actual cluster output

```
$ kubectl get pod ledger-compactor -n kcna-lab10
NAME               READY   STATUS      RESTARTS   AGE
ledger-compactor   0/1     OOMKilled   0          15s

    State:          Terminated
      Reason:       OOMKilled
      Exit Code:    137
      Started:      10:41:02 +0800
      Finished:     10:41:04 +0800
    Ready:          False

reason=OOMKilled exitCode=137 limit=32Mi
```

✓ Exit Code 0 and the COMPACTION-COMPLETE log line appear once the limit covers the real working set

Source: Planet-verified lab run, courseware/labs (isolated kind cluster)

ROOT CAUSE

- The container writes a 64 MiB working set into a memory-backed volume while its memory limit is 32Mi.
- 137 is 128 + 9: the process was killed by signal 9, SIGKILL, sent by the kernel OOM killer — not by Kubernetes.
- The Pod is Guaranteed. Its class gives eviction protection, not permission to exceed its own limit.

FIX

- Raise the memory limit to the measured working set plus headroom, or reduce what the workload buffers.
- Remember memory-backed emptyDir pages count against the writing container's memory limit.
- restartPolicy decides what happens next — Never leaves it Failed, Always produces a CrashLoopBackOff cycle.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Node-Pressure Eviction Order

Eviction is the node protecting itself. It is a different mechanism from an OOMKill, with a different victim order.



WHAT IT MEANS

Exceeding your request is what makes you a target. A Guaranteed Pod within its request is the last thing the kubelet will evict — but its own limit still applies.

Source: kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A LimitRange, Field by Field

A LimitRange is how a namespace stops being full of BestEffort Pods without asking every author to cooperate.

labs/lab-10/manifests/30-limitrange.yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: mfs-container-limits
  namespace: kcna-lab10
spec:
  limits:
  - type: Container
    min:            {cpu: 10m,  memory: 16Mi}
    max:            {cpu: 1,    memory: 512Mi}
    default:        {cpu: 500m, memory: 256Mi}
    defaultRequest: {cpu: 50m,  memory: 64Mi}
```

READING IT

1. **default is the limit, defaultRequest is the request**
   The two names are easy to swap. default fills in .limits; defaultRequest fills in .requests.
2. **min and max are validation, not defaulting**
   A container asking for more than max cpu is rejected at admission with a clear Forbidden message.
3. **It applies per container, not per Pod**
   type: Pod is also available and constrains the sum across all containers in the Pod.
4. **It is not retroactive**
   Admission only sees creates and updates. Pods that existed before the LimitRange keep their original — possibly BestEffort — shape.

Source: kubernetes.io/docs/concepts/policy/limit-range/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

LimitRange Defaulting in Action

The clearest demonstration in the course that admission control both mutates and validates.

Lab 10 — captured on the isolated kind cluster

```
$ kubectl describe limitrange mfs-container-limits -n kcna-lab10
Type       Resource  Min   Max     Default Request  Default Limit
Container  cpu       10m   1       50m              500m
Container  memory    16Mi  512Mi   64Mi             256Mi

$ kubectl run late-arrival --image=registry.k8s.io/pause:3.9 \
    --restart=Never -n kcna-lab10
POD           QOS          CPU_REQ  CPU_LIM  MEM_REQ  MEM_LIM
late-arrival  Burstable    50m      500m     64Mi     256Mi
notice-board  BestEffort   <none>   <none>   <none>   <none>

$ kubectl run oversized --image=registry.k8s.io/pause:3.9 ...
Error from server (Forbidden): pods "oversized" is
forbidden: maximum cpu usage per Container is 1, but
limit is 2
```

WHAT TO LOOK AT

1. **Two identical commands, two different outcomes.** late-arrival declared nothing and was given the namespace defaults; notice-board predates the LimitRange.
2. **Defaulting changes the QoS class.** The same manifest that produced BestEffort now produces Burstable. Admission mutated the Pod on its way in.
3. **max is enforced, not defaulted.** A container asking for a 2-core limit is refused outright with the exact ceiling quoted back.
4. **Nothing was created on the failure.** Admission rejection means no object exists — unlike a scheduling failure, where a Pending Pod does exist.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

A ResourceQuota, Field by Field

A ResourceQuota turns a shared cluster into a set of bounded tenancies, enforced at admission.

labs/lab-10/manifests/31-resourcequota.yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: mfs-lab-quota
  namespace: kcna-lab10
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "3"
    limits.memory: 2Gi
    pods: "15"
    count/deployments.apps: "5"
```

READING IT

1. **Quota is namespace-wide, LimitRange is per container**
   One caps the total the namespace may consume; the other shapes each individual container.

2. **requests and limits are quota'd separately**
   requests.cpu 1 with limits.cpu 3 allows a 3x overcommit ratio inside this namespace and no more.

3. **Object counts are quota'd too**
   count/<resource>.<group> caps the number of objects, which is how you stop runaway controllers.

4. **Quota forces resources to be declared**
   Once a compute quota exists, a Pod with no request for a quota'd resource is rejected — pair it with a LimitRange.

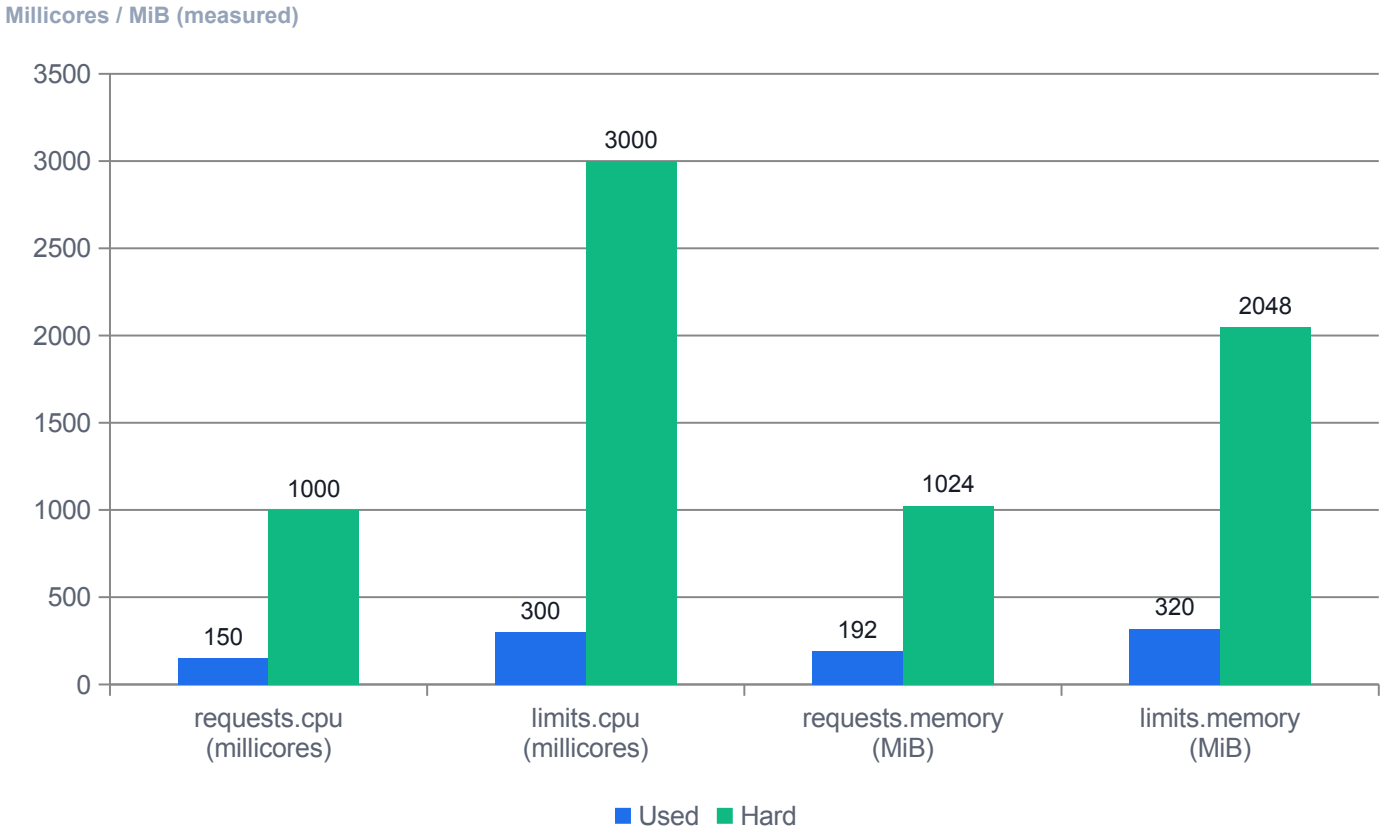
Source: kubernetes.io/docs/concepts/policy/resource-quotas/

© 2026 Tertiary Infotech Academy Pte Ltd

MEASURED LAB OBSERVATION

Namespace Quota: Used Against Hard

Real numbers from the parent-verified lab run. Note the unit normalisation — the raw output mixes m, Mi and Gi.



WHAT THE DATA SHOWS

Captured from describe quota

kubectl describe resourcequota mfs-lab-quota in Lab 10, with the three QoS Pods admitted. Units normalised to millicores and MiB for plotting.

requests.cpu is the binding constraint

15% consumed against the tightest headroom. Adding one 900m Pod exceeds it while every other bar stays low.

limits are quota'd three times higher

3000m of limits against 1000m of requests is a deliberate 3x overcommit allowance for this namespace.

Quota counts declarations, not usage

These Pods are nearly idle. The quota is spent the moment the Pod is admitted, whatever it actually consumes.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Rejected by ResourceQuota

Two rejections in this module look similar and are not: one is per-container policy, this one is namespace budget.

SYMPTOM

kubectl apply fails and no Pod object is created at all — there is nothing to describe and nothing Pending.

EVIDENCE — actual cluster output

```
$ kubectl apply -f 60-pod-quota-violation.yaml
Error from server (Forbidden): error when creating
"manifests/60-pod-quota-violation.yaml": pods
"cargo-modeller" is forbidden: exceeded quota:
mfs-lab-quota, requested: requests.cpu=900m,
used: requests.cpu=350m, limited: requests.cpu=1

$ kubectl get pod cargo-modeller -n kcna-lab10
Error from server (NotFound): pods
"cargo-modeller" not found
```

✓ **A request of 600m or less is admitted immediately with the same quota in place**

ROOT CAUSE

- 350m is already committed in the namespace and the new Pod requests a further 900m against a hard limit of 1000m.
- The Pod's 900m passes the LimitRange, whose per-container max cpu is 1 — two different admission plugins, two different verdicts.
- Quota is evaluated by the admission controller, so the rejection happens before persistence.

FIX

- Read all three numbers in the message: requested, used and limited. They tell you exactly how much room is left.
- Either lower the request, free capacity by removing other Pods, or negotiate a larger quota.
- Contrast with a scheduling failure, where the object exists and sits Pending until conditions change.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

TWO POLICY OBJECTS

LimitRange vs ResourceQuota

Aspect	LimitRange	ResourceQuota
Scope of the rule	One container, or one Pod	The whole namespace, in aggregate
Mutating?	Yes — supplies default and defaultRequest to containers that omit them	No — it only ever validates
Validating?	Yes — enforces min and max per container	Yes — rejects when the namespace total would exceed hard
Also limits object counts?	No	Yes — pods, services, and count/<resource>.<group>
Typical failure text	maximum cpu usage per Container is 1, but limit is 2	exceeded quota: ..., requested: ..., used: ..., limited: ...

CHOOSE

Use both: the LimitRange guarantees every Pod declares something, the quota caps what the namespace can declare in total.

Source: kubernetes.io/docs/concepts/policy/resource-quotas/

© 2026 Tertiary Infotech Academy Pte Ltd

SECOND-ORDER CONTROLLERS

Four Autoscalers, Four Signals

Ask two questions of any autoscaler: what does it change, and what signal does it read?

1 Horizontal Pod Autoscaler (HPA)

Changes .spec.replicas on a scale subresource. Signal: CPU or memory utilisation, or custom and external metrics. autoscaling/v2.

2 Vertical Pod Autoscaler (VPA)

Changes requests and limits for containers. Signal: observed usage history. An add-on, not core Kubernetes.

3 Cluster Autoscaler

Adds and removes nodes. Signal: Pods that are Pending because no node can fit them, and nodes that are underused.

4 KEDA

Event-driven scaling, including to and from zero. Signal: queue depth, topic lag, cron and dozens of other scalers. A CNCF graduated project.

5 They compose in one direction

HPA adds Pods, those Pods go Pending, the Cluster Autoscaler adds a node. That is the intended chain.

6 HPA and VPA conflict on CPU

Both would react to the same signal. Run VPA in recommendation-only mode, or scope them to different resources.

Source: kubernetes.io/docs/concepts/workloads/autoscaling/

© 2026 Tertiary Infotech Academy Pte Ltd

CHOOSING AN AUTOSCALER

What Each One Changes

Autoscaler	Changes	Reads	Ships with Kubernetes?
HPA	Replica count of a scalable workload	metrics.k8s.io, custom.metrics.k8s.io, external.metrics.k8s.io	Yes — autoscaling/v2 in-tree
VPA	Container requests and limits	Historical usage from the metrics pipeline	No — autoscaler add-on
Cluster Autoscaler	Number of nodes in a node group	Unschedulable Pods and node utilisation	No — add-on, provider-specific
KEDA	Replica count, including 0	External event sources via scalers; presents an HPA underneath	No — CNCF graduated project
metrics-server	Nothing — it is a data source	kubelet summary API	No — required add-on for HPA on CPU or memory

CHOOSE

HPA for load, VPA for right-sizing, Cluster Autoscaler for capacity, KEDA for events and scale-to-zero.

Source: kubernetes.io/docs/concepts/workloads/autoscaling/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

An HPA metrics Block, Field by Field

Every field here is load-bearing. The most common HPA defect is a Utilization target on a container with no CPU request.

labs/lab-10/manifests/50-hpa-quote-engine.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: {apiVersion: apps/v1,
    kind: Deployment, name: quote-engine}
  minReplicas: 2
  maxReplicas: 6
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 200Mi
```

READING IT

1. **autoscaling/v2 is the current API**
   The v2beta1 and v2beta2 versions are long removed. v1 still exists but can express only a single CPU target.
2. **Utilization is a percentage of the request**
   60% of a 100m CPU request is 60m per Pod. With no CPU request declared, Utilization cannot be computed at all.
3. **AverageValue is an absolute per-Pod figure**
   200Mi per Pod, independent of any request. Use it when a percentage would be meaningless.
4. **Multiple metrics take the maximum**
   The controller computes a desired replica count for each metric and applies the largest.

Source: kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

© 2026 Tertiary Infotech Academy Pte Ltd

READING REAL YAML

The HPA behavior Block and the Formula

Without behavior, an HPA reacts to every spike. With it, you are declaring how nervous the controller is allowed to be.

labs/lab-10/manifests/50-hpa-quote-engine.yaml (cont.)

```
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
      - type: Pods
        value: 2
        periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Percent
        value: 50
        periodSeconds: 60

# desired = ceil( current x (currentMetric
#                            / targetMetric) )
```

READING IT

1. **The formula is a simple ratio, rounded up**
   Two Pods at 120% of a 60% target gives ceil(2 x 120/60) = 4. Then min, max and behavior are applied.

2. **A tolerance suppresses small corrections**
   Ratios close to 1.0 are ignored, so a target of 60% against an observed 62% produces no scaling event at all.

3. **Asymmetric stabilisation is deliberate**
   Scale up after 30 seconds, scale down only after 300. Up fast to protect users, down slowly to avoid flapping.

4. **Policies cap the rate of change**
   At most 2 Pods added per minute; at most 50% removed per minute. The formula proposes, the policy disposes.

Source: kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED MODEL

HPA Response to a Load Curve (min 2, max 6, target 60%)

Clearly-labelled arithmetic model. It shows why an HPA's observed behaviour rarely matches the naive formula alone.



WHAT THE DATA SHOWS

Worked model, not measured data

Each point is the documented formula ceil(current x observed/target) evaluated by hand against the Lab 10 HPA settings.

min and max clamp the result

t0's formula output of 1 is raised to minReplicas 2; t2's output of 7 is capped at maxReplicas 6.

t3 shows the tolerance

62% against a 60% target is within the controller's tolerance, so no scaling event occurs and 6 is held.

t4 shows the scale-down window

The formula wants 4, but the 300-second stabilisation window holds the recent maximum until t5.

Source: kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

An HPA With No Metrics Pipeline

This is the most common real-world HPA problem and it is not an HPA problem at all.

Lab 10 — captured on the isolated kind cluster

```
$ kubectl get hpa quote-engine -n kcna-lab10
NAME          TARGETS                         MIN MAX REPLICAS
quote-engine  cpu: <unknown>/60%,
              memory: <unknown>/200Mi         2   6   2

Conditions:
  AbleToScale     True   SucceededGetScale
  ScalingActive   False  FailedGetResourceMetric
    the HPA was unable to compute the replica count:
    unable to fetch metrics from resource metrics API:
    the server could not find the requested resource
    (get pods.metrics.k8s.io)

$ kubectl top pods -n kcna-lab10
error: Metrics API not available
```

WHAT TO LOOK AT

1. **<unknown> is the diagnosis**
   The HPA object is valid and the controller is running; it simply has no numbers to act on.

2. **Two conditions, two different answers**
   AbleToScale True means the scale subresource works. ScalingActive False means the metric lookup does not.

3. **pods.metrics.k8s.io is the missing API**
   metrics-server registers that APIService. A stock kind cluster does not run it.

4. **Replicas stay at minReplicas**
   The HPA does not guess. With no signal it makes no change — a safe failure mode.

Source: Parent-verified lab run, courseware/labs (isolated kind cluster)

© 2026 Tertiary Infotech Academy Pte Ltd

TRAPS

Autoscaling Failure Modes Worth Memorising

Six ways an autoscaler quietly does nothing. Each one has a specific field or object you can check.

1 Utilization target with no CPU request

There is no denominator, so utilisation is undefined and the HPA never scales. Always declare a request on an HPA target.

2 A hard-coded replicas fighting the HPA

Re-applying a manifest that pins .spec.replicas overwrites the HPA's decision on every sync. Remove the field.

3 DaemonSets cannot be HPA targets

They expose no scale subresource. Their replica count is the node count by definition.

4 HPA scales Pods, not nodes

New Pods with no room go Pending. Without a Cluster Autoscaler that is where the story ends.

5 Scale-to-zero is not native HPA

minReplicas below 1 requires the HPAScaleToZero feature gate; KEDA and Knative are the usual routes.

6 Slow readiness defeats fast scaling

If a new Pod takes 90 seconds to become Ready, a 30-second stabilisation window buys you nothing.

Source: kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 10 — Resource Requests, Limits, QoS and Autoscaling

GOAL

Apply the MFS workload resource profiles, prove all three QoS classes from the live API, watch a container get OOMKilled by its own memory limit, then bound the namespace with a LimitRange and ResourceQuota and read an HPA honestly on a cluster with no metrics-server.

YOU WILL PRODUCE

- Three Pods proved to be Guaranteed, Burstable and BestEffort via .status.qosClass
- An OOMKilled container showing reason=OOMKilled and exit code 137
- A LimitRange plus ResourceQuota that reject an over-budget Pod at admission, and an autoscaling/v2 HPA on the quote-engine Deployment

CONCEPTS EXERCISED

- requests drive scheduling and are the denominator of HPA utilisation; limits drive throttling and OOMKill
- The three QoS classes are derived, never declared - the API server computes .status.qosClass
- LimitRange sets per-container defaults and bounds; ResourceQuota sets an aggregate namespace budget
- An HPA without metrics-server reports <unknown> and ScalingActive=False - the pipeline is missing, not the config

LAB	NAMESPACE	MAPS TO	TIME
Lab 10	kcna-lab10	LO2 · A3 · K3	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-10-resources-requests-limits-autoscaling/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM-FACING RECAP

Day 2 Facts You Should Be Able to State Without Looking

Deployment strategy default	RollingUpdate with maxSurge 25% and maxUnavailable 25%
Rollout ceiling / floor	replicas + maxSurge / replicas - maxUnavailable
Rounding	maxSurge rounds up, maxUnavailable rounds down
revisionHistoryLimit default	10 old ReplicaSets kept
Scaling replicas	Never creates a new revision or ReplicaSet
Rollback mechanism	Revives an existing ReplicaSet and re-stamps its revision number

Scheduler stages	PreFilter, Filter, Score, Reserve, Permit, PreBind, Bind
Only Filter causes Pending	A low score never blocks scheduling
Taint effects	NoSchedule, PreferNoSchedule, NoExecute
QoS classes	Guaranteed, Burstable, BestEffort — computed, not declared
Exit code 137	SIGKILL — in a container, usually OOMKill
HPA API version	autoscaling/v2; requires metrics-server for CPU or memory

Source: kubernetes.io/docs/concepts/workloads/controllers/deployment/

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 2 RECAP

What You Can Now Do, and What Comes Next

Everything today reduced to one sentence: declare the shape, and let a controller close the gap. Day 3 applies the same sentence to traffic and access.

YOU CAN NOW EXPLAIN AND DO

- Describe the reconciliation loop and why level-triggered control makes the cluster self-healing.
- Predict a rollout's Pod ceiling and availability floor from maxSurge and maxUnavailable, and verify it live.
- Diagnose a wedged rollout from Available and Progressing, and roll back understanding what the revision numbers do.
- Choose between Deployment, StatefulSet, DaemonSet, Job and CronJob from the shape of the work.
- Read a FailedScheduling message and say whether it is affinity, a taint or capacity.
- Set requests and limits deliberately, name the resulting QoS class, and explain throttling versus OOMKill.
- Read an autoscaling/v2 HPA and say what it will do and why it might do nothing.

DAY 3 BUILDS DIRECTLY ON THIS

- Services: the same label selectors you used for ReplicaSets now bind traffic to Pods.
- EndpointSlice: readiness — the state that made a Pod count as Available — decides who receives traffic.
- Ingress and HTTP routing on top of Services, with the stable networking.k8s.io/v1 API.
- Namespaces, ServiceAccounts and RBAC: who is allowed to create the workloads you wrote today.
- Secrets and ConfigMaps: the configuration those Pod templates consume.
- NetworkPolicy: label selectors again, this time deciding which Pods may talk to which.
- Bring today's mental model — declare, reconcile, observe status — with you unchanged.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 3 · REGISTERED TOPICS 3 AND 4

03

Services, Networking & Cluster Security

Two registered topics in one day: how traffic finds a Pod, and who is allowed to ask. Six labs, all evidence-based.

© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN

Day 3 — Services, Networking & Cluster Security

Instructional time 8.0 h. Breaks are not instructional time. Labs 11-16 run inside Sessions 2, 3 and 4.

Time	Duration	Session	Method
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 3 MAP

What today has to leave you able to do

Registered Topic 3 (Services and Networking) and Topic 4 (Security), mapped to LO3/A2 and LO4/A4.

1 Name a workload

Put a Service in front of Pods and explain what its selector produced — the EndpointSlice — rather than guessing.

2 Expose a workload

Choose ClusterIP, NodePort, LoadBalancer or headless from the requirement, and know what each one costs.

3 Route at layer 7

Write a networking.k8s.io/v1 Ingress and separate the declaration from the controller that must execute it.

4 Package a deployment

Install Helm and deploy a first chart; understand chart, values and release before Day 4 goes deeper.

5 Scope and grant

Namespaces, ServiceAccounts, Roles and ClusterRoles — and prove every allow and deny with kubectl auth can-i.

6 Protect the data path

Secrets that are encoded rather than encrypted, and NetworkPolicy as the network-layer control.

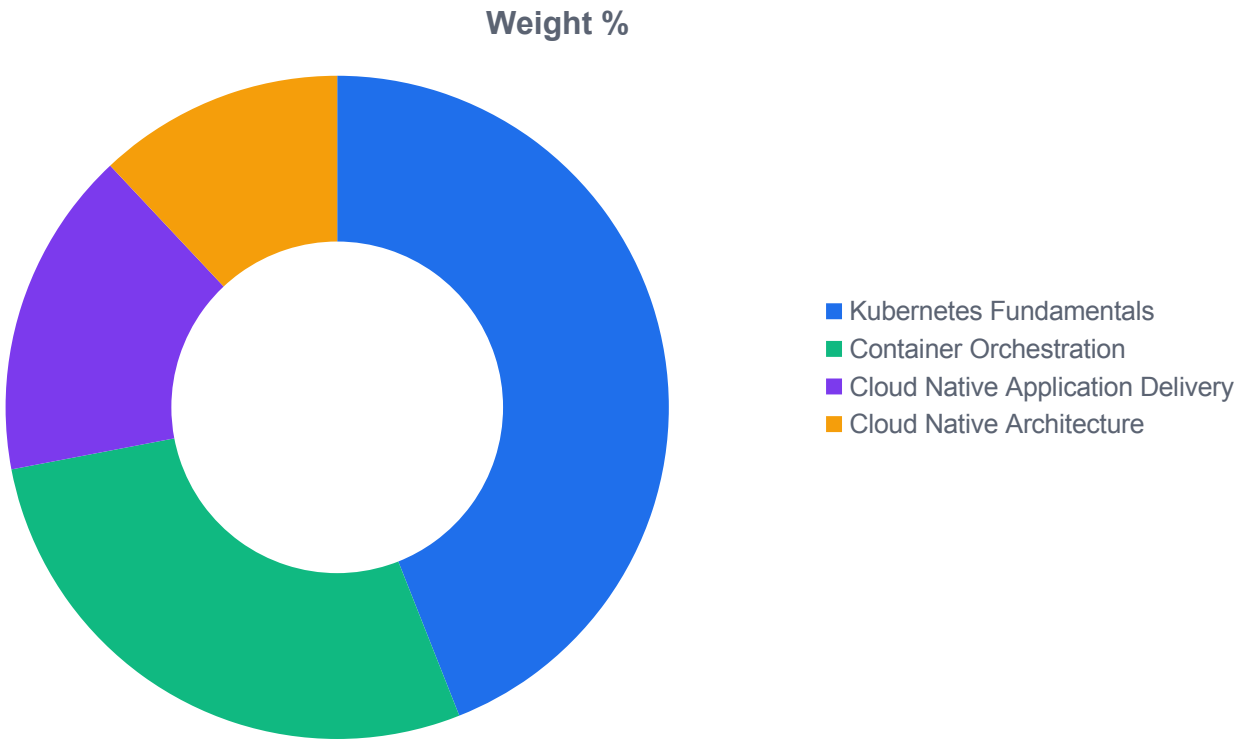
Source: course_data.TOPICS and course_data.LO_MAP (approved TSC mapping ICT-DES-4006-1.1)

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM WEIGHTING

Where today sits in the KCNA blueprint

Published CNCF domain weights, revision effective 24 Nov 2025. Container Orchestration is 28% of the exam.



WHAT THE DATA SHOWS

Container Orchestration = 28%

Its four sub-topics are Networking, Security, Troubleshooting and Storage. Day 3 takes two of the four.

Not the largest domain

Kubernetes Fundamentals at 44% is bigger — but networking and security supply most of the scenario questions.

Sub-topics are not separately weighted

CNCF publishes a weight per domain, not per sub-topic. Treat the four as roughly co-equal, not as 7% each.

Published, not modelled

These four numbers are read straight from the current curriculum PDF — no estimation is involved.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

THE FOUNDATIONAL RULE

Every Pod gets its own IP address, and nothing is network-address-translated between Pods.

That single sentence is the whole Kubernetes network model. Everything else today — Services, DNS, Ingress, NetworkPolicy — is built on top of it.

© 2026 Tertiary Infotech Academy Pte Ltd

NETWORK MODEL

The four conditions any cluster network must satisfy

Kubernetes specifies a contract, not a mechanism. Any network that meets these conditions is a valid cluster network.

1 Pods talk to Pods without NAT

Any Pod can reach any other Pod on any node using the target Pod's own IP. No port mapping, no address rewriting.

2 Nodes talk to Pods without NAT

Node agents — kubelet, the runtime, monitoring daemons — reach Pods on that node directly.

3 A Pod sees its own real IP

The address a Pod sees on its interface is the address other Pods use to reach it. No split-horizon identity.

4 The Pod is the network unit

All containers in a Pod share one network namespace, one IP and one port space; they reach each other on localhost.

5 Implementation is deliberately absent

The model states the requirements. How they are met is left to a CNI plugin — this is the extension point.

6 Nothing here is a security control

A flat, fully-reachable network is the default. Restriction is opt-in and arrives later today as NetworkPolicy.

Source: kubernetes.io/docs/concepts/cluster-administration/networking/

© 2026 Tertiary Infotech Academy Pte Ltd

ADDRESS SPACES

Three separate address spaces in one cluster

Pod IPs are real and pingable. ClusterIPs are not — they exist only as packet-rewriting rules.

NODE NETWORK	**Node InternalIP** Real routable addresses on your infrastructure network	**hostPort / hostNetwork** Escapes the Pod network and binds the node's own interface	**NodePort listener** kube-proxy opens a port on every node in this space
POD NETWORK (CNI)	**Pod IP** Assigned by the CNI plugin's IPAM from the cluster pod CIDR	**veth pair** One end in the Pod's netns, one end on the node bridge or routed	**Overlay or routed** VXLAN/IPIP encapsulation, or plain routes / eBPF — plugin's choice
SERVICE NETWORK (VIRTUAL)	**ClusterIP** A VIP from the service CIDR that no interface ever owns	**kube-proxy rules** iptables / IPVS / nftables entries that DNAT the VIP to a Pod IP	**CoreDNS record** Maps the Service name onto the VIP so clients never learn Pod IPs

MECHANISM

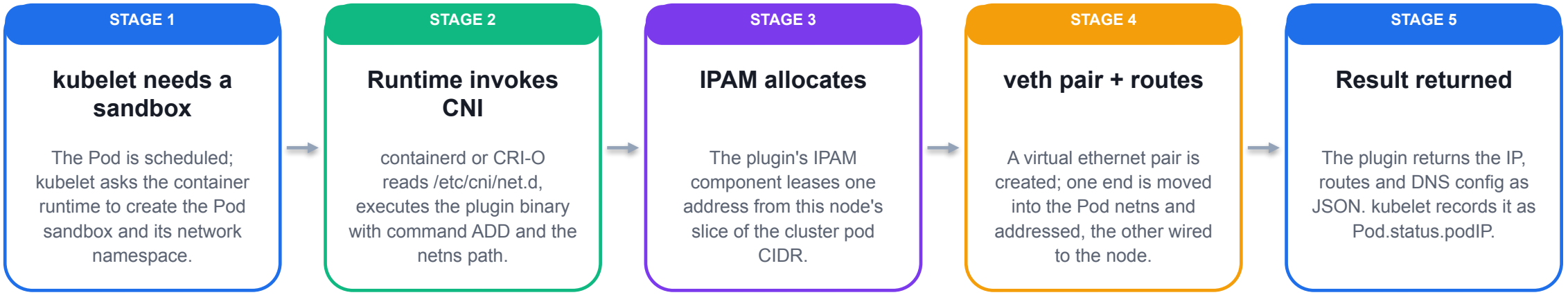
A ClusterIP is never configured on any network interface. That is why you can reach it and still never see it in `ip addr` anywhere in the cluster.

Source: kubernetes.io/docs/concepts/cluster-administration/networking/ · kubernetes.io/docs/reference/networking/virtual-ips/ (Service virtual IPs and kube-proxy modes)

© 2026 Tertiary Infotech Academy Pte Ltd

What a CNI plugin actually does when a Pod starts

CNI is a specification and an executable contract — the runtime literally runs a binary and reads JSON back.



WHAT IT MEANS

No CNI plugin configured means no ADD result, which means the Pod never leaves ContainerCreating and the node stays NotReady. That is the fingerprint of a missing CNI.

Source: kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/ · github.com/containernetworking/cni SPEC.md

© 2026 Tertiary Infotech Academy Pte Ltd

PLUGIN LANDSCAPE

CNI plugins differ in one thing that matters today

Every plugin satisfies the connectivity contract. Not every plugin enforces NetworkPolicy — and the API accepts your policy either way.

Plugin	Data path	NetworkPolicy	Typical reason to pick it
Calico	Routed (BGP) or overlay (VXLAN/IPIP)	Enforces v1 NetworkPolicy plus its own CRDs	Policy-first clusters; mature, widely deployed
Cilium	eBPF programs in the kernel data path	Enforces v1 NetworkPolicy plus L7-aware CRDs	eBPF observability, kube-proxy replacement, mesh ambitions
Flannel	Simple overlay (VXLAN by default)	No policy engine on its own	Minimal connectivity when policy is handled elsewhere
kindnet (kind default)	Simple routed CNI for local kind clusters	Stores NetworkPolicy objects; enforces none	Local training and CI — connectivity only, by design

CHOOSE

For a training cluster, connectivity is enough — but never demonstrate 'policy blocked it' on kindnet.

Source: kubernetes.io/docs/concepts/extend-kubernetes/compute-storage-net/network-plugins/ · github.com/containernetworking/cni SPEC.md · parent runtime note, qa/PARENT-VERIFIED-FACTS.md

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Pod IPs are real, per-Pod and node-aware

The addresses change every run; the relationships between the columns do not.

Representative shape — addresses are runtime values and will differ on your cluster

```
$ kubectl get pods -n kcna-lab11 -o wide
NAME                      READY  STATUS   IP            NODE
quote-api-6c9b4f7d55-7lqm2  1/1   Running  10.244.0.14   kcna-qa-worker
quote-api-6c9b4f7d55-jd8vx  1/1   Running  10.244.0.15   kcna-qa-worker
quote-api-6c9b4f7d55-x2t9k  1/1   Running  10.244.1.9    kcna-qa-worker2
dns-client                  1/1   Running  10.244.0.21   kcna-qa-worker

$ kubectl get nodes -o wide
NAME                  STATUS  INTERNAL-IP   VERSION
kcna-qa-control-plane Ready   172.18.0.3    v1.37.0
kcna-qa-worker        Ready   172.18.0.2    v1.37.0
kcna-qa-worker2       Ready   172.18.0.4    v1.37.0
```

WHAT TO LOOK AT

1. **Two different address spaces**
   Pod IPs are 10.244.x.y; node InternalIPs are 172.18.x.y. They never overlap.

2. **Per-node pod CIDR slices**
   10.244.0.0/24 belongs to one node, 10.244.1.0/24 to another. IPAM leases only from the local slice.

3. **Cross-node reachability is assumed**
   10.244.0.14 can open a socket to 10.244.1.9 with no translation. That is the model, not a feature.

4. **Pod IPs are ephemeral**
   Delete one Pod and the replacement gets a different address. This is precisely why Services exist.

Source: courseware/labs/lab-11-services-endpoints/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Cluster network facts worth memorising

Ten facts that resolve most networking questions before you ever open a terminal.

Pod CIDR	Cluster-wide pod address pool; sliced per node by the controller manager
Service CIDR	Separate pool for ClusterIP VIPs; must not overlap the pod CIDR
Pod IP field	Pod.status.podIP (and .podIPs for dual-stack)
Node address	Node.status.addresses[] — InternalIP is the one NodePort uses
CNI config dir	/etc/cni/net.d on each node; the runtime reads it, not the API server

CNI binaries	/opt/cni/bin — plugin executables invoked with ADD / DEL / CHECK
Containers in a Pod	Share one netns: same IP, same port space, reachable on localhost
hostNetwork: true	Pod uses the node's namespace directly — no Pod IP of its own
Dual-stack	IPv4/IPv6 families set per Service via ipFamilies and ipFamilyPolicy
What is NOT provided	Encryption, authentication or isolation between Pods by default

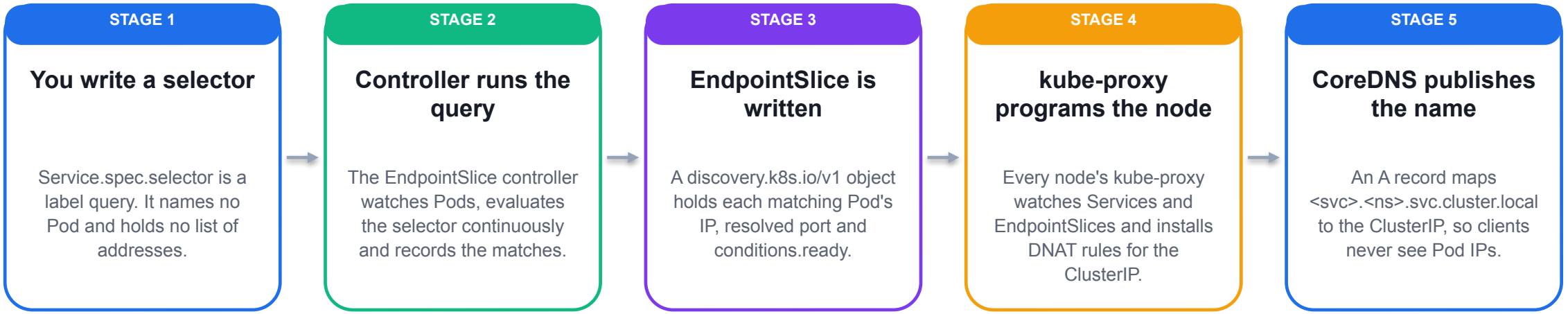
Source: kubernetes.io/docs/concepts/cluster-administration/networking/ · kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

A Service stores a selector — the cluster derives everything else

Five moving parts, one declarative input. Knowing which part failed is the whole of Service troubleshooting.



WHAT IT MEANS

Nothing in this chain is triggered by an apply. It is a continuous reconciliation — relabel a Pod and the EndpointSlice changes with no restart anywhere.

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The four ports of a Service, read in order

port, targetPort, nodePort and the port name each answer a different question. Confusing them is the classic error.

30-service-quote-api.yaml (NodePort variant)

```
apiVersion: v1
kind: Service
metadata:
  name: quote-api
  namespace: kcna-lab11
spec:
  type: NodePort
  selector:
    app: quote-api          # a label QUERY
  ports:
    - name: http
      port: 80              # the Service's own port
      targetPort: http      # named container port
      nodePort: 30080       # 30000-32767 only
      protocol: TCP
```

READING IT

1. **port — what clients dial**
   The port on the ClusterIP VIP. `quote-api:80` and nothing else is what callers inside the cluster use.
2. **targetPort — where it lands**
   The port on the Pod. A number, or the NAME of a containerPort. Named is the resilient choice.
3. **nodePort — the node-level door**
   Only meaningful for type NodePort or LoadBalancer. Drawn from the cluster-wide 30000-32767 pool.
4. **name — required once you have two**
   With a single port, `name` is optional. With two or more it is mandatory and Ingress refers to it.
5. **selector — not a Pod list**
   Change a Pod's labels and this Service's backend set changes immediately. No edit to the Service is needed.

Source: kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

NAMED PORTS

A named port is a contract between two objects

Named ports are on the registered outline because they decouple the Service contract from the container's listening port.

Named port, end to end across three objects

```
# Deployment — the container declares the name
    ports:
      - name: http          # <=15 chars, IANA_SVC_NAME
        containerPort: 8080
        protocol: TCP

# Service — refers to the NAME, not the number
  ports:
    - port: 80
      targetPort: http

# EndpointSlice — the controller resolved it
  ports:
    - name: http
      port: 8080            # the RESOLVED number
      protocol: TCP
```

READING IT

1. **The name lives on the container**
   containerPort.name is lowercase alphanumeric plus hyphens, at most 15 characters, unique within the Pod.

2. **The Service never learns 8080**
   targetPort: http is late-bound. The number is resolved per Pod, at endpoint-computation time.

3. **Resolution is visible**
   The EndpointSlice records the resolved number. If it shows <unset>, the name did not resolve on that Pod.

4. **Why it matters operationally**
   Two Deployments behind one Service can listen on different numbers, provided both use the same port name.

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The EndpointSlice object, field by field

This object is the answer to 'what is actually behind that Service name right now?'

kubectl get endpointslice quote-api-t8hkb -o yaml (abridged)

```
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: quote-api-t8hkb      # generated suffix
  namespace: kcna-lab11
  labels:
    kubernetes.io/service-name: quote-api
addressType: IPv4
ports:
  - name: http
    port: 8080
    protocol: TCP
endpoints:
  - addresses: ["10.244.0.14"]
    conditions:
      ready: true
    nodeName: kcna-qa-worker
    targetRef: {kind: Pod, name: quote-api-...}
```

READING IT

1. **It is owned, not authored**
   You never write this. The EndpointSlice controller creates, updates and garbage-collects it.
2. **The service-name label is the join key**
   kubernetes.io/service-name links slice to Service. A Service may own several slices.
3. **conditions.ready gates traffic**
   Only ready endpoints receive load-balanced connections. serving and terminating exist for graceful shutdown.
4. **addressType splits the object**
   IPv4, IPv6 and FQDN each get their own slices — this is how dual-stack is represented.
5. **Slices are capped and sharded**
   The controller splits at a configured maximum endpoints per slice, which is what makes large Services scale.

Source: kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Populated versus empty: the same command, two verdicts

Two shapes to recognise on sight. Everything else in Service debugging is elaboration of these.

Service backend diagnostics on kcna-lab11

```
# Healthy — selector matched three ready Pods
$ kubectl get endpointslices -n kcna-lab11
NAME             ADDRESSTYPE  PORTS  ENDPOINTS
quote-api-t8hkb  IPv4         8080   10.244.0.14,10.244.0.15,10.244.0.16

# Broken — selector app=quote-apis matched nothing
$ kubectl get endpointslices -n kcna-lab11
NAME                    ADDRESSTYPE  PORTS    ENDPOINTS
quote-api-broken-4zq6n  IPv4         <unset>  <unset>

# Root-cause proof — run the selector as a query
$ kubectl get pods -n kcna-lab11 -l app=quote-apis
No resources found in kcna-lab11 namespace.
```

WHAT TO LOOK AT

1. **The slice exists either way**
   An empty slice is not a missing slice. The Service was created correctly; the query returned nothing.

2. **<unset> PORTS is a second signal**
   No endpoints means no Pod to resolve the named port against, so the port column is unset too.

3. **Re-run the selector yourself**
   `kubectl get pods -l <selector>` is the decisive test. It reproduces exactly what the controller did.

4. **Use endpointslices, not endpoints**
   v1 Endpoints still prints today but is deprecated since v1.33. Build the habit on the current API.

Source: courseware/labs/lab-11-services-endpoints/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

CURRENCY

Endpoints and EndpointSlice — know which one to reach for

The registered outline says 'Services and Endpoints'. We teach the concept under its current object.

Aspect	v1 Endpoints (deprecated)	discovery.k8s.io/v1 EndpointSlice
Status	Deprecated as of Kubernetes v1.33	The current API; what controllers and kube-proxy consume
Shape	One object holding every address for a Service	Many slices per Service, each capped at a configured size
Scale behaviour	A single large object rewritten on every Pod change	Only the affected slice is rewritten — far less watch churn
Per-endpoint state	ready only, implied by the subset list	conditions.ready, .serving and .terminating
Topology and dual-stack	No address family split; limited topology data	addressType per slice; zone and nodeName per endpoint
Command	kubectl get endpoints — still prints, do not build on it	kubectl get endpointslices — use this for diagnostics

CHOOSE

EndpointSlice for everything you write or debug; recognise Endpoints only as legacy you may inherit.

Source: kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

PACKET PATH

Client → ClusterIP → kube-proxy → Pod

No proxy process is in this path. kube-proxy writes rules; the kernel does the work.

CLIENT SIDE

Application
Opens a socket to quote-api:80 — a name, never an address

Pod resolver
/etc/resolv.conf sends the query to the cluster DNS VIP

CoreDNS answer
Returns the ClusterIP 10.96.114.23; the client now has a VIP

NODE KERNEL (KUBE-PROXY'S RULES)

Netfilter hook
The packet to 10.96.114.23:80 is matched before it can be routed

Backend chosen
One ready endpoint is selected from the EndpointSlice-derived rule set

DNAT applied
Destination rewritten to 10.244.0.15:8080; conntrack pins the flow

POD SIDE

CNI data path
Overlay or route delivers the packet to the target node and veth

Container listens
The process is bound to 8080 and sees the client's real Pod IP

Reply un-DNATs
conntrack rewrites the source back to the VIP so the client is satisfied

MECHANISM

Load balancing happens once, at connection setup, in the client's own kernel. There is no central bottleneck — and no L7 awareness anywhere in this path.

Source: kubernetes.io/docs/reference/networking/virtual-ips/ (Service virtual IPs and kube-proxy modes)

© 2026 Tertiary Infotech Academy Pte Ltd

IMPLEMENTATION

kube-proxy modes — same API, different kernel machinery

Nothing in your Service manifest changes across these modes. Only the failure signatures and the scaling behaviour change.

Mode	How the VIP is realised	Practical character
iptables	Chains of netfilter rules; a service chain per port and a rule per endpoint	The long-standing Linux default. Rule evaluation is sequential, so very large clusters feel it
IPVS	In-kernel L4 load balancer; a virtual server per Service port with real servers behind it	Hash-table lookup instead of chain walking; offers scheduling algorithms such as rr, lc and sh
nftables	The successor netfilter subsystem, using sets and maps rather than long rule chains	Introduced as alpha in v1.29 and promoted through beta in v1.31 — confirm status against your cluster's release notes
kernelspace (Windows)	Windows Host Networking Service data path	The Windows equivalent; the Service API is identical
No kube-proxy at all	Some CNIs (for example Cilium in eBPF mode) replace it entirely	Services still work; the rules live in eBPF programs instead of netfilter

CHOOSE

Learn iptables mode as the reference model; recognise the others as the same contract, differently built.

Source: kubernetes.io/docs/reference/networking/virtual-ips/ (Service virtual IPs and kube-proxy modes)

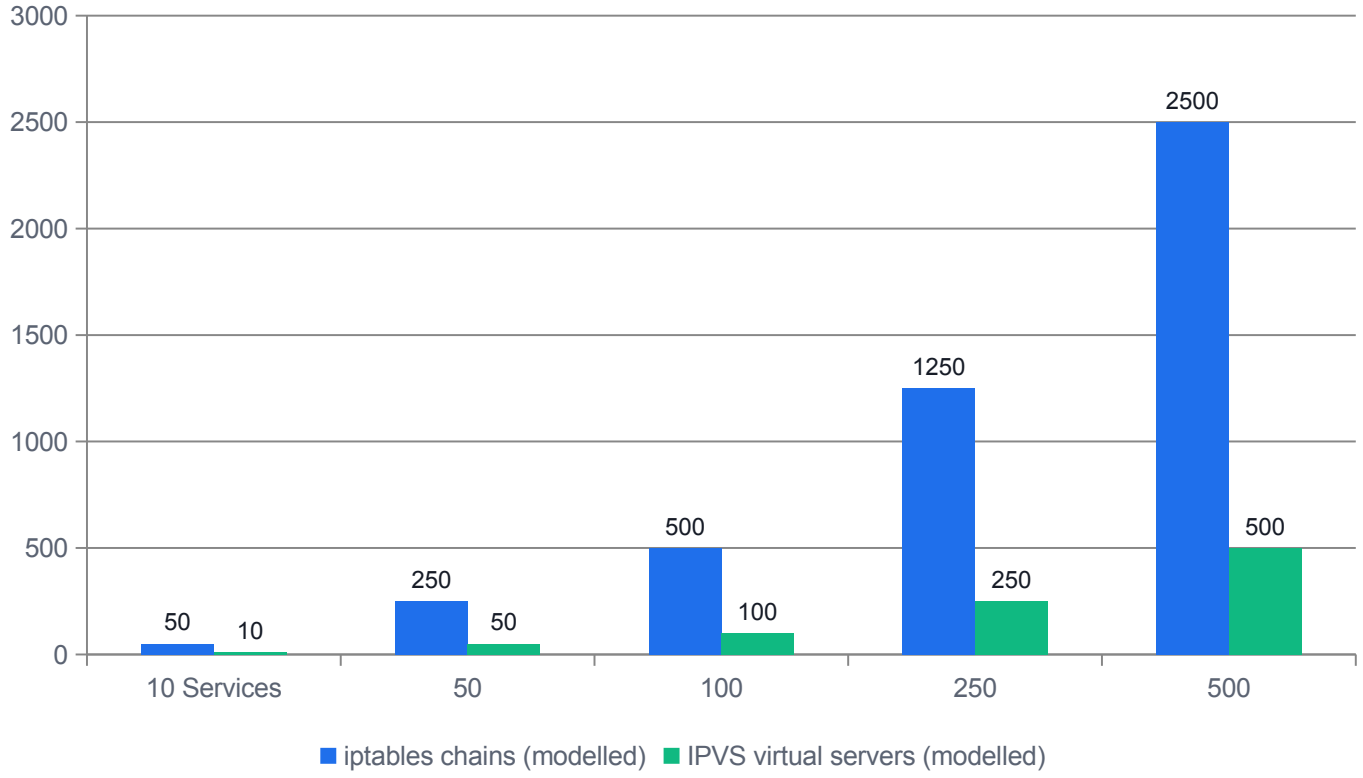
© 2026 Tertiary Infotech Academy Pte Ltd

WORKED MODEL

Why the proxy mode stopped being an implementation detail

WORKED MODEL, not a measurement. Arithmetic only: one service chain plus one endpoint chain per backend, at four ready backends per Service.

Kernel objects installed per node



WHAT THE DATA SHOWS

The formula, stated

chains = Services × (1 service chain + 4 endpoint chains) = Services × 5. IPVS = one virtual server per Service port.

500 Services → 2,500 chains

Every node carries the whole set, because every node must be able to resolve every ClusterIP locally.

Growth is linear, cost is not

Sequential rule evaluation makes the cost of a linear structure non-linear in practice.

This is a model, not a benchmark

It shows the SHAPE of the problem that IPVS and nftables were built to solve. No latency is claimed here.

Source: kubernetes.io/docs/reference/networking/virtual-ips/ (Service virtual IPs and kube-proxy modes) (chain structure). Values are arithmetic from the stated model, not measured.

© 2026 Tertiary Infotech Academy Pte Ltd

SELECTORS

Using selectors in a Service — five rules that prevent most incidents

A selector is a live query evaluated continuously — not a membership list captured at creation time.

1 Equality-based only

Service.spec.selector is a simple map of key: value, ANDed. No matchExpressions, no set-based operators.

2 Every key must match

Two keys in the selector means a Pod needs both labels. Adding a key silently narrows the backend set.

3 An empty selector is legal

Omit it entirely and no controller manages the endpoints — you supply them yourself for external backends.

4 Namespace-bound

A selector only ever matches Pods in the Service's own namespace. There is no cross-namespace selection.

5 Readiness still applies

A matched Pod that fails its readiness probe appears with conditions.ready false and receives no traffic.

6 Verify it as a query

`kubectl get pods -l <the exact selector>` is the only honest test. Reading the YAML is not verification.

Source: kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

MISCONCEPTIONS

Three things a Service is not

Each of these misconceptions produces a specific, recurring class of production surprise.

Not a proxy process

- No pod, no daemon and no hop sits on the data path
- kube-proxy programs rules and then steps aside
- Killing kube-proxy does not kill existing connections
- It does stop NEW rules from being programmed

no data-plane hop

Not a layer-7 load balancer

- Balancing is per connection, at TCP/UDP/SCTP level
- One HTTP keepalive connection pins to one Pod
- No path routing, no host routing, no header awareness
- That is what Ingress, Gateway API or a mesh are for

L4 only

Not a security boundary

- A ClusterIP is reachable from every Pod in the cluster
- Namespaces do not restrict who may dial the VIP
- Any Pod can also reach the backend Pod IPs directly
- NetworkPolicy is the control — and needs an enforcing CNI

not isolation

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/reference/networking/virtual-ips/ (Service virtual IPs and kube-proxy modes)

© 2026 Tertiary Infotech Academy Pte Ltd

BEYOND THE SELECTOR

Services that deliberately have no selector

The registered outline says 'creating Services and Endpoints'. This is where you genuinely create endpoints yourself.

1 Omit the selector entirely

No controller manages the backends. The Service exists, has a ClusterIP, and waits for you to supply endpoints.

2 You author the EndpointSlice

A hand-written discovery.k8s.io/v1 slice labelled kubernetes.io/service-name gives the Service its backends.

3 Use case: an external database

In-cluster clients dial a normal Service name; the addresses point at a machine outside the cluster.

4 Use case: a migration seam

Point the name at the old system, then swap the addresses for Pods without any client changing a line.

5 ExternalName is the DNS-only alternative

It publishes a CNAME instead. No VIP, no endpoints, no kube-proxy involvement at all.

6 Know the trade-off

A CNAME hands the client the external name; manual endpoints keep the indirection inside the cluster.

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/concepts/services-networking/endpoint-slices/ · Endpoints deprecation: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Four questions that localise any connectivity fault

Localise before you fix. Four questions cover essentially every Service connectivity ticket.

Does the NAME resolve?

1

If not, the problem is DNS, the namespace or the Service object. Nothing downstream matters yet.

Does the SERVICE have backends?

2

kubectl get endpointslices. Empty means the selector matched nothing or no Pod is Ready.

Do the SELECTOR and PORT agree?

3

Re-run the selector as a label query, and compare targetPort with the container's actual port.

Is something BLOCKING it?

4

NetworkPolicy on either end, or a mesh authorization rule. Check both directions under default-deny.

The signature tells you where you are

5

Refused immediately: usually no endpoints. Timeout: often policy, routing or a wrong port.

Resolve before you speculate

6

Each question has a command that answers it definitively. None of them requires guessing.

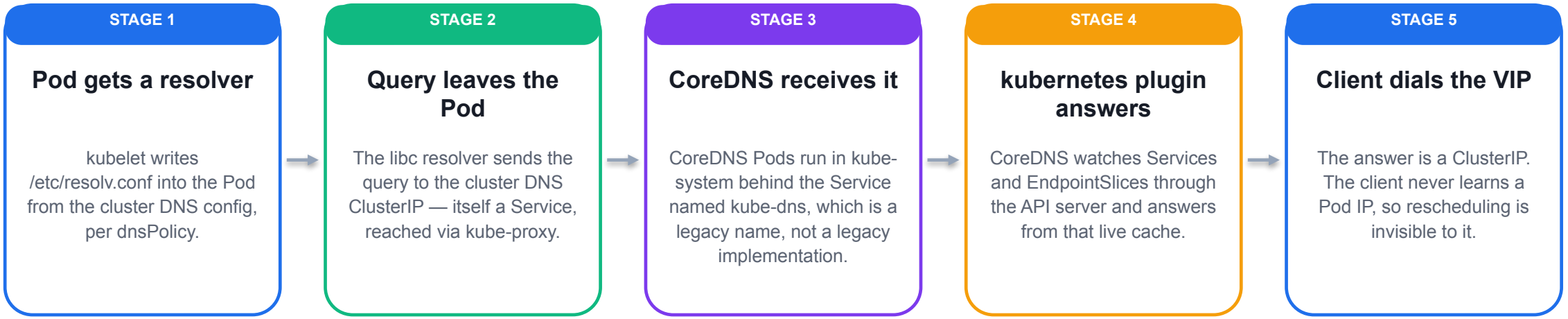
Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/concepts/services-networking/network-policies/

© 2026 Tertiary Infotech Academy Pte Ltd

CLUSTER DNS

How a Pod turns a Service name into an address

Cluster DNS is itself a Service behind a ClusterIP — DNS resolution depends on kube-proxy working first.



WHAT IT MEANS

CoreDNS answers from a watch cache, not from etcd lookups per query. That is why a new Service is resolvable within moments of being created.

Source: kubernetes.io/docs/concepts/services-networking/dns-pod-service/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

The resolver config and one successful lookup

Four lines of resolv.conf explain almost every DNS question you will be asked.

From inside kcna-lab11/dns-client

```
$ kubectl exec -n kcna-lab11 dns-client -- cat /etc/resolv.conf
search kcna-lab11.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5

$ kubectl exec -n kcna-lab11 dns-client -- \
    nslookup quote-api.kcna-lab11.svc.cluster.local
Server:    10.96.0.10
Address:   10.96.0.10:53

Name:      quote-api.kcna-lab11.svc.cluster.local
Address:   10.96.114.23

# Wrong namespace — correctly does NOT resolve
*** Can't find quote-api.default.svc.cluster.local: No answer
```

WHAT TO LOOK AT

1. **Three search suffixes**
   Own namespace first, then svc.cluster.local, then cluster.local. Short names work because of line 1.

2. **10.96.0.10 is a Service**
   It is the kube-dns ClusterIP in kube-system — commonly the tenth address of the service CIDR.

3. **The answer is a ClusterIP**
   10.96.114.23 is a VIP. It will not answer ICMP, and that is not a fault.

4. **The negative is correct**
   A Service record exists only under its own namespace. This 'failure' is the API behaving exactly as specified.

5. **BusyBox nslookup caveat**
   It ignores the search list for names that already contain a dot, so partial names look broken under nslookup only.

Source: courseware/labs/lab-11-services-endpoints/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/dns-pod-service/

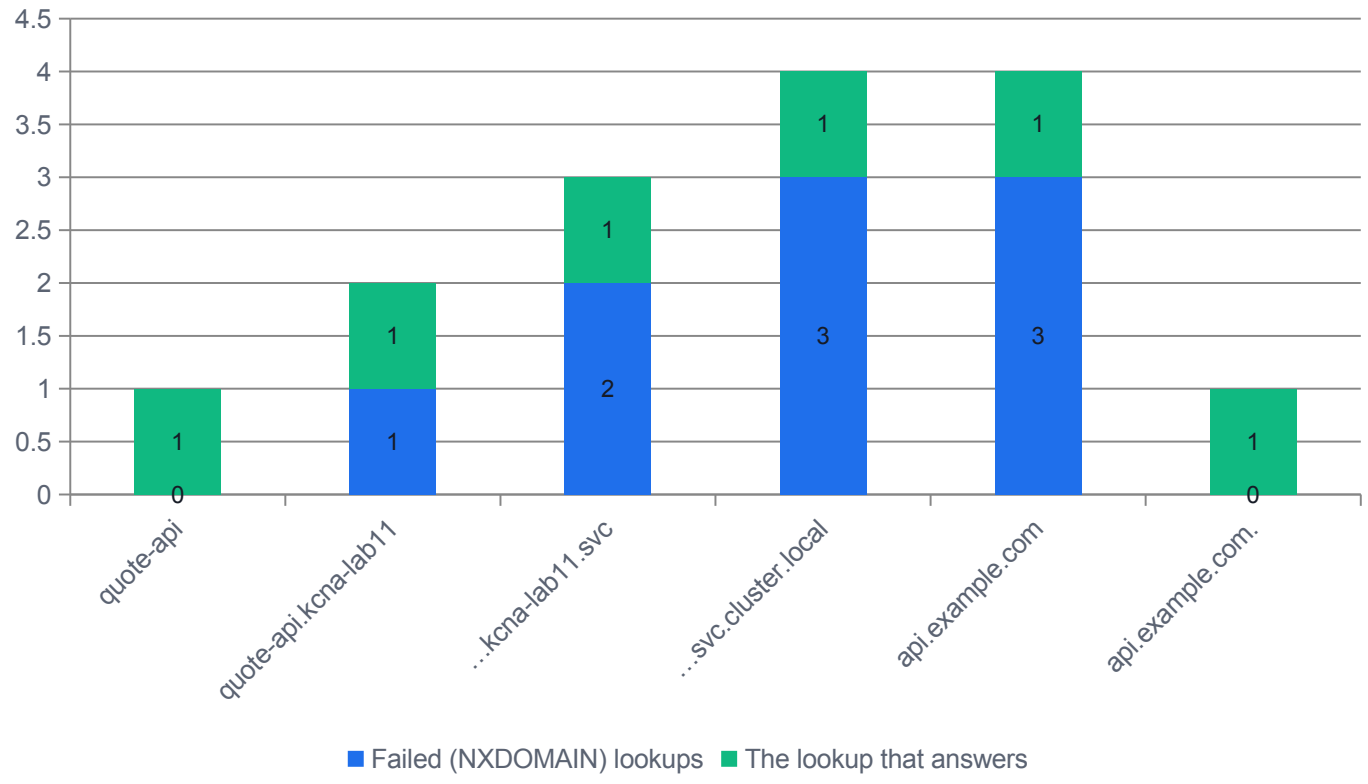
© 2026 Tertiary Infotech Academy Pte Ltd

WORKED MODEL

The cost of ndots:5, counted honestly

WORKED MODEL from the actual three-entry search list above. Counting A-record queries only; arithmetic is on the right.

DNS queries issued per resolution



WHAT THE DATA SHOWS

The rule being applied

Fewer dots than ndots (5) means the search list is tried FIRST, in order, before the name is tried as absolute.

The in-cluster FQDN is the trap

It has 4 dots, so it is still 'partial'. Three suffixes are tried and fail before the correct name is used.

External names pay the same tax

api.example.com has 2 dots: three wasted queries, then the answer. Multiply by A and AAAA and it is six wasted.

The trailing dot ends it

api.example.com. is absolute — one query, no search path. So is lowering ndots via dnsConfig.

Model, not measurement

No latency or throughput figure is claimed. This is query counting from a documented resolver rule.

Source: kubernetes.io/docs/concepts/services-networking/dns-pod-service/. Counts derived arithmetically from the lab's resolv.conf; not a measurement.

© 2026 Tertiary Infotech Academy Pte Ltd

DNS RECORDS

What cluster DNS actually publishes

Six record shapes. Recognising which one a client needs answers most 'why can't it find the database' tickets.

1 Service A / AAAA

<svc>.<ns>.svc.cluster.local resolves to the ClusterIP. One record, one address, regardless of backend count.

2 Headless Service A / AAAA

With clusterIP: None the same name returns one record per ready Pod — the client load-balances itself.

3 Per-Pod names in a headless set

<pod-hostname>.<svc>.<ns>.svc.cluster.local gives each Pod a stable name. StatefulSets depend on this.

4 SRV records

<port-name>.<proto>.<svc>.<ns>.svc.cluster.local returns port and target — this is why named ports matter.

5 ExternalName CNAME

A Service of type ExternalName publishes a CNAME to an outside host. No proxying, no VIP, no endpoints.

6 Pods are not named by default

Pod A records exist only for headless-Service members or when hostname/subdomain are set explicitly.

Source: kubernetes.io/docs/concepts/services-networking/dns-pod-service/

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARISON

ClusterIP DNS versus headless DNS

Headless is not a lesser Service. It is a deliberate choice to move load balancing to the client.

Aspect	Normal ClusterIP Service	Headless (clusterIP: None)
DNS answer	One A record — the virtual IP	One A record per READY endpoint, returned as a set
Load balancing	Kernel DNAT chooses a backend per connection	The client chooses; DNS just hands over the list
kube-proxy involvement	Programs rules for the VIP	None — there is no VIP to program
Per-Pod addressing	Not available through this Service	<pod>.<svc>.<ns>.svc.cluster.local for each member
Typical use	Stateless HTTP and gRPC backends	StatefulSets, clustered databases, client-side balancing
Failure signature	Refused or timed out at the VIP when the backend set is empty	NXDOMAIN or an empty answer — the name itself goes away

CHOOSE

Headless when identity matters (Day 4 StatefulSets); ClusterIP when only the name matters.

Source: kubernetes.io/docs/concepts/services-networking/dns-pod-service/ · kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A headless Service, and what changes

One field removes the virtual IP and changes what DNS returns — nothing else in the object changes.

50-service-headless.yaml (Lab 12)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: booking-edge-headless
  namespace: kcna-lab12
spec:
  clusterIP: None          # the whole point
  selector:
    app: booking-edge
  ports:
    - port: 80
      targetPort: 5678

# Result
#   CLUSTER-IP column shows  None
#   No nodePort is allocated
#   EndpointSlice is still produced normally
```

READING IT

1. **clusterIP: None is not empty**
   It is an explicit literal. Leaving the field out means 'allocate one for me' — the opposite instruction.

2. **Still a Service, still selected**
   The selector and the EndpointSlice controller behave exactly as before. Only the VIP disappears.

3. **port still matters**
   It is reported in SRV records and used by clients that ask DNS for the port, even with no DNAT.

4. **Immutable after creation**
   You cannot toggle a Service between headless and ClusterIP; you replace the object.

Source: courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

DNS resolves, the connection is refused — the classic

Name resolution succeeding proves the Service object exists. It proves nothing about backends.

SYMPTOM

A one-character selector typo. The Service is healthy, the name resolves, and every request fails instantly.

EVIDENCE — actual cluster output

```
$ kubectl exec -n kcna-lab11 dns-client -- \
    wget -qO- http://quote-api-broken
wget: can't connect to remote host (10.96.203.77): Connection refused
command terminated with exit code 1

$ kubectl describe service quote-api-broken -n kcna-lab11
Selector:   app=quote-apis
Endpoints:

$ kubectl get pods -n kcna-lab11 -l app=quote-apis
No resources found in kcna-lab11 namespace.
```

✓ **kubectl get endpointslices -n kcna-lab11 shows three ready addresses again**

Source: courseware/labs/lab-11-services-endpoints/verification/expected-output.md

ROOT CAUSE

- The selector says app=quote-apis; the Pods carry app=quote-api
- The label query returns zero Pods, so the EndpointSlice is empty
- In iptables mode kube-proxy REJECTs a VIP with no ready endpoints — hence an instant refusal, not a timeout

FIX

- Correct the selector to the label the Pods actually carry
- Verify by re-running it as a query, not by re-reading YAML
- The slice repopulates with no Pod restart and no rollout
- On IPVS or some CNIs the same fault presents as a TIMEOUT

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Headless DNS returns the Pods themselves

The difference between the two Service shapes is visible in one DNS answer.

Representative shape — Lab 12, kcna-lab12

```
# Normal ClusterIP Service — ONE answer, the VIP
$ nslookup booking-edge-clusterip.kcna-lab12.svc.cluster.local
Name:    booking-edge-clusterip.kcna-lab12.svc.cluster.local
Address: 10.96.44.10

# Headless Service — one answer PER READY POD
$ nslookup booking-edge-headless.kcna-lab12.svc.cluster.local
Name:    booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.0.31
Name:    booking-edge-headless.kcna-lab12.svc.cluster.local
Address: 10.244.1.17

$ kubectl get svc booking-edge-headless -n kcna-lab12
NAME                   TYPE        CLUSTER-IP   PORT(S)
booking-edge-headless  ClusterIP   None         80/TCP
```

WHAT TO LOOK AT

1. **One name, two record sets**
   The same query form returns a single VIP or a list of Pod IPs, depending only on clusterIP: None.

2. **Readiness gates the answer**
   A Pod that is not Ready is removed from the DNS answer, exactly as it is removed from the endpoint set.

3. **The client now balances**
   It picks from the list. Many resolvers use the first answer, so distribution depends on the client library.

4. **CLUSTER-IP shows the literal None**
   Not an empty cell — the string None is the confirmation that no VIP was allocated.

5. **This is what StatefulSets need**
   Per-Pod addressing is the basis of stable network identity on Day 4.

Source: courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/dns-pod-service/

© 2026 Tertiary Infotech Academy Pte Ltd

Morning Break

11:00 – 11:10 AM

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 11 — Services, Endpoints and Cluster DNS

GOAL

Give the Meridian Freight quote-api a stable cluster identity by putting a ClusterIP Service in front of it, then prove what that name actually points at by reading the EndpointSlice the selector produced - and break it with a one-character selector typo.

YOU WILL PRODUCE

- A quote-api Deployment (3 replicas) serving data/quote-api-index.html from a ConfigMap on named container port http/8080
- A ClusterIP Service mapping port 80 to the named targetPort http, with an EndpointSlice holding exactly the three Ready Pod IPs
- A dns-client Pod that resolves every name form in data/dns-fixtures.csv, including the cross-namespace case that correctly fails

CONCEPTS EXERCISED

- A Service stores a selector, not a Pod list; the endpoints controller derives the EndpointSlice from it
- port vs targetPort vs a named container port - and why only Ready Pods get conditions.ready true
- Cluster DNS: <svc>.<ns>.svc.cluster.local, the Pod search path and ndots:5
- Empty-endpoint signature: DNS resolves but TCP is refused - always re-run the selector as a label query

LAB	NAMESPACE	MAPS TO	TIME
Lab 11	kcna-lab11	LO3 · A2 · K2 K4	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-11-services-endpoints/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

SERVICE TYPES

The types are nested, not parallel

Setting type: LoadBalancer does not replace the ClusterIP — it wraps it. Read the -o wide output and you will see all three layers at once.

ClusterIP — the base

1

A virtual IP reachable only from inside the cluster. Every other type is this type plus something extra.

NodePort — ClusterIP + a node door

2

Everything ClusterIP does, plus a port opened on EVERY node in the 30000-32767 range.

LoadBalancer — NodePort + a request

3

Everything NodePort does, plus a request to an external controller to provision a real load balancer.

ExternalName — the odd one out

4

No selector, no endpoints, no VIP. Publishes a CNAME to an outside hostname and nothing more.

Headless — a modifier, not a type

5

clusterIP: None. The type is still ClusterIP; the VIP is simply not allocated.

Why the nesting matters

6

A LoadBalancer Service still has a ClusterIP and still has a nodePort. Deleting the LB does not remove those.

Source: kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARISON

Choosing between the Service shapes

Reachability, allocation and dependency. Answer those three and the type chooses itself.

Type	Reachable from	Allocates	Depends on
ClusterIP	Inside the cluster only	One VIP from the service CIDR	kube-proxy on each node
NodePort	Any node IP, from wherever nodes are reachable	VIP + one port on every node	A free port in 30000-32767 and node-level firewall rules
LoadBalancer	The external address, once provisioned	VIP + nodePort + an external address	A cloud-controller-manager or an implementation such as MetalLB
ExternalName	Wherever the target host is reachable	Nothing — no VIP, no port, no endpoints	The external DNS name resolving for your clients
Headless (clusterIP: None)	Inside the cluster only	Nothing — DNS returns Pod IPs directly	Client-side load balancing in the caller

CHOOSE

Default to ClusterIP. Escalate only when an identified caller genuinely sits outside the cluster.

Source: kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A NodePort Service, and what it inherits

NodePort is the first type that punches a hole in the node network — treat it as a firewall decision.

30-service-nodeport.yaml (Lab 12)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: booking-edge-nodeport
  namespace: kcna-lab12
spec:
  type: NodePort
  selector:
    app: booking-edge
  ports:
    - port: 80          # on the ClusterIP, still allocated
      targetPort: 5678  # on the Pod
      nodePort: 30080   # on EVERY node
# Optional: externalTrafficPolicy: Local
#   preserves the client source IP, but drops traffic
#   on nodes that host no ready backend
```

READING IT

1. **Three ports, three scopes**
   80 is on the VIP, 5678 is in the container, 30080 is on every node's network interface.

2. **Pinning nodePort is optional**
   Omit it and the API server allocates one from the pool. Pin it only when a fixed external contract demands it.

3. **Every node opens the port**
   Including nodes running none of the backend Pods. Traffic arriving there is forwarded to a node that has one.

4. **externalTrafficPolicy is the trade-off**
   Cluster (default) preserves reachability but SNATs the source IP; Local preserves the client IP but not reachability from every node.

Source: courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

NodePort facts you will be examined on

Ten reference facts, each of which has produced a real incident somewhere.

Default range	30000-32767 — 2,768 ports, cluster-wide
Where it is set	kube-apiserver flag --service-node-port-range
Who allocates	The API server, at admission time — not kube-proxy
Pinning	spec.ports[].nodePort; omit it for automatic allocation
Collision	A second Service asking for the same number is rejected outright
Out of range	Rejected by the API server's validation, not by the OpenAPI schema
Scope	Opened on every node, whether or not it runs a backend Pod
Still has a ClusterIP	NodePort does not remove the VIP; both paths work simultaneously
Reaching it from a laptop	Needs node reachability — on kind, extraPortMappings at create time, or simply kubectl port-forward
Security posture	It is an unauthenticated door on every node; pair it with firewall rules

Source: kubernetes.io/docs/concepts/services-networking/service/ · courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

nodePort 8080 — rejected before the object ever exists

Two distinct allocator errors share one message shape: out of range, and already allocated.

SYMPTOM

A perfectly valid-looking manifest is refused at apply time because the requested port is outside the allocator's range.

EVIDENCE — actual cluster output

```
$ kubectl apply -f manifests/70-service-nodeport-invalid.yaml
The Service "booking-edge-nodeport-invalid" is invalid:
  spec.ports[0].nodePort: Invalid value: 8080: provided port
  is not in the valid range. The range of valid ports is
  30000-32767

$ kubectl get service booking-edge-nodeport-invalid -n kcna-lab12
Error from server (NotFound): services
  "booking-edge-nodeport-invalid" not found

# A different rejection, same allocator:
Invalid value: 30080: provided port is already allocated
```

ROOT CAUSE

- 8080 is outside the cluster's --service-node-port-range
- This is API-server validation plus allocator state, not schema validation — so static linters pass the file happily
- Nothing was created: there is no partial object to clean up

FIX

- Choose a free port inside 30000-32767, or omit nodePort entirely and let the allocator pick
- If a fixed low port is a hard requirement, that is a LoadBalancer or Ingress requirement, not a NodePort one
- Widening the range is a cluster-wide API-server change

✓ **kubectl get svc -n kcna-lab12 -o jsonpath shows the allocated nodePort inside the range**

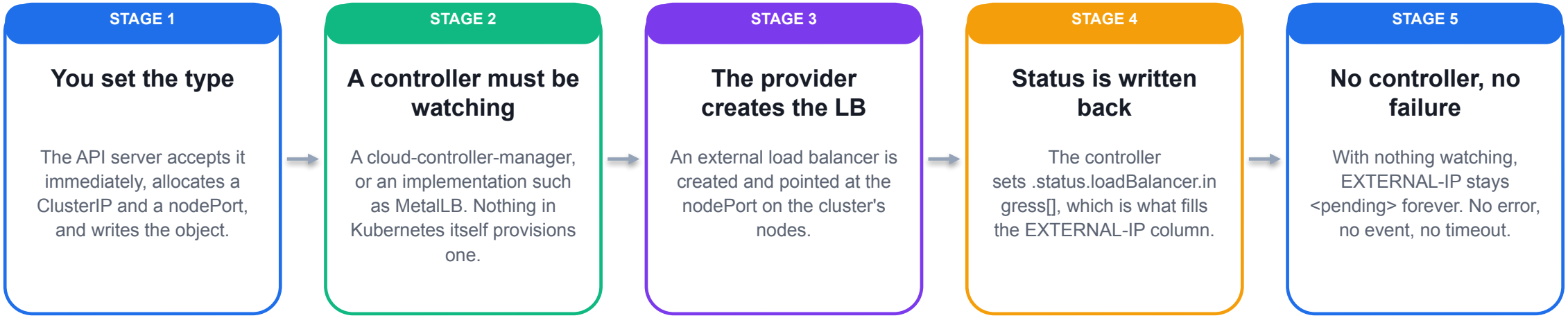
Source: courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

LOADBALANCER

What has to exist for type: LoadBalancer to mean anything

type: LoadBalancer is a request, not a capability. The cluster records the request whether or not anyone can fulfil it.



WHAT IT MEANS

Permanent <pending> is the documented outcome on a cluster with no load-balancer controller — and it is exactly what a stock kind cluster shows.

Source: kubernetes.io/docs/concepts/services-networking/service/ · courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Four Service shapes, one Deployment, one endpoint set

Read the columns as layers: VIP, then node door, then external address.

Lab 12 — kcna-lab12 on a stock kind cluster

```
$ kubectl get svc -n kcna-lab12 -o wide
NAME                    TYPE          CLUSTER-IP    EXTERNAL-IP  PORT(S)
booking-edge-clusterip  ClusterIP     10.96.44.10   <none>       80/TCP
booking-edge-nodeport   NodePort      10.96.51.203  <none>       80:30080/TCP
booking-edge-lb         LoadBalancer  10.96.77.9    <pending>    80:31544/TCP
booking-edge-headless   ClusterIP     None          <none>       80/TCP

$ kubectl describe svc booking-edge-lb -n kcna-lab12
Events:  <none>

# All four resolve to the same two endpoint addresses
$ kubectl get endpointslices -n kcna-lab12 \
    -o custom-columns=NAME:.metadata.name,EP:.endpoints[*].addresses
```

WHAT TO LOOK AT

1. **The nesting is visible**
   The LoadBalancer row still shows a ClusterIP and still shows an auto-allocated nodePort, 31544.

2. **<pending> is not an error state**
   It is the honest representation of an unfulfilled request on a cluster with no LB controller.

3. **Events: <none> is the corroboration**
   Nothing has even looked at the object. Combine the two signals rather than relying on either alone.

4. **None is a literal**
   The headless row shows the string None in CLUSTER-IP, not an empty cell.

5. **One selector, one backend set**
   All four Services select the same Pods, so all four produce identical endpoint addresses.

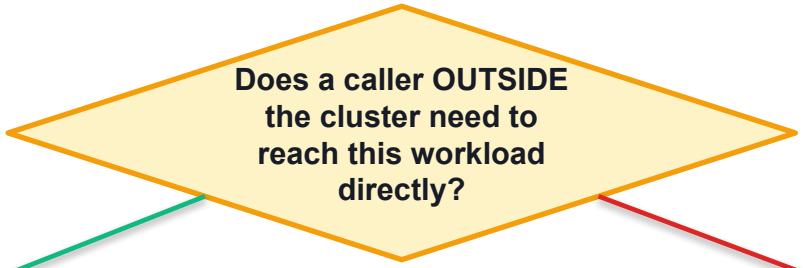
Source: courseware/labs/lab-12-service-types-nodeport-lb/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

DECISION

Which Service type does this requirement actually need?

Externally reachable is a requirement, not a preference. Everything else follows from it.



YES — it must be externally reachable

- Managed cloud and one L4 endpoint per app → LoadBalancer
- Many HTTP apps sharing one entry point → an Ingress or Gateway in front of ClusterIP Services
- Bare metal with no LB implementation → NodePort plus your own external load balancer
- Cost check: one cloud LB per Service adds up fast

NO — callers are inside the cluster

- Stateless backend consumed by other Pods → ClusterIP
- Clients need per-Pod identity or do their own balancing → headless (clusterIP: None)
- Fronting something outside the cluster by name → ExternalName, or ClusterIP with manual EndpointSlices
- Ad-hoc human access for debugging → kubectl port-forward

Source: kubernetes.io/docs/concepts/services-networking/service/

© 2026 Tertiary Infotech Academy Pte Ltd

CLI GENERATION

Generating Service manifests without writing YAML

Both generators exist to save keystrokes, not to replace understanding of the fields.

Registered outline: generating Service manifests via the CLI

```
# Generate, do not create: --dry-run=client
$ kubectl create service clusterip quote-api \
    --tcp=80:8080 --dry-run=client -o yaml
apiVersion: v1
kind: Service
metadata:
  name: quote-api
spec:
  ports:
  - name: 80-8080
    port: 80
    targetPort: 8080
  selector:
    app: quote-api
  type: ClusterIP

# Derive from an existing workload instead:
$ kubectl expose deployment quote-api --port=80 \
    --target-port=http --dry-run=client -o yaml
```

WHAT TO LOOK AT

1. **--dry-run=client renders locally**
   Nothing is sent for creation. You get YAML on stdout to review, edit and commit.

2. **create service invents the selector**
   It assumes app: <name>. Check that against the labels your Pods actually carry before applying.

3. **expose reads the real object**
   It copies the selector from the Deployment, which is why it is the safer generator of the two.

4. **Generated port names are ugly**
   80-8080 is machine-made. Rename it to something an Ingress backend can refer to readably.

5. **Generation is a starting point**
   The output still needs review — it is a draft, not an authored manifest.

Source: kubernetes.io/docs/reference/generated/kubectl/kubectl-commands#create-service and #expose

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 12 — Service Types: ClusterIP, NodePort and LoadBalancer

GOAL

Expose one Meridian Freight backend through all four Service shapes at once - ClusterIP, NodePort, LoadBalancer and headless - and record honestly which ones a stock kind cluster can actually deliver.

YOU WILL PRODUCE

- Four Services over a single booking-edge Deployment, proven by kubectl get svc -o wide to share one selector and one endpoint set
- A pinned NodePort on 30080 reached on the node InternalIP from inside the cluster, plus the kind extraPortMappings config in data/ that would expose it to the host
- A LoadBalancer Service documented and asserted as permanently <pending>, with kubectl describe showing Events: <none> as the proof no controller is watching

CONCEPTS EXERCISED

- The types are nested, not parallel: ClusterIP is inside NodePort is inside LoadBalancer
- nodePort comes from a cluster-wide pool, default 30000-32767 via --service-node-port-range
- type: LoadBalancer is a request that only a cloud-controller-manager or MetalLB can fulfil - kind has neither
- clusterIP: None removes the virtual IP so DNS returns one A record per ready Pod

LAB	NAMESPACE	MAPS TO	TIME
Lab 12	kcna-lab12	LO3 · A2 · K2 K4	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-12-service-types-nodeport-lb/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

CHECKPOINT

What Services and DNS have established

Two registered topics meet here: routing gets richer, and then access gets restricted.

SETTLED BEFORE LUNCH

- Pods have real IPs; ClusterIPs are rules, not interfaces
- A Service stores a selector; the controller derives the EndpointSlice from it continuously
- kube-proxy programs the kernel — nothing proxies on the data path
- DNS gives a stable name; the search list and ndots:5 explain both the convenience and the cost
- The four Service shapes are nested layers over one base

CARRY INTO THE AFTERNOON

- Everything so far balances at layer 4 only — no host, path or header awareness anywhere
- Nothing so far restricts who may connect; the network is still flat and open
- A declaration with no controller behind it is inert — you have now seen that with LoadBalancer
- That exact pattern repeats with Ingress after lunch
- Then Topic 4 asks the other half: who is allowed to ask?

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/concepts/services-networking/dns-pod-service/

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1:10 – 1:50 PM

© 2026 Tertiary Infotech Academy Pte Ltd

MOTIVATION

Why layer 4 runs out, and what Ingress adds

Ingress exists to consolidate many HTTP services behind one entry point and one certificate story.

1 One LoadBalancer per Service is wasteful

Forty HTTP services means forty external addresses and forty cloud load-balancer bills.

2 Services cannot see a hostname

shop.example.com and tracking.example.com arrive on the same socket. Layer 4 cannot tell them apart.

3 Services cannot see a path

/api and /static must reach different backends. A ClusterIP has no idea what a path is.

4 TLS has to terminate somewhere

Certificates, SNI and HTTP/2 negotiation belong at the edge, not in every application container.

5 Ingress is the L7 declaration

networking.k8s.io/v1: host rules, path rules with a pathType, a default backend and a TLS block.

6 An Ingress controller is the implementation

The object is inert until a controller claims it. This is the LoadBalancer lesson again, one layer up.

Source: kubernetes.io/docs/concepts/services-networking/ingress/

© 2026 Tertiary Infotech Academy Pte Ltd

DATA PATH

Where an Ingress controller actually sits

Traffic enters through the controller's OWN Service. The Ingress object never carries a packet.

DECLARATION (API OBJECTS)

- **Ingress** — Host and path rules, pathType, backend service+port, tls block
- **IngressClass** — Names a controller; created BY the controller, not by you
- **Secret (kubernetes.io/tls)** — tls.crt and tls.key that the controller will serve

IMPLEMENTATION (RUNS IN THE CLUSTER)

- **Controller Pod** — Watches Ingress objects it is entitled to claim, via its IngressClass
- **Reverse proxy config** — Rules are compiled into the proxy's own configuration and reloaded
- **Its own Service** — Usually LoadBalancer or NodePort — the controller needs its own way in

BACKENDS (UNCHANGED)

- **ClusterIP Service** — The Ingress backend the controller resolves and forwards to
- **EndpointSlice** — Many controllers bypass the VIP and send straight to endpoint IPs
- **Pods** — Receive plain HTTP; TLS was terminated at the controller

MECHANISM

Ingress routing is a two-hop story: something external reaches the controller, and the controller reaches your backend. Both hops must exist before anything works.

Source: kubernetes.io/docs/concepts/services-networking/ingress/ · kubernetes.io/docs/concepts/services-networking/ingress-controllers/ · retirement notice: kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

An Ingress rule, read field by field

Creating an Ingress resource is a registered outline item. These are the fields that decide its behaviour.

40-ingress-shop.yaml (Lab 13, abridged)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: meridian-shop
  namespace: kcna-lab13
spec:
  ingressClassName: meridian-edge
  defaultBackend:
    service:
      name: route-fallback-svc
      port: {number: 80}
  rules:
    - host: shop.meridianfreight.internal
      http:
        paths:
          - path: /catalog
            pathType: Prefix
            backend:
              service:
                name: catalog-svc
                port: {name: http}
```

READING IT

1. **ingressClassName binds the object**
   It names an IngressClass. A class that does not exist yet is legal, silent and never reconciled.
2. **defaultBackend catches the rest**
   Anything matching no rule goes here. Without it, unmatched requests are the controller's problem.
3. **host is optional per rule**
   Omit it and the rule matches any host. Present, it matches that host only.
4. **pathType is a required enum**
   Exact, Prefix or ImplementationSpecific — case-sensitive, and validated by the API server.
5. **port by name or number**
   Referring to the Service's named port survives a backend port change without editing the Ingress.

Source: courseware/labs/lab-13-ingress-routing/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/ingress/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

pathType and matching semantics

The whole matching model in ten lines. Most Ingress bugs are one of rows 3, 5 or 9.

Exact	The request path must equal the value exactly, case-sensitively
Prefix	Matches on whole path ELEMENTS split by /, not on string prefix
Prefix trap	/catalog matches /catalog/shoes but NEVER /catalogue
ImplementationSpecific	Meaning is delegated to the controller — often regex
Case sensitivity	The enum itself is case-sensitive: "prefix" is rejected

Longest match wins	Where multiple paths match, the longest matching path is preferred
Exact beats Prefix	At equal length, an Exact match is preferred over a Prefix match
Trailing slash	/foo and /foo/ are distinct for Exact; Prefix ignores the trailing slash
Host wildcards	*.example.com matches one label only — not example.com itself
No rewriting in the API	Path rewriting is an annotation, hence controller-specific and not portable

Source: kubernetes.io/docs/concepts/services-networking/ingress/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Two Ingress failures: one loud, one completely silent

Loud failures teach syntax. Silent ones teach architecture — and cost far more time.

SYMPTOM

A lowercase pathType is rejected instantly. An Ingress with no usable class is accepted and then ignored forever.

EVIDENCE — actual cluster output

```
# LOUD — API server validation
$ kubectl apply -f manifests/60-ingress-invalid-pathtype.yaml
The Ingress "meridian-shop-broken" is invalid:
  spec.rules[0].http.paths[0].pathType: Unsupported value:
  "prefix": supported values: "Exact",
  "ImplementationSpecific", "Prefix"

# The same file passes STATIC validation:
$ kubeconform -strict -summary 60-ingress-invalid-pathtype.yaml
Valid: 1, Invalid: 0, Errors: 0

# SILENT — created, classless, never reconciled
NAME                  CLASS    HOSTS      ADDRESS  PORTS
meridian-shop-broken  <none>   shop...             80
```

ROOT CAUSE

- pathType is a closed, case-sensitive enum enforced by the API server, so a linter with the schema still lets it through
- The silent case is different: the object is valid, but no controller has an IngressClass entitling it to claim it
- Class handling when ingressClassName is omitted is controller-dependent — a default IngressClass is one mechanism, not the only one

FIX

- Fix the enum casing — the error names the three legal values
- For the silent case, confirm which controller is installed and set ingressClassName to a class it actually serves
- Diagnose with the whole picture: IngressClasses present, controller Pods running, and the object's events

Source: courseware/labs/lab-13-ingress-routing/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/ingress/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Diagnosing an Ingress precisely — and what NOT to conclude

This is a lesson in evidential discipline as much as in Ingress.

Lab 13 — a correct Ingress on a cluster with no controller installed

```
$ kubectl get ingress -n kcna-lab13
NAME              CLASS          HOSTS        ADDRESS  PORTS
meridian-shop     meridian-edge  shop,tracking          80
meridian-shop-tls  meridian-edge  secure...              80, 443

$ kubectl get ingressclass
No resources found

$ kubectl describe ingress meridian-shop -n kcna-lab13
Address:
Ingress Class:   meridian-edge
Default backend: route-fallback-svc:80 (10.244.0.44:5678)
Events:          <none>
```

WHAT TO LOOK AT

1. **Blank ADDRESS is NOT proof on its own**
   A local controller can route perfectly without ever publishing an external status address.

2. **Zero IngressClasses is the strong signal**
   Combined with Events: <none>, that is a sound conclusion: nothing can claim this object.

3. **PORTS 443 proves nothing**
   It appears purely because the object carries a tls block. It is not evidence TLS is being served.

4. **The backend resolved fine**
   The parenthesised endpoint IP shows the Service and its backends are healthy. The gap is the controller.

5. **Diagnose with three facts**
   IngressClasses present, controller Pods running, events on the object. Never one column alone.

Source: courseware/labs/lab-13-ingress-routing/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The TLS block, and what it does not do

Creating a TLS certificate for authentication appears twice today: here for servers, and later for client identity in Topic 4.

50-ingress-tls-example.yaml plus its Secret

```
spec:
  tls:
    - hosts:
        - secure.meridianfreight.internal
      secretName: meridian-shop-tls-cert
  rules:
    - host: secure.meridianfreight.internal
      ...
---
apiVersion: v1
kind: Secret
metadata:
  name: meridian-shop-tls-cert
type: kubernetes.io/tls
data:
  tls.crt: <base64 PEM certificate>
  tls.key: <base64 PEM private key>
```

READING IT

1. **Two fields only**
   hosts and secretName. There is no cipher list, no protocol version and no certificate content in the Ingress.
2. **The Secret type is enforced**
   kubernetes.io/tls requires exactly the keys tls.crt and tls.key. The API validates that.
3. **The controller terminates TLS**
   It loads the Secret and serves the certificate. The Ingress object never handles a byte of traffic.
4. **A missing Secret is not an error**
   The Ingress applies cleanly before the Secret exists. A live controller would log a warning and fall back.
5. **Same namespace only**
   The Secret must live in the Ingress's namespace. There is no cross-namespace certificate reference.

Source: courseware/labs/lab-13-ingress-routing/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/ingress/

© 2026 Tertiary Infotech Academy Pte Ltd

INGRESSCLASS

How an Ingress finds its controller

IngressClass is the entire ownership model. Get it wrong and you get silence, never an error.

1 IngressClass is a real object

Cluster-scoped, naming a controller string such as example.com/ingress-controller.

2 Controllers install it

You do not normally author IngressClass objects; the controller's own installation creates them.

3 ingressClassName is the binding

The Ingress names the class; the controller claims only objects naming a class it owns.

4 A forward reference is legal

Naming a class that does not exist produces no error and no event. It simply never gets claimed.

5 Default class behaviour

An IngressClass annotated as default can claim class-less Ingresses — but the exact handling is controller-dependent.

6 Multiple controllers coexist

Internal and external edges, or blue/green migrations, are expressed as two classes in one cluster.

Source: kubernetes.io/docs/concepts/services-networking/ingress/ · kubernetes.io/docs/concepts/services-networking/ingress-controllers/ · retirement notice: kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/

© 2026 Tertiary Infotech Academy Pte Ltd

CURRENCY

The controller landscape you are choosing from in 2026

A retirement changes which implementation you deploy. It does not change the API you were taught.

1 The API is not the project

The community ingress-nginx project was retired in March 2026. networking.k8s.io/v1 Ingress is unaffected and stable.

2 Do not conflate two NGINXes

The retired item is a CONTROLLER project. The nginx web-server image used elsewhere in this course is a different thing.

3 Controllers are pluggable by design

That is the point of IngressClass. Replacing a controller is a deployment change, not an application rewrite.

4 This course's runnable path pins Traefik

Traefik ships an Ingress controller, installs by Helm chart and can be reached with port-forward — no new cluster needed.

5 Gateway API projects are a different surface

Some maintained projects target gateway.networking.k8s.io rather than Ingress. Verify which API a project serves.

6 Verify before you standardise

Check the project's own current documentation for maintenance status and supported Kubernetes versions.

Source: kubernetes.io/docs/concepts/services-networking/ingress-controllers/ · retirement notice: kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/ · Traefik chart 41.4.0 / app v3.7.12 (traefik.github.io/charts), pinned and recorded in qa/PARENT-VERIFIED-FACTS.md

© 2026 Tertiary Infotech Academy Pte Ltd

DIRECTION OF TRAVEL

Ingress and Gateway API side by side

Both are current. One is finished; the other is where new capability is being added.

Aspect	Ingress (networking.k8s.io/v1)	Gateway API (gateway.networking.k8s.io)
Status	Stable, and feature-frozen — new features go to Gateway API	The successor surface; developed as a Kubernetes SIG project
Object model	One object mixing infrastructure and routing concerns	GatewayClass, Gateway and *Route split by role
Protocols	HTTP and HTTPS	HTTPRoute plus TCP, TLS, UDP and gRPC route kinds
Extensibility	Annotations — powerful but controller-specific and non-portable	Typed fields and typed policy attachment instead of annotation strings
Multi-tenancy	Namespace boundaries are awkward to express	Explicit cross-namespace references with permission granted by the target
What to learn now	Required: it is what most clusters run and what KCNA asks about	Recognise the model and the direction; expect migration work over time

CHOOSE

Learn Ingress properly, read Gateway API's role split — and never assume a controller serves both.

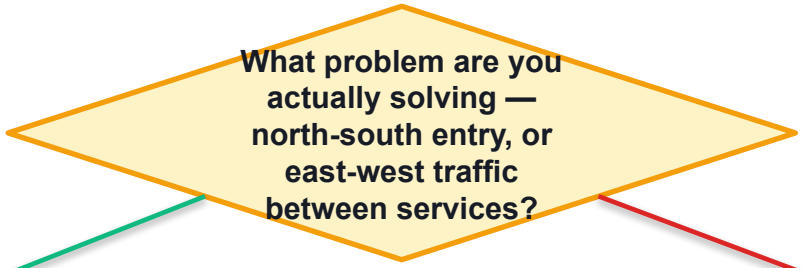
Source: kubernetes.io/docs/concepts/services-networking/ingress/ · gateway-api.sigs.k8s.io · kubernetes.io/docs/concepts/services-networking/gateway/

© 2026 Tertiary Infotech Academy Pte Ltd

DECISION

Ingress, Gateway API or a service mesh?

These are not competitors. Most mature clusters run an edge AND a mesh, solving different problems.



YES

NO

NORTH-SOUTH — traffic entering the cluster

- Straightforward HTTP host/path routing → Ingress
- Non-HTTP protocols, header-based splits or typed policy → Gateway API
- Several teams sharing one edge with delegated control → Gateway API's role split
- Either way you need a controller installed and running

EAST-WEST — traffic between your own services

- Automatic mTLS between workloads → service mesh
- Retries, timeouts, circuit breaking without app changes → service mesh
- Uniform golden metrics per service call → service mesh
- Mesh is additive: it does not remove the need for an edge

Source: kubernetes.io/docs/concepts/services-networking/ingress/ · istio.io/latest/docs/overview/ · linkerd.io/2/overview/ · envoyproxy.io

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The IngressClass object that makes claiming possible

Deploying an Ingress controller is a registered outline item. This object is how the deployment becomes usable.

An IngressClass, annotated

```yaml
apiVersion: networking.k8s.io/v1
kind: IngressClass          # CLUSTER-SCOPED
metadata:
  name: meridian-edge
  annotations:
    # optional: claim class-less Ingresses
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: example.com/ingress-controller
  parameters:                # optional, controller-defined
    apiGroup: example.com
    kind: EdgeConfig
    name: meridian-edge-settings
# Ingress objects then set:
#   spec.ingressClassName: meridian-edge
```

READING IT

1. **spec.controller is the real identity**
   A controller only claims classes whose controller string it recognises. The class NAME is just a label for humans.
2. **Cluster-scoped on purpose**
   One class can be referenced by Ingress objects in every namespace.
3. **Installed by the controller**
   You normally get this object from the controller's own installation, not from your application manifests.
4. **The default-class annotation**
   One mechanism by which class-less Ingresses get claimed. Exact behaviour is still controller-dependent.
5. **parameters is an escape hatch**
   A typed reference to controller-specific configuration — and therefore not portable between controllers.

Source: kubernetes.io/docs/concepts/services-networking/ingress/ · kubernetes.io/docs/concepts/services-networking/ingress-controllers/ · retirement notice: kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/

© 2026 Tertiary Infotech Academy Pte Ltd

SERVICE MESH

What a mesh adds that Services cannot

A mesh moves cross-cutting network concerns out of every application and into the platform.

1 Identity per workload

Each workload gets a cryptographic identity — commonly a SPIFFE ID — issued and rotated by the mesh.

2 mTLS by default

Both ends authenticate with certificates, so traffic is encrypted and the caller is provably identified.

3 L7 traffic control

Retries, timeouts, circuit breaking and weighted splits, applied without touching application code.

4 Uniform telemetry

Request rate, error rate and latency for every call, measured identically across languages.

5 Authorization at layer 7

Policy expressed as 'service A may call POST /pay on service B' rather than as IP ranges.

6 It is not free

Extra hops, extra CPU and memory, a control plane to run and a new failure domain to understand.

Source: istio.io/latest/docs/overview/ · linkerd.io/2/overview/ · envoyproxy.io

© 2026 Tertiary Infotech Academy Pte Ltd

MESH ANATOMY

Control plane, data plane and your workload

The mesh is an interception layer. Your application code and your Service objects are unchanged.

CONTROL PLANE	**Config distribution** Translates mesh policy into proxy configuration and pushes it	**Certificate authority** Issues and rotates short-lived workload identity certificates	**Kubernetes watch** Reads Services, EndpointSlices and mesh CRDs from the API server
DATA PLANE	**Sidecar proxy** A proxy container in each Pod, intercepting inbound and outbound traffic	**Node-level proxy** Ambient mode instead puts an L4 proxy per node, with L7 only where needed	**Traffic capture** iptables redirection or eBPF steers the workload's traffic into the proxy
WORKLOAD	**Application container** Unmodified — it still dials the ordinary Service name	**Cluster DNS + Service** Still resolve as normal; the mesh sits underneath, not instead	**NetworkPolicy** Still applies at L3/L4 — the two controls are complementary

MECHANISM

Envoy is the most common data-plane proxy (Istio, Contour, Cilium's L7 path), but it is not universal — Linkerd ships its own purpose-built micro-proxy.

Source: istio.io/latest/docs/overview/ · linkerd.io/2/overview/ · envoyproxy.io

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARISON

Sidecar mode versus ambient mode

Two deployment models for the same idea. The trade-off is cost and friction against isolation.

Aspect	Sidecar	Ambient / node-level
Where the proxy runs	One proxy container inside every meshed Pod	A shared L4 proxy per node, plus optional L7 proxies where policy needs them
Resource cost	Proportional to Pod count — every replica pays	Proportional to node count for L4; L7 cost only where enabled
Adoption friction	Pods must be restarted to inject or remove the sidecar	Workloads can be enrolled without redeploying them
Blast radius	Isolated per Pod — one proxy failure affects one Pod	Shared per node — a node proxy failure affects more workloads
Maturity	The long-established model, widely operated	The newer model; verify status in the project's own current documentation

CHOOSE

For KCNA, know both models exist and why ambient was introduced — not which product to buy.

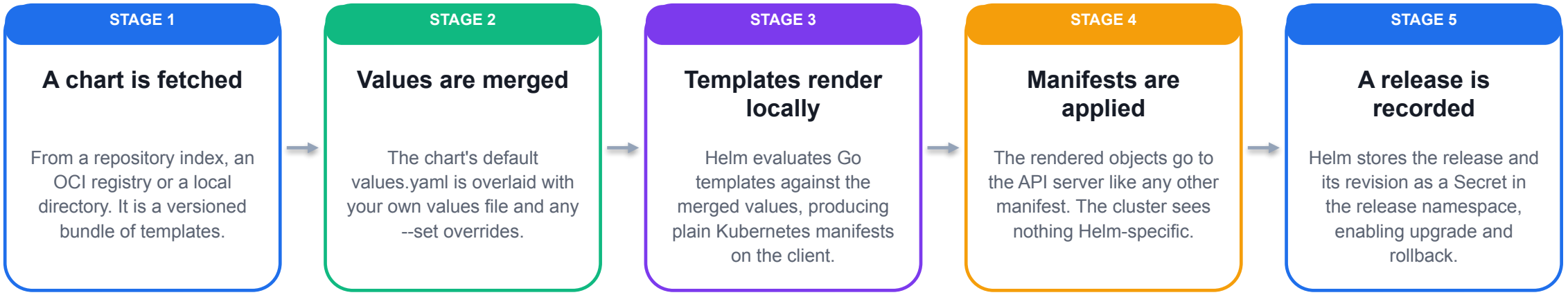
Source: istio.io/latest/docs/overview/ · linkerd.io/2/overview/ · envoyproxy.io

© 2026 Tertiary Infotech Academy Pte Ltd

PACKAGING

What happens when you deploy your first chart

Helm is a client-side template engine plus a release ledger. Nothing Helm-specific runs in the cluster.



WHAT IT MEANS

Because rendering is local, `helm template` shows you exactly what will be applied — review the output before you trust any third-party chart.

Source: helm.sh/docs/topics/charts/ · helm.sh/docs/topics/architecture/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The three files that define a chart

Installing Helm and deploying a first chart is a registered outline item. Understand these three files and the rest is command syntax.

Minimal chart structure

```
# Chart.yaml — identity and version
apiVersion: v2
name: quote-api
version: 0.1.0          # the CHART version
appVersion: "1.4.2"     # the APP version

# values.yaml — the defaults you may override
replicaCount: 3
image:
  repository: registry.example.com/quote-api
  tag: ""               # empty -> fall back to appVersion
service:
  type: ClusterIP
  port: 80

# templates/service.yaml — a template, not a manifest
  type: {{ .Values.service.type }}
  ports:
    - port: {{ .Values.service.port }}
```

READING IT

1. **Two versions, two meanings**
   version is the chart's own release number; appVersion is the software it installs. They move independently.
2. **values.yaml is the API of the chart**
   It is the documented surface a consumer may change. Anything not exposed there is not configurable.
3. **templates/ holds Go templates**
   {{ .Values.x }} is substituted at render time on the client, never by the cluster.
4. **The empty tag idiom**
   A blank default that falls back to appVersion is the conventional way to keep image and chart in step.
5. **Charts can depend on charts**
   Chart.yaml dependencies pull in sub-charts, which is how one install can bring a whole stack.

Source: helm.sh/docs/topics/charts/ · helm.sh/docs/topics/architecture/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Helm vocabulary, in one panel

Ten terms. Day 4's Lab 21 assumes every one of them.

Chart	A versioned package of templates plus default values
Release	One installation of a chart into one namespace, under a name you choose
Revision	An incrementing number per release change; rollback targets a revision
Repository	An index.yaml listing chart versions; charts may also live in OCI registries
values.yaml	The chart's defaults — the documented configuration surface

--set / -f	Override values on the command line or from your own values file
helm template	Render locally and print; applies nothing. The review step
Release storage	A Secret in the release namespace holds the release record
No in-cluster agent	Helm 3 removed Tiller; the client talks to the API server directly
Toolchain pinned here	Helm v4.2.4, verified against the official get.helm.sh archive checksum

Source: helm.sh/docs/topics/charts/ · helm.sh/docs/topics/architecture/ · toolchain pin recorded in qa/PARENT-VERIFIED-FACTS.md

© 2026 Tertiary Infotech Academy Pte Ltd

SCOPE

What Helm does, and what it deliberately does not

Knowing Helm's boundaries is what stops it being blamed for problems it was never designed to solve.

It packages and parameterises

- One chart, many environments, via values files
- Versioned and shareable through repositories or OCI
- Dependencies let one install bring a whole stack
- Templating removes copy-paste manifest drift

packaging

It tracks releases

- Named installation with a revision history
- helm upgrade and helm rollback operate on revisions
- State lives in a Secret in the release namespace
- helm list shows what a namespace actually has installed

lifecycle

It does NOT do these

- It is not a continuous reconciler — nothing reverts drift
- It does not validate that a chart is safe to run
- It does not replace RBAC, policy or admission control
- GitOps tools add the reconciliation loop — Day 4

boundaries

Source: helm.sh/docs/topics/charts/ · helm.sh/docs/topics/architecture/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 13 — Ingress Resources and HTTP Routing

GOAL

Model Meridian Freight's host and path routing with Ingress, then verify the rules through a pinned Traefik controller. Distinguish accepted API configuration from working HTTP routing.

YOU WILL PRODUCE

- Ingress host, Prefix and Exact rules aligned with data/routing-table.csv
- A TLS declaration and lab TLS Secret
- Controller inventory and observed routing evidence
- Optional Traefik 41.4.0 chart installation with an explicit IngressClass
- Seven live HTTP checks, including element boundaries and the standalone fallback Ingress

CONCEPTS EXERCISED

- Ingress declares routing; a controller implements it
- Controller discovery and end-to-end probes establish actual behaviour; an empty ADDRESS alone does not
- Explicit ingressClassName and controller-specific adoption
- Prefix path elements, Exact matching and strictPrefixMatching
- TLS termination at the controller; Service and resource backend types
- Traefik's rule-free fallback Ingress and routing through port-forward

LAB	NAMESPACE	MAPS TO	TIME
Lab 13	kcna-lab13	LO3 · A2 · K2 K4	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-13-ingress-routing/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

NAMESPACES

What a Namespace scopes — and what it does not

Namespaces are a naming and authorisation boundary. Assuming more than that is a recurring security error.

1 It scopes NAMES

Two Services may both be called api, one per namespace. Uniqueness is per namespace, per resource kind.

2 It scopes RBAC

A RoleBinding grants inside one namespace only. This is the primary reason namespaces exist operationally.

3 It scopes QUOTAS and LIMITS

ResourceQuota and LimitRange are namespaced objects and cap the whole namespace's consumption.

4 It scopes DNS

A Service record lives under <ns>.svc.cluster.local. Wrong namespace means the name does not resolve.

5 It is NOT a network boundary

Any Pod can reach any other Pod's IP across namespaces. Only NetworkPolicy changes that.

6 It is NOT a node or kernel boundary

Pods from different namespaces share nodes and kernels. Isolation there is a scheduling and runtime question.

Source: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Namespace that carries policy labels

The namespace is where Pod Security Standards are switched on. It carries policy, not just a name.

00-namespace.yaml (Lab 14)

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: kcna-lab14
  labels:
    kubernetes.io/metadata.name: kcna-lab14
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/enforce-version: latest
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/audit: restricted
    course: kcna
# The enforce label REJECTS violating Pods.
# The warn label only prints a warning on create.
# The audit label only records an audit annotation.
```

READING IT

1. **The metadata.name label is automatic**
   The control plane sets it on every namespace, which is what makes namespaceSelector usable in NetworkPolicy.
2. **Three independent modes**
   enforce, warn and audit can each name a different level. Warn first, enforce later, is the safe rollout.
3. **enforce-version pins behaviour**
   latest tracks the cluster; a pinned version keeps the checks stable across upgrades.
4. **Labels, not a separate object**
   Pod Security admission is configured entirely by namespace labels — there is no policy CRD to install.
5. **Your own labels belong here too**
   course: kcna is how the labs select and clean up everything they created.

Source: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ · kubernetes.io/docs/concepts/security/pod-security-standards/ · courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

SCOPE

Namespaced versus cluster-scoped resources

This distinction decides what `kubectl delete namespace` actually cleans up — and what it silently leaves.

Question	Namespaced	Cluster-scoped
Examples	Pod, Service, Deployment, ConfigMap, Secret, Role, RoleBinding, NetworkPolicy	Node, PersistentVolume, StorageClass, Namespace, ClusterRole, ClusterRoleBinding, IngressClass
Deleting the namespace	Removes them — garbage collected with the namespace	Leaves them behind; they must be deleted by name
Name uniqueness	Unique within the namespace	Unique across the whole cluster
How to check	kubectl api-resources --namespaced=true	kubectl api-resources --namespaced=false
RBAC grant	Role + RoleBinding, or ClusterRole + RoleBinding for reusable rules	Requires a ClusterRole bound by a ClusterRoleBinding

CHOOSE

Before deleting a namespace, ask which cluster-scoped objects your work created.

Source: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Namespaces in practice

Four built-in namespaces and six operational facts.

default	Where objects land with no -n. Not special otherwise
kube-system	Control-plane and add-on workloads. Treat as read-only
kube-public	Readable by unauthenticated users; holds cluster bootstrap info
kube-node-lease	Node heartbeat Lease objects — one per node
Terminating state	A namespace stuck here usually has a finalizer waiting on something

Cross-namespace access	Use the full DNS name; RBAC and policy still apply separately
Quota per namespace	ResourceQuota caps CPU, memory, storage and object counts
Default context	kubectl config set-context --current --namespace=<ns>
Not a hard tenant boundary	For untrusted tenants, consider separate clusters or stronger isolation
Labels matter	kubernetes.io/metadata.name is what namespaceSelector matches on

Source: kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/

© 2026 Tertiary Infotech Academy Pte Ltd

Afternoon Break

3:50 – 4:00 PM

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTERED TOPIC 4

04

Cluster Security

Authentication, authorization, admission, Secrets and NetworkPolicy — the other half of 'can this request succeed?'

© 2026 Tertiary Infotech Academy Pte Ltd

THE 4C'S

Four nested layers of cloud native security

Each layer can only be as strong as the one outside it. Hardening Code cannot compensate for an open Cluster.

CLOUD / INFRASTRUCTURE

- **Provider and network** — VPC design, firewalls, private API endpoints, node placement
- **Identity and access** — Who may create clusters, read state and reach the control plane
- **Physical and tenancy** — Data residency, isolation and the provider's own attestations

CLUSTER

- **API access** — Authentication, RBAC and admission control on every request
- **Component config** — etcd encryption, kubelet authorization, audit logging, certificate rotation
- **Workload guardrails** — Pod Security Standards, NetworkPolicy, quotas

CONTAINER AND CODE

- **Image provenance** — Signed images, SBOMs, scanned bases, pinned digests
- **Runtime posture** — Non-root, read-only rootfs, dropped capabilities, seccomp
- **Application code** — TLS, input validation, dependency hygiene, no secrets in source

MECHANISM

Kubernetes controls the Cluster layer and gives you tools for Container. Cloud and Code remain your responsibility and are where most real breaches start.

Source: kubernetes.io/docs/concepts/security/overview/ (the 4C's)

© 2026 Tertiary Infotech Academy Pte Ltd

DEFAULTS

What a stock cluster does and does not protect

Two things are secure by default. Four are explicitly your job — and all four appear in today's labs.

Protected: the API server

1

Every request is authenticated and authorized. Anonymous access is limited and unprivileged.

Protected: etcd on the wire

2

Control-plane components use mutual TLS between each other by default in managed and kubeadm clusters.

NOT protected: Pod-to-Pod traffic

3

Flat, unauthenticated and unencrypted unless you add NetworkPolicy or a mesh.

NOT protected: Secret contents at rest

4

Base64 encoding only, unless encryption at rest is explicitly configured on the API server.

NOT protected: what a container may do

5

Without Pod Security admission or policy, containers can ask for root, hostPath and privileged.

NOT protected: image provenance

6

Any image the node can pull will run. Signature verification is admission-time work you must add.

Source: kubernetes.io/docs/concepts/security/overview/ (the 4C's)

© 2026 Tertiary Infotech Academy Pte Ltd

REQUEST PIPELINE

What every API request passes through

Four gates, always in this order. Knowing which gate rejected you tells you which configuration to fix.

1 · AUTHENTICATION	**Who is this?** Client certificate, bearer token or OIDC ID token — first module to succeed wins	**Result** A username and a set of groups. Kubernetes stores no user object anywhere	**Failure** 401 Unauthorized. The request never reaches authorization
2 · AUTHORIZATION	**May they do this?** Node, RBAC, ABAC or Webhook modules, consulted in configured order	**Decision inputs** user, groups, verb, resource, subresource, namespace, name	**Failure** 403 Forbidden, with a message naming every one of those fields
3 · ADMISSION → 4 · PERSIST	**Mutating admission** Webhooks and controllers may CHANGE the object — defaults, injection	**Validating admission** Schema validation, then validating webhooks and CEL policies — accept or reject	**etcd write** Only now is the object persisted and made visible to watchers

MECHANISM

Mutation happens before validation, deliberately — so a webhook that injects a sidecar still has its result checked by every validating policy.

Source: kubernetes.io/docs/reference/access-authn-authz/authentication/ · kubernetes.io/docs/reference/access-authn-authz/authorization/ · kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

© 2026 Tertiary Infotech Academy Pte Ltd

THE FACT THAT SURPRISES EVERYONE

Kubernetes has no User object. There is nothing to create, list or delete.

Human identity is asserted by an external credential — a certificate, an OIDC token, a proxy header. The cluster only ever derives a username and groups from it.

© 2026 Tertiary Infotech Academy Pte Ltd

AUTHENTICATION

How the API server learns who you are

Several modules, tried in turn. The first to accept the credential decides the identity; none of them stores it.

1 X.509 client certificates

The certificate's CN becomes the username and each O becomes a group. Signed by a CA the API server trusts.

2 ServiceAccount bearer tokens

Signed JWTs for in-cluster workloads. Modern tokens are short-lived, audience-bound and Pod-bound.

3 OpenID Connect

An external identity provider issues an ID token; the API server validates it and maps claims to user and groups.

4 Webhook token authentication

The API server calls an external service to validate an opaque token and receive the identity back.

5 Authenticating proxy

A trusted front proxy asserts identity in headers. Only usable behind a properly secured proxy.

6 Anonymous requests

Unmatched requests may be treated as system:anonymous in group system:unauthenticated — and then authorized normally.

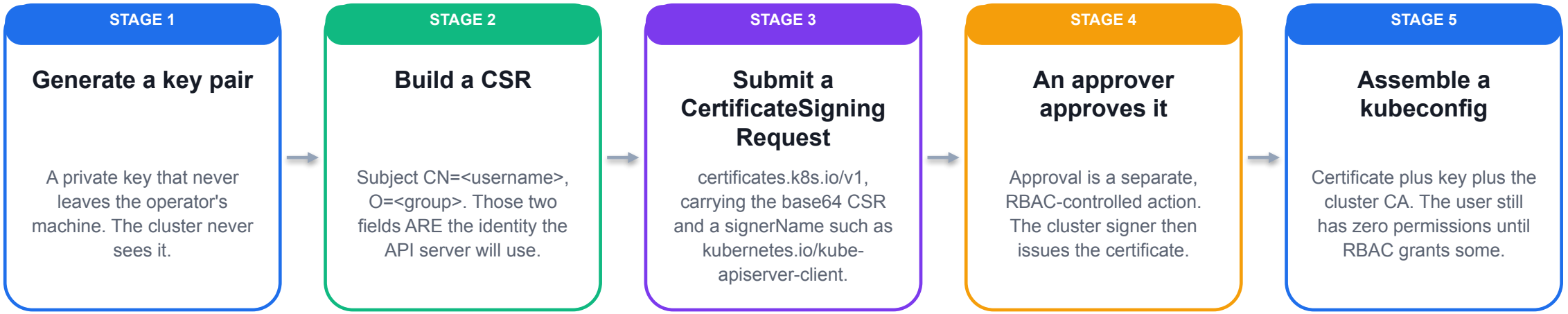
Source: kubernetes.io/docs/reference/access-authn-authz/authentication/

© 2026 Tertiary Infotech Academy Pte Ltd

CLIENT CERTIFICATES

Creating a TLS certificate for authentication

The registered outline's 'creating a TLS certificate for authentication' — mechanism, not keystrokes.



WHAT IT MEANS

A signed certificate proves identity and nothing else. Note the operational weakness: certificates cannot be revoked by Kubernetes, only expired.

Source: kubernetes.io/docs/reference/access-authn-authz/certificate-signing-requests/

© 2026 Tertiary Infotech Academy Pte Ltd

SERVICEACCOUNTS

Overview, and the points to note

A ServiceAccount is an identity for in-cluster workloads — and an identity is not an authorisation.

1 It is a real namespaced object

Unlike users, ServiceAccounts exist in the API. Their username is system:serviceaccount:<ns>:<name>.

2 Every namespace gets one named default

Pods that name no ServiceAccount use it. It is created automatically and holds no permissions.

3 Zero permissions until bound

Creating a ServiceAccount grants nothing at all. A RoleBinding or ClusterRoleBinding is what grants.

4 Tokens are projected, not stored

kubelet requests a short-lived, audience-bound token and projects it into the Pod; it is rotated automatically.

5 automountServiceAccountToken

Set false on the Pod or the ServiceAccount when the workload never calls the API. Free attack-surface reduction.

6 Group membership is automatic

Every ServiceAccount is in system:serviceaccounts and system:serviceaccounts:<namespace>.

Source: kubernetes.io/docs/concepts/security/service-accounts/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The projected ServiceAccount token volume

Authenticating with tokens, seen from the workload's side — a registered outline item.

The volume kubelet adds when a token is automounted

```
spec:
  serviceAccountName: ops-reader
  automountServiceAccountToken: true
  volumes:
    - name: kube-api-access
      projected:
        sources:
          - serviceAccountToken:
              path: token
              expirationSeconds: 3607
              audience: ""     # API server default
          - configMap:
              name: kube-root-ca.crt
              items: [{key: ca.crt, path: ca.crt}]
          - downwardAPI:
              items:
                - path: namespace
                  fieldRef:
                    fieldPath: metadata.namespace
```

READING IT

1. **Three sources, one directory**
   Token, cluster CA bundle and namespace are projected together into /var/run/secrets/kubernetes.io/serviceaccount.

2. **expirationSeconds drives rotation**
   kubelet refreshes the file before expiry. A client that caches the token at start-up eventually fails.

3. **audience binds the token**
   An empty audience means the API server. A named audience produces a token only that recipient should accept.

4. **The token is bound to the Pod**
   It carries the Pod's identity and stops being valid when the Pod is gone — unlike legacy Secret-based tokens.

5. **kubelet writes this implicitly**
   You rarely author it. Seeing it explicitly is how you understand what automounting actually does.

Source: kubernetes.io/docs/concepts/security/service-accounts/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

What a Pod's identity looks like from inside

The identity is a file. Protecting the file is protecting the identity.

Lab 14 — token-inspector Pod running as ops-reader

```
$ kubectl exec -n kcna-lab14 token-inspector -- \
    ls -1 /var/run/secrets/kubernetes.io/serviceaccount/
ca.crt
namespace
token

$ kubectl exec -n kcna-lab14 token-inspector -- \
    cat /var/run/secrets/kubernetes.io/serviceaccount/namespace
kcna-lab14

# Decoded JWT payload (claims, abridged):
  "aud": ["https://kubernetes.default.svc.cluster.local"],
  "exp": 1789200000,
  "kubernetes.io": {
     "namespace": "kcna-lab14",
     "pod": {"name": "token-inspector"},
     "serviceaccount": {"name": "ops-reader"}},
  "sub": "system:serviceaccount:kcna-lab14:ops-reader"
```

WHAT TO LOOK AT

1. **Three files, no more**
   CA bundle to trust the API server, namespace for convenience, and the bearer token itself.

2. **sub is the RBAC subject**
   system:serviceaccount:<ns>:<name> is exactly the string you put in a RoleBinding and in --as=.

3. **aud restricts the audience**
   A token minted for the API server should not be accepted by anything else.

4. **exp makes it short-lived**
   Modern projected tokens expire and are rotated in place by kubelet.

5. **A JWT is not encrypted**
   Anyone who can read the file can read the claims. Confidentiality comes from filesystem and RBAC controls.

Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md · kubernetes.io/docs/concepts/security/service-accounts/

© 2026 Tertiary Infotech Academy Pte Ltd

SCENARIOS

Practical ServiceAccount scenarios

Three recurring situations. Each is a registered-outline 'practical scenario' for ServiceAccounts.

In-cluster controller

- An operator watching CRDs needs list/watch on them
- Give it a dedicated ServiceAccount, never default
- Bind the narrowest Role that its watch actually needs
- Token is projected, rotated and audience-bound for free

least privilege

CI/CD deployer

- A pipeline applies manifests into one namespace
- One ServiceAccount per pipeline, per namespace
- RoleBinding scoped to that namespace only
- Rotate by re-requesting a bound token, not by long-lived Secrets

scoped grant

Workload with no API access

- Most application Pods never call the API server at all
- Set automountServiceAccountToken: false
- A compromised container then has no cluster credential
- Costs nothing and removes a whole escalation path

reduce surface

Source: kubernetes.io/docs/concepts/security/service-accounts/

© 2026 Tertiary Infotech Academy Pte Ltd

AUTHORIZATION

The modules that answer 'may they do this?'

Configured with --authorization-mode on the API server. Most clusters run Node,RBAC.

1 RBAC

Role-based access control through API objects. The mode you will configure, audit and be examined on.

2 Node

A special-purpose authorizer restricting each kubelet to the objects its own Pods actually need.

3 Webhook

Delegates the decision to an external service. Used by cloud providers to bridge their own IAM.

4 ABAC

Attribute-based policy from a file on the control-plane node. Legacy; requires an API server restart to change.

5 AlwaysAllow / AlwaysDeny

Test-only modes. AlwaysAllow makes every permission check meaningless — verify it is not enabled.

6 Order matters

Modules are consulted in the configured order; the first module to allow or deny decides the request.

Source: kubernetes.io/docs/reference/access-authn-authz/authorization/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Role and the RoleBinding that activates it

Two objects, five fields that matter. Everything else in RBAC is a variation on this pair.

20-role-pod-reader.yaml + its RoleBinding (Lab 14)

```
kind: Role                      # namespaced
metadata: {name: pod-reader, namespace: kcna-lab14}
rules:
  - apiGroups: [""]             # "" = core group
    resources: ["pods"]
    verbs: ["get", "list", "watch"]
  - apiGroups: [""]
    resources: ["pods/log"]     # SUBRESOURCE, separate
    verbs: ["get"]
---
kind: RoleBinding
metadata: {name: ops-reader-pods, namespace: kcna-lab14}
subjects:
  - kind: ServiceAccount
    name: ops-reader
    namespace: kcna-lab14
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

READING IT

1. **A rule is apiGroups × resources × verbs**
   All three are lists, and the rule is their cross product. "" is the core group, not a wildcard.
2. **Subresources are separate resources**
   pods/log must be listed explicitly. Granting pods does not grant pods/log, exec or portforward.
3. **The Role alone grants nothing**
   Rules are inert until a binding attaches them to a subject. Two objects, always.
4. **roleRef is immutable**
   To repoint a binding you delete and recreate it. This is deliberate — it prevents silent privilege changes.
5. **A dangling roleRef fails silently**
   Binding to a Role that does not exist is accepted and grants nothing. No error is ever printed.

Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md · kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

ClusterRole, and the two ways to bind it

ClusterRole and ClusterRoleBindings are a named registered-outline item. The binding decides the scope.

30-clusterrole-configmap-viewer.yaml (Lab 14)

```
kind: ClusterRole               # cluster-scoped rules
metadata: {name: kcna-lab14-configmap-viewer}
rules:
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "watch"]
---
# BINDING A — RoleBinding: grants ONLY in this namespace
kind: RoleBinding
metadata: {namespace: kcna-lab14, name: cm-view}
roleRef: {kind: ClusterRole,
          name: kcna-lab14-configmap-viewer}
---
# BINDING B — ClusterRoleBinding: grants CLUSTER-WIDE
kind: ClusterRoleBinding
metadata: {name: cm-view-everywhere}
roleRef: {kind: ClusterRole,
          name: kcna-lab14-configmap-viewer}
```

READING IT

1. **One rule set, two blast radii**
   The identical ClusterRole grants one namespace under A and every namespace under B. Read the BINDING, not the role.
2. **ClusterRole + RoleBinding is the idiom**
   Write reusable rules once, grant them namespace by namespace. This is the pattern to prefer.
3. **ClusterRoleBinding is the sharp edge**
   It is the only construct that grants cluster-wide. Treat every one as a security review item.
4. **Cluster-scoped resources need B**
   Nodes, PersistentVolumes and StorageClasses live outside namespaces, so only a ClusterRoleBinding can grant them.
5. **It survives namespace deletion**
   A ClusterRole is cluster-scoped. Deleting kcna-lab14 leaves it behind — delete it by name.

Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md · kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

THE MATRIX

Four combinations, and exactly what each grants

Read the binding first. The role supplies the verbs; the binding supplies the scope.

Role kind	Binding kind	Where it grants	Typical use
Role (namespaced)	RoleBinding (same namespace)	That one namespace only	A grant specific to one team's namespace
ClusterRole	RoleBinding	That one namespace only — the binding wins	Reusable rule sets applied namespace by namespace
ClusterRole	ClusterRoleBinding	Every namespace, plus cluster-scoped resources	Platform administrators and cluster-wide controllers
Role (namespaced)	ClusterRoleBinding	Not permitted — a ClusterRoleBinding cannot reference a Role	Rejected; the API enforces this
Any	Binding to a non-existent role	Nothing — accepted and silently useless	The dangling roleRef bug, seen in Lab 14

CHOOSE

ClusterRole + RoleBinding by default; reserve ClusterRoleBinding for genuinely cluster-wide need.

Source: kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

RBAC fields and verbs

Ten reference facts. Rows 1, 5 and 9 cause the most real-world surprises.

Read verbs	get, list, watch — list leaks whole objects, so treat it as read-all
Write verbs	create, update, patch, delete, deletecollection
apiGroups	"" is the core group; apps, batch, rbac.authorization.k8s.io and so on
resources	Plural, lowercase, as shown by kubectl api-resources
Subresources	pods/log, pods/exec, pods/portforward, deployments/scale — granted separately

resourceNames	Restrict a rule to named objects; does not work with list or watch
nonResourceURLs	Grants paths such as /healthz; only valid in a ClusterRole
Aggregation	aggregationRule composes a ClusterRole from labelled ClusterRoles
Additive only	There is no deny rule anywhere in RBAC; permissions are a union
Escalation guard	You cannot grant permissions you do not hold, absent the escalate verb

Source: kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Proving a grant instead of asserting it

RBAC questions have a definitive answer available on demand. Never argue about a grant; ask.

Lab 14 — replaying data/rbac-matrix.csv

```
# Impersonate the ServiceAccount and ask the API server
$ kubectl auth can-i list pods \
    --as=system:serviceaccount:kcna-lab14:ops-reader \
    -n kcna-lab14
yes

$ kubectl auth can-i list pods \
    --as=system:serviceaccount:kcna-lab14:ops-reader \
    -n default
no

$ kubectl auth can-i list configmaps \
    --as=system:serviceaccount:kcna-lab14:ops-reader \
    -n default
no

$ kubectl auth can-i --list -n kcna-lab14 \
    --as=system:serviceaccount:kcna-lab14:ops-reader
```

WHAT TO LOOK AT

1. **The subject string is exact**
   system:serviceaccount:<ns>:<name>. Any typo silently impersonates a different, unbound identity.

2. **The last two lines are the lesson**
   configmaps came from a ClusterRole — and still returns no outside the RoleBinding's namespace.

3. **Exit status is machine-readable**
   yes exits 0, no exits 1, so a permission matrix can be replayed as an automated test.

4. **--list enumerates everything**
   It prints every rule that applies to that subject in that namespace — the fastest audit you have.

5. **Impersonation needs privilege**
   --as requires the impersonate verb. On a shared cluster, test from inside a Pod instead.

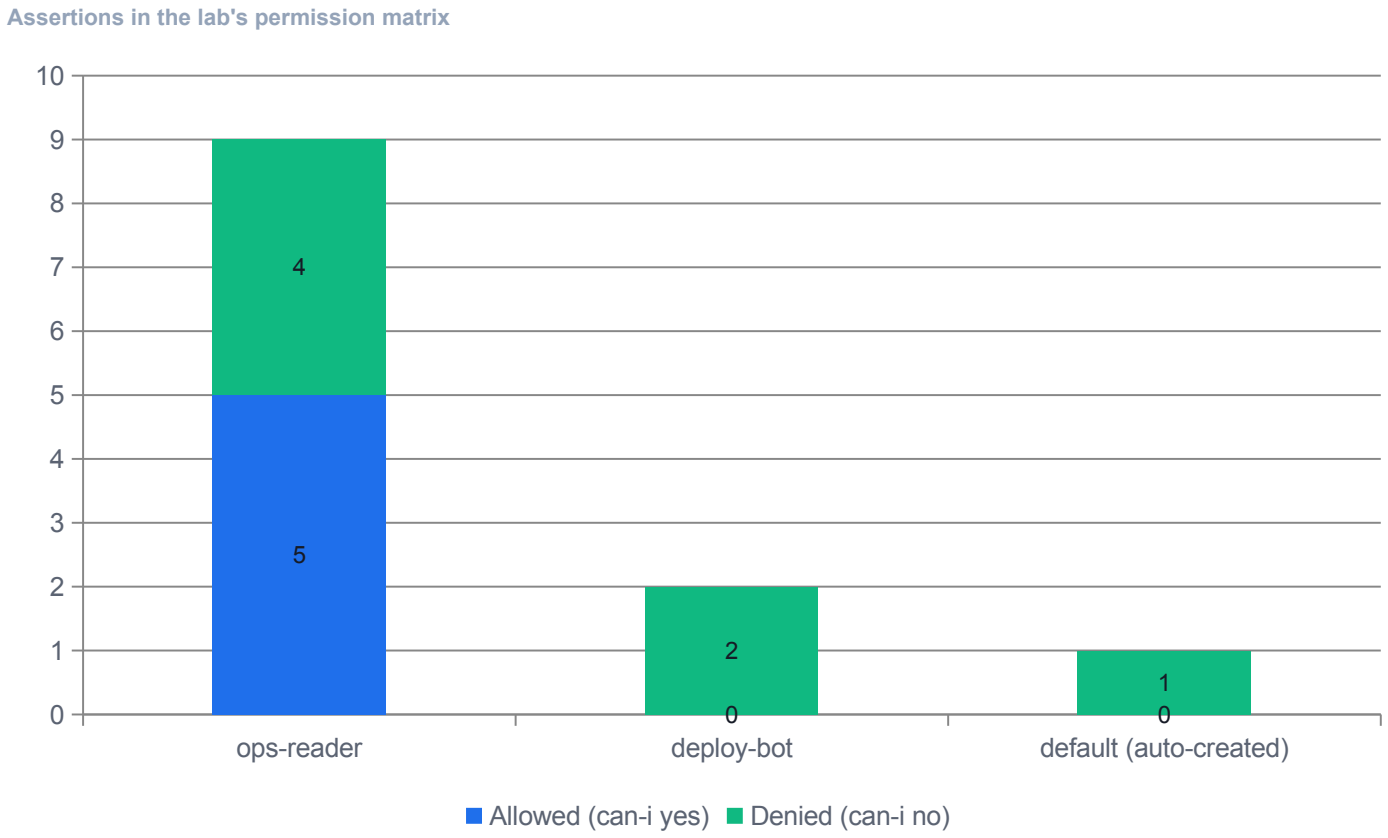
Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

LAB EVIDENCE

The Lab 14 permission matrix, counted

Twelve verb/resource/namespace assertions from data/rbac-matrix.csv, each replayed against the live API server.



WHAT THE DATA SHOWS

Five allows, seven denies

The denies are the valuable half: they are what proves the grant is actually bounded.

ops-reader: 5 of 9 allowed

get/list/watch on pods, get on pods/log, list on configmaps. Everything else is refused.

deploy-bot: 0 of 2

The ServiceAccount exists and has no binding at all. Existence is not permission.

default: 0 of 1

The auto-created default ServiceAccount is unprivileged — which is why Pods should not silently rely on it.

Counts, not estimates

These are row counts from the shipped CSV, replayed by the lab's verifier. Nothing here is modelled.

Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md · courseware/labs/lab-14-namespaces-serviceaccounts-rbac/data/rbac-matrix.csv

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Reading a Forbidden message as a specification

Subject, verb, resource, apiGroup, namespace. Read them in that order and the fix writes itself.

SYMPTOM

An in-cluster client is refused. The error names every field you need to write the fix.

EVIDENCE — actual cluster output

```
$ kubectl get secrets -n kcna-lab14 \
    --as=system:serviceaccount:kcna-lab14:ops-reader
Error from server (Forbidden): secrets is forbidden:
  User "system:serviceaccount:kcna-lab14:ops-reader"
  cannot list resource "secrets" in API group ""
  in the namespace "kcna-lab14"

# The five fields map 1:1 onto an RBAC rule:
#   subject   -> RoleBinding.subjects[]
#   verb list -> rules[].verbs
#   secrets   -> rules[].resources
#   group ""  -> rules[].apiGroups
#   namespace -> the Role/RoleBinding's namespace
```

ROOT CAUSE

- No bound rule grants list on secrets to that subject in that namespace
- RBAC is additive and has no deny rule, so this is an absence of a grant, never an explicit refusal
- 403 means authentication SUCCEEDED — the identity was accepted and then found insufficient

FIX

- Decide first whether the grant SHOULD exist; list on secrets reads every secret value in scope
- If it should, add the resource to an existing Role rather than creating a broader one
- Re-verify with kubectl auth can-i, not by re-running the original command by hand

Source: courseware/labs/lab-14-namespaces-serviceaccounts-rbac/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

DECISION

Role or ClusterRole — and which binding?

The question is never 'which object is easier'. It is 'what is the smallest blast radius that works'.



YES — genuinely cluster-wide

- Nodes, PersistentVolumes, StorageClasses, IngressClasses → ClusterRole + ClusterRoleBinding
- A controller that must watch every namespace → the same pair, with the narrowest possible verbs
- Record it: every ClusterRoleBinding deserves a review
- Prefer aggregating existing ClusterRoles over new wide ones

NO — one or a few namespaces

- Rules used in one namespace only → Role + RoleBinding
- The same rules reused in many namespaces → ClusterRole + a RoleBinding per namespace
- Never reach for ClusterRoleBinding to 'make it work'
- Verify the boundary with can-i in a namespace you did NOT intend to grant

Source: kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROL POINTS

Three places a request can be stopped

These are independent layers. A gap in one is not covered by strength in another.

NETWORK LAYER (L3/L4)	**NetworkPolicy** Selector-based allow rules on Pod ports; enforced by the CNI plugin	**kube-proxy** Not a control — it rewrites destinations, it does not authorise them	**ipBlock / cloud firewall** CIDR-level control for traffic entering or leaving the cluster
APPLICATION LAYER (L7)	**Ingress / Gateway** Host and path routing; TLS termination at the edge	**Service mesh policy** Identity-based rules — service A may call POST /pay on service B	**The app itself** Its own authentication and authorization remain necessary
CONTROL PLANE (API)	**Authentication** Certificate, token or OIDC identity for every API request	**RBAC** Verb and resource authorization, scoped by the binding	**Admission** Pod Security, quotas and policy engines, immediately before the write

MECHANISM

RBAC controls what may be ASKED OF THE API. NetworkPolicy controls what may be SENT BETWEEN PODS. Neither implies the other, and confusing them is a common audit finding.

Source: kubernetes.io/docs/concepts/security/overview/ (the 4C's) · kubernetes.io/docs/concepts/services-networking/network-policies/ · kubernetes.io/docs/reference/access-authn-authz/rbac/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 14 — Namespaces, ServiceAccounts and RBAC

GOAL

Grant Meridian Freight's night-shift operators read-only access to one namespace and nothing else, then prove every allow and every deny with kubectl auth can-i instead of asserting it.

YOU WILL PRODUCE

- A namespaced Role/RoleBinding giving ops-reader get, list, watch on pods plus the separate pods/log subresource
- A read-only ClusterRole bound by a RoleBinding - reusable rules, namespaced grant, and zero ClusterRoleBindings
- A 12-row permission matrix in data/rbac-matrix.csv replayed through kubectl auth can-i --as=system:serviceaccount:kcna-lab14:<sa>

CONCEPTS EXERCISED

- A Namespace scopes names, bindings and quotas - it is not a network, node or privilege boundary
- A ServiceAccount is an identity with zero permissions until a binding grants them
- ClusterRole + RoleBinding grants only inside that namespace; only a ClusterRoleBinding is cluster-wide
- RBAC is additive with no deny rule, and roleRef is immutable while a dangling roleRef fails silently

LAB	NAMESPACE	MAPS TO	TIME
Lab 14	kcna-lab14	LO4 · A4 · K6	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-14-namespaces-serviceaccounts-rbac/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

CONFIGURATION

ConfigMap and Secret — what actually differs

The difference is real but smaller than most people assume — and it is not encryption.

1 Both are key/value API objects

Both are namespaced, both cap at roughly 1 MiB, both mount as volumes or inject as environment variables.

2 Encoding differs

ConfigMap holds plain strings in data. Secret holds base64 in data, or plain strings in stringData on write.

3 Handling differs

kubectl describe masks Secret values; kubelet keeps mounted Secret volumes in memory rather than on node disk.

4 RBAC treats them the same

Both are just resources. get on secrets is what protects them — not the object kind.

5 Secret has meaningful types

kubernetes.io/tls, dockerconfigjson and others carry required keys the API server validates.

6 Neither is encrypted by default

Encryption at rest is an API server configuration. Without it, both sit in etcd in plaintext.

Source: kubernetes.io/docs/concepts/configuration/secret/

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Secret, and the two ways to write its values

Everything in this object is recoverable by anyone who can read it. That is the design, not a defect.

A Secret manifest, with DUMMY values only

```
apiVersion: v1
kind: Secret
metadata:
  name: booking-credentials
  namespace: kcna-lab15
type: Opaque
data:                       # values MUST be base64
  DB_USERNAME: RFVNTVktbm90LWEtcmVhbC11c2VybmFtZQ==
  DB_PASSWORD: RFVNTVktbm90LWEtcmVhbC1wYXNzd29yZA==
stringData:                 # write-only convenience
  DB_HOST: booking-db.kcna-lab15.svc.cluster.local
immutable: true             # optional, one-way
# base64 -d on either value returns the plaintext.
# There is no key, no passphrase and no cipher here.
```

READING IT

1. **data is base64, always**
   The API server rejects data values that are not valid base64. That is an encoding rule, not a security control.
2. **stringData is write-only sugar**
   You supply plaintext; the API server encodes it and merges it into data. Read the object back and stringData is gone.
3. **type carries required keys**
   Opaque means no constraints. kubernetes.io/tls demands tls.crt and tls.key, and is validated.
4. **immutable: true is one-way**
   It blocks updates, reduces kubelet watch load, and can only be undone by deleting the object.
5. **Committing this file is the risk**
   Anyone with repository access has the credential. This is why the lab generates its Secret rather than shipping it.

Source: kubernetes.io/docs/concepts/configuration/secret/ · courseware/labs/lab-15-secrets-configmaps/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

The one-line disproof of 'Kubernetes encrypts Secrets'

Learners should perform this themselves once. It permanently changes how they treat Secrets.

Lab 15 — demonstrated, not asserted

```
$ kubectl get secret booking-credentials -n kcna-lab15 \
    -o jsonpath='{.data.BOOKING_DB_PASSWORD}' | base64 -d
DUMMY-not-a-real-password-change-me

# describe only LOOKS protective:
$ kubectl describe secret booking-credentials -n kcna-lab15
Data
====
BOOKING_DB_PASSWORD:  35 bytes

# And on this cluster, at rest:
$ kubectl get pod -n kube-system -l component=kube-apiserver \
    -o jsonpath='{...spec.containers[0].command}' \
    | grep -i encryption-provider \
    || echo "NOT SET -> plaintext in etcd"
NOT SET -> plaintext in etcd
```

WHAT TO LOOK AT

1. **No key was required**
   No password, no passphrase, no privileged operation. Only the RBAC verb get on secrets.
2. **describe masks, it does not protect**
   Printing a byte count is a display convenience. The value is one jsonpath away.
3. **base64 is an encoding**
   It exists so binary values survive YAML and JSON. It has never been a confidentiality mechanism.
4. **Check your own cluster**
   No --encryption-provider-config means Secret values sit in etcd in plaintext.
5. **The real control is RBAC**
   Who holds get and list on secrets is the question that actually protects them.

Source: courseware/labs/lab-15-secrets-configmaps/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

MITIGATIONS

What actually protects a Secret

Six controls. Note that not one of them is 'because it is a Secret object rather than a ConfigMap'.

1 RBAC first

Restrict get, list and watch on secrets. list is the dangerous one — it returns every value in scope.

2 Encryption at rest

EncryptionConfiguration on the API server, ideally with a KMS provider so the key lives outside the cluster.

3 External secret stores

Vault, cloud secret managers or the Secrets Store CSI driver keep the material outside etcd entirely.

4 Never commit Secrets to git

If you must, use sealed/encrypted forms so the repository never holds recoverable plaintext.

5 Prefer volumes over environment variables

Env values are frozen at start, visible in /proc/<pid>/environ and often captured by crash dumps and logs.

6 Reduce who has a token at all

automountServiceAccountToken: false removes an entire class of credential-theft path.

Source: kubernetes.io/docs/concepts/configuration/secret/ · kubernetes.io/docs/tasks/administer-cluster/encrypt-data/

© 2026 Tertiary Infotech Academy Pte Ltd

INJECTION

Mounting Secrets inside Pods — three routes, three behaviours

Mounting Secrets inside Pods is a registered outline item. The route you pick decides whether rotation works.

Route	How it behaves	When to use it
env / envFrom	Frozen at container start; visible in /proc/<pid>/environ and in any process listing that dumps the environment	Legacy apps that only read environment variables
Volume (directory mount)	One file per key; kubelet refreshes the files after a change, within about one sync period	The default choice — supports rotation without a restart
Volume with subPath	A point-in-time copy of one file; it NEVER updates for the life of the container	Only when a single file must land beside unrelated content
Projected volume	Combines Secret, ConfigMap, downwardAPI and token sources into one directory	Service account tokens, and bundling related material
imagePullSecrets	Consumed by kubelet to authenticate to a registry; never exposed to the container	Private image registries

CHOOSE

Volume mount by default; env only for applications that leave you no choice; never subPath for anything that rotates.

Source: kubernetes.io/docs/concepts/configuration/secret/ · courseware/labs/lab-15-secrets-configmaps/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

ENCRYPTION AT REST

How Secret values stop being plaintext in etcd

A control-plane configuration, not an object you can create with kubectl.

STAGE 1

Write an EncryptionConfiguration

A file listing resources (typically secrets) and an ordered list of providers.

STAGE 2

Point the API server at it

--encryption-provider-config. The first provider in the list is the one used to ENCRYPT.

STAGE 3

New writes are encrypted

Every create or update of a Secret is encrypted on its way into etcd.

STAGE 4

Existing objects are not

They stay as they were until rewritten — commonly forced with a read-and-replace of every Secret.

STAGE 5

Decryption tries each provider

All listed providers are available for reading, which is what makes key rotation possible.

WHAT IT MEANS

With a KMS provider the key material lives outside the cluster, so an etcd backup on its own is no longer enough to recover Secret values.

Source: kubernetes.io/docs/tasks/administer-cluster/encrypt-data/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A key that does not exist — CreateContainerConfigError

Configuration references are resolved late. That is why this class of bug survives every static check.

SYMPTOM

The Pod is admitted, scheduled and then never starts. Neither apply nor any static validator saw the problem.

EVIDENCE — actual cluster output

```
$ kubectl apply -f manifests/50-pod-broken-key.yaml
pod/broken-key-consumer created

$ kubectl get pod broken-key-consumer -n kcna-lab15
NAME                 READY   STATUS
broken-key-consumer  0/1     CreateContainerConfigError

$ kubectl describe pod broken-key-consumer -n kcna-lab15
Warning  Failed  10s (x3 over 24s)  kubelet
  Error: couldn't find key APP_TIER in ConfigMap
  kcna-lab15/booking-app-config
```

✓ **kubectl get pod shows Running 1/1 and the event no longer recurs**

Source: courseware/labs/lab-15-secrets-configmaps/verification/expected-output.md

ROOT CAUSE

- configMapKeyRef names a key that is not in the object
- The reference is resolved by kubelet at container-creation time — long after admission has already accepted the Pod
- The same failure occurs for a missing secretKeyRef key, and for a missing ConfigMap or Secret entirely

FIX

- Correct the key name, or set optional: true if absence is genuinely acceptable
- Read the event text — it names the exact key and the exact object
- Distinguish this from CreateContainerError (runtime refused) and ImagePullBackOff (registry problem)

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Secret types and limits

Ten reference rows. The type field is not cosmetic — it is validated and consumed by other controllers.

Opaque	The default. Arbitrary keys, no validation
kubernetes.io/tls	Requires exactly tls.crt and tls.key — used by Ingress
kubernetes.io/dockerconfigjson	Registry credentials for imagePullSecrets
kubernetes.io/basic-auth	username and password keys
kubernetes.io/ssh-auth	ssh-privatekey key

kubernetes.io/service-account-token	Legacy long-lived token; projected tokens are preferred
bootstrap.kubernetes.io/token	Node bootstrap tokens in kube-system
Size limit	Roughly 1 MiB per object, as for ConfigMap
immutable: true	Blocks updates and reduces kubelet watch load; irreversible
Node-scoped delivery	kubelet only receives Secrets used by Pods scheduled to that node

Source: kubernetes.io/docs/concepts/configuration/secret/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 15 — Secrets, ConfigMaps and Safe Injection

GOAL

Split the Meridian Freight booking-api configuration into ConfigMaps and a Secret, inject each by the safest route, and prove with your own hands that a Kubernetes Secret is base64-encoded rather than encrypted.

YOU WILL PRODUCE

- A booking-app-config ConfigMap built three ways - from-file, from-env-file and from-literal - kept byte-identical to data/app-settings.json and data/feature-flags.env
- A booking-credentials Secret generated from data/dummy-credentials.env and deliberately never committed as a manifest, decoded live with base64 -d
- Two consumer Pods contrasting env injection with volume injection, including a subPath mount that provably never updates

CONCEPTS EXERCISED

- A Secret is base64-encoded, not encrypted; RBAC, encryption at rest and external stores are the real controls
- env injection freezes values at container start and leaks through /proc/<pid>/environ; volume mounts update live
- A subPath mount is a point-in-time copy and never tracks ConfigMap changes, unlike the directory mount beside it
- A missing key is invisible to apply and to static validation - it surfaces as CreateContainerConfigError

LAB	NAMESPACE	MAPS TO	TIME
Lab 15	kcna-lab15	LO4 · A4 · K6	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-15-secrets-configmaps/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

THE DEFAULT

Before any policy exists, everything is open

The auditors' finding in Lab 16 is the normal state of every cluster that has never had a policy written.

1 Every Pod can reach every Pod

Across namespaces, across nodes, on any port the target listens on. This is the network model working as designed.

2 Services do not restrict anything

A ClusterIP is dialable from every Pod, and the backend Pod IPs are dialable directly as well.

3 Namespaces do not restrict anything

They scope names and RBAC. They have no effect on packets whatsoever.

4 NetworkPolicy is the L3/L4 control

networking.k8s.io/v1: select Pods, declare policy types, list allowed peers and ports.

5 The API is universal; enforcement is not

The API server accepts your policy on any cluster. Only an enforcing CNI plugin actually applies it.

6 So verify enforcement, never assume it

kind's default kindnet stores NetworkPolicy objects and enforces none of them.

Source: kubernetes.io/docs/concepts/services-networking/network-policies/ · courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Default-deny, and one selective allow

Two objects express 'nothing may talk, except this one path'. That is the whole pattern.

40-networkpolicy-default-deny.yaml + 60-allow-frontend

```
# A — deny everything in this namespace
kind: NetworkPolicy
metadata: {name: default-deny-all, namespace: kcna-lab16}
spec:
  podSelector: {}          # {} = ALL pods, not none
  policyTypes: [Ingress, Egress]
  # no ingress: and no egress: rules -> nothing allowed
---
# B — allow one source to one destination port
kind: NetworkPolicy
metadata: {name: allow-frontend-to-payments}
spec:
  podSelector:
    matchLabels: {app: payments-api}
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: {role: frontend}
      ports: [{protocol: TCP, port: 8080}]
```

READING IT

1. **podSelector: {} means ALL Pods**
   An empty selector is no constraint. It is the single most misread line in the whole API.
2. **policyTypes decides the directions**
   Listing Egress with no egress rules denies all egress — including DNS, which breaks name resolution.
3. **Selection implies isolation**
   A Pod becomes deny-by-default for a direction the moment ANY policy selects it for that direction.
4. **Allows are a union**
   Multiple policies selecting the same Pod add their allowed peers together. There is no deny rule to override them.
5. **The port is the POD port**
   8080 is the container's port. The Service's port 80 is irrelevant — kube-proxy already rewrote the destination.

Source: courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md · kubernetes.io/docs/concepts/services-networking/network-policies/

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

kubectl describe netpol, read literally

Five lines of describe output that are routinely read backwards.

Lab 16 — kcna-lab16

```
$ kubectl describe networkpolicy default-deny-all -n kcna-lab16
Spec:
  PodSelector:  <none> (Allowing the specific traffic to
                all pods in this namespace)
  Allowing ingress traffic:
    <none> (Selected pods are isolated for ingress)
  Allowing egress traffic:
    <none> (Selected pods are isolated for egress)
  Policy Types: Ingress, Egress

$ kubectl describe networkpolicy allow-frontend-to-payments
Spec:
  PodSelector:  app=payments-api
  Allowing ingress traffic:
    To Port: 8080/TCP
    From:
      PodSelector: role=frontend
  Not affecting egress traffic
  Policy Types: Ingress
```

WHAT TO LOOK AT

1. **PodSelector: <none> means ALL pods**
   The parenthesised text kubectl prints is the authoritative reading. It does not mean zero Pods.
2. **'Selected pods are isolated' is the deny**
   There is no deny keyword in the API. Isolation with no allow rules IS the denial.
3. **'Not affecting egress traffic' matters**
   The second policy lists only Ingress in policyTypes, so it makes no egress statement at all.
4. **To Port is the Pod port**
   8080, not the Service's 80. Policy is evaluated after kube-proxy's DNAT.
5. **describe shows intent, not enforcement**
   Everything above prints identically on a CNI that enforces nothing. Determine enforcement separately.

Source: courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

SEMANTICS

Six rules that decide what a policy actually does

Rules 4 and 5 differ by two characters of YAML indentation and change the security outcome completely.

1 No deny rule exists

The API can only express allows. Denial is what happens to traffic no allow rule covers.

2 Isolation is per direction

A Pod can be isolated for ingress and wide open for egress, depending on the policyTypes that select it.

3 Rules are ORed across policies

Two policies selecting one Pod produce the union of their allowed peers — never the intersection.

4 Peers in one list item are ANDed

namespaceSelector and podSelector inside ONE from[] entry must both match.

5 Split across items they are ORed

Two separate from[] entries mean either may match — which silently widens the policy.

6 Ports are Pod ports

Policy is applied below the Service abstraction, after the destination has already been rewritten.

Source: kubernetes.io/docs/concepts/services-networking/network-policies/ · courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

PEER TYPES

Selecting the other end of the connection

Five peer forms. Choosing the wrong one usually widens access rather than breaking it, so it fails silently.

Peer	Matches	Watch out for
podSelector	Pods in the POLICY'S OWN namespace carrying those labels	It never reaches other namespaces on its own
namespaceSelector	All Pods in namespaces carrying those labels	Alone, it allows EVERY Pod in the matched namespaces
namespaceSelector + podSelector (one item)	Pods with those labels, in namespaces with those labels	Indentation decides AND versus OR — verify with describe
ipBlock	A CIDR, with optional except ranges	For traffic outside the cluster; Pod IPs churn, so do not use it for in-cluster peers
No from / to at all	With the direction listed in policyTypes, nothing is allowed in that direction	This is how default-deny is written — deliberately empty

CHOOSE

Label namespaces deliberately — kubernetes.io/metadata.name is set for you on every namespace.

Source: kubernetes.io/docs/concepts/services-networking/network-policies/

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A policy that is accepted, valid and matches nothing

An empty-matching NetworkPolicy is more dangerous than no policy: it looks like a control.

SYMPTOM

Three selector typos in one object. It applies without a warning and protects precisely nothing.

EVIDENCE — actual cluster output

```
$ kubectl apply -f manifests/70-networkpolicy-broken-selector.yaml
networkpolicy.networking.k8s.io/allow-broken created

$ kubectl describe networkpolicy allow-broken -n kcna-lab16
Spec:
  PodSelector:  app=payments      <- pods are app=payments-api
  Allowing ingress traffic:
    To Port: 80/TCP               <- pod port is 8080
    From:
      PodSelector: role=frontends <- pods are role=frontend

$ kubectl get pods -n kcna-lab16 -l app=payments
No resources found in kcna-lab16 namespace.
```

ROOT CAUSE

- Selectors are live label queries; a query that matches nothing is perfectly valid
- The port names the Service's 80 instead of the Pod's 8080, so even a correct selector would not match the traffic
- Nothing in the API validates that a selector is USEFUL — only that it is well-formed

FIX

- Re-run every selector in the policy as a kubectl label query before trusting it
- Take ports from the container spec, never from the Service
- Then test the intended allow AND the intended deny with a real connection attempt

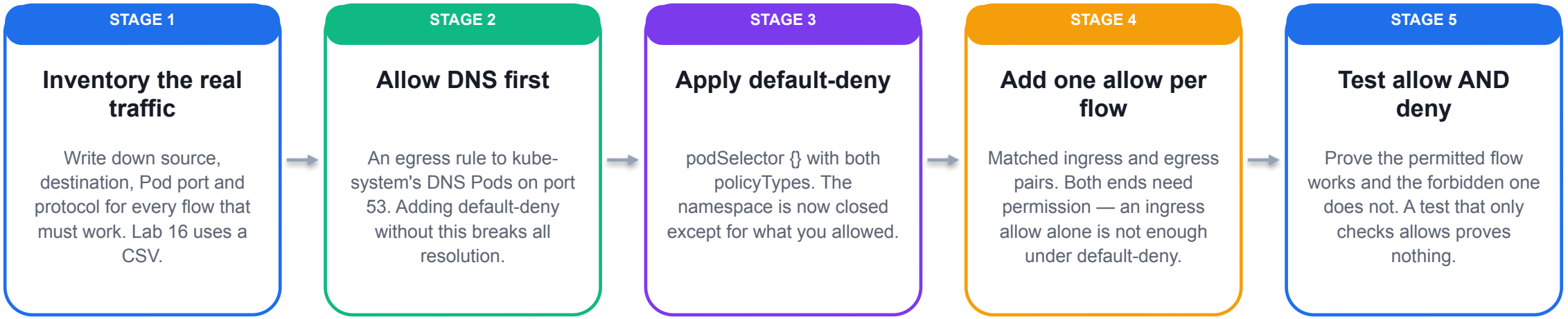
Source: courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

METHOD

Building a policy set that does not cause an outage

Mechanism, not a run-book: the order exists because two of these steps cause outages if reversed.



WHAT IT MEANS

Under default-deny both directions are evaluated: the caller needs an egress allow and the target needs an ingress allow. Forgetting one is the usual cause of a 'policy broke everything' incident.

Source: kubernetes.io/docs/concepts/services-networking/network-policies/ · courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

NetworkPolicy fields and honest caveats

Ten rows. The last four are the ones that keep a claim about your cluster honest.

apiVersion	networking.k8s.io/v1 — namespaced object
podSelector: {}	Selects ALL Pods in the namespace
policyTypes	[Ingress], [Egress] or both; decides which directions are isolated
Empty ingress list	Direction isolated, nothing allowed
Ports	Pod ports; port ranges via endPort where supported

Protocols	TCP, UDP and SCTP — SCTP support is plugin-dependent
Enforcement	Performed by the CNI plugin, not by Kubernetes itself
kindnet	Stores policies and enforces none — verified on this course's training cluster
Managed clusters	Many enforce, but some require an explicit add-on or option — check your provider
How to verify	Inventory the CNI DaemonSet, then run a real connection probe and report the result

Source: kubernetes.io/docs/concepts/services-networking/network-policies/ · courseware/labs/lab-16-networkpolicy-isolation/verification/expected-output.md · qa/PARENT-VERIFIED-FACTS.md

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 16 — NetworkPolicy and Default-Deny Isolation

GOAL

Close the auditors' finding that any Pod can reach Meridian Freight's payments-api by building default-deny plus one selective allow - and then determine honestly whether this cluster's CNI enforces any of it.

YOU WILL PRODUCE

- A default-deny-all NetworkPolicy (podSelector {}, Ingress and Egress) plus the DNS egress rule that default-deny otherwise breaks
- A matched ingress/egress pair admitting only role=frontend to app=payments-api on the POD port 8080, diffed against data/traffic-matrix.csv
- A recorded enforcement determination: the CNI DaemonSet inventory and a live probe whose result is reported rather than assumed

CONCEPTS EXERCISED

- kind's default CNI kindnet stores NetworkPolicies and enforces none - the API is universal, enforcement is the CNI's job
- There is no deny rule: a Pod becomes deny-by-default the moment any policy selects it, and allows are the union of all matching rules
- NetworkPolicy ports are Pod ports, evaluated below the Service abstraction after kube-proxy's DNAT
- namespaceSelector and podSelector in one peer item are ANDed; split across two items they are ORed

LAB	NAMESPACE	MAPS TO	TIME
Lab 16	kcna-lab16	LO4 · A4 · K6	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-16-networkpolicy-isolation/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

POD SECURITY STANDARDS

Three levels that replaced PodSecurityPolicy

Built-in, namespace-scoped and free. There is no reason for a production namespace to be unlabelled.

1 privileged

Entirely unrestricted. Intended for trusted, infrastructure workloads such as CNI and CSI DaemonSets.

2 baseline

Blocks known privilege escalations: no privileged containers, no hostPath, no host namespaces, no hostPort.

3 restricted

Baseline plus hardening: run as non-root, drop ALL capabilities, seccomp RuntimeDefault, no privilege escalation.

4 Applied by namespace label

pod-security.kubernetes.io/enforce, warn and audit. No CRD, no controller to install.

5 PodSecurityPolicy is gone

PSP was removed in Kubernetes v1.25. Pod Security admission is the built-in successor.

6 Policy engines go further

Kyverno or OPA Gatekeeper express rules PSS cannot — required labels, registry allow-lists, image digests.

Source: kubernetes.io/docs/concepts/security/pod-security-standards/

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Pod Security admission, in one panel

Three modes, two label keys and one very common blocker.

enforce	Rejects violating Pods outright at admission
audit	Records a violation annotation in the audit log; Pod is admitted
warn	Prints a warning to the client; Pod is admitted
Label form	pod-security.kubernetes.io/<mode>: <level>
Version pinning	pod-security.kubernetes.io/<mode>-version: v1.31 or latest

Scope	Evaluated per namespace, at Pod admission time
Existing Pods	Are not evicted when a label is added — only new Pods are checked
restricted essentials	runAsNonRoot, allowPrivilegeEscalation: false, capabilities drop ALL, seccomp RuntimeDefault
Common blocker	An image whose process insists on binding a port below 1024
Exemptions	Configurable by username, RuntimeClass or namespace in the admission configuration

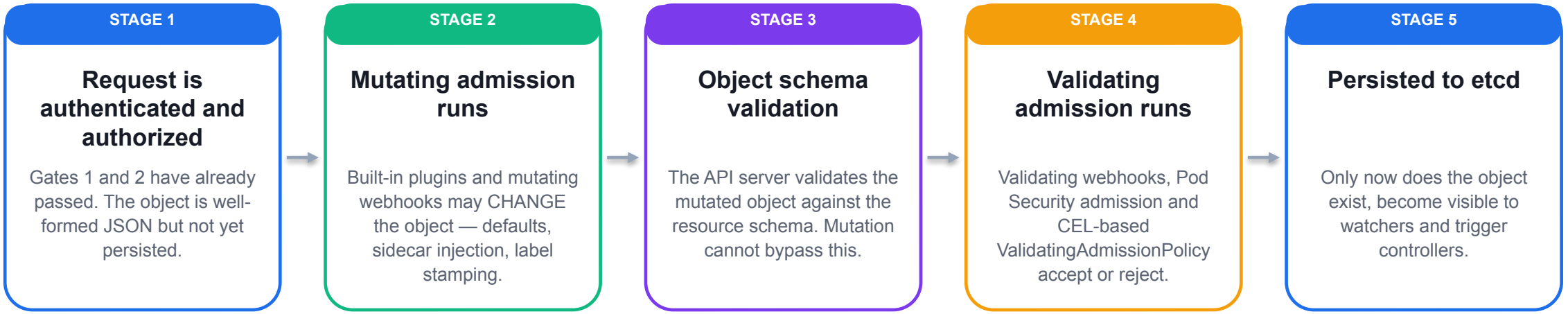
Source: kubernetes.io/docs/concepts/security/pod-security-standards/

© 2026 Tertiary Infotech Academy Pte Ltd

ADMISSION CONTROL

The last gate before an object exists

Mutate first, validate second — deliberately, so injected changes are still policed.



WHAT IT MEANS

ValidatingAdmissionPolicy expresses rules in CEL inside the API server, avoiding a webhook service on the critical path of every write.

Source: kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

© 2026 Tertiary Infotech Academy Pte Ltd

SUPPLY CHAIN

Trusting what actually runs

Kubernetes will run any image the node can pull. Every control here is something you add.

1 Pin by digest, not by tag

Tags are mutable pointers. image@sha256:… names exactly one set of bytes, forever.

2 Sign images

Sigstore/cosign signatures let admission verify the publisher before a Pod is allowed to run.

3 Publish an SBOM

SPDX or CycloneDX inventories what is inside the image, so a new CVE can be answered by query rather than rebuild.

4 Record provenance

in-toto attestations and SLSA levels describe how the artifact was built and by which pipeline.

5 Enforce at admission

Registry allow-lists, digest requirements and signature verification belong in an admission policy.

6 Scan continuously

An image that was clean at build time is not clean forever. Re-scan what is actually running.

Source: kubernetes.io/docs/concepts/security/supply-chain-security/ · slsa.dev

© 2026 Tertiary Infotech Academy Pte Ltd

POLICY ENGINES

Where a rule can live

Three tiers, increasing in power and in operational responsibility.

Built-in admission

- Pod Security admission for the three PSS levels
- ResourceQuota and LimitRange for consumption
- No installation, no extra failure domain
- Limited to what the built-in plugins express

no install

ValidatingAdmissionPolicy

- Rules written in CEL, evaluated inside the API server
- No webhook service on the write path
- Bound to resources by a policy binding object
- Good fit for straightforward structural constraints

in-process CEL

Webhook policy engines

- Kyverno and OPA Gatekeeper are the CNCF-project options
- Can mutate as well as validate, and generate objects
- Richest expressiveness — and an extra service to operate
- Mind failurePolicy: Fail can block writes during an outage

most expressive

Source: kubernetes.io/docs/reference/access-authn-authz/admission-controllers/ · kyverno.io · openpolicyagent.org/docs/gatekeeper

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

Projects behind today's concepts

Twelve CNCF projects that implement the mechanisms taught today. Recognising the names is KCNA-relevant.



CoreDNS

Cluster DNS



Cilium

eBPF CNI + policy



Envoy

Proxy data plane



Istio

Service mesh



Linkerd

Service mesh



Contour

Ingress / Gateway



Helm

Chart packaging



SPIFFE

Workload identity



OPA

Policy engine



Kyverno

Kubernetes policy



Falco

Runtime security



Notary

Content trust

Source: cncf.io/projects — project marks from courseware/assets/logos/MANIFEST.json

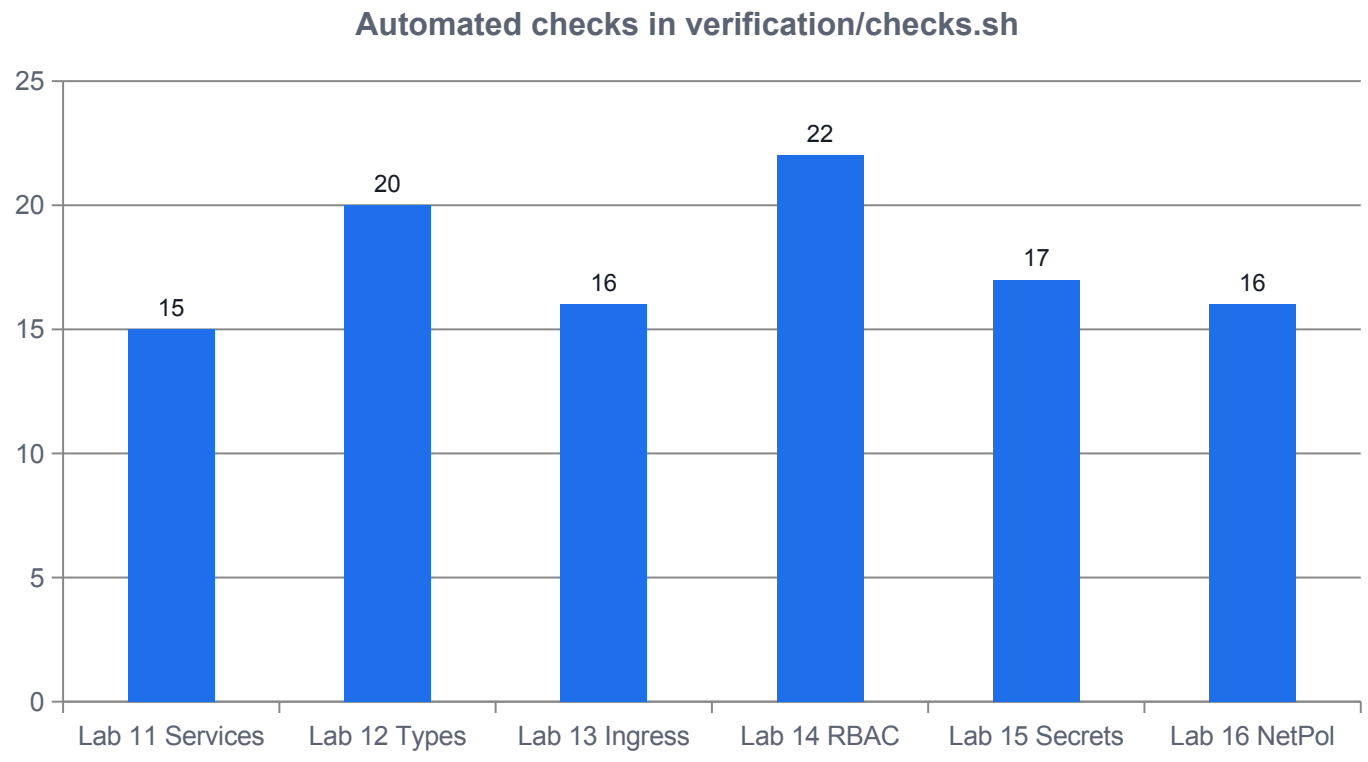
© 2026 Tertiary Infotech Academy Pte Ltd

LAB EVIDENCE

What Day 3's six labs actually verify

Counted from the shipped verifiers, not estimated: 106 automated assertions across the day.

PASS assertions on a clean run



Source: courseware/labs/lab-11-services-endpoints/verification/expected-output.md and the equivalent files for Labs 12-16

WHAT THE DATA SHOWS

106 checks in one day

Every claim a learner makes today is machine-checkable against their own cluster.

Lab 14 carries the most

22 checks, because a 12-row permission matrix is replayed one assertion at a time.

Denials are counted too

Several checks assert that something correctly does NOT work — the wrong-namespace DNS name, the unbound ServiceAccount.

Lab 16 reports rather than assumes

On an enforcing CNI the same script asserts more; on kindnet it records non-enforcement honestly.

Measured, not modelled

These are PASS counts from each lab's expected-output reference on a clean run.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 3 RECAP

Networking and security, consolidated

Two registered topics, twelve load-bearing facts, six labs of evidence.

TOPIC 3 — SERVICES AND NETWORKING

- IP per Pod, no NAT; ClusterIP is a rule, not an interface
- A Service holds a selector; EndpointSlice holds the result — v1 Endpoints is deprecated since v1.33
- port / targetPort / nodePort / named port each answer a different question
- CoreDNS, the search path and ndots:5 explain both the convenience and the cost of short names
- Ingress declares L7 routing; a controller must implement it — and Gateway API is where new features go
- Namespaces scope names, RBAC, quota and DNS. Nothing else

TOPIC 4 — SECURITY

- The 4C's: Cloud, Cluster, Container, Code — nested, not parallel
- Every request: authenticate, authorize, admit, persist. 401, 403 and admission errors are different gates
- No User object exists; ServiceAccounts do, and hold zero permission until bound
- The binding decides RBAC scope; prove grants with kubectl auth can-i, including the denies
- Secrets are base64-encoded, not encrypted — RBAC, encryption at rest and external stores are the controls
- NetworkPolicy is the L3/L4 control, and needs an enforcing CNI to mean anything

Source: kubernetes.io/docs/concepts/services-networking/service/ · kubernetes.io/docs/reference/access-authn-authz/rbac/ · kubernetes.io/docs/concepts/services-networking/network-policies/

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM FOCUS

How today's material is asked about

Six recurring question shapes drawn from the Container Orchestration domain's Networking and Security sub-topics.

1 Definition swaps

ClusterIP versus NodePort versus LoadBalancer; Role versus ClusterRole. Know what each one adds.

2 Which object does X?

Selector lives on the Service; addresses live on the EndpointSlice; L7 rules live on the Ingress.

3 Which layer enforces it?

kube-proxy is L4. Ingress and mesh are L7. NetworkPolicy is L3/L4 and is enforced by the CNI.

4 Security true/false

Secrets are not encrypted by default; namespaces are not network boundaries; RBAC has no deny rule.

5 Order questions

Authenticate, authorize, mutate, validate, persist. And default-deny before allows, with DNS allowed first.

6 Current versus legacy

EndpointSlice over Endpoints; Pod Security Standards over PodSecurityPolicy; Gateway API as the successor surface.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY FOUR

04

Storage, Cluster Architecture & Application Delivery

Where data outlives Pods, how a cluster is actually built, and how declarative software reaches it.

© 2026 Tertiary Infotech Academy Pte Ltd

LESSON PLAN

Day 4 — Storage, Cluster Architecture & Delivery

Four instructional sessions, five labs, 8.0 hours of instructional time.

Time	Duration	Session	Method
9:30 – 11:00 AM	1 h 30 m	Session 1	Lecture · Demo
11:00 – 11:10 AM	10 m	Morning Break	Break
11:10 AM – 1:10 PM	2 h 00 m	Session 2	Lecture · Practical
1:10 – 1:50 PM	40 m	Lunch Break	Break
1:50 – 3:50 PM	2 h 00 m	Session 3	Practical · Didactic questioning
3:50 – 4:00 PM	10 m	Afternoon Break	Break
4:00 – 6:30 PM	2 h 30 m	Session 4	Practical · Demonstration

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 4 ORIENTATION

What Today Adds to Your Model of Kubernetes

Two registered topics and the 16% KCNA application-delivery domain, in one day.

1 Data outlives the Pod

PersistentVolume, PersistentVolumeClaim and StorageClass decouple a workload from the disk it happens to be using.

2 Storage is pluggable

The Container Storage Interface moved every driver out of the Kubernetes tree and behind a gRPC contract.

3 Identity plus storage

volumeClaimTemplates give each StatefulSet ordinal its own PVC that deliberately outlives the controller.

4 A cluster is built, not conjured

kubeadm produces PKI, static Pod manifests, bootstrap tokens, a kubelet config and add-ons — nothing more magical than that.

5 etcd is the cluster's memory

Raft quorum, snapshot save/status, and a restore that rewinds every object in the API.

6 Git becomes the deployment interface

Helm and Kustomize render; a GitOps controller reconciles and self-heals against drift.

© 2026 Tertiary Infotech Academy Pte Ltd

COVERAGE MAP

How Day 4 Maps to the Registered Outline

Every slide today serves a registered topic or a current KCNA domain objective.

Topic 5 — Storage	Registered · LO4 · A4 · K6
Topic 6 — Cluster Architecture	Registered · LO5 · A5 · K5
KCNA module — App Delivery	Cloud Native Application Delivery, 16%
Lab 17	Volumes: emptyDir, hostPath, container filesystem
Lab 18	PersistentVolumes, Claims and StorageClasses
Lab 19	StatefulSets, headless Services, stable identity
Lab 20	Control plane anatomy and etcd backup/restore
Lab 21	Packaging and delivery: Helm, Kustomize, GitOps
Assessment link	PP Task 4 (A4) and Task 5 (A5) draw on today
Exam link	Storage sits inside Container Orchestration (28%)

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 5 · REGISTERED

05

Storage

Storage in container orchestration, from the Docker vocabulary that preceded it to the CSI contract that replaced it.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY STORAGE EXISTS

A container's filesystem dies with the container. Every storage feature in Kubernetes follows from that one fact.

The image is read-only. The writable layer is scratch space owned by one container instance. Volumes are the only bytes that survive.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Where a Container's Bytes Actually Live

Three storage regions with three different lifetimes — the whole topic is knowing which region a file landed in.

IMAGE — READ-ONLY

- **Base layer** — Distro rootfs from the registry
- **Dependency layer** — Runtime and libraries
- **Application layer** — Your binary and assets
- **Layer digests** — Content-addressed, shared, cached

CONTAINER — EPHEMERAL

- **Writable layer** — Copy-on-write upper directory
- **Merged view** — What the process sees as /
- **Scope** — One container instance only
- **On exit** — Discarded by the runtime

VOLUME — OUTSIDE THE UNION

- **emptyDir** — Lives as long as the Pod
- **PVC-backed** — Lives independently of the Pod
- **configMap / secret** — Projected, read-only
- **hostPath** — A path on one specific node

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Why the Writable Layer Is Not Storage

Copy-on-write is an efficiency mechanism, not a durability one.



WHAT IT MEANS

Restart-safety is not persistence. Only a volume survives the container that wrote to it.

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB EVIDENCE

Proving the Filesystem Is Ephemeral

Same manifest, same path, different bytes.

terminal — kcna-lab17

```
$ kubectl -n kcna-lab17 exec lane-scratch -- cat /data/first-boot
2026-09-05T04:11:23Z

$ kubectl -n kcna-lab17 delete pod lane-scratch --now
$ kubectl -n kcna-lab17 apply -f manifests/lane-scratch.yaml

$ kubectl -n kcna-lab17 exec lane-scratch -- cat /data/first-boot
2026-09-05T04:12:57Z

$ kubectl -n kcna-lab17 get pod lane-scratch \
    -o jsonpath='{.status.containerStatuses[0].containerID}'
containerd://8f1c...  # a different container ID
```

WHAT TO LOOK AT

1. **The timestamp moved**
   The file the first container wrote was never seen by the second.
2. **The container ID changed**
   A new container instance means a new writable layer, always.
3. **Nothing errored**
   There is no warning event for lost scratch data — the API is behaving exactly as designed.
4. **The fix is structural**
   Declare a volume; do not rely on the container filesystem.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

PRIOR ART

Docker Storage — the Vocabulary Kubernetes Inherited

Kubernetes did not invent these ideas; it generalised them across a cluster.

1 Storage driver

overlay2 and friends assemble image layers plus a writable layer into one merged filesystem. Not a persistence feature.

2 Bind mount

A host path spliced into the container. Fast, powerful, and tied to one machine's directory layout.

3 Named volume

Docker-managed storage with a lifecycle independent of the container: docker volume create / ls / rm.

4 tmpfs mount

RAM-backed mount that never reaches disk and vanishes with the container.

5 Volume driver plugin

An out-of-process plugin implementing Create / Mount / Unmount / Remove so third-party storage could back a named volume.

6 Why it mattered

The plugin API proved the idea of pluggable storage — and exposed how per-engine and non-portable that idea was.

Source: Docker documentation — Storage overview, volumes, storage drivers and the Volume plugin API (docs.docker.com).

© 2026 Tertiary Infotech Academy Pte Ltd

TRANSLATE

Docker Concept → Kubernetes Equivalent

The mapping is close enough to be useful and different enough to be dangerous.

Docker	Kubernetes	What changes in a cluster
Bind mount (-v /host:/ctr)	hostPath volume	Now node-specific: the Pod must land on the node that has the path
Named volume	PersistentVolumeClaim + PersistentVolume	The claim is portable; the backing volume is provisioned for you
tmpfs mount	emptyDir with medium: Memory	Counts against the writing container's memory limit
Volume driver plugin	CSI driver (controller + node plugins)	A versioned gRPC contract, not an engine-specific plugin
docker volume create	StorageClass + dynamic provisioning	Volumes are created on demand by a controller, not by an operator

CHOOSE

Learn the Kubernetes object, not the Docker flag — the cluster has to answer 'which node?' as well as 'which path?'.

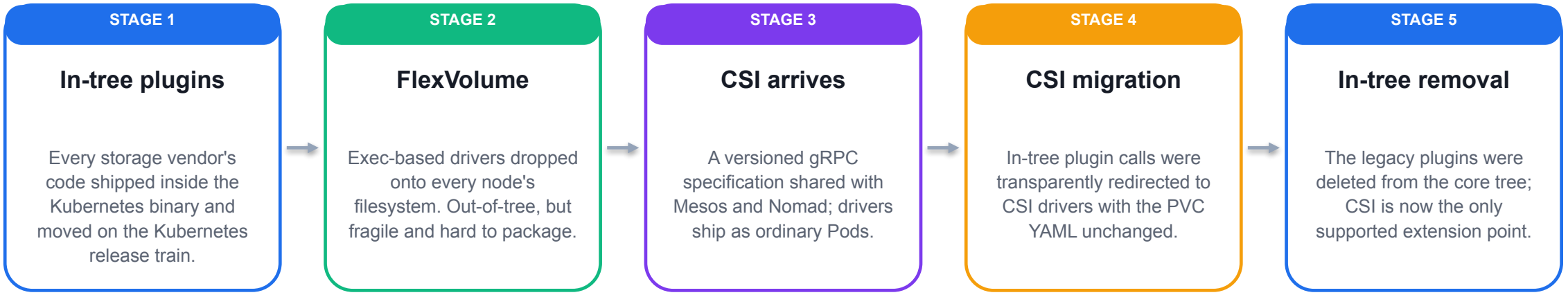
Source: Docker documentation — Storage overview, volumes, storage drivers and the Volume plugin API (docs.docker.com).

© 2026 Tertiary Infotech Academy Pte Ltd

HISTORY

From Volume Plugins to the Container Storage Interface

Three attempts at the same problem: how do you extend storage without extending Kubernetes itself?



WHAT IT MEANS

A PVC written for an in-tree plugin still reads the same — the compatibility was preserved deliberately at the API surface.

Source: Kubernetes documentation — Volume Plugin Migration / CSI migration; kubernetes-csi.github.io external sidecar docs.

© 2026 Tertiary Infotech Academy Pte Ltd

TAXONOMY

The Kubernetes Volume Types You Must Recognise

Pod.spec.volumes is a discriminated union — the key you set decides everything else.

1 **emptyDir**

Created empty when the Pod is assigned to a node; deleted when the Pod is removed. Shared by every container in the Pod.

2 **hostPath**

A file or directory from the node's own filesystem. Node-coupled and a documented security hazard.

3 **persistentVolumeClaim**

The portable option: the Pod references a claim, and the claim is bound to a PersistentVolume.

4 **configMap / secret**

Key-value data projected as read-only files. Updated in place by the kubelet for non-subPath mounts.

5 **downwardAPI**

Pod metadata — labels, annotations, resource limits — exposed as files without an API call.

6 **projected**

One mount point combining configMap, secret, downwardAPI and bound serviceAccountToken sources.

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Reading an emptyDir — Disk-Backed and Memory-Backed

The most common volume in Kubernetes, and the most commonly misunderstood lifetime.

lane-report.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: lane-report
spec:
  containers:
  - name: report-builder
    volumeMounts:
    - name: report
      mountPath: /shared
  - name: report-web
    volumeMounts:
    - name: report
      mountPath: /shared
      readOnly: true
  volumes:
  - name: report
    emptyDir:
      sizeLimit: 64Mi
```

READING IT

1. **One volume, two mounts**
   Both containers name the same volume; the sidecar pattern is just a shared emptyDir.
2. **readOnly is per mount**
   The same volume can be writable in one container and read-only in another.
3. **sizeLimit is an eviction bound**
   Exceeding it makes the Pod a candidate for eviction; it is not a hard filesystem quota on every backend.
4. **Lifetime is the Pod**
   A container restart keeps the data; deleting the Pod destroys it.

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

EXACT SEMANTICS

emptyDir: The Facts That Get Examined

Two of these are exam favourites: the lifetime is the Pod, and memory-backed files are charged to the writing container.

Created when	The Pod is assigned to a node — not before
Destroyed when	The Pod is removed from the node, for any reason
Container restart	Data survives; the volume outlives the container
Default backing	The node's ephemeral-storage medium (disk)
medium: Memory	A tmpfs — RAM-backed, cleared on node reboot

Memory accounting	Files count against the memory limit of the container that writes them
sizeLimit	Bounds the volume; drives eviction, not a hard quota
SizeMemoryBackedVolumes	Feature gate: beta in v1.22, generally available in v1.32
Sharing	Every container in the Pod may mount it; no cross-Pod sharing
Not a substitute for	PersistentVolumeClaim — it is never durable

Source: Kubernetes reference — Feature Gates. SizeMemoryBackedVolumes: beta v1.22, GA v1.32. Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB EVIDENCE

A Memory-Backed emptyDir Is Really a tmpfs

Declared 32Mi of medium: Memory; the container sees a 32.0M tmpfs.

terminal — kcna-lab17

```
$ kubectl -n kcna-lab17 exec lane-cache -- df -h /cache
Filesystem                Size      Used Available Use% Mounted on
tmpfs                    32.0M         0     32.0M   0% /cache

$ kubectl -n kcna-lab17 exec lane-cache -- mount | grep /cache
tmpfs on /cache type tmpfs (rw,relatime,size=32768k,inode64)

# the Pod spec that produced it
  volumes:
  - name: cache
    emptyDir:
      medium: Memory
      sizeLimit: 32Mi
```

WHAT TO LOOK AT

1. **Size matches sizeLimit exactly**
   32Mi declared, 32.0M reported — the tmpfs is sized from the spec.

2. **Filesystem type is tmpfs**
   Not overlay, not ext4 — this mount is served from page cache.

3. **Charged to the writer**
   Bytes written here count against the memory limit of the container doing the writing.

4. **Use it for**
   Secrets materialised at runtime, hot caches and scratch space you want gone at Pod teardown.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

hostPath and Its type Field

The one volume type that reaches outside the cluster's abstractions and touches a specific machine.

node-log-peek.yaml

```
  volumes:
  - name: node-logs
    hostPath:
      path: /var/log
      type: Directory

  containers:
  - name: peek
    volumeMounts:
    - name: node-logs
      mountPath: /host/var/log
      readOnly: true

# type values you should know:
#   ""                 no check at all
#   Directory          must already exist
#   DirectoryOrCreate  kubelet creates it, 0755
#   File / FileOrCreate  same, for files
#   Socket             must be a unix socket
```

READING IT

1. **type is a precondition**
   The kubelet checks it before mounting; an unmet check fails the mount rather than silently creating a path.

2. **Directory vs DirectoryOrCreate**
   The first refuses to invent state on a node; the second happily creates a directory you may not have intended.

3. **readOnly on the mount**
   Read-only is enforced at the mount, and is the only sane default for node paths.

4. **Empty type is the trap**
   No check means a typo mounts a newly created empty directory and the Pod starts anyway.

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

hostPath type: Directory Refuses to Invent State

A useful failure: it proves the check runs before the container is created.

SYMPTOM

Pod stays ContainerCreating; no container image is ever pulled.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab17 get pod archive-peek
NAME           READY   STATUS              RESTARTS   AGE
archive-peek   0/1     ContainerCreating   0          46s

$ kubectl -n kcna-lab17 describe pod archive-peek
Events:
  Type     Reason       Age   From     Message
  ----     ------       ----  ----     -------
  Warning  FailedMount  12s   kubelet  MountVolume.SetUp failed for
    volume "archive" : hostPath type check failed:
    /mnt/tracklane-archive is not a directory
```

✓ **STATUS reaches Running and the mount appears in describe**

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

ROOT CAUSE

The manifest declared type: Directory, but no such directory exists on the node the scheduler chose. The kubelet refuses the mount rather than fabricating the path.

FIX

- Read the FailedMount event — the message names the exact path
- Decide deliberately: pre-create on the node, or switch to DirectoryOrCreate and accept that the kubelet will create it
- Better: stop using hostPath for application data and use a PVC

© 2026 Tertiary Infotech Academy Pte Ltd

RISK

Why hostPath Is Treated as Dangerous

Not forbidden; deliberately uncomfortable, so you have to justify each use.

1 It escapes the container boundary

The Pod reads and writes the node's filesystem directly, outside everything the container isolation model promises.

2 It is a privilege-escalation path

Mounting the kubelet's directories, container socket or a writable /etc can hand an attacker the node.

3 It breaks portability

The Pod is only correct on nodes that happen to have that path with that content and those permissions.

4 It defeats the scheduler

Nothing in the scheduler knows the path exists; you must pin the Pod yourself with a selector or affinity.

5 Pod Security Standards restrict it

The baseline and restricted policies disallow hostPath volumes; only privileged permits them.

6 Legitimate uses exist

Node agents — log shippers, CNI plugins, monitoring DaemonSets — genuinely need node paths and are deployed knowingly.

Source: Kubernetes documentation — Volumes (kubernetes.io/docs/concepts/storage/volumes/). Retrieved 5 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 17 — Volumes: emptyDir, hostPath and the Container Filesystem

GOAL

Prove that a container's writable layer is discarded with the Pod, then use emptyDir to share scratch state between containers in one Pod and hostPath to reach the node's filesystem — while learning exactly why hostPath is a security liability.

YOU WILL PRODUCE

- A destroyed-and-recreated Pod that demonstrably loses its container-layer data, and an emptyDir-backed report served over HTTP by a sibling container.
- A memory-backed emptyDir (tmpfs) with an enforced sizeLimit, evidenced by mount output and an ENOSPC write failure.
- A read-only hostPath mount of the node's /var/log plus a diagnosed FailedMount event from a hostPath type assertion.

CONCEPTS EXERCISED

- Container filesystem ephemerality and the copy-on-write writable layer
- emptyDir lifecycle: created on Pod scheduling, destroyed on Pod removal
- emptyDir.medium Memory (tmpfs) versus node-disk backing, and sizeLimit enforcement
- hostPath type assertions, readOnly mounts, and node-escape risk

LAB	NAMESPACE	MAPS TO	TIME
Lab 17	kcna-lab17	LO4 · A4 · K6	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-17-volumes-emptydir-hostpath/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Morning Break

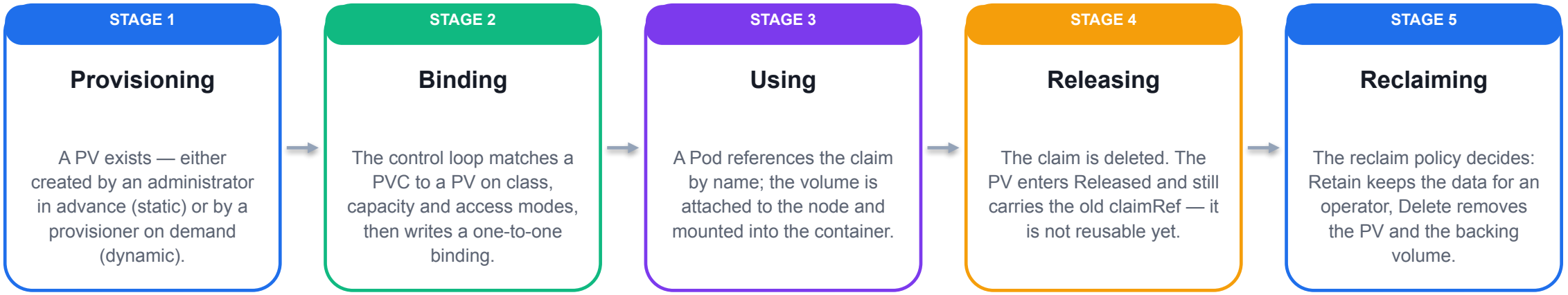
11:00 – 11:10 AM

© 2026 Tertiary Infotech Academy Pte Ltd

LIFECYCLE

The PersistentVolume Lifecycle, End to End

Five phases, and the exam asks about the boundaries between them.



WHAT IT MEANS

A PVC never binds to two PVs and a PV never serves two claims — binding is exclusive and one-to-one.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

SCOPING

Who Owns Which Storage Object

The namespace boundary is exactly where the tenant/operator split sits in storage.

CLUSTER-SCOPED

- **PersistentVolume** — A real volume registered in the API
- **StorageClass** — A named provisioning recipe
- **CSIDriver / CSINode** — Driver capability registration
- **VolumeSnapshot Class** — Snapshot recipe per driver
- **VolumeAttachment** — Node-to-volume attach record

NAMESPACED

- **PersistentVolume Claim** — A tenant's request for storage
- **Pod** — References the claim by name
- **VolumeSnapshot** — A point-in-time copy request
- **ResourceQuota** — Caps storage per class per namespace
- **ConfigMap / Secret** — Driver credentials and parameters

MECHANISM

Tenants create claims; only cluster administrators create the PersistentVolumes and StorageClasses that satisfy them.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Reading a PersistentVolume Field by Field

A static PersistentVolume: the administrator's half of the contract.

tracklane-archive-pv.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: tracklane-archive-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: tracklane-manual
  claimRef:
    namespace: kcna-lab18
    name: archive-claim
  hostPath:
    path: /mnt/tracklane-archive
    type: DirectoryOrCreate
```

READING IT

1. **capacity is an advertisement**
   It tells the binder how big this volume is. A claim asking for less still gets the whole volume.
2. **accessModes are capabilities**
   What this volume is able to support — not what the Pod will do with it.
3. **persistentVolumeReclaimPolicy: Retain**
   When the claim goes away the data stays and an administrator must act. The safe default for real data.
4. **claimRef pre-binds it**
   Naming the claim in advance reserves this PV so no other claim in the cluster can take it.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The Claim and the Fields That Prove It Bound

Requested 512Mi. Bound to 1Gi. Both are correct.

archive-claim.yaml (spec + observed status)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: archive-claim
  namespace: kcna-lab18
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: tracklane-manual
  resources:
    requests:
      storage: 512Mi
status:
  phase: Bound
  accessModes: [ReadWriteOnce]
  capacity:
    storage: 1Gi        # not 512Mi
  # spec.volumeName is filled in by the binder
```

READING IT

1. **requests.storage is a minimum**
   It is the smallest acceptable volume, not an allocation. The binder may hand you a larger one.
2. **status.capacity is the truth**
   Requested 512Mi, granted 1Gi — always read status, never assume spec was honoured verbatim.
3. **storageClassName must match**
   An empty string means 'no class, static only'; omitting the field means 'use the default class'.
4. **spec.volumeName appears after binding**
   The binder writes it. You normally do not set it by hand.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

kubectl get pv,pvc — Bound Next to Pending

Four columns answer almost every storage question you will be asked in the exam.

terminal — kcna-lab18

```
$ kubectl -n kcna-lab18 get pv,pvc
NAME                          CAPACITY  ACCESS  RECLAIM  STATUS
pv/tracklane-archive-pv       1Gi       RWO     Retain   Bound
pv/pvc-8b2e17d4-05c9-4f36     256Mi     RWO     Delete   Bound

NAME                    STATUS    VOLUME                   CAPACITY
pvc/archive-claim       Bound     tracklane-archive-pv     1Gi
pvc/render-cache        Bound     pvc-8b2e17d4-05c9-4f36   256Mi
pvc/report-claim        Pending

# STORAGECLASS column (trimmed above):
#   archive-claim  tracklane-manual   report-claim  tracklane-manul
```

WHAT TO LOOK AT

1. **Two PVs, two origins**
   The named one was authored by hand; the pvc-<uid> one was created by a dynamic provisioner.

2. **RECLAIM differs per PV**
   Retain on the static volume, Delete inherited from the StorageClass on the dynamic one.

3. **CAPACITY reflects the PV**
   archive-claim shows 1Gi even though the claim asked for 512Mi.

4. **Pending has no VOLUME**
   An empty VOLUME column is the whole diagnosis — nothing bound. describe tells you why.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Access Modes — and Why They Are Node-Scoped

Three of the four modes are counted per node; only ReadWriteOncePod is counted per Pod.

Mode	Short	What it actually permits	Typical backend
ReadWriteOnce	RWO	Read-write by many Pods, but only on ONE node at a time	Cloud block storage, iSCSI, local disks
ReadOnlyMany	ROX	Read-only, mounted by many nodes simultaneously	Pre-populated NFS exports, read-only datasets
ReadWriteMany	RWX	Read-write, mounted by many nodes simultaneously	NFS, CephFS, Azure Files — a file, not block, protocol
ReadWriteOncePod	RWOP	Read-write by exactly ONE Pod in the whole cluster	CSI drivers that opt in; the only Pod-scoped mode

CHOOSE

RWO does not mean 'one Pod'. Two Pods on the same node can both mount an RWO volume — that surprise is a standard exam distractor.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Reclaim Policies — What Happens to the Data

The single field most likely to destroy production data by accident.

Policy	When the claim is deleted	The data	Use it when
Retain	PV moves to Released and keeps the old claimRef	Preserved until an administrator acts	The data has value: archives, databases, anything auditable
Delete	PV object and the backing volume are both removed	Destroyed, usually irrecoverably	Genuinely disposable caches, ephemeral CI workspaces
Recycle (deprecated)	A scrubber Pod ran rm -rf on the volume, then re-offered it	Wiped, then reused	Never — superseded by dynamic provisioning; do not design with it

CHOOSE

Dynamic provisioning defaults to Delete. If your data matters, create a StorageClass with reclaimPolicy: Retain and use it deliberately.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Retain in Action — a Released Volume Will Not Rebind

Retain is doing its job precisely when the new claim stays Pending.

terminal — kcna-lab18

```
$ kubectl delete pvc archive-claim -n kcna-lab18
persistentvolumeclaim "archive-claim" deleted

$ kubectl get pv tracklane-archive-pv
NAME                   CAPACITY  RECLAIM  STATUS     CLAIM
tracklane-archive-pv   1Gi       Retain   Released   kcna-lab18/...

$ kubectl apply -f archive-claim.yaml   # same name, new object
$ kubectl -n kcna-lab18 get pvc archive-claim
NAME            STATUS    VOLUME   CAPACITY
archive-claim   Pending

$ kubectl get pv tracklane-archive-pv -o jsonpath='{.spec.claimRef.uid}'
3c9a1f60-77bd-4b32-9d21-0e5c6f4a1b88   # the OLD claim's UID
```

WHAT TO LOOK AT

1. **Released is not Available**
   Retain deliberately parks the volume so nobody else can bind to it and read your data.
2. **The claimRef holds a UID**
   Recreating a claim with the same name creates a different object with a different UID, so the reference no longer matches.
3. **Recovery is an admin action**
   An administrator clears the stale claimRef; the data on the volume was never touched.
4. **This is the safety feature**
   Automatic rebinding would leak one tenant's data to the next claimant of the same name.

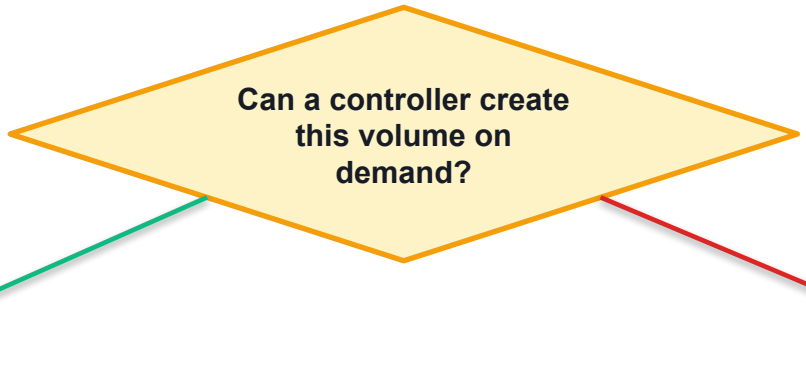
Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN DECISION

Static or Dynamic Provisioning?

The question is not which is better; it is whether a driver exists that can create the volume for you.



Yes — dynamic provisioning

- Define a StorageClass with a provisioner and parameters
- Tenants create PVCs; PVs appear named pvc-<uid>
- Reclaim policy and binding mode come from the class
- Scales to many namespaces without operator involvement

No — static provisioning

- An administrator creates each PersistentVolume by hand
- Use for pre-existing NFS exports, LUNs, licensed appliances
- Pre-bind with claimRef so the right claim gets the right volume
- Operational cost is per-volume and does not scale

Source: Kubernetes documentation — Storage Classes (kubernetes.io/docs/concepts/storage/storage-classes/).

© 2026 Tertiary Infotech Academy Pte Ltd

THREE PATHS

How a Volume Actually Reaches a Pod

Same mountPath in the container; three very different operational models behind it.

Static PersistentVolume

- Administrator authors the PV up front
- PVC matches on class, size, access mode
- claimRef can reserve it for one claim
- Data survives the claim under Retain

operator-created | Retain

Dynamic via StorageClass

- PVC names a class; provisioner creates the volume
- PV name is pvc-<claim-uid>
- Class supplies reclaim policy and binding mode
- The default class applies when the field is omitted

on demand | pvc-<uid>

Inline ephemeral CSI volume

- Declared inside Pod.spec.volumes, no PVC object
- Generic ephemeral volumes still create a PVC for you
- Lifetime is bound to the Pod
- For per-Pod scratch on real storage hardware

Pod-scoped | no PVC to manage

Source: Kubernetes documentation — Ephemeral Volumes; Images (kubernetes.io/docs/concepts/storage/ephemeral-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A StorageClass Is a Provisioning Recipe

Change the class, change the behaviour of every volume it creates — retroactively for none of them.

storageclass-tracklane-retain.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: tracklane-retain
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: rancher.io/local-path
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  nodePath: /opt/local-path-provisioner
---
# the cluster default in this training environment
# name: standard
# provisioner: rancher.io/local-path
# reclaimPolicy: Delete
# volumeBindingMode: WaitForFirstConsumer
```

READING IT

1. **provisioner selects the driver**
   Usually a CSI driver name. It is the only field a claim cannot override.
2. **reclaimPolicy is inherited**
   Every PV this class creates gets this policy. Retain here is what makes the class safe for real data.
3. **volumeBindingMode changes when**
   Immediate binds as soon as the claim exists; WaitForFirstConsumer waits for a Pod so topology can be honoured.
4. **allowVolumeExpansion: true**
   Without it, editing requests.storage on a bound PVC is rejected.

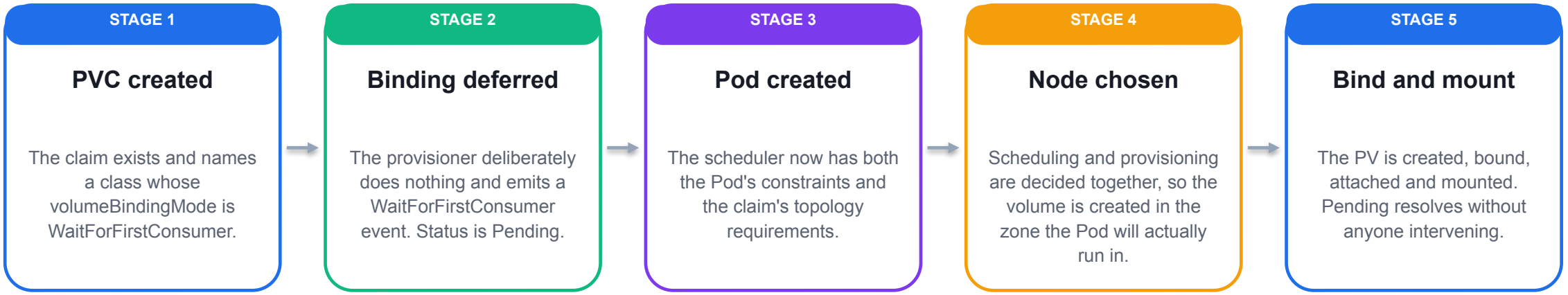
Source: Kubernetes documentation — Storage Classes (kubernetes.io/docs/concepts/storage/storage-classes/).

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

WaitForFirstConsumer: Why Pending Is the Correct State

A Pending PVC with no Pod is not a fault — it is the design.



WHAT IT MEANS

Immediate binding can strand a volume in zone A while the only schedulable node is in zone B. WaitForFirstConsumer exists to prevent exactly that.

Source: Kubernetes documentation — Storage Classes (kubernetes.io/docs/concepts/storage/storage-classes/).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

describe pvc — the Event That Means 'Working as Intended'

Learn to read this event; it stops you 'fixing' a healthy cluster.

terminal — kcna-lab18

```
$ kubectl -n kcna-lab18 describe pvc render-cache
Name:          render-cache
Namespace:     kcna-lab18
StorageClass:  standard
Status:        Pending
Volume:
Capacity:
Access Modes:
Used By:       <none>
Events:
  Type    Reason                Age   From                Message
  ----    ------                ----  ----                -------
  Normal  WaitForFirstConsumer  14s   persistentvolume-   waiting
          (x3 over 44s)               controller          for first
                                       consumer to be created
                                       before binding
```

WHAT TO LOOK AT

1. **Type is Normal, not Warning**
   The event class tells you immediately this is expected behaviour, not an error.

2. **Used By: <none>**
   No Pod references this claim yet, which is precisely the condition the binder is waiting on.

3. **The reporter is the PV controller**
   Not the scheduler and not the driver — the binding decision is being held open on purpose.

4. **Resolution needs no action**
   Create a Pod that mounts the claim and the volume is provisioned in the same scheduling decision.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

PVC Pending Because the Class Name Has a Typo

Two Pending claims, two completely different meanings — the event type separates them.

SYMPTOM

A claim stays Pending forever and no PersistentVolume is ever created.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab18 get pvc report-claim
NAME           STATUS    VOLUME   CAPACITY   STORAGECLASS
report-claim   Pending                       tracklane-manul

$ kubectl -n kcna-lab18 describe pvc report-claim
Events:
  Type     Reason              Age   Message
  ----     ------              ----  -------
  Warning  ProvisioningFailed  8s    storageclass.storage.k8s.io
                                     "tracklane-manul" not found

$ kubectl get storageclass
NAME                 PROVISIONER            RECLAIMPOLICY
standard (default)   rancher.io/local-path  Delete
tracklane-retain     rancher.io/local-path  Retain
```

ROOT CAUSE

storageClassName is a free-text reference with no foreign-key check. 'tracklane-manul' is missing an 'a', so no class matches and nothing provisions the volume.

FIX

- Compare the claim's STORAGECLASS column against kubectl get sc
- Note that Warning + ProvisioningFailed distinguishes this from the Normal WaitForFirstConsumer case
- storageClassName is immutable on a bound claim — recreate the PVC with the correct name

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

DEFAULTS

The Default StorageClass and Its Three-Way Trap

Omitted, empty and named are three distinct states of one field.

1 Marked by an annotation

storageclass.kubernetes.io/is-default-class: "true". There should be exactly one such class in the cluster.

2 Field omitted entirely

The default class is substituted into the claim at admission time and written into spec.storageClassName.

3 Field set to "" (empty string)

Explicitly opts out of dynamic provisioning. Only a matching pre-created PV can satisfy the claim.

4 Field set to a name

That class is used, whether or not it exists — see the previous failure slide.

5 Two defaults is a defect

If more than one class is marked default, the behaviour is not one you want to depend on. Audit it.

6 In this training cluster

standard / rancher.io/local-path / Delete / WaitForFirstConsumer — know your environment's default before you design against it.

Source: Kubernetes documentation — Storage Classes (kubernetes.io/docs/concepts/storage/storage-classes/).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Using a PVC in a Pod

Four lines connect a workload to durable storage — and one of them is a common scaling trap.

render-cache-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: render-cache}
spec:
  template:
    spec:
      containers:
      - name: renderer
        volumeMounts:
        - name: cache
          mountPath: /cache
          subPath: renders
          readOnly: false
      volumes:
      - name: cache
        persistentVolumeClaim:
          claimName: render-cache
          readOnly: false
```

READING IT

1. **The Pod names the claim, not the volume**
   claimName is a same-namespace reference. The Pod never knows which PersistentVolume it got.
2. **mountPath is inside the container**
   It has no relationship to any path on the node or on the backing storage.
3. **subPath mounts a subdirectory**
   Useful to share one volume between containers without collisions — but subPath mounts do not receive configMap updates.
4. **A Deployment can use a PVC**
   But every replica mounts the SAME claim. With RWO and more than one node that will not schedule.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 18 — PersistentVolumes, Claims and StorageClasses

GOAL

Give the TrackLane customs archive storage that outlives the Pod that wrote it — first by hand-binding a static PersistentVolume, then by letting a StorageClass provision one dynamically — and diagnose a Pending claim from describe events alone.

YOU WILL PRODUCE

- A statically provisioned PV bound to a namespaced PVC, seeded with 30 archive rows by one Pod and read back by a second Pod after the writer was deleted.
- A dynamically provisioned volume from kind's default `standard` StorageClass, observed transitioning from Pending (WaitForFirstConsumer) to Bound the moment a consumer is scheduled.
- A diagnosed Pending PVC whose root cause is a StorageClass name typo, evidenced by a ProvisioningFailed event.

CONCEPTS EXERCISED

- PV/PVC binding: capacity, access modes, volumeMode, storageClassName and claimRef pre-binding
- Reclaim policies Retain vs Delete, and the Released phase
- StorageClass, dynamic provisioning and volumeBindingMode WaitForFirstConsumer
- CSI as the out-of-tree plugin interface between Kubernetes and storage vendors

LAB	NAMESPACE	MAPS TO	TIME
Lab 18	kcna-lab18	LO4 · A4 · K6	55 min

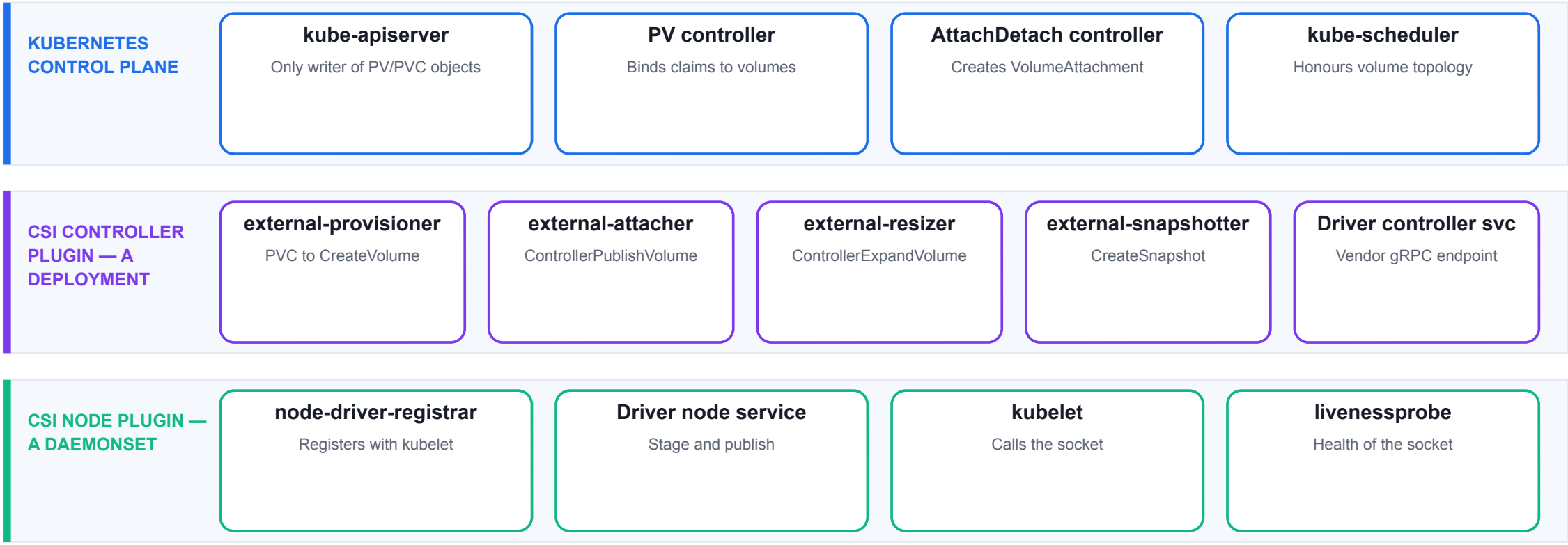
Full step-by-step procedure: Learner Guide · courseware/labs/lab-18-pv-pvc-storageclass/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

The CSI Controller / Node Split and Its Sidecars

One driver, two deployments: cluster-wide volume management and per-node mounting are different jobs with different privileges.



Source: Container Storage Interface specification (github.com/container-storage-interface/spec) and Kubernetes documentation — Volume Plugin FAQ / CSI Drivers.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPONENTS

Each Sidecar Watches One Kubernetes Object

Kubernetes never speaks CSI directly — sidecars translate API objects into gRPC calls.

external-provisioner

1 Watches PersistentVolumeClaims. Calls CreateVolume, then creates the PersistentVolume object that the binder will use.

external-attacher

2 Watches VolumeAttachment objects and calls ControllerPublishVolume / ControllerUnpublishVolume.

external-resizer

3 Watches for a change to PVC spec.resources.requests.storage and calls ControllerExpandVolume.

external-snapshotter

4 Watches VolumeSnapshot and VolumeSnapshotContent; calls CreateSnapshot and DeleteSnapshot.

node-driver-registrar

5 Registers the driver's unix socket with the kubelet plugin registry so the kubelet knows the driver exists.

livenessprobe

6 Exposes the driver's gRPC Probe response as an HTTP endpoint the kubelet can use for a liveness probe.

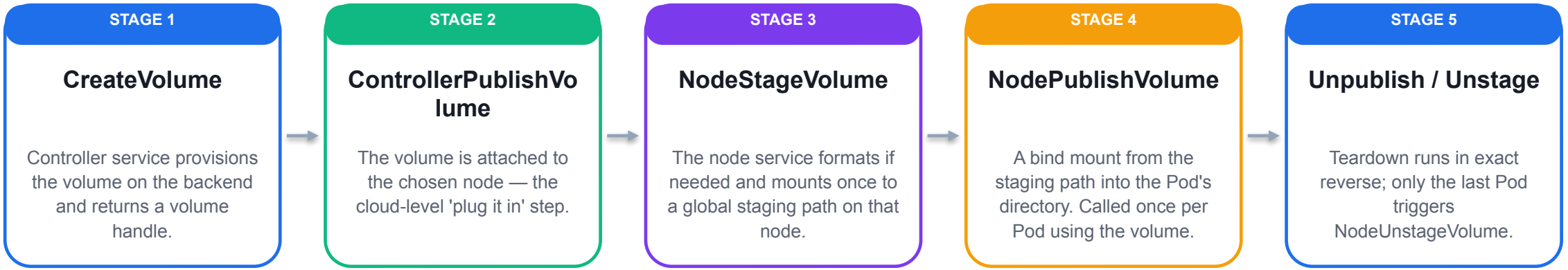
Source: Kubernetes documentation — Volume Plugin Migration / CSI migration; kubernetes-csi.github.io external sidecar docs.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

The RPC Sequence Behind One Mounted Volume

Two attach steps and two mount steps, each idempotent by specification.



WHAT IT MEANS

Staging exists so that several Pods on one node can share a single mount of the same volume without mounting it twice.

Source: Container Storage Interface specification (github.com/container-storage-interface/spec) and Kubernetes documentation — Volume Plugin FAQ / CSI Drivers.

© 2026 Tertiary Infotech Academy Pte Ltd

RATIONALE

Why the In-Tree Drivers Had to Go

An architectural clean-up done without breaking a single user-authored manifest.

The problems with in-tree storage

- Vendor code shipped inside the kube-controller-manager and kubelet
- A driver bug fix had to wait for a Kubernetes release
- Every cluster carried every vendor's code and its CVE surface
- Cloud credentials had to be readable by core components
- The core repository accumulated code nobody in SIG could test

What CSI migration preserved

- The same PVC and StorageClass YAML keeps working
- In-tree plugin names were transparently redirected to CSI drivers
- Migration graduated per provider, then the in-tree code was removed
- Drivers now release on their own schedule as ordinary workloads
- The extension point is a published spec, not a Kubernetes internal

Source: Kubernetes documentation — Volume Plugin Migration / CSI migration; kubernetes-csi.github.io external sidecar docs.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Capabilities CSI Unlocked

Every one of these is a driver capability — the API accepts the field, the driver decides whether it works.

Dynamic provisioning	CreateVolume driven by a StorageClass
Attach / detach	VolumeAttachment objects, per node
Topology awareness	allowedTopologies + WaitForFirstConsumer
Volume snapshots	snapshot.storage.k8s.io/v1: VolumeSnapshot, VolumeSnapshotClass, VolumeSnapshotContent
Volume cloning	PVC dataSource kind: PersistentVolumeClaim

Online expansion	allowVolumeExpansion + ControllerExpandVolume
Inline ephemeral volumes	csi: block inside Pod.spec.volumes
Generic ephemeral volumes	ephemeral.volumeClaimTemplate — a PVC scoped to the Pod's life
Raw block volumes	volumeMode: Block instead of Filesystem
Capability discovery	CSIDriver and CSINode objects in the API

Source: Kubernetes documentation — Volume Snapshots (snapshot.storage.k8s.io/v1).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Snapshot, Then Restore Into a New Claim

Backup at the storage layer, expressed as ordinary Kubernetes objects.

snapshot-and-restore.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: archive-2026-09-05
spec:
  volumeSnapshotClassName: csi-hostpath-snapclass
  source:
    persistentVolumeClaimName: archive-claim
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: archive-restore}
spec:
  accessModes: [ReadWriteOnce]
  resources: {requests: {storage: 1Gi}}
  dataSource:
    apiGroup: snapshot.storage.k8s.io
    kind: VolumeSnapshot
    name: archive-2026-09-05
```

READING IT

1. **Snapshots are namespaced**
   VolumeSnapshot lives beside the claim; VolumeSnapshotContent is the cluster-scoped counterpart, like PV to PVC.

2. **readyToUse gates the restore**
   status.readyToUse must be true before the snapshot can be used as a data source.

3. **dataSource creates a new volume**
   The restore is a fresh PVC and a fresh PV — it never overwrites the source claim.

4. **Requires CRDs and a controller**
   The snapshot API is not built into the API server; it is CRDs plus the external-snapshotter.

Source: Kubernetes documentation — Volume Snapshots (snapshot.storage.k8s.io/v1).

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Volume Expansion — Grow Only, Never Shrink

Four actors touch one number: you, the resizer, the backend and the node.

1 The class must permit it

allowVolumeExpansion: true on the StorageClass. Without it the API server rejects the edit outright.

2 You edit the claim

Increase spec.resources.requests.storage on the bound PVC. The PV is updated for you; you never edit it directly.

3 Controller expands the volume

external-resizer calls ControllerExpandVolume against the storage backend.

4 Then the filesystem grows

The node service runs NodeExpandVolume. Until it does, the PVC carries the FileSystemResizePending condition.

5 Some drivers need a restart

Offline expansion requires the Pod to be recreated; online expansion does not. This is a driver property.

6 Shrinking is not supported

Reducing requests.storage is rejected. Plan capacity, or restore into a smaller volume from a snapshot.

Source: Kubernetes documentation — Persistent Volumes (kubernetes.io/docs/concepts/storage/persistent-volumes/).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

volumeClaimTemplates — One PVC per Ordinal

The only workload API that provisions storage per replica.

depot-ledger-statefulset.yaml

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: depot-ledger}
spec:
  serviceName: depot-ledger      # the headless Service
  replicas: 3
  podManagementPolicy: OrderedReady
  persistentVolumeClaimRetentionPolicy:
    whenDeleted: Retain
    whenScaled: Retain
  template:
    spec:
      containers:
      - name: ledger
        volumeMounts:
        - {name: ledger, mountPath: /var/lib/ledger}
  volumeClaimTemplates:
  - metadata: {name: ledger}
    spec:
      accessModes: [ReadWriteOnce]
      resources: {requests: {storage: 128Mi}}
```

READING IT

1. **It is a template, not a claim**
   The controller creates one real PVC per ordinal, named <template>-<statefulset>-<ordinal>.
2. **The binding is sticky**
   depot-ledger-1 always remounts ledger-depot-ledger-1, even after deletion and rescheduling to another node.
3. **Retention policy is the safety net**
   Retain/Retain means the claims outlive both scale-down and the StatefulSet itself.
4. **serviceName is mandatory**
   It names the headless Service that gives each ordinal its stable DNS record.

Source: Kubernetes documentation — StatefulSets; StatefulSet.spec.persistentVolumeClaimRetentionPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

One Claim per Ordinal, Named Predictably

Stable name plus stable claim equals stable identity.

terminal — kcna-lab19

```
$ kubectl -n kcna-lab19 get pod,pvc
NAME                  READY   STATUS    AGE
pod/depot-ledger-0    1/1     Running   3m
pod/depot-ledger-1    1/1     Running   2m52s
pod/depot-ledger-2    1/1     Running   2m44s

NAME                              STATUS   VOLUME       CAPACITY
pvc/ledger-depot-ledger-0         Bound    pvc-4a1c..   128Mi
pvc/ledger-depot-ledger-1         Bound    pvc-9e77..   128Mi
pvc/ledger-depot-ledger-2         Bound    pvc-2b30..   128Mi

$ kubectl -n kcna-lab19 exec depot-ledger-1 -- cat /var/lib/ledger/mounts.log
2026-09-05T07:14:02Z mounted by depot-ledger-1
2026-09-05T07:21:36Z mounted by depot-ledger-1   # after deletion
```

WHAT TO LOOK AT

1. **The name encodes the identity**
   <claimTemplate>-<statefulSet>-<ordinal>: entirely derivable, which is what makes reattachment deterministic.
2. **Ages are staggered**
   OrderedReady creates 0, then 1, then 2 — each waits for the previous Pod to become Ready.
3. **Two mount records, one volume**
   The Pod was deleted and recreated on a different IP, yet the same volume came back with its data.
4. **Capacity is per ordinal**
   Three claims of 128Mi, not one shared 384Mi volume.

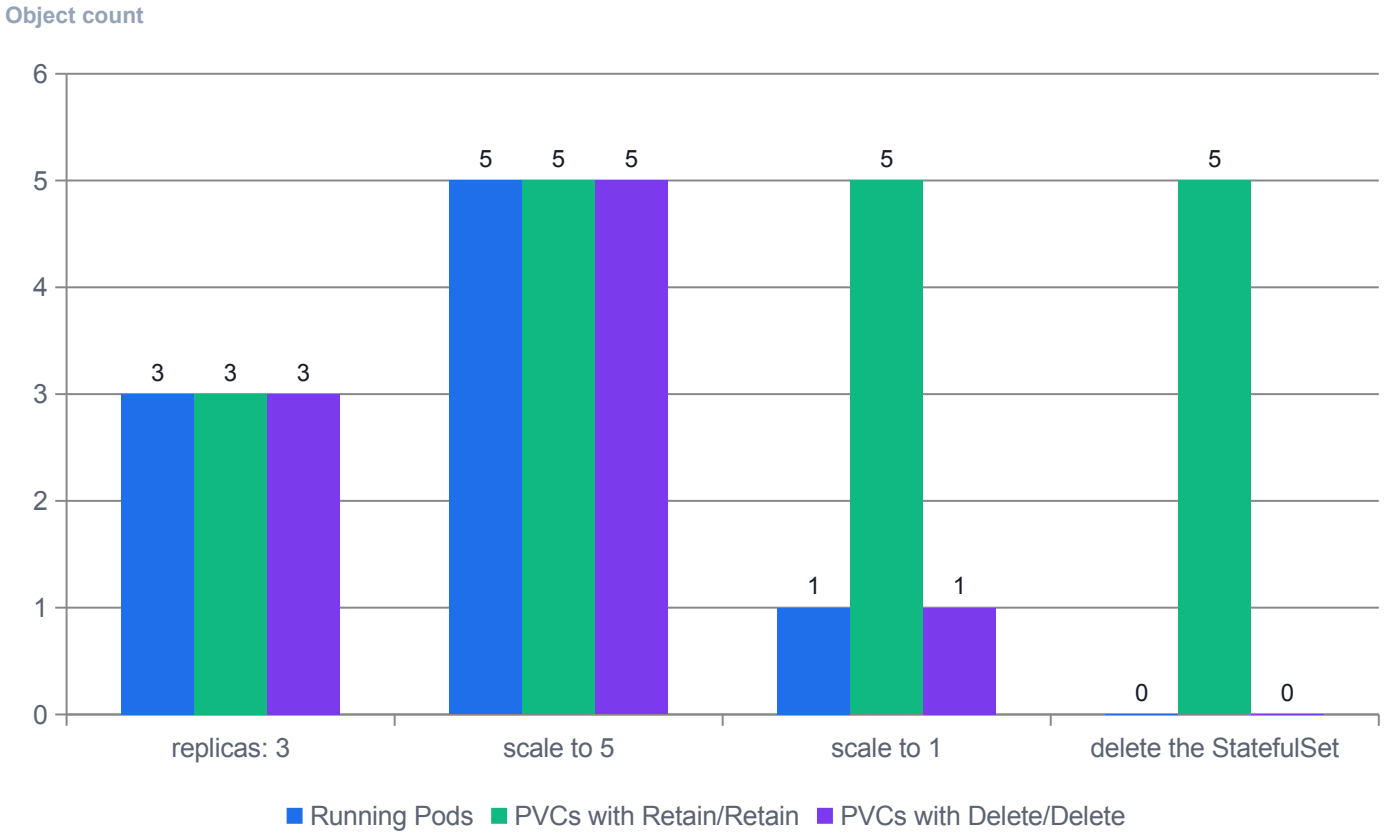
Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

ARITHMETIC

PVCs Do Not Disappear When You Scale Down

A worked model of the documented retention policy, not a measurement.



WHAT THE DATA SHOWS

Exact, not measured

Derived from the documented persistentVolumeClaimRetentionPolicy semantics applied to a 3-to-5-to-1 scaling sequence.

Retain is the default

Both whenScaled and whenDeleted default to Retain, so claims accumulate unless you opt out.

Scale-up re-adopts

Returning to 3 replicas reuses ledger-...-2, data intact, rather than provisioning a new claim.

The cost is real

Five 128Mi claims persist after scaling to one Pod. Storage quota is consumed by Pods that no longer exist.

Source: Kubernetes documentation — StatefulSets; StatefulSet.spec.persistentVolumeClaimRetentionPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd

CONCEPT

Stable Identity Has Exactly Three Parts

Identity is the reason to choose a StatefulSet; replicas alone never are.

1 A stable name

Ordinal Pod names 0..N-1. Deleting depot-ledger-1 creates another Pod with the same name, not a random suffix.

2 A stable network identity

<pod>.<service>.<ns>.svc.cluster.local, published by the headless Service named in spec.serviceName.

3 A stable claim

One PVC per ordinal from volumeClaimTemplates, reattached on every reschedule.

4 OrderedReady

Default policy: create and scale up in order, terminate in reverse order, one at a time.

5 Parallel

podManagementPolicy: Parallel starts and stops all Pods at once. Identity is kept; ordering is abandoned.

6 What is NOT stable

The Pod IP. It changes on every reschedule — which is exactly why the DNS name exists.

Source: Kubernetes documentation — StatefulSets; StatefulSet.spec.persistentVolumeClaimRetentionPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Deployment Storage vs StatefulSet Storage

The difference is not durability; it is whether replicas are interchangeable.

Question	Deployment + one PVC	StatefulSet + volumeClaimTemplates
How many volumes?	One, shared by every replica	One per ordinal, created automatically
Scaling to 3 on RWO	Fails to schedule beyond the node holding the volume	Works — each replica gets its own volume
Pod identity after restart	New random name, no volume affinity	Same ordinal name, same claim, same data
Deleting the controller	The PVC you authored remains	Claims remain by default (whenDeleted: Retain)
Right for	Shared read-mostly data on RWX, or no data at all	Databases, queues, ledgers — anything with per-replica state

CHOOSE

If each replica needs its own disk, you need volumeClaimTemplates — no amount of Deployment configuration substitutes for it.

Source: Kubernetes documentation — StatefulSets; StatefulSet.spec.persistentVolumeClaimRetentionPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A StatefulSet With No Headless Service Looks Healthy

Ready status measures the container, not the contract.

SYMPTOM

All replicas report Running and Ready, yet per-Pod DNS does not resolve and peers cannot find each other.

EVIDENCE — actual cluster output

```
$ kubectl -n kcna-lab19 get statefulset ghost-ledger
NAME           READY   AGE
ghost-ledger   1/1     54s        # no warning of any kind

$ kubectl -n kcna-lab19 get endpointslices -l \
    kubernetes.io/service-name=ghost-ledger
No resources found in kcna-lab19 namespace.

$ kubectl -n kcna-lab19 exec ledger-client -- \
    nslookup ghost-ledger-0.ghost-ledger.kcna-lab19.svc.cluster.local
** server can't find ghost-ledger-0.ghost-ledger.kcna-lab19
   .svc.cluster.local: NXDOMAIN
```

✓ **nslookup returns one A record per ordinal**

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

ROOT CAUSE

spec.serviceName names a headless Service that was never created. The StatefulSet controller does not validate that the Service exists, so the workload runs while its identity guarantee is silently absent.

FIX

- Create the headless Service with clusterIP: None and a selector matching the Pod labels
- Confirm with kubectl get endpointslices — one address per ready Pod
- Re-resolve <pod>.<service>.<ns>.svc.cluster.local from a client Pod

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 19 — StatefulSets, Headless Services and Stable Identity

GOAL

Give the TrackLane depot ledger stable per-replica identity — ordinal Pod names, per-Pod DNS through a headless Service, and one automatically created PersistentVolumeClaim per ordinal — then prove that identity survives Pod deletion and that the volumes deliberately outlive the StatefulSet.

YOU WILL PRODUCE

- A three-replica StatefulSet whose Pods each serve a different ledger shard from their own PVC, resolvable at <pod>.<svc>.<ns>.svc.cluster.local.
- Evidence of ordered creation and reverse-ordered termination, and of a deleted Pod reattaching to its original volume with an incremented mount counter.
- A diagnosed NXDOMAIN caused by a StatefulSet whose serviceName names a headless Service that was never created.

CONCEPTS EXERCISED

- StatefulSet versus Deployment: stable ordinals and identity versus interchangeable replicas
- Headless Services (clusterIP: None) and per-Pod DNS records
- volumeClaimTemplates: one PVC per ordinal, and why they are not garbage-collected
- podManagementPolicy OrderedReady versus Parallel, and rolling updates with partition

LAB	NAMESPACE	MAPS TO	TIME
Lab 19	kcna-lab19	LO4 · A4 · K6	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-19-statefulset-headless/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1:10 – 1:50 PM

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 6 · REGISTERED

06

Cluster Architecture, Installation & Configuration

What kubeadm actually builds, how upgrades are sequenced, and how etcd keeps the whole thing consistent.

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE INSTALLATION

The Decisions You Make Before kubeadm Runs

Installation is mostly decisions; the commands are the easy part and live in the Learner Guide.

1 Control-plane endpoint

A stable DNS name or VIP in front of the API servers. Adding it later means reissuing certificates — decide up front.

2 Topology

Stacked etcd on the control-plane nodes, or an external etcd cluster with its own failure domain.

3 Networking CIDRs

Pod and Service CIDRs must not overlap each other, the node network, or anything you might peer with later.

4 CNI plugin

Chosen before the first workload, and it determines whether NetworkPolicy is enforced at all.

5 Container runtime

A CRI implementation on every node, with the cgroup driver matching the kubelet's.

6 Certificate strategy

kubeadm issues a 10-year CA and one-year leaf certificates. Renewal has to be somebody's scheduled job.

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

What kubeadm Leaves Behind on a Control-Plane Node

The control plane is not special software management — it is four containers the kubelet starts from files on disk.

NODE FILESYSTEM	**/etc/kubernetes/manifests** Static Pod manifests	**/etc/kubernetes/pki** CA and leaf certificates	**admin.conf / kubelet.conf** kubeconfig files	**/var/lib/kubelet/config.yaml** Kubelet configuration
STATIC PODS — KUBELET-MANAGED	**etcd** Consistent key-value store	**kube-apiserver** The only writer to etcd	**kube-controller-manager** Reconciliation loops	**kube-scheduler** Binds Pods to nodes
ADD-ONS — API-MANAGED	**CoreDNS** Deployment; cluster DNS	**kube-proxy** DaemonSet; Service dataplane	**CNI plugin** DaemonSet you install	**Storage provisioner** Optional, per environment

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

The Phases kubeadm init Runs

Understand the phases; the exact command lines belong in the Learner Guide, not on a slide.



WHAT IT MEANS

Every phase is separately invocable, which is why kubeadm is described as a toolkit rather than an installer.

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

OWNERSHIP

The Three Things kubeadm Really Owns

Not an installer of Kubernetes so much as a producer of certificates, files and tokens.

Public key infrastructure

- A self-signed cluster CA plus a separate etcd CA
- Server certs with the right SANs for the endpoint
- Client certs for kubelet, scheduler, controller-manager
- The service-account signing key pair (sa.key / sa.pub)

/etc/kubernetes/pki | 10-year CA

Static Pod manifests

- Four YAML files the kubelet watches on disk
- Rewritten in place by kubeadm upgrade apply
- Mirror Pods appear in the API as read-only reflections
- Owned by the Node object, not by any controller

kubelet-managed | no scheduler

Identity for joining

- Bootstrap tokens with a TTL and their RBAC bindings
- The discovery CA hash a joiner uses to trust the endpoint
- TLS bootstrap and automatic CSR approval
- The kubelet configuration ConfigMap every node reads

bootstrap-token | CSR approval

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

The PKI a kubeadm Cluster Depends On

Two independent trust roots: one for Kubernetes, one for etcd.

ca.crt / ca.key	Cluster root CA — signs the Kubernetes leaf certs
apiserver.crt	Serving cert; SANs must cover every endpoint used
apiserver-kubelet-client	API server's identity when calling kubelets
front-proxy-ca	Separate CA for aggregated API servers
etcd/ca.crt	A distinct CA — etcd trust is not the cluster CA

etcd/server + etcd/peer	Client-facing and member-to-member certs
sa.key / sa.pub	Signs and verifies ServiceAccount tokens
Default CA lifetime	10 years
Default leaf lifetime	1 year — renewed by kubeadm on upgrade
Rotation risk	Clusters that never upgrade quietly expire

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

How a Node Proves Who It Is When Joining

Joining is an authentication problem, not a networking one.

join-config.yaml

```
apiVersion: kubeadm.k8s.io/v1beta4
kind: JoinConfiguration
discovery:
  bootstrapToken:
    apiServerEndpoint: k8s.tracklane.internal:6443
    token: abcdef.0123456789abcdef
    caCertHashes:
      - sha256:1a2b3c4d...   # hash of the cluster CA public key
nodeRegistration:
  criSocket: unix:///run/containerd/containerd.sock
  kubeletExtraArgs:
    - name: node-labels
      value: topology.kubernetes.io/zone=ap-southeast-1a
---
# the token object itself lives in kube-system as
# Secret/bootstrap-token-abcdef  type: bootstrap.kubernetes.io/token
```

READING IT

1. **The token is mutual**
   It authenticates the node to the cluster; caCertHashes lets the node verify the cluster before trusting it.
2. **Tokens expire**
   Default TTL is 24 hours. A stale token is the most common reason a join fails months later.
3. **nodeRegistration is durable**
   Labels and the CRI socket set here end up in the kubelet's own configuration on that node.
4. **It is only bootstrap credentials**
   The token is used once, to request a real kubelet client certificate — then it is no longer needed.

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB EVIDENCE

Static Pods and Their Mirror Objects

The API object is a reflection; the manifest on disk is the source of truth.

terminal — kcna-lab20

```
$ kubectl -n kube-system get pod kube-scheduler-kcna-control-plane \
    -o jsonpath='{.metadata.annotations}' | tr ',' '\n'
kubernetes.io/config.source: file
kubernetes.io/config.hash: 9a4c1f7e0b3d2856ac9f4b17e05d3c82
kubernetes.io/config.mirror: 9a4c1f7e0b3d2856ac9f4b17e05d3c82

$ kubectl -n kube-system get pod kube-scheduler-kcna-control-plane \
    -o jsonpath='{.metadata.ownerReferences[0].kind}'
Node

$ kubectl -n kube-system delete pod kube-scheduler-kcna-control-plane
pod "kube-scheduler-kcna-control-plane" deleted
$ kubectl -n kube-system get pod kube-scheduler-kcna-control-plane
NAME                               READY   STATUS    RESTARTS   AGE
kube-scheduler-kcna-control-plane 1/1     Running   0          7s
```

WHAT TO LOOK AT

1. **config.source: file**
   The definitive marker of a static Pod — the kubelet read it from disk, not from the API server.
2. **Owned by the Node**
   Not by a Deployment or DaemonSet. No controller will reschedule it anywhere else.
3. **Deleting it changes nothing**
   AGE resets because a new mirror object was created; RESTARTS stays 0 because the container never stopped.
4. **To actually stop it**
   Move the manifest out of /etc/kubernetes/manifests on the node — which is exactly what an etcd restore does.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

CONCEPT

Static Pods vs Mirror Pods vs Everything Else

A deliberate bootstrapping trick, and a recurring exam question.

Static Pod

1

Defined by a file in the kubelet's staticPodPath. The kubelet starts it with no involvement from the scheduler or API server.

Mirror Pod

2

A read-only API object the kubelet creates so the static Pod is visible to kubectl. It is a reflection, never a controller.

Name suffix

3

The node name is appended: kube-apiserver-<node>. That is how you recognise one at a glance.

Not evictable

4

kubectl drain will not move it; kubectl delete only removes the mirror. The container keeps running.

Why the control plane uses them

5

Bootstrapping: the API server cannot be scheduled by a scheduler that needs the API server to exist.

Legitimate other uses

6

Node-local agents that must run even when the API server is unreachable.

Source: Kubernetes documentation — Creating a cluster with kubeadm; kubeadm init phases; kubeadm PKI certificates and requirements.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN

Stacked etcd vs External etcd

Both are supported HA topologies; the difference is how correlated your failures are.

Dimension	Stacked etcd	External etcd
Where etcd runs	As a static Pod on each control-plane node	On dedicated hosts with their own lifecycle
Node count for HA	3 control-plane nodes total	3 control-plane nodes plus 3 etcd hosts
Failure coupling	Losing a node loses an API server and an etcd member together	The two failure domains are independent
Certificate management	kubeadm generates the etcd PKI for you	You supply the etcd CA and client certificates to kubeadm
Operational cost	Lower — this is the kubeadm default	Higher, but etcd can be tuned, backed up and upgraded separately

CHOOSE

Start stacked. Move to external etcd when etcd's storage profile, backup cadence or upgrade risk genuinely differs from the control plane's.

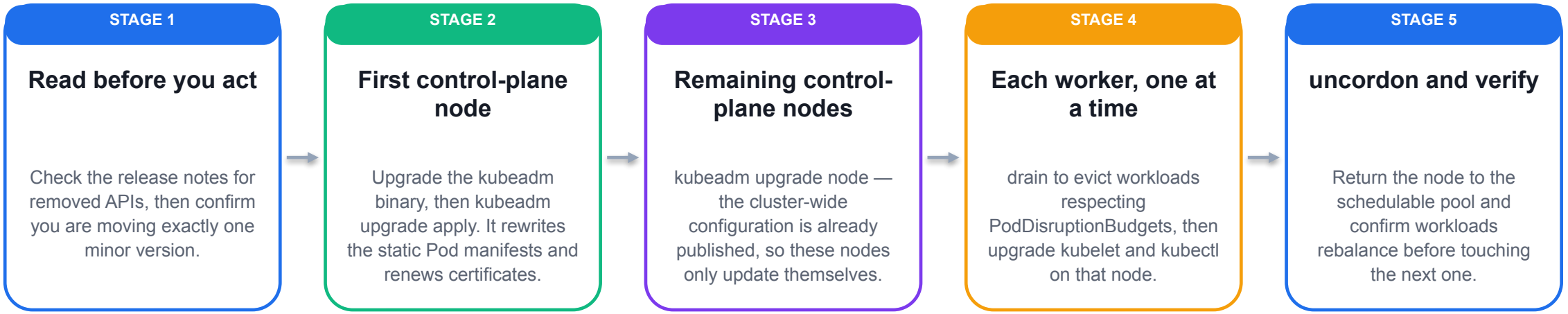
Source: Kubernetes documentation — Options for Highly Available topology (stacked etcd vs external etcd).

© 2026 Tertiary Infotech Academy Pte Ltd

SEQUENCE

Cluster Upgrade Order — and Why That Order

The order is not convention — it is forced by the skew policy.



WHAT IT MEANS

The control plane goes first because the version skew policy permits an older kubelet against a newer API server, but never the reverse.

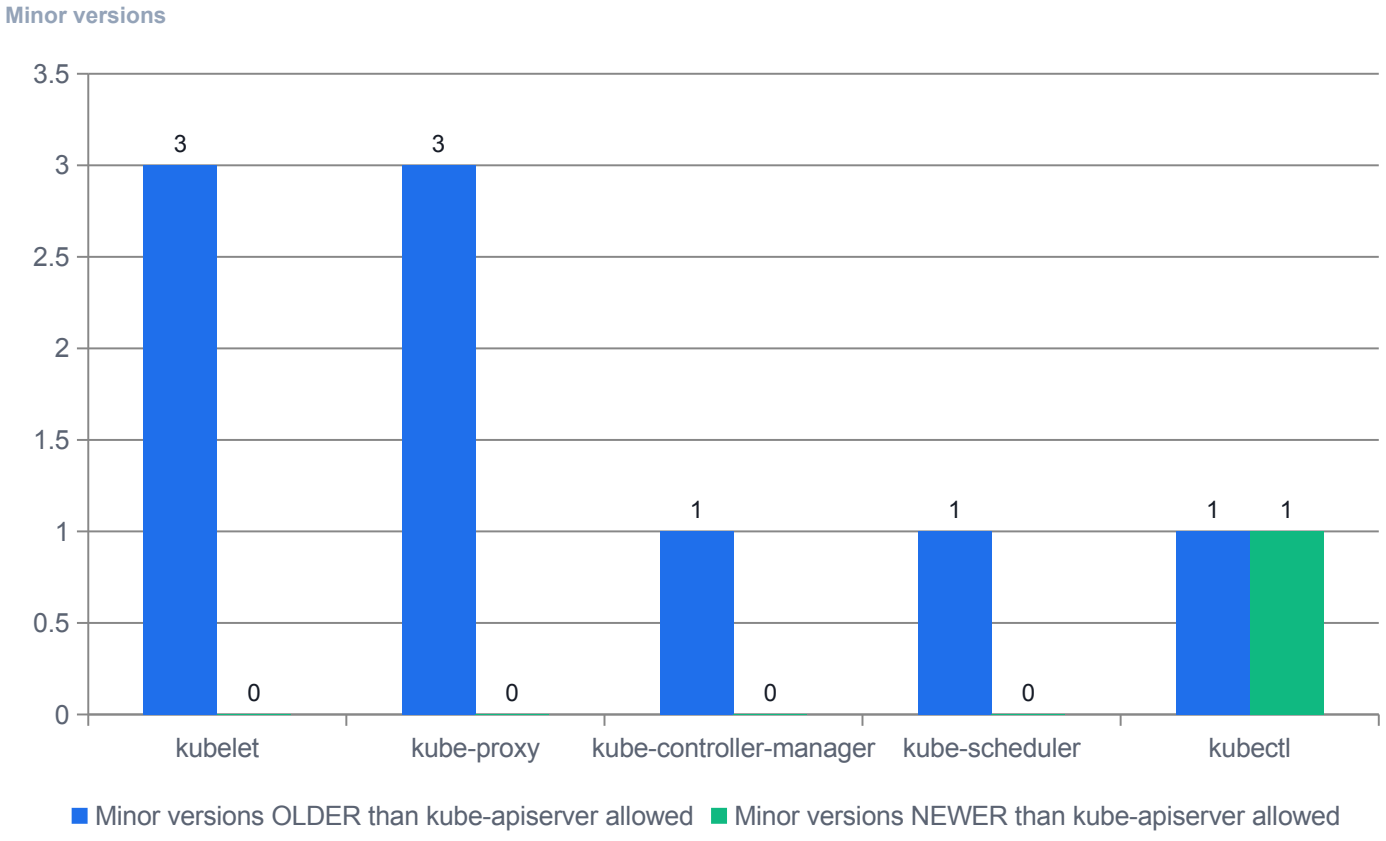
Source: Kubernetes documentation — Upgrading kubeadm clusters; Kubernetes Version Skew Policy (kubernetes.io/releases/version-skew-policy/).

© 2026 Tertiary Infotech Academy Pte Ltd

PUBLISHED POLICY

Version Skew: How Far Apart Components May Be

Read the upgrade order straight off this chart.



WHAT THE DATA SHOWS

Published, not estimated

These are the allowances in the Kubernetes version skew policy, not measured behaviour.

Nothing may outrun the API server

Except kubectl, every component must be at or below the kube-apiserver minor version.

kubelet gets three

The wide kubelet window is what makes rolling node upgrades across a large fleet practical.

Scheduler and controllers get one

They are tightly coupled to API behaviour, so their window is deliberately narrow.

Source: Kubernetes documentation — Upgrading kubeadm clusters; Kubernetes Version Skew Policy (kubernetes.io/releases/version-skew-policy/).

© 2026 Tertiary Infotech Academy Pte Ltd

OPERATIONAL

Upgrade Rules That Bite in Practice

Six rules that account for most upgrade incidents.

1 One minor version at a time

1.30 to 1.32 is two upgrades. Skipping is unsupported and the control plane will refuse it.

2 Never upgrade a kubelet past the API server

The skew policy is one-directional. Control plane first, always.

3 Deprecated APIs break silently

Objects stored under a removed API version fail on read after the upgrade. Audit before, not after.

4 drain respects PodDisruptionBudgets

A PDB that can never be satisfied will block the drain forever — that is the PDB working, not a bug.

5 Static Pod manifests are rewritten

Local edits to /etc/kubernetes/manifests are overwritten by kubeadm upgrade apply. Keep customisation in configuration.

6 Snapshot etcd first

The upgrade path forward is well supported; the path backward is a restore from a snapshot you took beforehand.

Source: Kubernetes documentation — Upgrading kubeadm clusters; Kubernetes Version Skew Policy (kubernetes.io/releases/version-skew-policy/).

© 2026 Tertiary Infotech Academy Pte Ltd

NODE VERBS

cordon, drain and uncordon Are Three Different Things

Draining is a request that other people's disruption budgets are allowed to refuse.

What each verb changes

- cordon sets node.spec.unschedulable: true — nothing is evicted
- drain cordons, then evicts every eligible Pod on the node
- Eviction goes through the Eviction API, so PDBs are honoured
- uncordon clears the flag; it does not bring workloads back
- Nothing here moves a Pod — controllers recreate them elsewhere

What refuses to be drained

- Static and mirror Pods — the kubelet owns them, not a controller
- DaemonSet Pods, unless you pass --ignore-daemonsets
- Bare Pods with no controller, unless you accept losing them
- Pods using local storage, unless you accept deleting the data
- Anything a PodDisruptionBudget currently protects

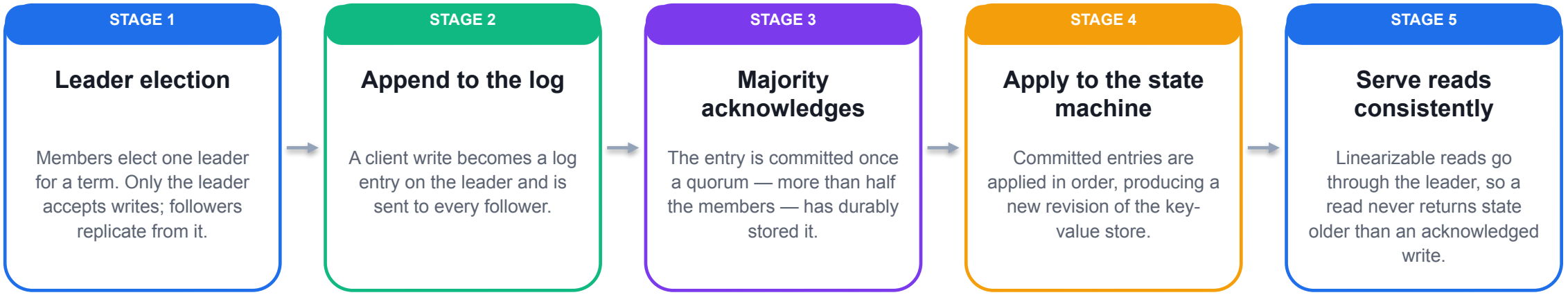
Source: Kubernetes documentation — Upgrading kubeadm clusters; Kubernetes Version Skew Policy (kubernetes.io/releases/version-skew-policy/).

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

How etcd Agrees With Itself — Raft in Five Steps

etcd chooses consistency over availability, and the whole control plane inherits that choice.



WHAT IT MEANS

Availability is bounded by the majority rule: a cluster that cannot form a quorum stops accepting writes rather than risk divergence.

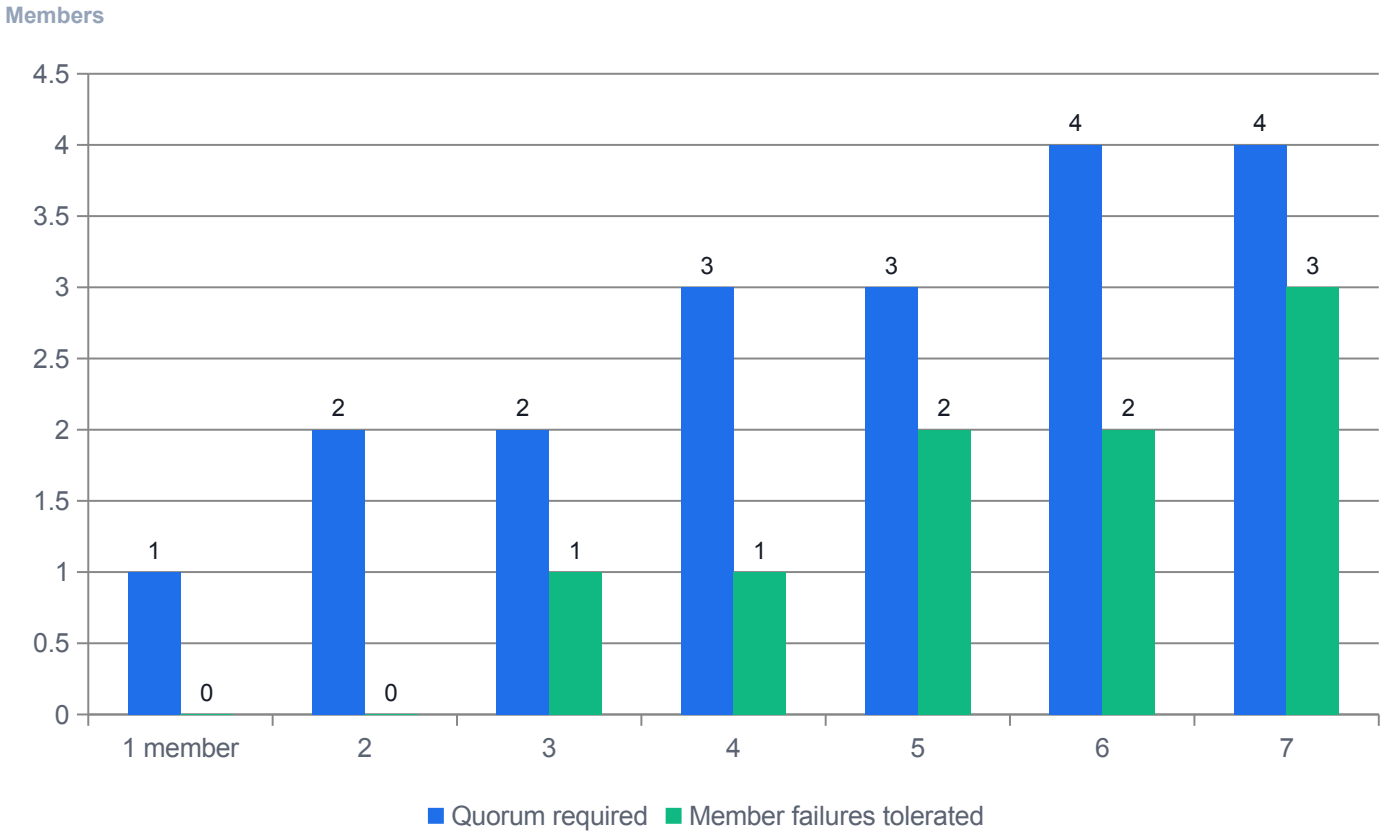
Source: Kubernetes documentation — Operating etcd clusters for Kubernetes; etcd.io documentation (FAQ, Raft, etcdutl).

© 2026 Tertiary Infotech Academy Pte Ltd

EXACT ARITHMETIC

Why etcd Clusters Have an Odd Number of Members

Quorum = floor(n/2) + 1. Everything else on this slide follows from that.



WHAT THE DATA SHOWS

Pure arithmetic

Quorum is floor(n/2)+1 and tolerance is n minus quorum. No measurement is involved — this is the Raft majority rule.

Even counts add nothing

4 members tolerate the same single failure as 3, while adding another thing that can fail.

3 is the common answer

It survives one member loss, which covers node maintenance and a single host failure.

5 for larger estates

Two simultaneous failures tolerated, at the cost of more replication traffic per write.

Source: Kubernetes documentation — Operating etcd clusters for Kubernetes; etcd.io documentation (FAQ, Raft, etcdutl).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Reading the etcd Static Pod Manifest

Nine flags explain the entire storage layer of a Kubernetes cluster.

etcd.yaml — as read from the runtime cluster

```
# /etc/kubernetes/manifests/etcd.yaml  (extract)
spec:
  containers:
  - image: registry.k8s.io/etcd:3.5.15-0
    command:
    - etcd
    - --data-dir=/var/lib/etcd
    - --listen-client-urls=https://127.0.0.1:2379,
                           https://172.18.0.2:2379
    - --advertise-client-urls=https://172.18.0.2:2379
    - --listen-peer-urls=https://172.18.0.2:2380
    - --initial-cluster=kcna-control-plane=https://172.18.0.2:2380
    - --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
    - --cert-file=/etc/kubernetes/pki/etcd/server.crt
    - --key-file=/etc/kubernetes/pki/etcd/server.key
    - --client-cert-auth=true
```

READING IT

1. **Two ports, two purposes**
   2379 serves clients (only the API server should be one); 2380 carries member-to-member replication.

2. **--data-dir is the whole cluster**
   Every object in your Kubernetes API lives under this directory. Back it up, and never share it between members.

3. **--initial-cluster lists members**
   One entry here means a single-member cluster: quorum 1, zero failures tolerated.

4. **Client cert auth is mandatory**
   etcd trusts a separate CA from the cluster CA, and requires a client certificate for every connection.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

Verifying a Snapshot Before You Trust It

A backup you have never verified is a hypothesis, not a backup.

terminal — kcna-lab20

```
$ etcdctl --endpoints=https://127.0.0.1:2379 \
    --cacert=/etc/kubernetes/pki/etcd/ca.crt \
    --cert=/etc/kubernetes/pki/etcd/server.crt \
    --key=/etc/kubernetes/pki/etcd/server.key \
    snapshot save /var/lib/etcd/kcna-lab20-snapshot.db
{"level":"info","msg":"saved",
 "path":"/var/lib/etcd/kcna-lab20-snapshot.db"}
Snapshot saved at /var/lib/etcd/kcna-lab20-snapshot.db

$ etcdutl snapshot status \
    /var/lib/etcd/kcna-lab20-snapshot.db -w table
+----------+----------+------------+------------+
|   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
+----------+----------+------------+------------+
| a41f7b2e |    38412 |       1187 |     7.4 MB |
+----------+----------+------------+------------+
```

WHAT TO LOOK AT

1. **HASH is the integrity check**
   It is computed from the file contents. A snapshot that will not hash is a snapshot you cannot restore.

2. **REVISION is the point in time**
   38412 is the global etcd revision captured. Restoring rewinds the cluster to exactly this revision.

3. **TOTAL KEYS is a sanity signal**
   1187 keys for a small single-node cluster. A sudden drop between backups is worth investigating.

4. **etcdutl, not etcdctl**
   snapshot status and snapshot restore moved to etcdutl — they are file operations and need no running cluster.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

UNDERSTAND, DO NOT RUN

Restore Is a Node Operation, Not an API Call

Read this as a mechanism. The keystrokes belong in a change window with an approved runbook, not in a training lab.

STAGE 1

Stop the control plane

The static Pod manifests are moved out of /etc/kubernetes/manifests so the kubelet tears the containers down. Nothing is writing to etcd.

STAGE 2

Restore into a new directory

etcdutl snapshot restore writes a fresh data directory. The original is left untouched so the attempt is reversible.

STAGE 3

Repoint the data directory

The etcd manifest's --data-dir (or the hostPath behind it) is moved to the restored copy.

STAGE 4

Restore the manifests

The kubelet sees the files return and restarts etcd, then the API server, controller-manager and scheduler.

STAGE 5

Verify before declaring success

/readyz, node readiness and a spot check of workloads. Only then is the cluster considered recovered.

WHAT IT MEANS

NOT RUNNABLE IN CLASS. This is a node-level, cluster-wide, destructive procedure; it is presented so you can reason about it, and it is never executed against a shared cluster.

Source: Kubernetes documentation — Operating etcd clusters for Kubernetes; etcd.io documentation (FAQ, Raft, etcdutl).

© 2026 Tertiary Infotech Academy Pte Ltd

SCOPE

What an etcd Snapshot Does and Does Not Contain

A complete cluster recovery plan needs at least three backups, and etcd is only one of them.

1 Every API object

Pods, Deployments, Secrets, RBAC, CRDs and their instances, at one consistent revision.

2 Not your PersistentVolume data

The snapshot holds the PV and PVC objects, not the bytes on the storage backend. Those need their own backup.

3 Not container images

A restore assumes the registry still serves every image the restored objects reference.

4 Not node-local state

Certificates, kubelet configuration and static Pod manifests live on disks the snapshot never saw.

5 Encrypted Secrets stay encrypted

If encryption at rest is enabled, the snapshot is useless without the EncryptionConfiguration key material.

6 It rewinds deletions too

Objects created after the snapshot revision disappear on restore. That is the property people forget under pressure.

Source: Kubernetes documentation — Operating etcd clusters for Kubernetes; etcd.io documentation (FAQ, Raft, etcdutl).

© 2026 Tertiary Infotech Academy Pte Ltd

POLICY

Turning 'We Back Up etcd' Into Something Testable

RPO and RTO are the two numbers that turn a wish into an engineering requirement.

RPO	30 minutes — the maximum acceptable data loss
RTO	60 minutes — the maximum acceptable recovery time
Snapshot cadence	*/30 * * * * — derived from the RPO, not guessed
Snapshot command	etcdctl snapshot save against a live member
Verification	etcdutl snapshot status on every snapshot taken

On-node copy	/var/lib/etcd-backup for fast local restore
Off-site copy	Required — a node failure must not destroy the backup
Retention	48 hourly, 14 daily, 12 monthly
Restore rehearsal	On a disposable cluster, on a schedule
Owner	Named, because an unowned backup policy is not a policy

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTERED TOPIC

The NetworkPolicy Model in Six Facts

A small API with two behaviours that surprise almost everyone.

1 Namespaced and Pod-selecting

A policy lives in one namespace and applies to the Pods its podSelector matches in that namespace.

2 Additive allow-lists only

There is no deny rule. Policies union together; a connection is permitted if any policy allows it.

3 Selection flips the default

A Pod nobody selects accepts everything. The moment one policy selects it for a direction, everything else is denied.

4 policyTypes is explicit

Ingress and Egress are controlled independently. An ingress-only policy leaves egress completely open.

5 It operates on Pod IPs

Rules are evaluated against endpoints, not Service names — the Service VIP is not what the policy sees.

6 The CNI must enforce it

The API accepts a NetworkPolicy on any cluster. Whether packets are actually dropped depends entirely on the plugin.

Source: Kubernetes documentation — Network Policies (networking.k8s.io/v1 NetworkPolicy).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

Default-Deny Plus One Deliberate Allow

Two objects: one to close the namespace, one to open exactly the path you intended.

netpol-default-deny.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny-all, namespace: kcna-lab16}
spec:
  podSelector: {}           # every Pod in the namespace
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: allow-api-from-web, namespace: kcna-lab16}
spec:
  podSelector: {matchLabels: {app: api}}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: {matchLabels: {app: web}}
    ports:
    - {protocol: TCP, port: 8080}
```

READING IT

1. **Empty podSelector means all**
   {} selects every Pod in the namespace — the idiom for a namespace -wide default.

2. **No ingress key means deny**
   Listing the policyType without any rules is what produces the default-deny behaviour.

3. **The second policy adds back**
   Union semantics: web to api on 8080 is now allowed, and nothing else is.

4. **Remember DNS**
   A default-deny egress policy blocks CoreDNS on UDP/TCP 53. Almost every 'my app broke' report starts here.

Source: Kubernetes documentation — Network Policies (networking.k8s.io/v1 NetworkPolicy).

© 2026 Tertiary Infotech Academy Pte Ltd

DEBUGGING

Why a Correct-Looking Policy Does Nothing

There is no event, no condition and no status field to tell you a policy is inert. You have to test it.

1 The CNI does not enforce

kind's default CNI accepts NetworkPolicy objects without enforcing them. Calico or Cilium is required to see drops.

2 Egress DNS is blocked

Default-deny egress without a rule for UDP/TCP 53 to kube-dns breaks name resolution before any connection is attempted.

3 The selector matches nothing

A label typo means no Pod is selected, so the default-allow behaviour silently continues.

4 Wrong namespace

The policy is namespaced. Applied next door, it protects the wrong workload and reports no error.

5 hostNetwork Pods bypass it

Traffic on the node's own network stack is not subject to Pod network policy.

6 You are testing via a Service

Policy is evaluated on the Pod IP after Service translation — test with the endpoint in mind, not the VIP.

Source: Kubernetes documentation — Network Policies (networking.k8s.io/v1 NetworkPolicy).

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 20 — Control Plane Anatomy and etcd Backup/Restore

GOAL

Open the control plane that has been running every lab so far: read the static Pod manifests the kubelet acts on, trace how apiserver, etcd, scheduler and controller-manager interact, then take and verify a real etcd snapshot to a file — and read, without executing, the destructive restore procedure.

YOU WILL PRODUCE

- A verified etcd snapshot file with etcdctl snapshot save and a snapshot status table showing hash, revision, total keys and size.
- An annotated control-plane inventory generated from the platform team's dataset, naming the exact mirror-pod objects on this node.
- A version-controlled etcd backup policy (RPO/RTO, retention, off-node copy, what a restore does not recover) applied as a cluster object and read from inside the cluster.

CONCEPTS EXERCISED

- Control-plane components and how apiserver, etcd, scheduler, controller-manager and kubelet interact
- Static Pods, the kubelet manifest directory, and mirror Pods in the API server
- etcd as the single source of cluster state, and snapshot save / snapshot status
- Backup policy, RPO/RTO, and the read-only boundary around a destructive restore

LAB	NAMESPACE	MAPS TO	TIME
Lab 20	kcna-lab20	LO5 · A5 · K5	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-20-cluster-architecture-etcd/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Afternoon Break

3:50 – 4:00 PM

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA DOMAIN · 16%

07

Cloud Native Application Delivery

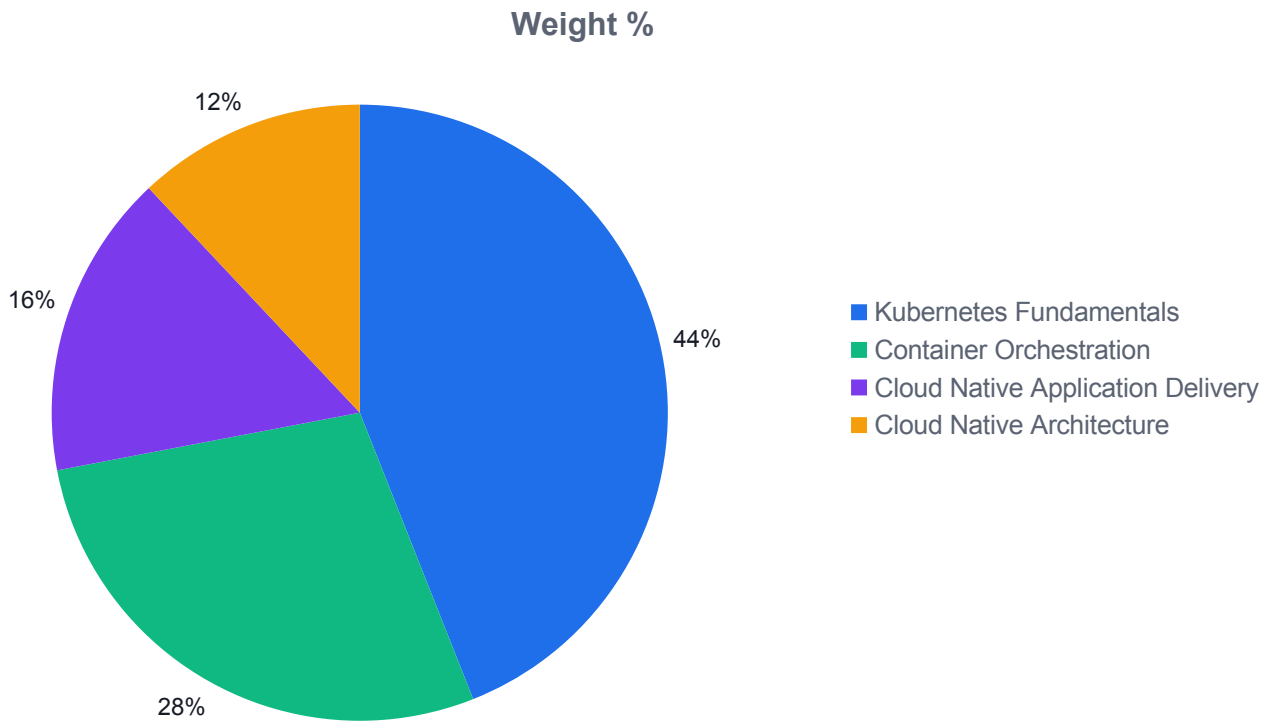
Packaging with Helm and Kustomize, delivering with GitOps, and debugging the gap between 'applied' and 'working'.

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM CONTEXT

Where Application Delivery Sits in the KCNA Blueprint

Two domains meet today: 28% Container Orchestration this morning, 16% Application Delivery now.



WHAT THE DATA SHOWS

16% is this afternoon

Cloud Native Application Delivery covers Application Delivery and Debugging — roughly ten of sixty questions.

Published weights

Taken from the CNCF curriculum revision effective 24 Nov 2025, not from an older four-way split.

Storage sits elsewhere

This morning's storage material is examined under Container Orchestration, which carries 28%.

Breadth over depth

Delivery questions test whether you can distinguish Helm, Kustomize and GitOps — not whether you can author charts.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

THE PROBLEM

Why 'kubectl apply -f' Stops Working at Scale

Packaging solves the first two problems; GitOps solves the rest.

1 Environments multiply

Dev, staging and production need the same twelve objects with different replicas, limits, hostnames and feature flags.

2 Copy-paste diverges

Three directories of near-identical YAML drift within weeks, and nobody can tell which difference was intentional.

3 Nobody knows what is deployed

The cluster reflects whoever last ran apply from whichever laptop, with whatever branch was checked out.

4 Rollback is improvised

Reverting means finding and reapplying the previous YAML — if anyone kept it.

5 Credentials spread

Every engineer and every pipeline that applies manifests needs cluster-admin-shaped access.

6 Manual changes are invisible

An emergency kubectl edit survives until the next deploy, then silently vanishes.

Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

VOCABULARY

CI, CD and GitOps Are Three Different Things

Continuous integration never deploys anything. That is the distinction most people blur.

Question	CI	CD (push)	GitOps (pull)
What triggers it	A commit or pull request	A successful CI run	A change in the Git repository, observed
What it produces	A tested, signed artifact and image	A cluster mutated by the pipeline	A cluster continuously converged toward Git
Who holds cluster credentials	Nobody — it never touches a cluster	The pipeline runner, from outside	The in-cluster controller only
Direction of travel	n/a	Pipeline pushes into the cluster	Controller pulls from the repository
What happens to drift	n/a	Nothing until the next deployment	Detected continuously and optionally reverted

CHOOSE

CI proves the artifact is good. CD or GitOps decides how it reaches a cluster. They are routinely conflated in interviews and in exams.

Source: OpenGitOps v1.0.0 Principles (opengitops.dev), CNCF App Delivery TAG.

© 2026 Tertiary Infotech Academy Pte Ltd

END TO END

The Delivery Chain From Commit to Running Pod

Two repositories, one direction of flow, one auditable record.



WHAT IT MEANS

The deployable unit is a Git commit, so 'what is running' and 'who changed it' are answered by git log rather than by memory.

Source: OpenGitOps v1.0.0 Principles (opengitops.dev), CNCF App Delivery TAG.

© 2026 Tertiary Infotech Academy Pte Ltd

Chart Anatomy — What Is in the Directory

A chart is a directory with a contract, not a magic format.

1 Chart.yaml

apiVersion: v2, the chart name, version (the package version) and appVersion (the software version). Two different numbers.

2 values.yaml

The default configuration surface. Everything a consumer may reasonably override belongs here, documented.

3 templates/

Go templates that render Kubernetes manifests. Text templating, which is both the power and the problem.

4 templates/_helpers.tpl

Named templates included elsewhere — fullname, selectorLabels, validation. Underscore means 'renders nothing itself'.

5 charts/ and Chart.lock

Vendored subchart dependencies and their pinned versions, resolved by helm dependency update.

6 crds/ and NOTES.txt

CRDs are installed before templates and never upgraded by Helm; NOTES.txt prints on install but is not part of the render.

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Helm Template With Values Interpolation

Four lines of templating and one genuinely important idiom.

templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "tracklane.fullname" . }}
  labels:
    {{- include "tracklane.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath
          "/configmap.yaml") . | sha256sum }}
    spec:
      containers:
      - name: {{ .Chart.Name }}
        image: {{ .Values.image | quote }}
        envFrom:
        - configMapRef:
            name: {{ include "tracklane.fullname" . }}-config
```

READING IT

1. **Three built-in objects**
   .Values comes from values files and --set; .Chart from Chart.yaml; .Release from the install invocation.

2. **include beats template**
   include returns a string, so it can be piped — which is why nindent works here and would not with template.

3. **checksum/config is the trick**
   Hashing the rendered ConfigMap into a Pod annotation forces a rollout whenever configuration changes.

4. **Without it, envFrom is stale**
   Environment variables are read once at container start; updating the ConfigMap alone changes nothing.

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

REFERENCE

Helm Values Precedence and Release State

Two mechanisms to memorise: how values merge, and where the release lives.

Lowest precedence	The chart's own values.yaml
Then	Values from a parent chart, for subcharts
Then	-f / --values files, applied left to right
Then	--set key=value on the command line
Highest precedence	--set-file key=path, for file contents

Where a release is stored	A Secret in the release namespace: sh.helm.release.v1.<name>.v<n>
Revision history	helm history shows every revision and its status
Rollback	helm rollback <release> <revision> reapplies a stored manifest
Chart version vs appVersion	The package version and the software version move independently
Server-side state	There is no Tiller: Helm 3 is a client that stores state as Secrets

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

helm template vs helm install vs helm upgrade

One of these is safe to run against production by accident.

Behaviour	helm template	helm install / upgrade
Contacts the cluster	No — pure local rendering	Yes; authenticates with your kubeconfig
Records a release	No	Yes; a new revision Secret every time
Runs hooks	No	Yes; pre-install, post-upgrade and the rest
lookup function	Returns empty — nothing to look up	Queries live objects during rendering
Right place for it	CI: render, diff, validate with kubeconform, commit the output	The moment you actually intend to change a cluster

CHOOSE

Render in CI with helm template; let something accountable apply the result. --dry-run is not the same thing: it still talks to the API server.

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

The Same Chart, Two Values Files

Conditional resources are what Kustomize cannot do and Helm can.

terminal — kcna-lab21

```
$ helm template tracklane ./chart -f values-dev.yaml \
    | grep -E 'replicas|cpu:|LOG_LEVEL|kind:'
kind: ConfigMap
kind: Service
kind: Deployment
  replicas: 1
            cpu: 50m
          value: debug

$ helm template tracklane ./chart -f values-prod.yaml \
    | grep -E 'replicas|cpu:|LOG_LEVEL|kind:'
kind: PodDisruptionBudget      # rendered only for prod
kind: ConfigMap
kind: Service
kind: Deployment
  replicas: 3
            cpu: 100m
          value: warn
```

WHAT TO LOOK AT

1. **One chart, two renders**
   No YAML was duplicated. The difference between environments is a values file, reviewable on its own.

2. **Values changed the numbers**
   replicas 1 to 3, cpu 50m to 100m, log level debug to warn — all from .Values interpolation.

3. **A template changed the shape**
   The PodDisruptionBudget exists only in prod, produced by an if guard around the whole file.

4. **This is CI's job**
   Render both, validate both against the schema, and diff them before anything reaches a cluster.

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

HONEST ASSESSMENT

Where Helm Hurts

Helm is the default for distributing software to strangers — and that is a different job from configuring your own.

1 It templates text, not YAML

The renderer has no idea it is producing YAML until the output is parsed, so indentation bugs are runtime surprises.

2 Whitespace control is a skill

{{- and -}} and nindent are a real learning curve that has nothing to do with Kubernetes.

3 Values sprawl

Mature charts expose hundreds of keys; consumers cannot tell which combinations were ever tested.

4 Diffs are hard to read

A pull request shows a changed value, not the resulting manifest change. Render in CI to recover the real diff.

5 CRD lifecycle is awkward

Files under crds/ are installed once and never upgraded or deleted by Helm.

6 Hook ordering surprises

Weights, deletion policies and failed hooks produce behaviour that is hard to reason about mid-incident.

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

KUSTOMIZE

Base Plus Overlays, With No Templating by Design

Everything is a structured edit to real YAML, which is why there is no template syntax to learn.

kustomization.yaml

1

The only entry point. It lists resources, transformers, generators and patches for one variant.

Base

2

Plain, valid, applyable Kubernetes YAML. You can kubectl apply the base directly and it works.

Overlay

3

A directory that references the base and describes only the differences for one environment.

Transformers

4

namePrefix, nameSuffix, namespace, images, replicas and labels — declarative field rewrites, not string substitution.

Generators

5

configMapGenerator and secretGenerator build objects from literals or files, with a content hash in the name.

Built into kubectl

6

kubectl kustomize renders and kubectl apply -k applies. No additional binary is required.

Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

A Production Overlay, Field by Field

Sixteen lines describe an entire production variant.

overlays/prod/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
nameSuffix: -prod
replicas:
  - name: tracklane
    count: 3
images:
  - name: nginx
    newName: docker.io/library/nginx
    newTag: 1.27-alpine
labels:
  - pairs: {app.kubernetes.io/instance: tracklane-prod}
    includeSelectors: true
patches:
  - path: patch-resources.yaml      # strategic merge
  - path: patch-probe-timing.yaml   # RFC 6902 JSON patch
```

READING IT

1. **resources points at the base**
   A relative path. The overlay never copies the base; it references and edits it.

2. **Transformers are typed**
   replicas and images are structured edits — Kustomize knows which field to change, so a rename cannot corrupt the manifest.

3. **includeSelectors: true is load-bearing**
   It writes the instance label into selectors as well, which is what stops dev and prod Deployments adopting each other's Pods.

4. **Two patch dialects, one list**
   Kustomize infers strategic-merge or JSON 6902 from the file's shape.

Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Strategic Merge Patch vs JSON 6902 Patch

Both are supported in one patches list; the file's shape decides which is used.

Dimension	Strategic merge patch	JSON 6902 patch
Shape of the file	A partial Kubernetes object, YAML map	A list of {op, path, value} operations
How it finds a list item	By merge key — containers match on name	By index: /spec/template/spec/containers/0
Can it remove a field	Not really; null handling is inconsistent	Yes — op: remove is explicit
If the list is reordered	Still correct; matching is by key	Silently patches the wrong element
Readability	High — it looks like the object you are editing	Low, but unambiguous and precise

CHOOSE

Reach for strategic merge first. Use JSON 6902 when you need removal, precise ordering, or a field the merge keys cannot express.

Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

A Kustomize Patch That Matches Nothing Says Nothing

The most instructive failure of the day: correct syntax, successful command, wrong cluster.

SYMPTOM

The overlay renders successfully, exits 0, and quietly emits the unpatched base.

EVIDENCE — actual cluster output

```
$ head -4 overlays/prod/patch-resources.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: tracklane-web        # the base is named 'tracklane'

$ kubectl kustomize overlays/prod | grep -A2 'resources:'
        resources:
          requests:
            cpu: 50m         # expected 100m

$ diff <(kubectl kustomize base) \
       <(kubectl kustomize overlays/prod) | grep cpu
$ echo $?
1                            # no difference at all
```

ROOT CAUSE

The patch target name does not match any resource in the base. Kustomize applies the patch to zero objects, which is not an error condition, so the render succeeds with the base values intact.

FIX

- Correct the patch target's kind, name and namespace to match the base
- Never trust exit code 0 from a renderer — assert on the rendered output instead
- Add a CI check that greps the render for the values the overlay is supposed to produce

Source: Course lab evidence captured on the isolated runtime cluster, Kubernetes v1.37.0 (courseware/qa runtime reports).

© 2026 Tertiary Infotech Academy Pte Ltd

THREE OPTIONS

Raw Manifests, Helm and Kustomize Side by Side

They are not rivals in practice — Argo CD and Flux will render either, and charts are frequently post-processed with Kustomize.

Raw manifests

- No tooling, no abstraction, no surprises
- Every environment is a full copy that will drift
- Perfect for a handful of objects or a demo
- Still the substrate everything else renders to

kubectl apply -f | no indirection

Helm

- Versioned, redistributable packages with dependencies
- Conditional resources and loops via Go templates
- Release history and helm rollback
- The default way third parties ship software

chart + values | release state

Kustomize

- Base plus overlays, no template language at all
- Structured, typed edits to real YAML
- Built into kubectl; nothing extra to install
- Best for your own applications across environments

overlays | in kubectl

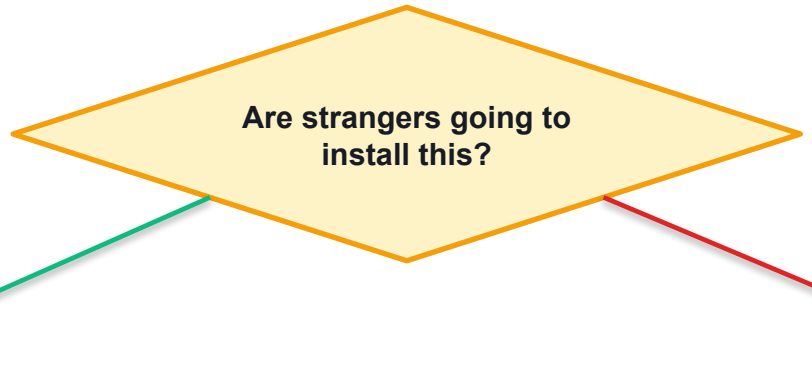
Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

DESIGN DECISION

Helm or Kustomize for This Repository?

Distribution is the axis that actually decides this, not personal preference.



Yes — package it as a Helm chart

- A versioned artifact you can publish to a registry
- Consumers configure through a documented values surface
- Conditional resources for optional components
- Dependencies and release history come for free

No — it is your own app: use Kustomize

- Base plus one overlay per environment, all in your repository
- Diffs stay readable because there is no template layer
- kubectl kustomize needs no extra tooling in CI
- Escalate to Helm only when you need conditionals or loops

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

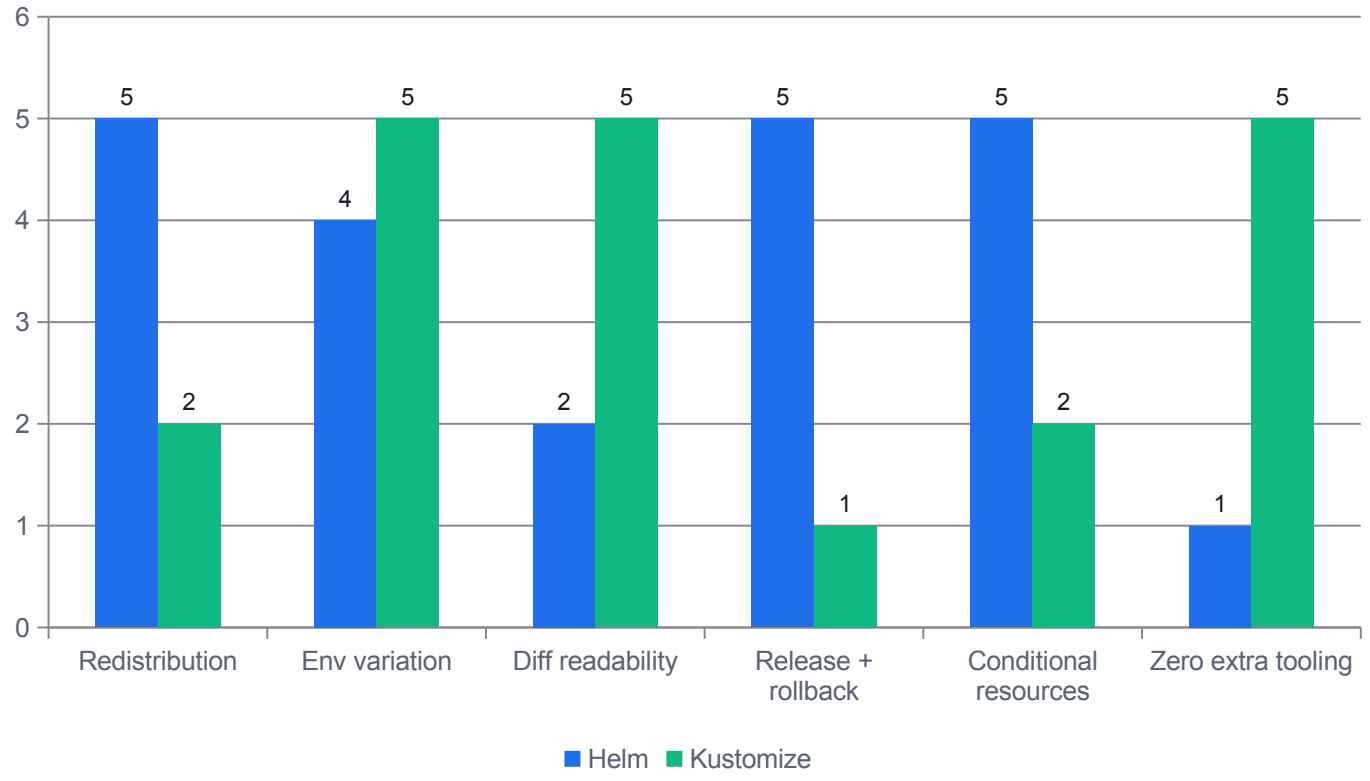
© 2026 Tertiary Infotech Academy Pte Ltd

EVALUATION FRAMEWORK

Scoring Helm Against Kustomize on Six Criteria

A decision aid you can defend in a design review — clearly labelled as judgement, not measurement.

Fit score (1 = poor, 5 = strong)



WHAT THE DATA SHOWS

This is a rubric, not data

Scores are an authored 1-5 evaluation framework for classroom comparison. Nothing here was measured.

The profiles are complementary

Each tool wins clearly on three criteria — which is why real platforms often use both.

Weight the criteria yourself

Redistribution dominates for a vendor; diff readability dominates for a platform team under review.

Use it as a conversation

The value is in arguing about the weights with your own constraints, not in the totals.

Source: Helm documentation — Charts, Values, Templates (helm.sh/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

THE DISTINCTION THAT MATTERS

GitOps is not a pipeline that runs kubectl. It is a controller inside the cluster that never stops making reality match Git.

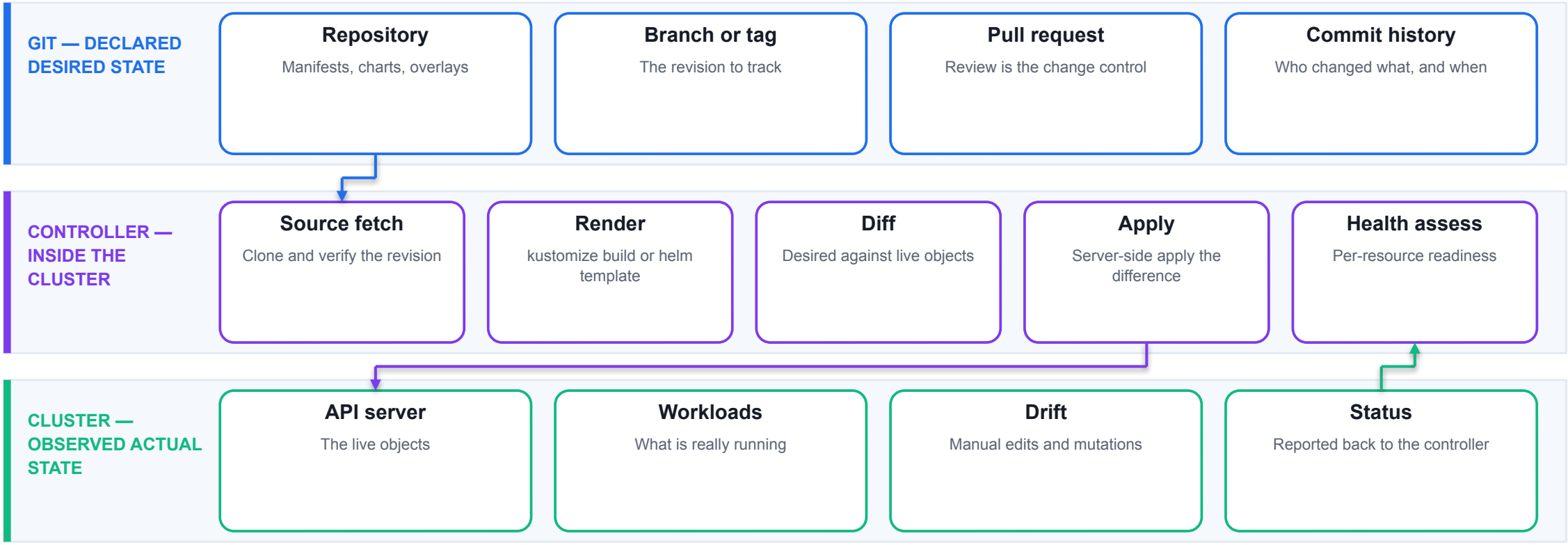
That single architectural change is what produces drift detection, self-healing and an audit trail you did not have to build.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE

The GitOps Reconciliation Loop

A closed loop with no external actor holding credentials — the same control-loop pattern as every other Kubernetes controller.



Source: OpenGitOps v1.0.0 Principles (opengitops.dev), CNCF App Delivery TAG.

© 2026 Tertiary Infotech Academy Pte Ltd

PRINCIPLES

The Four OpenGitOps Principles

Four sentences, maintained by the CNCF OpenGitOps project, and commonly examined.

1 Declarative

The entire desired state of the system is expressed declaratively — no imperative deployment scripts.

2 Versioned and immutable

Desired state is stored so that it is versioned, immutable and retains a complete version history.

3 Pulled automatically

Software agents automatically pull the declared state from source. Nothing external pushes into the cluster.

4 Continuously reconciled

Agents continuously observe actual state and attempt to apply the declared state — not once at deploy time.

5 What follows from these

Rollback is git revert, audit is git log, and the deployment record is the repository itself.

6 What is NOT implied

GitOps says nothing about Git specifically, nor about which renderer, nor about progressive delivery.

Source: OpenGitOps v1.0.0 Principles (opengitops.dev), CNCF App Delivery TAG.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Push Delivery vs Pull Delivery

Same YAML, same registry, very different blast radius.

Dimension	Push (pipeline applies)	Pull (controller reconciles)
Cluster credentials	Held by the CI system, outside the cluster's trust boundary	Held only by a controller running inside the cluster
Network direction	Inbound: CI must reach the API server	Outbound: the cluster reaches the repository and registry
Drift detection	None between deployments	Continuous, on the reconcile interval
Recovering a cluster	Re-run the pipeline and hope it is idempotent	Point a controller at the repository; it rebuilds the state
Scaling to many clusters	One pipeline target per cluster, with one credential each	One controller per cluster, each pulling the paths it owns

CHOOSE

Pull inverts the trust direction: no external system needs standing write access to your cluster. That is the security argument, and it is the strongest one.

Source: OpenGitOps v1.0.0 Principles (opengitops.dev), CNCF App Delivery TAG.

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

An Argo CD Application, Read Field by Field

Twenty lines that replace an entire deployment pipeline stage.

applications/tracklane-dev.yaml

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tracklane-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: default
  source:
    repoURL: https://github.com/tertiarycourses/...
    targetRevision: main
    path: kustomize/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: kcna-lab21
  syncPolicy:
    automated: {prune: true, selfHeal: true, allowEmpty: false}
    syncOptions: [CreateNamespace=true, ServerSideApply=true]
```

READING IT

1. **The Application is the contract**
   One custom resource names a source revision, a path and a destination. That triple is the entire reconciliation contract.

2. **path implies the renderer**
   Argo CD detects kustomization.yaml and runs Kustomize; a Chart.yaml would make it render Helm instead.

3. **prune and selfHeal are the teeth**
   prune deletes objects removed from Git; selfHeal reverts manual edits. Without both, drift simply accumulates.

4. **targetRevision: main is a smell**
   A mutable branch means the deployed revision changes without a commit to this file. Pin a tag or a digest in production.

Source: Argo CD documentation — Application specification and Automated Sync Policy (argo-cd.readthedocs.io).

© 2026 Tertiary Infotech Academy Pte Ltd

WORKED EXAMPLE

The Same Contract Expressed in Flux

Different objects, identical contract: a revision, a path, a destination, and prune.

flux/tracklane-dev.yaml

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata: {name: tracklane, namespace: flux-system}
spec:
  interval: 1m                # how often to FETCH
  url: https://github.com/tertiarycourses/...
  ref:
    branch: main
---
apiVersion: kustomize.toolkit.fluxcd.io/v1
kind: Kustomization
metadata: {name: tracklane-dev, namespace: flux-system}
spec:
  interval: 5m                # how often to RECONCILE
  sourceRef: {kind: GitRepository, name: tracklane}
  path: ./kustomize/overlays/dev
  targetNamespace: kcna-lab21
  prune: true
```

READING IT

1. **Flux splits source from apply**
   GitRepository fetches and verifies; Kustomization renders and applies. Argo CD combines both in one Application.

2. **Two independent intervals**
   Fetching every minute and reconciling every five minutes are separate decisions with separate costs.

3. **prune is the same idea**
   Objects that leave the repository leave the cluster. It is opt-in in both tools for exactly the same reason.

4. **Not the kubectl Kustomization**
   This is kustomize.toolkit.fluxcd.io — a controller CR, not the kustomization.yaml in your repository.

Source: Flux documentation — GitRepository and Kustomization APIs (fluxcd.io/flux).

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Argo CD and Flux — Both CNCF Graduated

This is a genuine either/or, and the exam only expects you to know they are the two graduated implementations.

Dimension	Argo CD	Flux
Core custom resource	Application (and ApplicationSet)	GitRepository / OCIRepository plus Kustomization or HelmRelease
Shape of the design	One object per application, opinionated	Composable single-purpose controllers
Operator experience	A first-class web UI showing the resource tree	CLI and Kubernetes events; UI comes from third parties
Multi-tenancy	AppProject scopes sources, destinations and kinds	Namespaces plus per-Kustomization service accounts
Helm handling	Renders the chart, then applies the output	HelmRelease drives a real Helm release with history

CHOOSE

Both are CNCF graduated and both implement the same principles. Choose on operating model — a visual console, or composable controllers.

Source: CNCF Graduated project listing (cncf.io/projects); CNCF Artwork repository, Linux Foundation Trademark Usage Guidelines.

© 2026 Tertiary Infotech Academy Pte Ltd

DEBUGGING DELIVERY

Why 'Synced' and 'Healthy' Are Two Questions

Two orthogonal axes produce four states, and only one of them means 'finished'.

1 Sync compares manifests

It asks whether the live objects match the rendered desired state. It says nothing about whether they work.

2 Health assesses resources

A per-kind readiness judgement: a Deployment is healthy when its available replicas match the desired count.

3 Synced + Progressing

The apply succeeded and the rollout is still in flight. Wait, then look again.

4 Synced + Degraded

The most common confusion. Git was applied faithfully and the workload is still broken — the manifest itself is wrong.

5 OutOfSync + Healthy

Something works but does not match Git. Usually a manual edit, or a mutating webhook writing fields back.

6 Missing

The object is in Git but not in the cluster. Look for admission rejection, quota, or a namespace that does not exist.

Source: Argo CD documentation — Application specification and Automated Sync Policy (argo-cd.readthedocs.io).

© 2026 Tertiary Infotech Academy Pte Ltd

FAILURE EVIDENCE

Synced but Degraded: GitOps Did Its Job Perfectly

The whole debugging skill is knowing which layer a red status actually indicts.

SYMPTOM

The Application reports Synced with no sync errors, yet no Pod of the new release is serving traffic.

EVIDENCE — actual cluster output

```
$ kubectl -n argocd get application tracklane-dev
NAME            SYNC STATUS   HEALTH STATUS
tracklane-dev   Synced        Degraded

$ kubectl -n kcna-lab21 get deploy tracklane-dev
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
tracklane-dev   0/3     3            0           6m

$ kubectl -n kcna-lab21 get pvc
NAME            STATUS    VOLUME   CAPACITY   STORAGECLASS
tracklane-data  Pending                       fast-ssd

$ kubectl -n kcna-lab21 describe pvc tracklane-data | tail -2
  Warning  ProvisioningFailed   storageclass.storage.k8s.io
                                "fast-ssd" not found
```

ROOT CAUSE

Synced only means the live objects match Git. The committed manifest references a StorageClass that does not exist in this cluster, so the claim never binds and the Pods never schedule.

FIX

- Stop debugging the GitOps controller — it applied exactly what was committed
- Follow the health status down the owner chain: Application to Deployment to Pod to the blocking object
- Fix the manifest in Git; never patch the cluster, because selfHeal will revert it

Source: Argo CD documentation — Application specification and Automated Sync Policy (argo-cd.readthedocs.io).

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

Drift, Self-Heal and the Fields You Must Ignore

Self-healing is only pleasant once you have told the controller which fields it does not own.

1 Drift is any divergence

A manual kubectl edit, an operator writing status, or an admission webhook mutating a field the controller did not expect.

2 Detection is continuous

The controller re-renders and re-diffs on its interval whether or not anything changed in Git.

3 selfHeal reverts it

Automated sync reapplies the desired state. Emergency fixes made by hand disappear, sometimes mid-incident.

4 prune deletes orphans

Objects that were removed from Git are removed from the cluster. Off by default because it is destructive.

5 ignoreDifferences is essential

HPA-managed replicas and webhook-injected sidecars must be excluded or the controller fights another controller forever.

6 Server-side apply helps

Field managers make ownership explicit, so two controllers can own different fields of the same object without a war.

Source: Argo CD documentation — Application specification and Automated Sync Policy (argo-cd.readthedocs.io).

© 2026 Tertiary Infotech Academy Pte Ltd

READ THE OUTPUT

kubectl diff — Drift Without Applying Anything

The manual version of what a GitOps controller does on every reconcile.

terminal — kcna-lab21

```
$ kubectl -n kcna-lab21 diff -k kustomize/overlays/prod
diff -u -N /tmp/LIVE-1/apps.v1.Deployment.tracklane-prod \
           /tmp/MERGED-1/apps.v1.Deployment.tracklane-prod
--- LIVE
+++ MERGED
@@ -12,7 +12,7 @@
 spec:
-  replicas: 5
+  replicas: 3
@@ -34,7 +34,7 @@
           resources:
             requests:
-              cpu: 250m
+              cpu: 100m
$ echo $?
1                      # exit 1 means differences were found
```

WHAT TO LOOK AT

1. **It is a server-side dry run**
   The API server computes the merged result, so defaults and webhooks are already accounted for.
2. **LIVE versus MERGED**
   Minus lines are what the cluster has now; plus lines are what applying would produce.
3. **Exit code 1 is the signal**
   Not an error. It is how CI asserts 'the cluster no longer matches the repository'.
4. **Two drifts, two stories**
   replicas 5 is probably a real emergency scale-up; cpu 250m is probably an undocumented hand edit.

Source: Kubernetes documentation — Declarative Management of Kubernetes Objects Using Kustomize; kubectl.docs.kubernetes.io reference.

© 2026 Tertiary Infotech Academy Pte Ltd

COMPARE

Release Strategies and What Each One Costs

Four strategies, and only two of them need infrastructure you do not already have.

Strategy	How traffic moves	Rollback	What it costs
Rolling update	Pods replaced gradually; all traffic follows	Roll forward or undo — minutes	Two versions serve at once with no control over the ratio
Blue/green	Switch a Service selector from blue to green at once	Flip the selector back — seconds	Double the capacity for the duration of the cutover
Canary	A small weighted share to the new version, then increase	Shift the weight back to zero — seconds	Needs a traffic router: Ingress, Gateway API or a service mesh
Feature flag	All instances updated; behaviour toggled per request	Turn the flag off — immediate	Flag debt, and code paths that must both keep working

CHOOSE

Progressive delivery separates deploying code from releasing behaviour. Once those are separate, rollback stops being a deployment event.

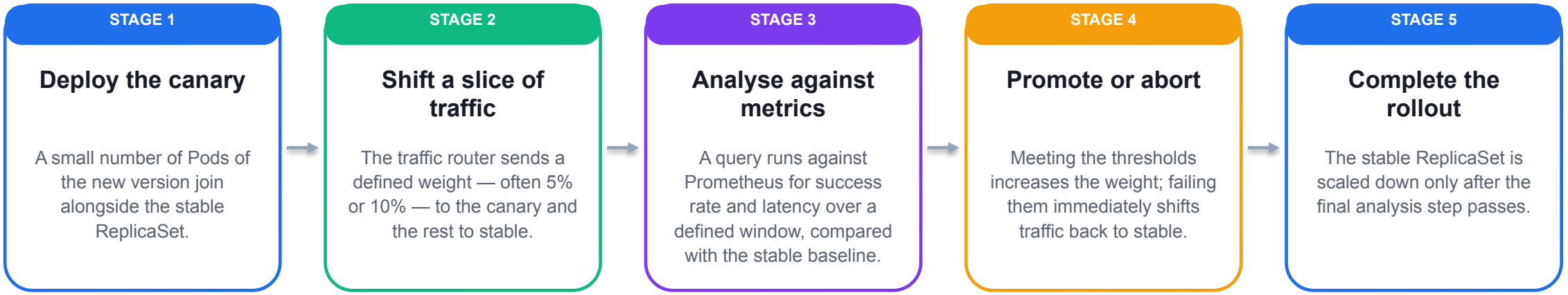
Source: Argo Rollouts documentation (argo-rollouts.readthedocs.io); Flagger documentation (docs.flagger.app).

© 2026 Tertiary Infotech Academy Pte Ltd

MECHANISM

How an Automated Canary Decides

Argo Rollouts and Flagger both implement this loop as a controller; the steps are the concept to remember.



WHAT IT MEANS

The judgement is a metric query with a threshold, which is why progressive delivery is inseparable from observability.

Source: Argo Rollouts documentation (argo-rollouts.readthedocs.io); Flagger documentation (docs.flagger.app).

© 2026 Tertiary Infotech Academy Pte Ltd

BUILDING BLOCKS

What Progressive Delivery Actually Requires

Half of progressive delivery is routing; the other half is observability you should already have.

1 A traffic router

Something that can split by weight: an Ingress controller with canary annotations, Gateway API HTTPRoute weights, or a mesh.

2 A metrics provider

Prometheus or an equivalent that can answer 'is the canary worse than stable' as a query, in seconds.

3 A workload abstraction

Argo Rollouts replaces the Deployment with a Rollout CR; Flagger drives a normal Deployment from the outside.

4 Automated abort

A rollback that requires a human to notice is not progressive delivery; it is a slower outage.

5 Feature flags

OpenFeature and similar decouple release from deploy entirely, including for changes that ship no new image.

6 Discipline about baselines

Comparing the canary to the stable version at the same moment, not to yesterday's numbers.

Source: Argo Rollouts documentation (argo-rollouts.readthedocs.io); Flagger documentation (docs.flagger.app).

© 2026 Tertiary Infotech Academy Pte Ltd

RELATED IDEAS

Infrastructure as Code and GitOps Are Not the Same

Both are declarative and both live in Git. The difference is who applies the change, and how often.

Infrastructure as Code

- Declarative definitions of infrastructure — networks, databases, clusters — in version control
- Applied by a runner: terraform apply, pulumi up, a pipeline job
- State is kept in a separate state file that must be locked
- Reconciliation happens when someone runs the tool
- Drift is discovered by running a plan, not continuously

GitOps

- Declarative definitions reconciled by a controller in the cluster
- Applied continuously, with no external actor holding credentials
- State is the Kubernetes API itself; no separate state file
- Reconciliation happens on an interval, forever
- Drift is detected continuously and optionally corrected

Source: Crossplane documentation — Composite Resources and Providers (crossplane.io/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

CONVERGENCE

Crossplane: Infrastructure as a Kubernetes Control Plane

The 'control plane for infrastructure' idea: stop running apply, and let a controller hold the desired state.

Providers install CRDs

1

A cloud provider package adds custom resources for buckets, databases and networks to your cluster's API.

Managed resources are objects

2

A database becomes a Kubernetes object with a spec and a status, reconciled by a controller like anything else.

Compositions build platform APIs

3

A CompositeResourceDefinition exposes a simple claim that fans out into many provider resources.

Continuous reconciliation

4

Manual changes in the cloud console are corrected, because the control loop never stops.

GitOps now covers infrastructure

5

Because it is all Kubernetes objects, Argo CD or Flux delivers infrastructure with the same mechanism as applications.

The trade-off

6

Your cluster becomes a dependency for provisioning. That is a deliberate architectural commitment.

Source: Crossplane documentation — Composite Resources and Providers (crossplane.io/docs).

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

The Delivery and Storage Projects Worth Knowing

Recognising the marks and one sentence of purpose is the KCNA ecosystem expectation.



Helm

Package manager · Graduated



Argo

Argo CD, Rollouts · Graduated



Flux

GitOps toolkit · Graduated



Crossplane

Infra control plane · Graduated

Backstage

Developer portal · Incubating



Kyverno

Policy as code · Graduated



Rook

Ceph storage operator · Graduated



Longhorn

Distributed block storage



etcd

Cluster datastore · Graduated



Harbor

Registry · Graduated



KEDA

Event-driven autoscaling · Graduated



CNCF Landscape

Where all of these are catalogued

Source: CNCF Graduated project listing (cncf.io/projects); CNCF Artwork repository, Linux Foundation Trademark Usage Guidelines.

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 21 — Packaging and Delivery: Helm, Kustomize and GitOps

GOAL

Replace hand-maintained TrackLane YAML with two real packaging approaches — a Helm chart rendered from environment values files, and a Kustomize base with dev and prod overlays — then read a GitOps Application manifest field by field to see how a controller turns Git into the cluster's desired state.

YOU WILL PRODUCE

- A minimal but complete Helm chart (Chart.yaml, values.yaml, _helpers.tpl, templates, NOTES.txt) that renders different dev and prod output from two values files, including a conditionally created PodDisruptionBudget.
- A Kustomize base plus dev and prod overlays that render to stdout with kubectl kustomize alone, differing in replicas, resources, config literals, image registry, probe timing and dataset size.
- An annotated Argo CD Application and the equivalent Flux GitRepository/Kustomization pair, read field by field, with the reconciliation model (prune, selfHeal, drift) explained against them.

CONCEPTS EXERCISED

- Helm chart anatomy: Chart.yaml versus appVersion, values precedence, named templates, the checksum/config annotation
- Kustomize base and overlay: strategic-merge versus JSON 6902 patches, transformers, generator hashes
- Templating versus patching — when each is the right tool, and the cost of each
- GitOps: desired state in Git, a controller reconciling drift, prune and self-heal

LAB	NAMESPACE	MAPS TO	TIME
Lab 21	kcna-lab21	LO5 · A5 · K5	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-21-helm-kustomize-gitops/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

RECALL SET

Day 4 Facts You Should Be Able to State Cold

Fourteen one-line answers that cover most of today's examinable surface.

emptyDir lifetime	The Pod — survives container restarts
SizeMemoryBackedVolumes	Beta v1.22, generally available v1.32
PV scope / PVC scope	Cluster-scoped / namespaced
RWO scope	One node, not one Pod (RWOP is the Pod-scoped mode)
Default dynamic reclaim policy	Delete, inherited from the class
Pending PVC with WaitForFirstConsumer	Correct behaviour, not a fault
CSI plugin split	Controller Deployment plus node DaemonSet

StatefulSet claim name	<template>-<statefulset>-<ordinal>
etcd quorum	floor(n/2) + 1; 3 members tolerate 1 failure
Kubelet version skew	Up to three minor versions behind the API server
helm template	Renders offline; never contacts the cluster
Kustomize templating	None by design — structured edits only
GitOps direction	Pull, by a controller inside the cluster
Synced but Degraded	Git applied faithfully; the manifest is wrong

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM TECHNIQUE

Distractors Built From Today's Material

Six statements that are plausible, common, and wrong.

1. **'RWO means one Pod'**
   It means one node. Two Pods co-scheduled on that node can both mount the volume.

2. **'A Pending PVC is broken'**
   Under WaitForFirstConsumer with no consumer, Pending is the designed state. Read the event type.

3. **'Deleting a StatefulSet frees the storage'**
   Retention defaults to Retain, so the claims and the data remain.

4. **'Four etcd members are safer than three'**
   Both tolerate exactly one failure; four just adds another thing that can fail.

5. **'helm template applies the chart'**
   It renders to stdout. Nothing reaches the cluster until something applies the output.

6. **'GitOps is CI/CD with Git'**
   The defining property is a controller pulling and reconciling continuously, not the use of Git.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

CONSOLIDATION

What You Can Do at the End of Day 4

Two registered topics closed, and the delivery domain covered end to end.

Design and explain

- Choose a volume type from a stated durability requirement
- Write a PV, a PVC and a StorageClass and predict the binding
- Explain why a PVC is Pending from the event type alone
- Draw the CSI controller and node plugins with their sidecars
- Justify a stacked or external etcd topology
- Sequence a cluster upgrade from the version skew policy

Evidence you produced

- A tmpfs-backed emptyDir sized exactly from its spec (Lab 17)
- A Retained volume whose data outlived its claim (Lab 18)
- Three ordinals, three claims, and a reattached volume (Lab 19)
- A verified etcd snapshot and a control-plane inventory (Lab 20)
- Two Helm renders and two Kustomize overlays that differ (Lab 21)
- A silently unpatched overlay you diagnosed by diffing

© 2026 Tertiary Infotech Academy Pte Ltd

TOMORROW

Day 5 — Observability, Troubleshooting and Assessment

One instructional session, then the two assessment instruments.

1. **Probes and self-healing**
   liveness, readiness and startup probes, and what each one actually causes the kubelet to do.

2. **Events, logs and metrics**
   Field selectors, the metrics-server pipeline, and why events expire long before you go looking for them.

3. **Prometheus and the exposition format**
   Scraping, metric types and how the observability pipeline is assembled from CNCF components.

4. **Troubleshooting triage**
   A repeatable order of investigation across application, node and control plane.

5. **Written Assessment**
   Seven short-answer questions in 60 minutes, mapped to K1 to K7.

6. **Practical Performance**
   Six tasks in 120 minutes; Task 4 and Task 5 draw directly on today's storage and delivery work.

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTERED TOPIC 7

07

Logging and Monitoring

Signals, health, Events, logs and metrics — what the cluster tells you, in what order, and for how long it keeps telling you.

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 5 · OUTCOMES

What Today Has To Leave You Able To Do

LO6 · A6 · K7 — monitor the system, then troubleshoot it.

1 Name the signal, then reach for it

Metrics, logs, traces and Events answer different questions. Choosing wrongly is the most expensive minute of an incident.

2 Read a probe stanza as a time budget

initialDelaySeconds + periodSeconds x failureThreshold is a number of seconds. Compare it against the application's real start time.

3 Query Events as API objects

--sort-by=.lastTimestamp and --field-selector, plus the fields that are not selectable and the one-hour default TTL.

4 Explain the metrics pipeline end to end

Instrument, expose, scrape, store, query, alert. Know where metrics-server stops and where Prometheus starts.

5 Work a triage tree, not a hunch

Scheduled? pulled? started? Ready? endpoints? Stop at the first NO — that is the phase. Cause comes second.

6 Place a project in the CNCF ecosystem

Sandbox, incubating, graduated — and which widely used tools are not CNCF projects at all.

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

The Signals — And The Question Each One Answers

Four established signals plus one emerging — and they are not interchangeable.

Metrics — 'how much, how often?'

1

Numeric samples over time, cheap to store, aggregatable. Good at trend and threshold. Cannot tell you what happened to one request.

Logs — 'what exactly happened?'

2

Discrete, timestamped records with arbitrary detail. Expensive per byte. Good at the specific case, poor at the aggregate.

Traces — 'where did the time go?'

3

A causally linked tree of spans across services. The only signal that shows latency attribution in a distributed call path.

Events — 'what did the platform decide?'

4

Kubernetes' own record of controller and kubelet decisions. Short lived, aggregated, namespaced — a signal source, not an audit log.

Profiles — the emerging signal

5

Continuous CPU/heap profiling correlated with the other signals. Answers 'which function burned the CPU', not just 'CPU was high'.

Correlation is the real product

6

One shared set of labels (service, namespace, pod, trace id) across all signals is what turns four data sets into one investigation.

Source: OpenTelemetry documentation, What is OpenTelemetry and the Collector: opentelemetry.io/docs/what-is-opentelemetry/ ; opentelemetry.io/docs/collector/

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

Monitoring vs Observability

Not competing products — one is a practice, the other is a property of the system.

Dimension	Monitoring	Observability
Question shape	Known-unknowns: you predicted the failure and built a check for it	Unknown-unknowns: you must ask a question nobody anticipated
Built from	Dashboards and alerts on a fixed set of series	High-cardinality, high-dimension telemetry you can slice at query time
Typical artefact	'CPU > 80% for 5 minutes' alert rule	'Which tenant, on which build, in which zone, is slow?'
Fails when	The outage is a shape you did not predict	Cardinality and cost are not controlled
Relationship	A subset — every monitored signal is telemetry	A property of the system, not a product you can buy

CHOOSE

Monitor the handful of symptoms users feel; instrument richly enough that you can still investigate the outage you did not predict.

Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

Three Signals, One Incident

Meridian Freight INC-4417, read through each signal in turn.

Metrics told you it broke

- Error ratio for the dispatch endpoint crossed 5%
- sum(rate(...{status="502"}[2m])) / sum(rate(...[2m]))
- Cheap enough to keep at 10s resolution for months
- Tells you WHEN and HOW BAD — never WHY

symptom · alert · trend

Traces told you where

- The dispatch span spends 3.0s waiting on manifest-indexer
- Parent/child spans carry one trace id across services
- Sampling means you may not have the exact bad request
- Tells you WHICH HOP — rarely the line of code

latency · attribution

Logs told you why

- ERROR ... upstream=manifest-indexer error=upstream_timeout
- Full detail for one request id, at the highest cost per byte
- Only stdout/stderr is captured by the cluster
- Tells you WHAT the process actually did

detail · evidence

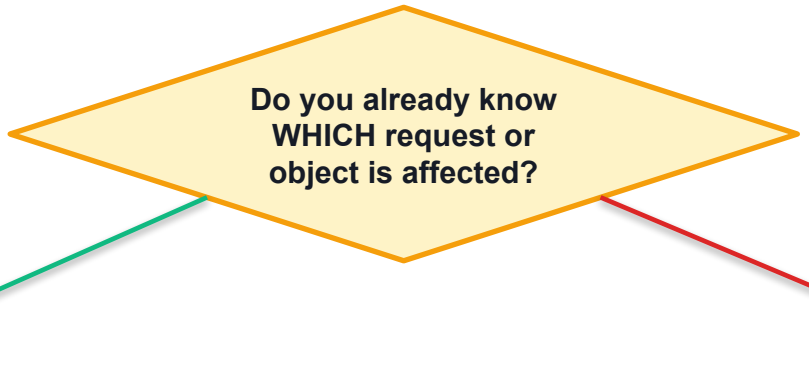
Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

Which Signal Do You Reach For First?

The cost of the wrong first move is measured in SLA minutes.



YES — go narrow

- Logs for that Pod, with --previous if it restarted
- Events scoped with --field-selector involvedObject.name=...
- The trace for that request id, if tracing is instrumented
- You are confirming a hypothesis, not searching

NO — go wide first

- Metrics: which service, which status code, since when
- Aggregate rate() ratios, never absolute counts
- Narrow to one object, THEN switch to logs and Events
- Searching logs cluster-wide first is the classic time sink

Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

The Four Golden Signals

Google SRE: if you can only measure four things about a user-facing system, measure these.

1 Latency

Time to serve a request. Split successful from failed latency — a fast 500 will otherwise flatter your average.

2 Traffic

Demand on the system in its own units: HTTP requests per second, sessions, messages consumed.

3 Errors

Rate of failed requests. Explicit (500), implicit (200 with wrong content) and policy (over the latency budget).

4 Saturation

How full the most constrained resource is. The leading indicator — it degrades before latency or errors move.

Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

© 2026 Tertiary Infotech Academy Pte Ltd

OBSERVABILITY

RED and USE — Two Instrumentation Recipes

Two checklists that make instrumentation a decision, not an afterthought.

	RED	USE
Applies to	Request-driven services (your application)	Resources (CPU, memory, disk, network, queues)
R / U	Rate — requests per second	Utilisation — % time the resource was busy
E / S	Errors — failed requests per second	Saturation — queue depth or waiting work
D / E	Duration — distribution of request time	Errors — error counts for that resource
Origin	Tom Wilkie, derived from the golden signals	Brendan Gregg, from systems performance analysis
Use together	Answers 'are users suffering?'	Answers 'which resource is the cause?'

CHOOSE

Instrument services with RED and nodes with USE; the golden signals are the union of the two.

Source: RED method — Tom Wilkie, Grafana Labs; USE method — Brendan Gregg, brendangregg.com/usemethod.html

© 2026 Tertiary Infotech Academy Pte Ltd

RELIABILITY

SLI, SLO, SLA and the Error Budget

An SLO without an error-budget policy is a wish, not a target.

SLI	Service Level Indicator — a measured ratio of good events to valid events
Example SLI	successful dispatch requests / all dispatch requests over 28 days
SLO	Service Level Objective — the internal target for that SLI, e.g. 99.9%
SLA	Service Level Agreement — a contract with a customer, with consequences
Relationship	SLO must be strictly tighter than any SLA it supports
Error budget	1 - SLO. The permitted unreliability, expressed as time or events
Budget policy	Budget spent -> feature work stops, reliability work starts
Burn rate	How fast the budget is being consumed relative to even burn (1.0)

Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

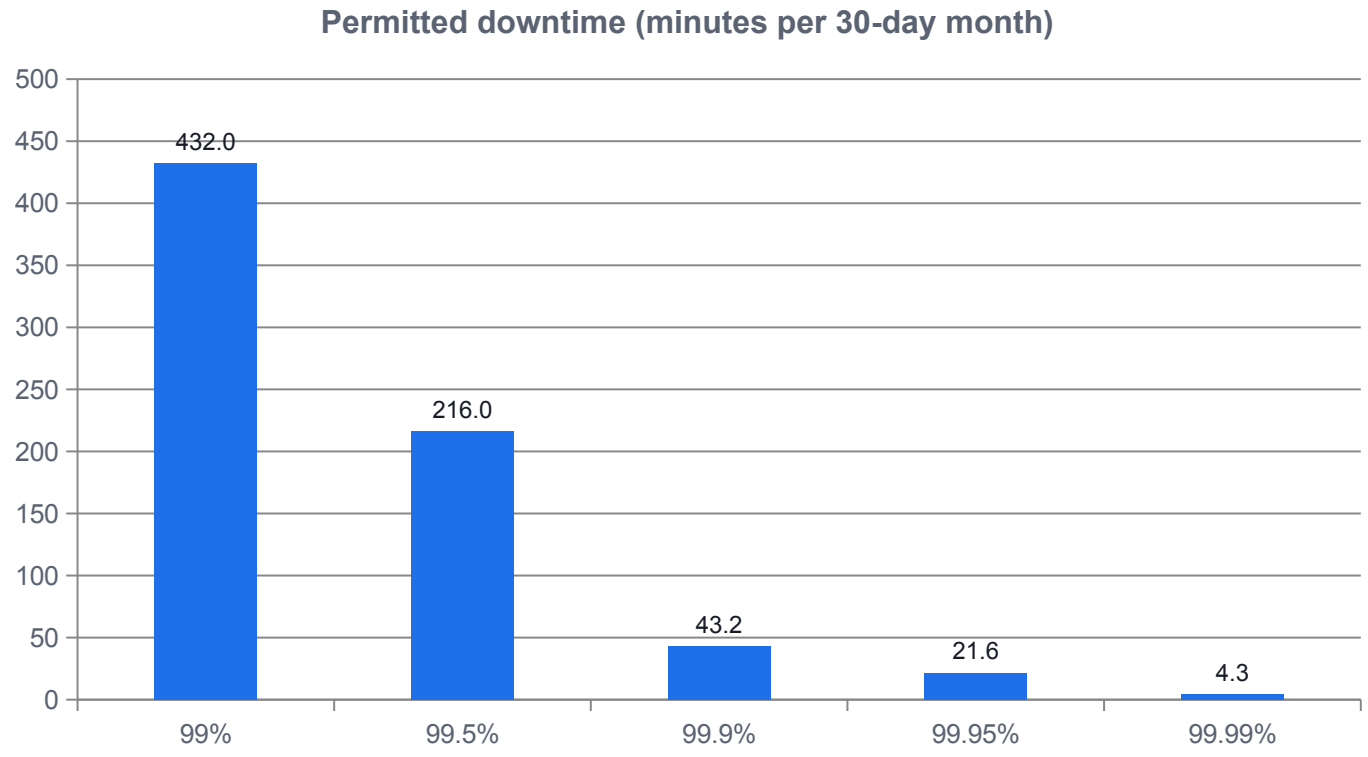
© 2026 Tertiary Infotech Academy Pte Ltd

RELIABILITY

What Each Nine Actually Costs You

Exact arithmetic on a 30-day month = 43,200 minutes. No measured data, no estimates.

Minutes of permitted unavailability per 30-day month



WHAT THE DATA SHOWS

The arithmetic

43,200 min x (1 - SLO). 99.9% -> 43,200 x 0.001 = 43.2 minutes per month.

Each nine costs 10x

Every added nine divides the budget by ten. 99.99% leaves 4.32 minutes — less than one bad rollout.

Deploys spend budget too

A 3-minute failed rollout is 7% of a 99.9% monthly budget. Four of those and the month is gone.

Pick the SLO you can staff

99.99% implies automated rollback and no manual step in the recovery path.

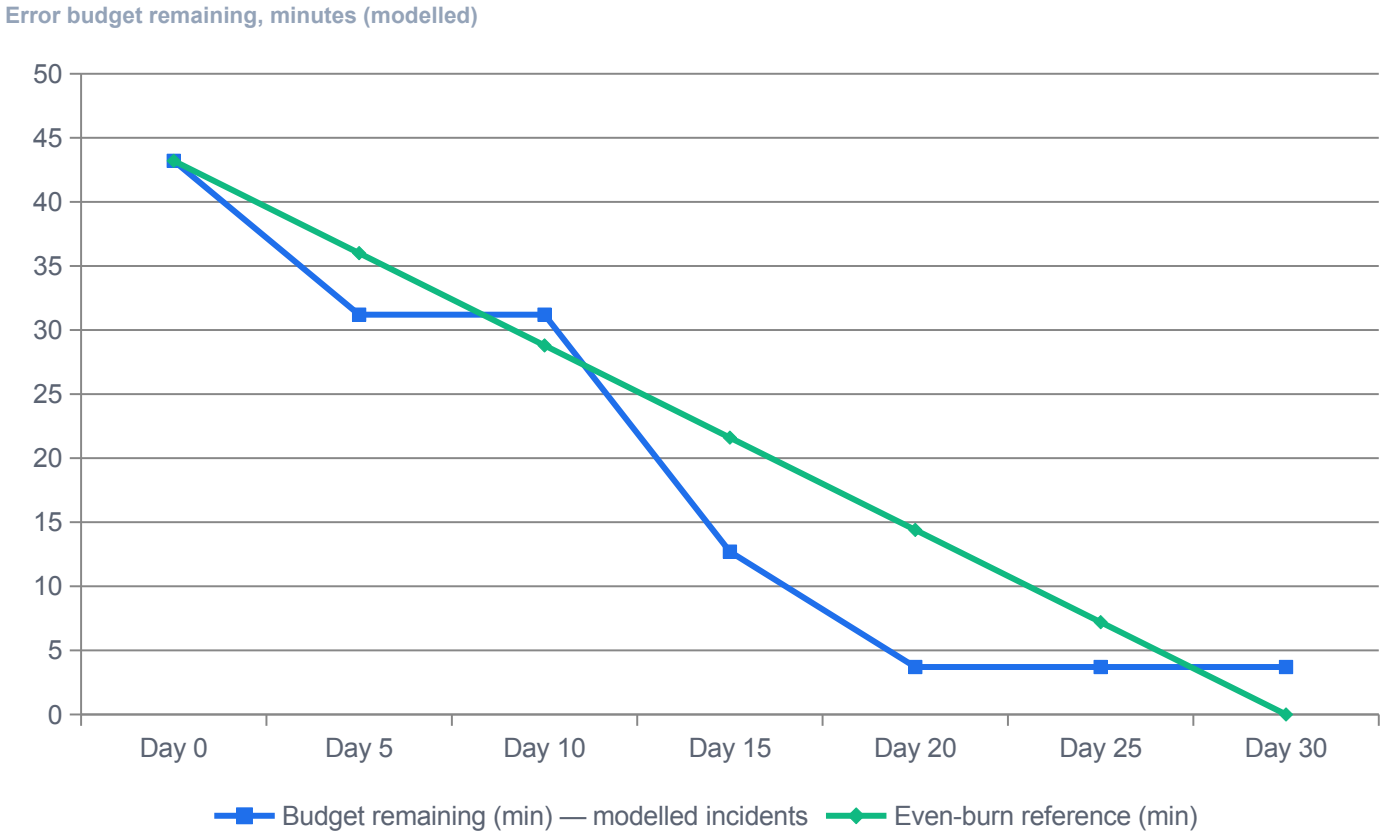
Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

© 2026 Tertiary Infotech Academy Pte Ltd

RELIABILITY

Error-Budget Burn — A Worked Model, 99.9% SLO

A LABELLED MODEL, not measured data. Three incidents debited from a 43.2-minute monthly budget.



WHAT THE DATA SHOWS

The three debits

Day 5: 12.0 min -> 31.2. Day 15: 18.5 min -> 12.7. Day 20: 9.0 min -> 3.7. Exact subtraction.

Even burn is 1.44 min/day

43.2 / 30. The dashed reference line is 43.2 x (1 - day/30).

Burn rate, not level, alerts

At day 15 the budget is below the reference line: burn rate > 1. That is the alert.

Budget left is not success

3.7 minutes with ten days to run means the next single incident breaches the SLO.

Source: Worked model built on the exact 43.2 min/month figure; incident magnitudes are illustrative and labelled as such.

© 2026 Tertiary Infotech Academy Pte Ltd

RELIABILITY

From User Symptom To Alert That Is Worth Waking For

Alert on symptoms the user feels, budgeted — not on every resource that twitches.



WHAT IT MEANS

Symptom-based, budget-referenced alerting is what stops an on-call rota drowning in cause-based alerts nobody acts on.

Source: Google SRE Book, Monitoring Distributed Systems (four golden signals) and Embracing Risk (error budgets): sre.google/sre-book/

© 2026 Tertiary Infotech Academy Pte Ltd

THE ALERTING RULE OF THUMB

If an alert does not correspond to something a user can feel, it belongs in a dashboard — not on a pager.

Cause-based alerts multiply with your architecture. Symptom-based, budget-referenced alerts scale with your user-visible surface, which is far smaller.

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

Who Reports Health, And Who Consumes It

Kubernetes health is not a dashboard — it is an input to controllers that take action without you.

REPORTERS

Container probes
kubelet runs httpGet / tcpSocket / exec / grpc against the container

kubelet node status
Posts node conditions and a heartbeat Lease to the API server

Controllers
Emit Events describing every decision they take

CONSUMERS

EndpointSlice controller
Adds or removes Pod addresses as readiness changes

kubelet restart logic
Restarts the container when liveness fails, per restartPolicy

Node controller
Taints and evicts when the heartbeat stops or pressure is set

MECHANISM

Readiness changes routing; liveness changes container lifecycle; node conditions change scheduling and eviction. Every probe you write is a control input.

Source: Kubernetes documentation, Pod Lifecycle — Container probes: kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

Liveness, Readiness and Startup — By Consequence

Identical syntax, three completely different consequences. The consequence is the exam answer.

	livenessProbe	readinessProbe	startupProbe
Question	Is the process still functional?	Should it receive traffic right now?	Has it finished booting yet?
On failure	Container killed and restarted; RESTARTS increments	Pod address removed from the Service EndpointSlice; container untouched	Container killed once the whole startup budget is exhausted
While it runs	Evaluated for the container's whole life	Evaluated for the container's whole life	Evaluated only until its first success, then never again
Side effect	Restart loop if the app is merely slow	Graceful removal from load balancing; no restart	Suspends liveness AND readiness for the whole boot window
Typical mistake	Pointing it at a dependency you do not own	Reusing the liveness path, so a busy Pod gets restarted	Omitting it, then loosening liveness until it means nothing

Source: Kubernetes documentation, Pod Lifecycle — Container probes: kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

Reading A Probe Block As A Time Budget

Every probe field is arithmetic. Do the arithmetic before you deploy, not during the incident.

composite of the two Lab 22 workloads · v1 Pod spec

```
containers:
- name: indexer
  image: busybox:1.36
  startupProbe:            # boot budget: 5s x 24 = 120s
    httpGet: {path: /index.html, port: http}
    periodSeconds: 5
    failureThreshold: 24
  readinessProbe:          # traffic gate: 3 + 5x2 = 13s
    httpGet: {path: /ready.html, port: http}
    initialDelaySeconds: 3
    periodSeconds: 5
    failureThreshold: 2
  livenessProbe:           # restart gate: 10 + 10x3 = 40s
    httpGet: {path: /index.html, port: http}
    initialDelaySeconds: 10
    periodSeconds: 10
    failureThreshold: 3
```

READING IT

1. **The budget formula**
   initialDelaySeconds + (periodSeconds x failureThreshold). That is a number of seconds you compare with the app's real start time.

2. **Separate paths on purpose**
   /ready.html gates traffic, /index.html gates life. Sharing one path means a temporarily busy Pod gets restarted, not drained.

3. **startupProbe suspends the others**
   Liveness and readiness are not evaluated at all until startup succeeds once. 5s x 24 = a 120s boot window with zero restarts.

4. **timeoutSeconds counts as a failure**
   A slow response is a failed probe. Default timeoutSeconds is 1 — on a loaded node that alone can produce a restart loop.

Source: Kubernetes documentation, Configure Liveness, Readiness and Startup Probes: kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

Probe Handlers And Where Each One Lies To You

Four handler types, and each has a failure mode where it passes while the application is dead.

1 httpGet

Any 200-399 is a pass. Cheap and the default choice — but a handler that returns 200 unconditionally proves nothing.

2 tcpSocket

Passes if the port accepts a connection. A listening socket on a wedged process still passes; use it only as a floor.

3 exec

Runs a command in the container; exit 0 passes. The most expensive handler — it forks a process on every period.

4 grpc

Native gRPC health checking protocol, stable since v1.27. Removes the grpc-health-probe binary from the image.

5 terminationGracePeriodSeconds

Probe-level override exists for liveness: a slow shutdown can be given its own grace period.

6 successThreshold

Must be 1 for liveness and startup. Only readiness may require several consecutive passes.

Source: Kubernetes documentation, Pod Lifecycle — Container probes: kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#container-probes

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

The Aggressive Liveness Probe

The signature pair: Warning Unhealthy followed by Normal Killing, with exactly one line in the previous log.

SYMPTOM

manifest-indexer is in CrashLoopBackOff. Nothing is wrong with the application — it simply needs 40 seconds to warm its index.

EVIDENCE — actual cluster output

```
$ kubectl get pods -n kcna-lab22 -l app=manifest-indexer-outage
NAME                           READY   STATUS             RESTARTS
manifest-indexer-outage-t8pnw 0/1     CrashLoopBackOff   2 (12s ago)

$ kubectl describe pod -n kcna-lab22 $BROKEN | tail -4
  Normal   Started    22s (x4 over 80s)  kubelet  Started container
  Warning  Unhealthy  12s (x8 over 74s)  kubelet  Liveness probe
     failed: Get "http://10.244.0.18:8080/index.html": dial tcp
     10.244.0.18:8080: connect: connection refused
  Normal   Killing    12s (x4 over 66s)  kubelet  Container indexer
     failed liveness probe, will be restarted

$ kubectl logs -n kcna-lab22 $BROKEN --previous
[indexer] 06:31:04 warming shipment manifest index...
```

ROOT CAUSE

- Liveness budget = 5 + (5 x 2) = 15s
- Real start time = 40s. 15 < 40
- No startupProbe, so liveness runs from second 5

FIX

- Add a startupProbe sized above the real boot time
- Keep liveness for hangs, not for boots
- Never loosen liveness to paper over a slow start

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

HEALTH

Reading Probe State Out Of describe pod

Probe configuration and probe outcome are both visible without reading a single YAML file.

Lab 22 · kcna-lab22 · Kubernetes v1.37.0

```
$ kubectl describe pod -n kcna-lab22 -l app=manifest-indexer \
    | grep -E 'Restart Count|Liveness|Readiness|Startup'
    Restart Count:  0
    Liveness:   http-get http://:http/index.html delay=0s
                timeout=2s period=10s #success=1 #failure=3
    Readiness:  http-get http://:http/index.html delay=0s
                timeout=2s period=5s #success=1 #failure=2
    Startup:    http-get http://:http/index.html delay=0s
                timeout=2s period=5s #success=1 #failure=24

$ kubectl describe pod -n kcna-lab22 $VICTIM | sed -n \
    '/^Conditions:/,/^Volumes:/p'
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
```

WHAT TO LOOK AT

1. **Restart Count: 0 is the headline**
   A Pod that is 0/1 with zero restarts failed READINESS. A Pod with a climbing count failed LIVENESS. One column separates them.
2. **#failure is failureThreshold**
   describe renders the probe as delay / timeout / period / #success / #failure. Startup #failure=24 at period 5s is the 120s budget.
3. **Ready False, PodScheduled True**
   The conditions localise the failure: scheduling succeeded, the container initialised, and only readiness is refusing.
4. **Conditions are the API's own triage**
   Read them top to bottom — they follow the same phase order as the troubleshooting tree you will use this afternoon.

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 22 — Probes, Health and Self-Healing

GOAL

Configure liveness, readiness and startup probes on the Depot Portal workloads and prove that each one has a different consequence: restart, endpoint withdrawal, or a grace window for a slow boot.

YOU WILL PRODUCE

- A depot-portal Deployment whose readiness probe demonstrably adds and removes its Pod address from the Service EndpointSlice without ever restarting the container
- A manifest-indexer Deployment that boots for 40 seconds and reaches Ready with zero restarts because a startupProbe grants it a 120-second budget
- A completed probe-tuning worksheet recording each probe's failure budget, observed timing and consequence

CONCEPTS EXERCISED

- Liveness restarts the container; readiness withdraws it from the Service; startup suspends both while the process boots
- Failure budget arithmetic: initialDelaySeconds + (periodSeconds x failureThreshold)
- EndpointSlice is the current API that kube-proxy consumes for Service backends
- A liveness budget shorter than real startup time produces a permanent restart loop on a perfectly healthy application

LAB	NAMESPACE	MAPS TO	TIME
Lab 22	kcna-lab22	LO6 · A6 · K7	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-22-probes-health-selfhealing/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Morning Break

11:00 – 11:10 AM

© 2026 Tertiary Infotech Academy Pte Ltd

EVENTS

An Event Is An API Object — Read Its Fields

Events are namespaced objects in etcd, not log lines — which is exactly why they can be queried, and why they expire.

core/v1 Event · kcna-lab23

```
apiVersion: v1
kind: Event
metadata:
  name: depot-gate-agent.186f2a1c9d4e
involvedObject:
  kind: Pod
  name: depot-gate-agent
  namespace: kcna-lab23
type: Warning
reason: FailedMount
message: 'MountVolume.SetUp failed for volume
  "gate-config" : configmap "depot-gate-config"
  not found'
source:
  component: kubelet
count: 3
firstTimestamp: "2026-09-05T06:12:04Z"
lastTimestamp:  "2026-09-05T06:16:21Z"
```

READING IT

1. **involvedObject is the join key**
   kind, name, namespace and uid. These are the indexed fields you filter on with --field-selector; nothing else about the object is.

2. **type is only Normal or Warning**
   There is no Error type. Warning is the entire severity vocabulary for triage, which is why type=Warning is the highest-value filter.

3. **reason is a stable CamelCase token**
   FailedMount, FailedScheduling, Unhealthy, BackOff, Killing. Machine-matchable; the free-text message is not.

4. **count + first/lastTimestamp = aggregation**
   Three occurrences, one object. Kubernetes never stores one Event per occurrence, so 'one event' can mean hundreds.

Source: Kubernetes API reference, core/v1 Event; and Field Selectors: kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/

© 2026 Tertiary Infotech Academy Pte Ltd

EVENTS

Sort Them, Or You Will Misread Cause And Effect

Same data, three renderings — only two of them are safe to reason from.

Lab 23 · kcna-lab23 · Kubernetes v1.37.0

```
$ kubectl get events -n kcna-lab23 | head -4
LAST SEEN  TYPE     REASON      OBJECT                MESSAGE
41s        Normal   Pulled      Pod/gate-scanner-w2xkc  image present
2m6s       Normal   Scheduled   Pod/depot-portal-jm4vd  assigned to...
19s        Warning  FailedMount Pod/depot-gate-agent    configmap not

$ kubectl get events -n kcna-lab23 --sort-by=.lastTimestamp | tail -6
LAST SEEN  TYPE     REASON      OBJECT                MESSAGE
2m30s      Normal   Scheduled   Pod/gate-scanner-w2xkc  assigned
2m28s      Normal   Pulled      Pod/gate-scanner-w2xkc  image present
2m28s      Normal   Created     Pod/gate-scanner-w2xkc  Created: scanner
2m28s      Normal   Started     Pod/gate-scanner-w2xkc  Started scanner
2m16s      Warning  BackOff     Pod/gate-scanner-w2xkc  Back-off restart
30s        Warning  FailedMount Pod/depot-gate-agent    configmap not

$ kubectl events -n kcna-lab23 --for pod/depot-gate-agent
2m59s (x2 over 3m1s)  Warning  FailedMount  Pod/depot-gate-agent
```

WHAT TO LOOK AT

1. **The default order is not time**
   kubectl get events returns the API's own order. During an incident that is actively harmful — you will invert cause and effect.
2. **--sort-by=.lastTimestamp**
   Build muscle memory for this exact flag. It is a client-side sort over the JSON path, so it works on any list command.
3. **kubectl events sorts by default**
   The dedicated subcommand (GA since v1.29) sorts for you and takes --for pod/NAME to scope to a single object.
4. **Read the (x2 over 3m1s)**
   That is count and the first/last timestamp pair rendered inline. One row, two occurrences, over three minutes.

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

EVENTS

Field Selectors — Cutting The Noise To One Question

The single highest-value triage filter in Kubernetes, and its hard limit.

Lab 23 · --field-selector on core/v1 Events

```
$ kubectl get events -n kcna-lab23 \
    --field-selector involvedObject.kind=Pod,type=Warning \
    --sort-by=.lastTimestamp
LAST SEEN   TYPE      REASON       OBJECT                 MESSAGE
2m55s       Warning   BackOff      Pod/gate-scanner-w2xkc   Back-off
69s         Warning   FailedMount  Pod/depot-gate-agent     configmap

$ kubectl get events -n kcna-lab23 --field-selector reason=FailedMount
84s         Warning   FailedMount  Pod/depot-gate-agent     configmap

$ kubectl get events -n kcna-lab23 --field-selector message=whatever
Error from server (BadRequest): Unable to find "/v1,
Resource=events" that match label selector "", field selector
"message=whatever": field label not supported: message
```

WHAT TO LOOK AT

1. **Comma means AND**

   involvedObject.kind=Pod,type=Warning is a conjunction. On a busy cluster this strips Node, PVC and Deployment noise in one step.

2. **Field selectors are server-side**

   Unlike grep, the filter runs in the API server, so it scales and does not depend on the output format you happened to request.

3. **Only indexed fields are selectable**

   message is not. The error names the unsupported field explicitly — read it rather than assuming the selector syntax is wrong.

4. **Events carry no useful labels**

   So -l does not help here. Field selection is the only server-side filter you have for Events.

Source: Kubernetes API reference, core/v1 Event; and Field Selectors: kubernetes.io/docs/concepts/overview/working-with-objects/field-selectors/

© 2026 Tertiary Infotech Academy Pte Ltd

EVENTS

What You May Select On, And What Expires

Events are a signal SOURCE for a pipeline, not the pipeline — and definitely not an audit log.

involvedObject.kind	Pod, Node, Deployment, PersistentVolumeClaim
involvedObject.name	Exact object name — no globbing, no regex
involvedObject.namespace	Combine with -A to search cluster-wide
involvedObject.uid	Survives a name being reused by a new object
involvedObject.fieldPath	e.g. spec.containers{portal}
reason	FailedScheduling, Unhealthy, BackOff, Killing, FailedMount
type	Normal or Warning — the whole severity vocabulary
reportingComponent / source	Which component raised it: kubelet, default-scheduler
NOT selectable	message, count, timestamps — filter these client-side with grep or jq
Retention	kube-apiserver --event-ttl, default 1 hour; kubeadm does not override it

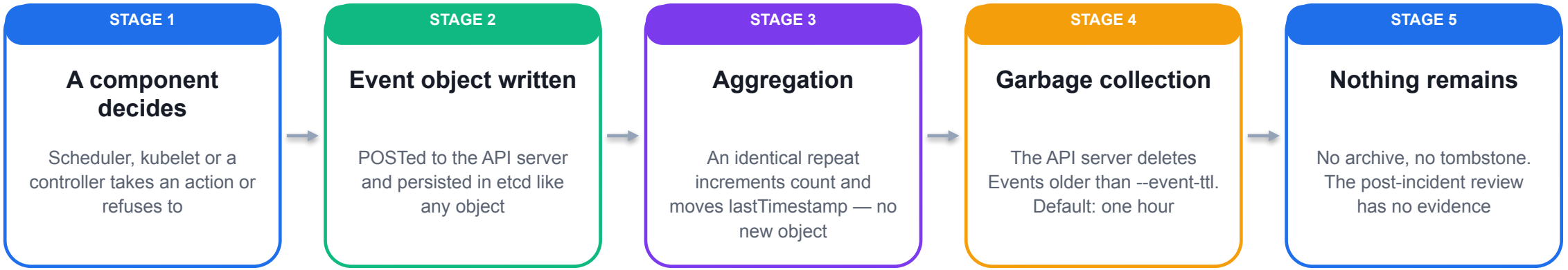
Source: Kubernetes reference, kube-apiserver command-line options (--event-ttl, default 1h): kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/

© 2026 Tertiary Infotech Academy Pte Ltd

EVENTS

The Life Of An Event

Five stages, and the fourth is the one that ruins post-incident reviews.



WHAT IT MEANS

If you need the decision trail after an hour, ship Events off the cluster — or read the component's own log, which is where the same decision still lives.

Source: Kubernetes reference, kube-apiserver command-line options (--event-ttl, default 1h): kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/

© 2026 Tertiary Infotech Academy Pte Ltd

LOGGING

The Cluster Logging Pipeline

ON THE POD

Container stdout/stderr
The ONLY stream the cluster captures

Files inside the container
Invisible to kubectl logs; die with the Pod

Sidecar re-emitter
tail -f a file back onto stdout so it becomes visible

ON THE NODE

CRI runtime
Writes /var/log/pods/<ns>_<pod>_<uid>/<container>/0.log

kubelet rotation
containerLogMaxSize 10Mi, containerLogMaxFiles 5 by default

Node agent DaemonSet
Fluent Bit or Fluentd tails those files on every node

OFF THE NODE

Transport / buffer
Parse, enrich with Pod metadata, buffer, retry

Log backend
Elasticsearch, Loki, OpenSearch, a managed service

Query and retention
The only place a log survives Pod deletion

Source: Kubernetes documentation, Logging Architecture: kubernetes.io/docs/concepts/cluster-administration/logging/

© 2026 Tertiary Infotech Academy Pte Ltd

LOGGING

--previous: The Most Important Flag In Triage

The container you can see is not the container that failed.

Lab 23 · CrashLoopBackOff diagnosis

```
$ kubectl logs -n kcna-lab23 $SCANNER --previous
06:18:02 INFO  gate-scanner starting, serial=SCN-0447
06:18:02 INFO  opening barcode reader on /dev/ttyUSB0
06:18:07 WARN  reader handshake retry 1/1
06:18:12 ERROR reader handshake failed: device not present
06:18:12 FATAL gate-scanner exiting with status 1

$ kubectl logs -n kcna-lab23 $SCANNER
06:19:44 INFO  gate-scanner starting, serial=SCN-0447
06:19:44 INFO  opening barcode reader on /dev/ttyUSB0

$ kubectl get pod -n kcna-lab23 $SCANNER -o jsonpath=\
  '{.status.containerStatuses[0].lastState.terminated.exitCode}'
1

$ kubectl logs -n kcna-lab23 $POD
error: a container name must be specified for pod depot-portal-...,
choose one of: [portal audit-sidecar]
```

WHAT TO LOOK AT

1. **The current log is useless mid-loop**
   A restarting container's current log shows a fresh boot. The reason it died is in the PREVIOUS container's stream.
2. **Only one generation is kept**
   --previous gives you the immediately preceding container, not a history. Two restarts while you fetch coffee and it is gone.
3. **lastState.terminated.exitCode is authoritative**
   The log text is what the app chose to print; the exit code is what the kernel observed. Cross-check them.
4. **Multi-container Pods refuse to guess**
   -c <container>, or -l <selector> with --prefix to read a whole Deployment at once.

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

LOGGING

kubectl logs — The Flags That Matter, And Why

kubectl logs is a reader for node-local files — every flag is constrained by that fact.

-c / --all-containers

1 Selects the container. Multi-container Pods error out rather than guess; the error lists the names.

--previous

2 The terminated generation of the container. The only way to see why a CrashLoopBackOff Pod died.

--since / --since-time

3 Server-side time bound. Cheaper and more honest than tailing everything and grepping.

--timestamps

4 Prefixes the runtime's receive time. Application timestamps and cluster time are different clocks — do not mix.

-l <selector> --prefix

5 Aggregates across the replicas of a Deployment and labels each line with its Pod and container.

--tail / -f

6 Bound the output. -f holds a watch open; it does not reconnect across a Pod restart.

Source: Kubernetes documentation, Logging Architecture: kubernetes.io/docs/concepts/cluster-administration/logging/

© 2026 Tertiary Infotech Academy Pte Ltd

LOGGING

Three Ways To Get Logs Off A Node

Fluent Bit and Fluentd occupy the first column; the second exists only because stdout is the sole captured stream.

	Node-level agent (DaemonSet)	Sidecar per Pod	App writes to backend
How	One agent per node tails /var/log/pods/*	A second container re-emits or ships the app's files	The application library posts to the backend directly
Cost	One agent per node — lowest overhead	One extra container per Pod — multiplies with replicas	No extra container, but a library dependency
Handles files?	No — only what reached stdout/stderr	Yes — this is the reason the pattern exists	Yes
Failure coupling	Agent down = that node stops shipping	Sidecar down = only that Pod stops shipping	Backend down = the app itself blocks or drops
Rotation risk	Must out-run containerLogMaxSize rotation	Must rotate the in-container file itself	None — nothing is written to disk

CHOOSE

Default to a node-level agent; add a sidecar only for a workload that genuinely cannot write to stdout.

Source: Kubernetes documentation, Logging Architecture: kubernetes.io/docs/concepts/cluster-administration/logging/

© 2026 Tertiary Infotech Academy Pte Ltd

METRICS

The Resource Metrics Pipeline

Four hops from a cgroup counter to kubectl top — and not one of them stores a time series.



WHAT IT MEANS

metrics-server answers 'what is it using now' for autoscaling. It cannot answer 'what was CPU at 03:00', and it is not a monitoring system.

Source: Kubernetes documentation, Resource Metrics Pipeline: kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/

© 2026 Tertiary Infotech Academy Pte Ltd

METRICS

kubectl top Fails — Add-on, Not A Broken Cluster

Two errors, two different diagnoses — learn to tell 'absent' from 'broken'.

SYMPTOM

kubectl top pods returns an error on a freshly built cluster. Nothing is broken: the resource metrics API is not installed.

EVIDENCE — actual cluster output

```
$ kubectl top pods -n kcna-lab23
error: Metrics API not available

$ kubectl get apiservice v1beta1.metrics.k8s.io
Error from server (NotFound): apiservices.apiregistration.k8s.io
"v1beta1.metrics.k8s.io" not found

$ kubectl api-resources --api-group=metrics.k8s.io
NAME   SHORTNAMES   APIVERSION   NAMESPACED   KIND

# after installing metrics-server, then scaling it to zero:
$ kubectl get apiservice v1beta1.metrics.k8s.io
NAME                     AVAILABLE                      AGE
v1beta1.metrics.k8s.io   False (FailedDiscoveryCheck)   6m
```

ROOT CAUSE

- metrics.k8s.io is an aggregated API, not core
- kind, kubeadm and many distributions omit it
- An empty api-resources table = the group does not exist

FIX

- Install the metrics-server add-on
- Then confirm the APIService reads AVAILABLE True
- False (FailedDiscoveryCheck) = installed but unreachable

Source: Kubernetes documentation, Resource Metrics Pipeline: kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/

© 2026 Tertiary Infotech Academy Pte Ltd

METRICS

metrics-server Is Not A Monitoring System

They look adjacent and are not substitutes. Confusing them is a standard KCNA distractor.

	metrics-server	Prometheus
Purpose	Feed the HPA/VPA and kubectl top	Store and query time series for humans and alerts
Retention	A short in-memory window — no history at all	A local TSDB on disk, plus remote_write for long term
Metrics	CPU and memory only, from cgroups	Anything a target exposes, including your own business metrics
Query	kubectl top, or the raw metrics.k8s.io endpoint	PromQL: rate, aggregation, quantiles, joins, arithmetic
Answers 'what was it at 03:00?'	No — nothing is stored	Yes, for the whole retention period
Failure impact	Autoscaling stalls at the last known state	You lose visibility, not scheduling

CHOOSE

Run metrics-server because the HPA requires it; run Prometheus because engineers require history.

Source: Kubernetes documentation, Resource Metrics Pipeline: kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 23 — Events, Logs, Field Selectors and the Metrics Server

GOAL

Build the incident evidence trail: query Events as API objects with server-side field selectors, drive kubectl logs across containers and restarts, and establish exactly what kubectl top requires on a kind cluster that does not ship metrics-server.

YOU WILL PRODUCE

- A namespace emitting a real mixed event stream (Normal, Warning FailedMount, Warning BackOff) filtered with compound --field-selector queries
- A two-container Pod proving that only stdout/stderr reaches kubectl logs, plus a crash-looping container whose cause is only visible via --previous
- A completed event-triage worksheet recording each query, what it returned, and whether kubectl top was available (Path A install or Path B skip)

CONCEPTS EXERCISED

- Events are aggregated (count + firstTimestamp) and garbage-collected after --event-ttl, one hour by default — a triage signal, never an audit log
- Field selectors are server-side but restricted to indexed fields: type, reason and involvedObject.* are selectable, message is not
- kubectl logs captures only stdout/stderr, retains one previous container generation, and is bounded by kubelet log rotation — the case for node-level shippers
- kubectl top is an add-on requiring metrics-server behind the aggregation layer; on kind it needs --kubelet-insecure-tls, and it holds no history

LAB	NAMESPACE	MAPS TO	TIME
Lab 23	kcna-lab23	LO6 · A6 · K7	50 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-23-events-logs-metrics/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

PIPELINE

The Observability Pipeline, End To End

PRODUCE

Instrumented app
Client library keeps counters, gauges and histograms in memory

Exporter
Translates a system that cannot be changed into the same format

/metrics endpoint
Plain text over HTTP, Content-Type text/plain; version=0.0.4

COLLECT AND STORE

Service discovery
kubernetes_sd_configs enumerates Pods, Services and Nodes

Scrape loop
An HTTP GET per target per scrape_interval. Nothing is pushed

TSDB
Samples appended per series; series identified by name + labels

CONSUME

PromQL
rate, aggregation, quantiles, ratios — computed at query time

Rules and Alertmanager
Recording rules precompute; alerts route, group, silence

Dashboards
Grafana is the usual front end — and is NOT a CNCF project

Source: Prometheus documentation, Configuration — scrape_config and kubernetes_sd_config: prometheus.io/docs/prometheus/latest/configuration/configuration/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Reading The Text Exposition Format

A scrape is an HTTP GET returning plain text. That is the entire protocol.

GET /metrics · text/plain; version=0.0.4

```
# HELP depot_portal_shipments_scanned_total Barcodes scanned.
# TYPE depot_portal_shipments_scanned_total counter
depot_portal_shipments_scanned_total{lane="inbound"} 1056
# TYPE depot_portal_queue_depth gauge
depot_portal_queue_depth{depot="sg-tuas"} 26
# TYPE depot_portal_scan_latency_seconds histogram
depot_portal_scan_latency_seconds_bucket{le="0.5"} 60
depot_portal_scan_latency_seconds_bucket{le="1"} 81
depot_portal_scan_latency_seconds_bucket{le="2.5"} 93
depot_portal_scan_latency_seconds_bucket{le="5"} 96
depot_portal_scan_latency_seconds_bucket{le="+Inf"} 97
depot_portal_scan_latency_seconds_sum 39
depot_portal_scan_latency_seconds_count 97
# TYPE depot_portal_build_info gauge
depot_portal_build_info{version="6.0.0"} 1
```

READING IT

1. **Name, label set, value**
   metric_name{k="v",...} value. Each distinct label combination is a SEPARATE time series. Values are 64-bit floats; no integer type.

2. **# TYPE is not decoration**
   It is what tells Prometheus — and you — whether rate() is legal on this series. # HELP is what an engineer reads at 3 a.m.

3. **_total marks a counter**
   Convention, not enforcement. Units belong in the name (_seconds, _bytes), never in a label.

4. **_info pattern: value always 1**
   build_info carries version and revision in LABELS so you can join them in PromQL instead of stamping them onto every metric.

Source: Prometheus documentation, Exposition formats: prometheus.io/docs/instrumenting/exposition_formats/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

The Four Metric Types

Type determines which functions are legal, and whether you can aggregate across replicas.

1 Counter

Monotonically increases; resets to 0 only when the process restarts. Never graph it raw — always rate() or increase().

2 Gauge

A current reading that may go up or down: queue depth, temperature, in-flight requests. Use the value directly.

3 Histogram

Cumulative _bucket series plus _sum and _count. Quantiles are estimated at QUERY time with histogram_quantile().

4 Summary

Quantiles computed in the CLIENT at observation time. Cheap to read, but you cannot aggregate quantiles across instances.

5 Histogram vs summary

Histograms are aggregatable across Pods; summaries are not. That single fact decides it in Kubernetes.

6 Native histograms

A newer, high-resolution encoding that removes the fixed-bucket compromise. Treat as an advanced topic for KCNA.

Source: Prometheus documentation, Metric types: prometheus.io/docs/concepts/metric_types/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Counter vs rate() — A Worked Example

Worked arithmetic on seven scrapes at a 10s interval — not measured cluster data.

Counter increase (units) and derived per-minute rate



WHAT THE DATA SHOWS

The samples

1000, 1009, 1018, 1028, 1037, 1046, 1056 at t = 0,10,...,60s. Deltas: 9, 9, 10, 9, 9, 10.

Per-second rate

delta / 10s = 0.9, 0.9, 1.0, 0.9, 0.9, 1.0. Multiplied by 60 for a readable per-minute series.

The counter line is useless

It rises forever. 1056 tells you only how long the process has been up, not how busy it is.

rate() drops __name__

It returns a NEW series, so the metric name disappears from the result labels.

Source: Prometheus documentation, Query functions — rate(), histogram_quantile(): prometheus.io/docs/prometheus/latest/querying/functions/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Decomposing A PromQL Expression

Four operations, each with its own failure mode.

PromQL · read inside out

```
# The expression
sum by (status) (
  rate(depot_portal_http_requests_total[1m])
)

# 1. selector -> depot_portal_http_requests_total
#    instant vector: one series per label set
# 2. [1m]     -> range vector: every sample in the
#    last minute. scrape_interval 10s => ~6 samples
# 3. rate()   -> per-second average increase over the
#    window, with counter resets corrected
# 4. sum by (status) -> collapse all other labels,
#    keep status only; __name__ is dropped

# WRONG — rate() on a gauge:
rate(depot_portal_queue_depth[1m])
# every fall is read as a counter reset
```

READING IT

1. **The range window rule**

   rate() needs at least two samples in the window. Make the range at least 4x the scrape interval; [5s] on a 10s scrape returns empty.

2. **Empty, not an error**

   A too-narrow window yields an empty result set. Silence in PromQL is a common and dangerous failure mode.

3. **Ratios, not absolute counts**

   sum(rate(...{status="502"}[2m])) / sum(rate(...[2m])) is dimensionless, so it never needs retuning as traffic grows.

4. **histogram_quantile needs le**

   sum by (le) (rate(..._bucket[5m])). Dropping le from the by clause is the classic mistake and yields NaN.

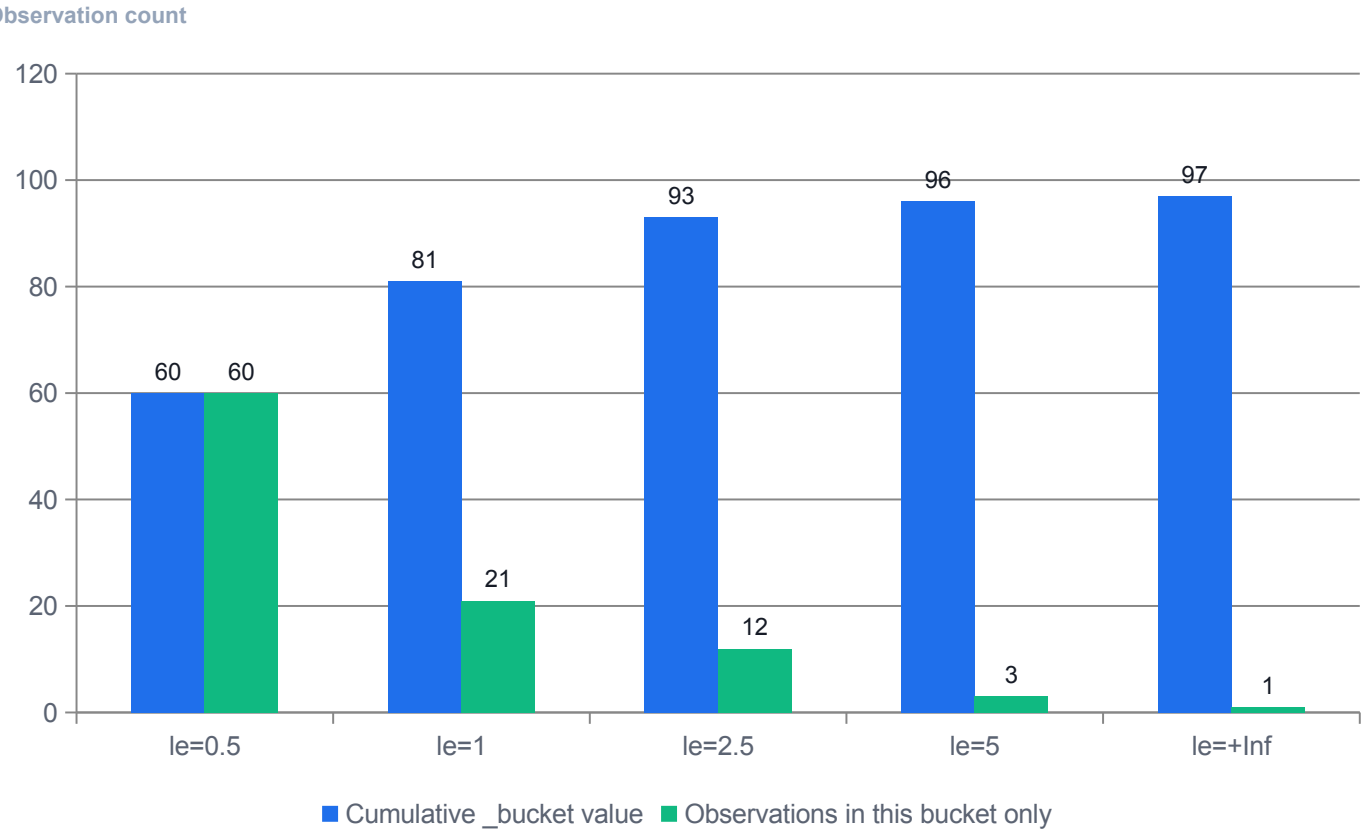
Source: Prometheus documentation, Query functions — rate(), histogram_quantile(): prometheus.io/docs/prometheus/latest/querying/functions/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Inside A Latency Histogram — Worked Example

Worked example over one scrape of depot_portal_scan_latency_seconds. Exact arithmetic, not measured production data.



WHAT THE DATA SHOWS

Buckets are cumulative

le means 'less than or equal to'. 60, 81, 93, 96, 97 must never decrease as le rises.

+Inf equals _count

97 = 97. If it does not, the exporter is broken. Per-bucket counts are the differences: 60,21,12,3,1.

The mean hides the tail

_sum / _count = 39 / 97 = 0.402 s. That number reassures nobody who waited.

p95 by interpolation

0.95 x 97 = 92.15, which falls in the le=2.5 bucket: 1 + 1.5 x (92.15-81)/12 = 2.39 s.

Source: Prometheus documentation, Query functions — rate(), histogram_quantile(): prometheus.io/docs/prometheus/latest/querying/functions/

© 2026 Tertiary Infotech Academy Pte Ltd

Reading A Scrape Configuration

Static targets on purpose — so the scrape anatomy is visible with nothing hidden behind a CRD.

prometheus.yml mounted from a ConfigMap · Lab 24

```
global:
  scrape_interval: 10s      # storage resolution
  evaluation_interval: 30s  # how often rules run
  external_labels:
    cluster: kcna-kind      # stamped on exported data
scrape_configs:
  - job_name: depot-portal  # becomes label job="..."
    metrics_path: /metrics  # this is the default
    scrape_timeout: 5s      # must be <= interval
    static_configs:
      - targets:
          - depot-metrics-sim.kcna-lab24.svc:9100
        labels:
          depot: sg-tuas
    relabel_configs:        # runs BEFORE the scrape
      - source_labels: [__address__]
        regex: '([^:]+):.*'
        target_label: instance
```

READING IT

1. **scrape_interval is your resolution**
   Every stored series has exactly this granularity. A rate() window must be several multiples of it or it returns nothing.

2. **job_name becomes a label**
   Along with instance, cluster and env, Prometheus composes labels you never wrote into the exposition. Learn to spot their origin.

3. **relabel vs metric_relabel**
   relabel_configs decides WHAT to scrape and how to label it. metric_relabel_configs runs after and decides what to KEEP.

4. **Static here, discovered in production**
   kubernetes_sd_configs enumerates Pods directly. Scraping through a Service load-balances across replicas and corrupts counters.

Source: Prometheus documentation, Configuration — scrape_config and kubernetes_sd_config: prometheus.io/docs/prometheus/latest/configuration/configuration/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Pull vs Push

The decisive advantage of pull is liveness detection you did not have to build.

	Pull (Prometheus scrape)	Push (StatsD, OTLP export)
Who initiates	The monitoring system, on a schedule	The application, whenever it chooses
Target discovery	Required — service discovery or static config	Not required; targets find the collector
Is the target up?	Free: a failed scrape IS the signal, up == 0	Needs a separate heartbeat; silence is ambiguous
Short-lived jobs	Awkward — needs a Pushgateway	Natural fit
Network direction	Monitoring must be able to reach targets	Targets must be able to reach the collector
Overload behaviour	The monitoring system controls the rate	Targets can flood the collector

CHOOSE

Pull for long-lived Kubernetes workloads; push via a gateway or OTLP for batch jobs and anything behind a one-way network.

Source: Prometheus documentation, Configuration — scrape_config and kubernetes_sd_config: prometheus.io/docs/prometheus/latest/configuration/configuration/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

Labels, Cardinality And Why Clusters Fall Over

Cardinality is the single operational constraint that decides whether a Prometheus survives.

1 One label set = one series

Every distinct combination of label values creates a new time series with its own memory and index.

2 Cardinality multiplies

10 statuses x 20 endpoints x 50 pods = 10,000 series from a single metric name.

3 Never label with an ID

user_id, request_id, session, full URL path or a raw error message. One series per user is unbounded.

4 Watch head series

prometheus_tsdb_head_series counts active series. A cardinality explosion shows there long before an OOMKill.

5 Drop late, not never

metric_relabel_configs with action: drop discards noisy series after the scrape — the standard control.

6 Aggregation is the answer

Record what you need with a recording rule, then drop the raw high-cardinality series.

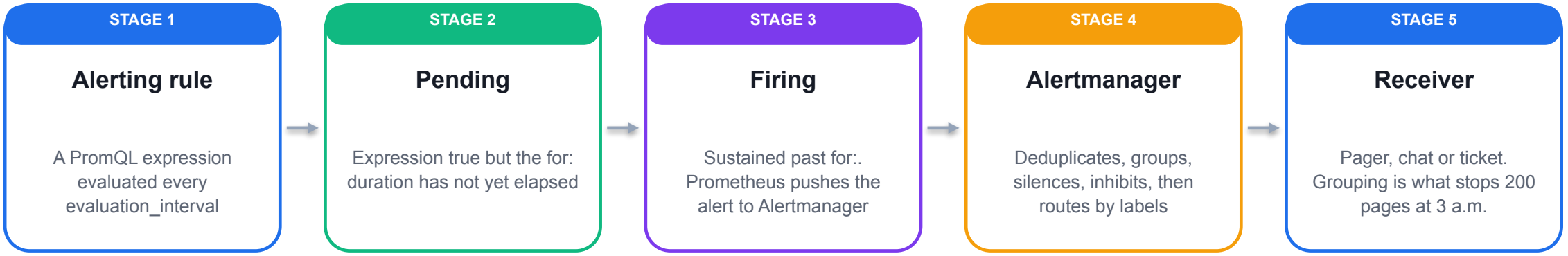
Source: Prometheus documentation, Metric types: prometheus.io/docs/concepts/metric_types/

© 2026 Tertiary Infotech Academy Pte Ltd

ALERTING

From Rule To Human

Prometheus evaluates; Alertmanager decides who is disturbed and how often.



WHAT IT MEANS

for: removes flapping, grouping removes duplicates, and inhibition suppresses the 40 downstream alerts caused by the one upstream failure you already paged for.

Source: Prometheus documentation, Alerting — Alertmanager: prometheus.io/docs/alerting/latest/alertmanager/

© 2026 Tertiary Infotech Academy Pte Ltd

PROMETHEUS

A Valid Config That Scrapes Nothing

The pull model's free liveness signal is only free if you actually alert on it.

SYMPTOM

The scrape config loads without complaint and the dashboard goes blank. up == 0 is the only thing that noticed.

EVIDENCE — actual cluster output

```
# the broken job points at port 9101 instead of 9100
$ promql 'up{job="depot-portal"}'
{"status":"success","data":{"resultType":"vector",
 "result":[{"metric":{"job":"depot-portal",...},
  "value":[<ts>,"0"]}]}}

# /api/v1/targets tells you WHY
"health":"down"
"lastError":"Get \"http://...:9101/metrics\": dial tcp
 connect: connection refused"

# and the series simply stop — no error anywhere
$ promql 'rate(depot_portal_http_requests_total[1m])'
{"status":"success","data":{"result":[]}}
```

ROOT CAUSE

- Schema-valid YAML pointing at the wrong port
- Prometheus loads it happily — nothing rejects it
- Absent series look identical to a healthy quiet service

FIX

- Alert on up == 0 for every job, always
- Check /targets health and lastError first
- Never alert on absence of data alone

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

OPENTELEMETRY

How A Trace Is Produced And Shipped

A span is a named, timed operation with attributes; a trace is the causal tree of spans sharing one trace id.



WHAT IT MEANS

OpenTelemetry standardises GENERATION and COLLECTION. It does not store or query anything — that is still Prometheus, Jaeger or a vendor backend.

Source: OpenTelemetry documentation, What is OpenTelemetry and the Collector: opentelemetry.io/docs/what-is-opentelemetry/ ; opentelemetry.io/docs/collector/

© 2026 Tertiary Infotech Academy Pte Ltd

OPENTELEMETRY

The Vocabulary You Will Be Asked For

OpenTelemetry is CNCF INCUBATING — widely adopted, and formally not yet graduated.

Trace and span

1

A trace is one request's journey. A span is one operation within it, with a parent, a start, a duration.

Context propagation

2

Injecting and extracting the trace context at every hop. W3C traceparent is the standard header.

API vs SDK

3

The API is what your code calls and is stable; the SDK is the swappable implementation that samples and exports.

Collector

4

A standalone binary: receivers in, processors in the middle, exporters out. Decouples the app from the backend.

OTLP

5

The convergence point. One protocol for all three signals, now accepted natively by most backends including Prometheus.

Semantic conventions

6

Agreed attribute names (http.request.method, k8s.pod.name) so telemetry from different teams can be joined.

Source: OpenTelemetry documentation, What is OpenTelemetry and the Collector: opentelemetry.io/docs/what-is-opentelemetry/ ; opentelemetry.io/docs/collector/

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

Prometheus and OpenTelemetry Are Not Rivals

One generates and moves telemetry; the other stores and queries it. Most clusters run both.

	Prometheus	OpenTelemetry
What it is	A monitoring system: scrape, store, query, alert	A standard plus implementations for generating and collecting telemetry
Signals	Metrics (and native support for exemplars)	Traces, metrics and logs
Data model	Pull, text exposition, its own TSDB	Push, OTLP, no storage of its own
Instrumentation	Client libraries per language	One API/SDK per language, plus auto-instrumentation
CNCF maturity	Graduated (2018, the second project to graduate)	Incubating
Together	Prometheus can ingest OTLP and scrape OTel Collectors	The Collector can export in Prometheus format

CHOOSE

Instrument with OpenTelemetry for portability; store and alert on metrics with Prometheus.

Source: CNCF project list and maturity levels: www.cncf.io/projects/ (checked 5 Sep 2026)

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

The Observability Neighbourhood — Stated Precisely

Grafana and Loki are NOT CNCF projects — they are open source from Grafana Labs, and they are absent from this grid for that reason.



Prometheus

CNCF GRADUATED · metrics, TSDB, alerting



OpenTelemetry

CNCF INCUBATING · API, SDK, Collector



Fluentd

CNCF GRADUATED · log collection



Fluent Bit

Sub-project of Fluentd · lightweight node agent



Jaeger

CNCF GRADUATED · distributed tracing backend



Thanos

CNCF INCUBATING · long-term Prometheus storage



Cortex

CNCF INCUBATING · horizontally scalable Prometheus

Source: CNCF project list and maturity levels: www.cncf.io/projects/ (checked 5 Sep 2026)

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 24 — Metrics, Prometheus Exposition and the Observability Pipeline

GOAL

Run a real scrape target and a single self-contained Prometheus, read the text exposition format line by line, and query PromQL against metric names you produced yourself — including the error-ratio query that would have detected INC-4417.

YOU WILL PRODUCE

- A working exporter serving depot_portal_* metrics in Prometheus text format at /metrics, scraped by hand with wget before Prometheus ever touches it
- A Prometheus Pod scraping two named static targets from a scrape config you wrote, with both targets reporting up == 1
- A completed exposition worksheet recording counter-versus-gauge behaviour, the measured rate against its predicted band, and the up == 0 signal during failure injection

CONCEPTS EXERCISED

- The four observability signals and which question each one answers
- Pull versus push: Prometheus synthesises the up metric for free, so a failed scrape is itself the liveness signal
- Counter, gauge and histogram: why rate() exists, why buckets are cumulative, and why le must survive aggregation
- OpenTelemetry generates and collects, Prometheus stores and queries; Prometheus is CNCF-graduated, OpenTelemetry incubating, Grafana not CNCF at all

LAB	NAMESPACE	MAPS TO	TIME
Lab 24	kcna-lab24	LO6 · A6 · K7	45 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-24-prometheus-observability/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

REGISTERED TOPIC 8

Troubleshooting

08

A repeatable triage method for application, worker-node and control-plane failure — carried straight into this afternoon's practical assessment.

© 2026 Tertiary Infotech Academy Pte Ltd

THE METHOD

Phase first, cause second.

Establish WHICH phase a workload failed in before you form any hypothesis about why. Asking for logs on a Pending Pod is not merely useless — the error you get back sounds like a container problem, and it is not.

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

The Triage Decision Tree — Stop At The First NO

Five questions, in this order. Each is only meaningful if the previous one answered YES.

STAGE 1

Q1 Scheduled?

kubectl get pod -o wide -> NODE column. NO = Pending; the scheduler refused and no container exists

→

STAGE 2

Q2 Image pulled?

kubectl describe pod -> events. NO = ErrImagePull / ImagePullBackOff; still nothing has run

→

STAGE 3

Q3 Started and stayed up?

STATUS and RESTARTS. NO = CrashLoopBackOff / Error / OOMKilled. Now logs exist — use --previous

→

STAGE 4

Q4 Ready?

READY n/n. NO = readiness failing; running but withdrawn from Service backends

→

STAGE 5

Q5 Endpoints?

kubectl get endpointslices -l kubernetes.io/service-name=<svc>. NO = selector or port mismatch

WHAT IT MEANS

All five YES? The workload is healthy — now look OUTSIDE it: NetworkPolicy, DNS, Ingress, or the application's own config.

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Symptom String -> Phase -> The One Command

The exact string is the diagnosis. Learn the strings, not the anecdotes.

Pending, NODE <none>	Q1 · kubectl describe pod -> Warning FailedScheduling
does not have a host assigned	Q1 · the unscheduled-Pod log error. Stop asking for logs
ErrImagePull / ImagePullBackOff	Q2 · kubectl describe pod -> the Failed event's message
waiting to start: trying and failing to pull image	Q2 · the image-pull log error. Different string, different phase
CrashLoopBackOff, RESTARTS climbing	Q3 · kubectl logs --previous, then lastState.terminated.exitCode
Warning Unhealthy + Normal Killing	Q3 · liveness-induced restart. Compare the probe budget with boot time
0/1 Running, RESTARTS 0	Q4 · readiness failing. describe pod -> Warning Unhealthy
ENDPOINTS <unset>, connection refused	Q5 · compare svc .spec.selector with the Pod labels
Request times out (not refused)	Past Q5 · suspect NetworkPolicy or a missing route, not endpoints
Pending with NO FailedScheduling event at all	Control plane · nothing is evaluating the Pod. Suspect the scheduler

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

One Command Localises Four Failures

Before any describe, any logs, any guessing — read the table.

Lab 25 · four broken workloads, one snapshot

```
$ kubectl get pods -n kcna-lab25 -o wide
NAME                     READY   STATUS             RESTARTS   NODE
depot-api-7d4b9c6f85     0/1     ImagePullBackOff   0          kind-cp
depot-portal-6f9c8d7b54  1/1     Running            0          kind-cp
depot-tracker-5c8f7b9d64 1/1     Running            0          kind-cp
depot-tracker-5c8f7b9d64 1/1     Running            0          kind-cp
manifest-indexer-8b7d5c  0/1     Pending            0          <none>
shipment-worker-59d4c8b7  0/1    CrashLoopBackOff   4 (21s)    kind-cp
triage-client            1/1     Running            0          kind-cp

$ kubectl get endpointslices -n kcna-lab25
NAME                 ADDRESSTYPE  PORTS    ENDPOINTS
depot-portal-4bp7q   IPv4         80       10.244.0.21,10.244.0.22
depot-tracker-9xz4k  IPv4         <unset>  <unset>
```

WHAT TO LOOK AT

1. **NODE <none> answers Q1 instantly**
   manifest-indexer never reached a node. Its failure is scheduling, and no amount of log reading will help.

2. **RESTARTS separates Q2 from Q3**
   shipment-worker has restarted four times, so the image obviously pulled. depot-api has zero restarts and never started.

3. **1/1 Running is not 'working'**
   Both depot-tracker Pods are green. The failure is one layer out, at the Service — which is why Q5 exists.

4. **<unset> endpoints is the answer**
   Compare against depot-portal's two addresses. A silent failure: no events, no restarts, no logs anywhere.

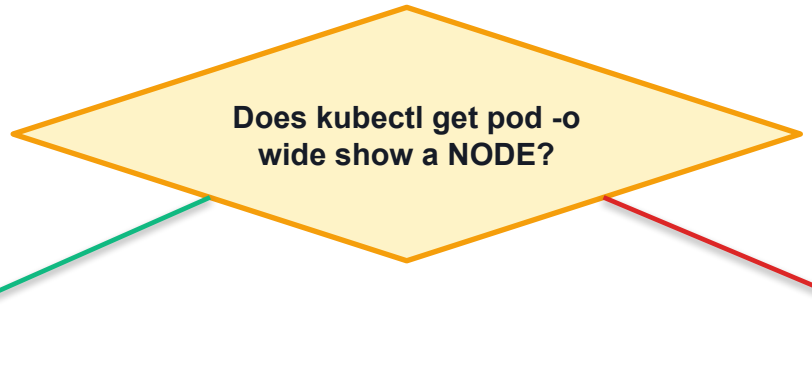
Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Q1 — Did The Scheduler Place It?

The scheduler decides on REQUESTS, not on live usage — a 5% busy node can still be fully committed.



YES — it was scheduled

- Containers exist; move to Q2 (image) then Q3 (start)
- describe pod events now come from the kubelet, not the scheduler
- Logs become meaningful from Q3 onward
- Restart counts are a reliable phase signal

NO — STATUS Pending

- The scheduler refused: read Warning FailedScheduling
- Insufficient cpu/memory · untolerated taint · affinity mismatch
- No matching PersistentVolume for a pending claim
- No FailedScheduling event AT ALL = suspect the scheduler itself

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Reading The Scheduler's Verdict

Same STATUS, at least five distinct causes. The event message is what separates them.

Lab 25 · DEP-1043 · Pending

```
$ kubectl describe pod -n kcna-lab25 -l app=manifest-indexer | tail -4
Events:
  Type     Reason            Age    From               Message
  Warning  FailedScheduling  4m5s   default-scheduler  0/1 nodes are
    available: 1 Insufficient cpu, 1 Insufficient memory.
    preemption: 0/1 nodes are available: 1 No preemption victims
    found for incoming pod.

$ kubectl get deploy manifest-indexer -o jsonpath=\
  '{.spec.template.spec.containers[0].resources.requests}'
{"cpu":"64","memory":"512Gi"}

$ kubectl get node $NODE -o jsonpath='{.status.allocatable.cpu}'
10

$ kubectl logs -n kcna-lab25 -l app=manifest-indexer
Error from server (BadRequest): pod manifest-indexer-... does not
have a host assigned
```

WHAT TO LOOK AT

1. **0/1 nodes are available, read literally**
   It is a per-node tally of reasons. On a large cluster the breakdown tells you how much of it each predicate ruled out.
2. **The five common predicates**
   Insufficient cpu/memory · untolerated taint · didn't match node affinity/selector · volume node affinity conflict · no free PV.
3. **Then do the arithmetic**
   64 cores and 512Gi requested against 10 allocatable cores. Requests, not usage, are what the scheduler compares.
4. **The log error is phase evidence**
   does not have a host assigned is the unscheduled signature — different from the image-pull Pod's waiting to start message.

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Q2 — Four Image-Pull Signatures, Four Different Fixes

Every one of these produces the same ImagePullBackOff status. Only the message distinguishes them.

not found / manifest unknown

1

The repository, name or tag is wrong. Compare against a known-good Deployment: the diff is the bug.

unauthorized / authentication required

2

Private registry with no credentials. Create an imagePullSecret and reference it in the Pod spec.

no such host / i/o timeout

3

The registry is unreachable FROM THE NODE. Network, DNS, egress proxy or firewall — not a manifest bug.

toomanyrequests

4

Registry rate limiting. Authenticate, or point at a mirror or pull-through cache.

Why there are no logs

5

No image means no container means no log stream. Reporters read that as the fault; it is a consequence.

imagePullPolicy matters

6

Always re-pulls every start; IfNotPresent trusts the node cache. A mutable tag plus IfNotPresent hides which build is actually running.

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Q3 — CrashLoopBackOff Diagnosed With --previous

The event sequence rules out Q2 before you read a single log line.

SYMPTOM

shipment-worker restarts endlessly. It prints one line and vanishes; the restart count is climbing.

EVIDENCE — actual cluster output

```
$ kubectl describe pod -n kcna-lab25 -l app=shipment-worker | tail -5
  Normal   Pulled   94s (x5 over 3m7s)  kubelet  image already present
  Normal   Created  94s (x5 over 3m7s)  kubelet  Created container
  Normal   Started  94s (x5 over 3m7s)  kubelet  Started container
  Warning  BackOff  12s (x14 over 3m4s) kubelet  Back-off restarting

$ kubectl logs -n kcna-lab25 -l app=shipment-worker --previous
[worker] starting shipment worker for depot sg-tuas
/bin/sh: /usr/local/bin/depot-shipment-worker: not found

$ kubectl get pod $WORKER -o jsonpath='{...lastState.terminated}'
worker  lastState.exitCode=127  reason=Error
```

ROOT CAUSE

- Pulled/Created/Started are all Normal
- So the image was fine and the container really ran
- Exit 127 = the shell could not find the command

FIX

- Correct command/args to match the image contents
- Verify with a throwaway --rm Pod from the same image
- Back-off is exponential, capped at 5 minutes

✓ **RESTARTS returns to 0 and the worker logs processed batches**

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Container Exit Codes You Will Meet Constantly

The exit code is what the kernel observed. The log is what the process chose to say.

0 — clean exit

1 The process finished. For a long-running workload this usually means the wrong restartPolicy or an entrypoint that returns.

1 — application error

2 Generic failure. Bad config, missing environment variable, unreachable dependency. Read the log; it is the app's own message.

127 — command not found

3 command/args disagree with the image contents. Prove it with a throwaway Pod from the same image.

137 — SIGKILL (128 + 9)

4 OOMKilled, or a failed liveness probe. Check reason: OOMKilled distinguishes memory from probe.

143 — SIGTERM (128 + 15)

5 Graceful stop requested. Normal during a rollout; a problem only if the app cannot finish in the grace period.

Where to read it

6 status.containerStatuses[].lastState.terminated.exitCode — authoritative, unlike the log text.

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Q5 — Does The Service Actually Have Backends?

v1 Endpoints was deprecated in v1.33 — inspect discovery.k8s.io/v1 EndpointSlice instead.



YES — routing is wired

- The failure is outside the workload and its Service
- Check NetworkPolicy: a deny gives a TIMEOUT, not a refusal
- Check DNS resolution from a client Pod in the same namespace
- Check Ingress class, path rules and the controller itself

NO — ENDPOINTS <unset>

- Selector mismatch: kubectl get pods -l <the svc selector>
- No Ready Pods: readiness is failing (that is Q4, not Q5)
- targetPort names a port the container never declared
- Symptom is an immediate connection refused, and total silence

Source: Kubernetes documentation, EndpointSlices; v1 Endpoints deprecated in v1.33: kubernetes.io/blog/2025/04/24/endpoints-deprecation/

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Everything Green, Nothing Works

The hardest of the four failures, because every dashboard in the building is green.

SYMPTOM

depot-tracker Pods are 1/1 Running with zero restarts and the dashboard is green, but every request to the Service times out.

EVIDENCE — actual cluster output

```
$ kubectl exec -n kcna-lab25 triage-client -- \
    wget -qO- --timeout=5 http://depot-tracker.kcna-lab25.svc/
wget: can't connect to remote host (10.96.184.22): Connection refused

$ kubectl get endpointslices -n kcna-lab25 \
    -l kubernetes.io/service-name=depot-tracker
NAME                 ADDRESSTYPE  PORTS    ENDPOINTS   AGE
depot-tracker-9xz4k  IPv4         <unset>  <unset>     6m45s

$ kubectl get svc depot-tracker -o jsonpath='{.spec.selector}'
{"app":"depot-tracking"}
$ kubectl get pods -l app=depot-tracker -o jsonpath='{...labels}'
{"app":"depot-tracker","pod-template-hash":"5c8f7b9d64"}

$ kubectl get pods -n kcna-lab25 -l app=depot-tracking
No resources found in kcna-lab25 namespace.
```

ROOT CAUSE

- Service selector says depot-tracking
- Pods are labelled depot-tracker
- A selector that matches nothing is silent — no event

FIX

- Correct the Service selector, not the Deployment
- kubectl get pods -l <selector> is the one-step proof
- Refused = no backend; timeout = policy or route

Source: Lab 22-25 transcripts, isolated kind cluster, Kubernetes v1.37.0

© 2026 Tertiary Infotech Academy Pte Ltd

TRIAGE

Past Q5 — Where The Path Can Still Break

Refused and timed out are different diagnoses. That distinction is worth real minutes in an incident.

Connection REFUSED

1

Something answered and said no: usually a ClusterIP with no backends, or nothing listening on the target port.

Connection TIMED OUT

2

Nothing answered at all: a NetworkPolicy deny, a missing route, or a wrong CIDR. Different fix entirely.

DNS: NXDOMAIN

3

Wrong name or wrong namespace. Fully qualify: svc.namespace.svc.cluster.local, and check the CoreDNS Pods.

Resolved but unreachable

4

DNS gave an IP, so name resolution is fine. The fault is routing, policy or the port — stop debugging DNS.

Default-deny is namespace-scoped

5

An empty-podSelector ingress policy denies everything in that namespace, including probes from peers.

Ingress needs a controller

6

An Ingress object with no controller reconciling it does nothing at all — and reports almost nothing.

Source: Kubernetes documentation, Debug Services: kubernetes.io/docs/tasks/debug/debug-application/debug-service/

© 2026 Tertiary Infotech Academy Pte Ltd

NODE FAILURE

Reading Node Conditions And Commitment

Four conditions and one commitment table answer most node-level questions.

kubectl describe node · Kubernetes v1.37.0

```
$ kubectl describe node $NODE | sed -n '/^Conditions:/,/^Addresses:/p'
Conditions:
  Type             Status  Reason                       Message
  MemoryPressure   False   KubeletHasSufficientMemory   sufficient
  DiskPressure     False   KubeletHasNoDiskPressure     no disk press.
  PIDPressure      False   KubeletHasSufficientPID      sufficient PID
  Ready            True    KubeletReady                 kubelet is
                                                        posting ready

$ kubectl describe node $NODE | sed -n '/Allocated resources/,/Events/p'
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource  Requests     Limits
  cpu       1050m (10%)  1300m (13%)
  memory    690Mi (8%)   1224Mi (15%)
```

WHAT TO LOOK AT

1. **Ready is one of four conditions**

   The three pressure conditions are the leading indicators — they flip before Ready does, and they are what trigger eviction.

2. **Reason is machine-readable**

   KubeletHasSufficientMemory / KubeletHasDiskPressure. Alert on the condition, not on a substring of the human message.

3. **Allocated is REQUESTS, not usage**

   10% committed on a node that may be 60% busy. The scheduler only ever compares requests against allocatable.

4. **The classic confusion**

   'There is plenty of free memory, why won't it schedule?' — because requests, not free memory, are what was exhausted.

Source: Kubernetes documentation, Nodes — Conditions and taint-based eviction: kubernetes.io/docs/concepts/architecture/nodes/#condition

© 2026 Tertiary Infotech Academy Pte Ltd

NODE FAILURE

A Node Goes NotReady

Illustrative multi-node transcript in the documented output shapes — the training cluster is single-node and this is not induced.

SYMPTOM

Every Pod on one worker becomes unreachable. The node reports NotReady and its kubelet has stopped posting status.

EVIDENCE — actual cluster output

```
$ kubectl get nodes
NAME       STATUS     ROLES    AGE   VERSION
worker-01  Ready      <none>   14d   v1.34.2
worker-02  NotReady   <none>   14d   v1.34.2

$ kubectl describe node worker-02 | grep -A2 'Ready '
  Ready   Unknown   Kubelet stopped posting node status.

$ kubectl get node worker-02 -o jsonpath='{.spec.taints}'
[{"effect":"NoSchedule","key":"node.kubernetes.io/unreachable"},
 {"effect":"NoExecute","key":"node.kubernetes.io/unreachable"}]

$ systemctl status kubelet      # ON THE NODE, not via kubectl
   Active: failed (Result: exit-code)
```

ROOT CAUSE

- kubelet stopped renewing its node Lease
- After the monitor grace period the node controller acts
- Status Unknown usually means node or network is gone

FIX

- Restore the kubelet or replace the node
- NoExecute taint evicts Pods; controllers reschedule
- Never delete the Node object to 'fix' the symptom

Source: Kubernetes documentation, Nodes — Conditions and taint-based eviction: kubernetes.io/docs/concepts/architecture/nodes/#condition

© 2026 Tertiary Infotech Academy Pte Ltd

NODE FAILURE

Pressure, Taints And Eviction Order

Node failure is a controller reaction, not a status message — know what reacts and in what order.

1 The heartbeat is a Lease

The kubelet renews a Lease object in kube-node-lease. Losing it — not losing Pods — is what marks a node NotReady.

2 Taints the node controller adds

node.kubernetes.io/not-ready and node.kubernetes.io/unreachable, each as NoSchedule then NoExecute.

3 tolerationSeconds

NoExecute evictions honour a per-Pod tolerationSeconds, so Pods are not removed the instant a node blips.

4 DiskPressure evicts and GCs

The kubelet reclaims by deleting unused images and evicting Pods. Container logs and images are the usual culprits.

5 MemoryPressure eviction order

BestEffort first, then Burstable over its requests, Guaranteed last. This is why you set requests and limits.

6 PIDPressure

Process-ID exhaustion on the node. Rare, ugly, and usually a runaway fork loop in one container.

Source: Kubernetes documentation, Node-pressure Eviction: kubernetes.io/docs/concepts/scheduling-eviction/node-pressure-eviction/

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROL PLANE

Read-Only Health: /readyz?verbose

The supported way to ask the control plane how it is — and it changes nothing.

API server health endpoints · read-only

```
$ kubectl get --raw='/readyz?verbose' | tail -12
[+]ping ok
[+]log ok
[+]etcd ok
[+]etcd-readiness ok
[+]informer-sync ok
[+]poststarthook/bootstrap-controller ok
[+]poststarthook/rbac/bootstrap-roles ok
[+]shutdown ok
readyz check passed

$ kubectl get --raw='/livez?verbose' | tail -3
[+]etcd ok
[+]autoregister-completion ok
livez check passed

$ kubectl get componentstatuses
Warning: v1 ComponentStatus is deprecated in v1.19+
```

WHAT TO LOOK AT

1. **livez vs readyz vs healthz**
   livez = is the process functional; readyz = should it take traffic. Exactly the container probe distinction. healthz is deprecated.
2. **[-]etcd failed is THE line**
   It means the API server cannot reach its datastore. Everything else in the cluster is about to fail behind it.
3. **Individual checks, not a summary**
   ?verbose lists every registered check with + or -. Without it you get one word and no diagnosis.
4. **ComponentStatus is not a fallback**
   Deprecated since v1.19; may be empty or absent. Recognise it in old documentation and do not depend on it.

Source: Kubernetes documentation, Kubernetes API health endpoints (/livez, /readyz, /healthz): kubernetes.io/docs/reference/using-api/health-checks/

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROL PLANE

Read-Only Inspection Surfaces

Everything on this slide reads. Nothing on it modifies, restarts, scales or deletes.

THROUGH THE API

--raw='/readyz?verbose'

Per-check control-plane readiness, including etcd reachability

kube-system Pods

kubectl -n kube-system get pods -l tier=control-plane

Component logs

kubectl -n kube-system logs kube-scheduler-<node>

ON THE NODE, WHEN THE API IS DOWN

Static Pod manifests

/etc/kubernetes/manifests/ — cat only, never an editor

Container runtime

crictl ps --name kube-apiserver, when kubectl cannot answer

kubelet service

systemctl status kubelet and its journal on that node

MECHANISM

The kubelet runs whatever it finds in /etc/kubernetes/manifests/ with no API server, no validation and no rollback — one typo there takes kubectl away from you entirely.

Source: Kubernetes documentation, Static Pods: kubernetes.io/docs/tasks/configure-pod-container/static-pod/

© 2026 Tertiary Infotech Academy Pte Ltd

CONTROL PLANE

Which Component Is Down? Read The Blast Radius

Discussed, never induced. On a shared training cluster, breaking the control plane ends the day for everyone.

Component down	Still works	Breaks	First check
kube-apiserver	Running Pods; kube-proxy keeps routing	All kubectl, every controller, every state change	crictl ps on the node; etcd reachability
etcd	Almost nothing meaningful	API server goes read-only then fails; everything follows	--raw='/readyz?verbose' -> [-]etcd failed
kube-scheduler	Existing Pods run normally	New Pods stay Pending with NO FailedScheduling event at all	kube-system logs kube-scheduler-<node>
kube-controller-manager	Existing Pods run	Deployments create no ReplicaSets; endpoints stop updating	kube-system logs kube-controller-manager-<node>
kubelet on one node	Every other node is fine	That node goes NotReady; its Pods evicted after the grace period	systemctl status kubelet on that node

CHOOSE

Pending with a FailedScheduling event = the scheduler is alive and refusing. Pending with no event at all = suspect the scheduler itself.

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

TROUBLESHOOTING

Three Altitudes Of Failure

Localise the altitude first; the evidence you should gather is completely different at each one.

Application failure

- Confined to one workload; the cluster is healthy
- Q1-Q5 localise it in one or two commands
- Evidence: events, --previous logs, exit codes, EndpointSlice
- You own the fix — it is almost always in your manifest

Lab 25 | Q1-Q5

Worker-node failure

- Every Pod on one node is affected; others are fine
- Evidence: node conditions, pressure taints, kubelet status
- The node controller reacts for you: taint, then evict
- Fix on the node or replace it; never delete the Node object

conditions | taints

Control-plane failure

- Cluster-wide symptoms with no workload-level cause
- Evidence: /readyz?verbose, kube-system logs, crictl
- Static Pods mean there is no rollout and no undo
- READ ONLY in this course — failure modes are discussed

readyz | read-only

Source: Kubernetes documentation, Troubleshooting Clusters: kubernetes.io/docs/tasks/debug/debug-cluster/

© 2026 Tertiary Infotech Academy Pte Ltd

HANDS-ON LAB

Lab 25 — Troubleshooting Triage: Application, Node and Control Plane

GOAL

Work a queue of four real incidents using a repeatable five-question triage method that identifies the failure phase before the cause, then inspect node and control-plane health strictly read-only.

YOU WILL PRODUCE

- Four diagnosed and repaired workloads: an image-pull typo, a CrashLoopBackOff exiting 127, a Pending Pod the scheduler refused, and a Service whose selector matched nothing
- A completed triage worksheet recording, per ticket, the first failing question, the decisive command, the root cause and the fix
- A reusable decision tree plus an exit-code reference the learner can carry into the practical assessment

CONCEPTS EXERCISED

- Phase before cause: scheduled, pulled, started, ready, endpoints — stop at the first NO
- Schema-valid is not working: the API server accepts all four broken manifests, so every failure is a runtime failure
- Match the tool to the phase — describe and events for scheduler and kubelet decisions, logs --previous only once a container has run, EndpointSlice for routing
- Control-plane components are static Pods with no controller, no validation and no undo; diagnose with /readyz?verbose and kube-system logs, and never write

LAB	NAMESPACE	MAPS TO	TIME
Lab 25	kcna-lab25	LO6 · A6 · K7	55 min

Full step-by-step procedure: Learner Guide · courseware/labs/lab-25-troubleshooting-triage/README.md

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

1:10 – 1:50 PM

© 2026 Tertiary Infotech Academy Pte Ltd

KCNA · CLOUD NATIVE ARCHITECTURE

09

Ecosystem, Principles and Community

An explicitly examinable sub-domain: how CNCF projects are governed, how maturity is earned, and how to make an accurate claim about the landscape.

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

The Landscape Is Not The Foundation

Being able to say precisely what CNCF membership does and does not imply is itself an exam objective.

1 CNCF Landscape

An interactive catalogue of the whole cloud native tooling space, organised into categories. Most entries are NOT CNCF projects.

2 Listing is not endorsement

An entry on landscape.cncf.io says 'this exists and fits this category'. It says nothing about maturity, quality or governance.

3 CNCF-hosted projects

A far smaller set with a defined maturity level, a governance document, a security process and TOC oversight.

4 The Linux Foundation

CNCF is a foundation within the Linux Foundation. Trademarks, legal, events and the certification programme sit there.

5 How to check, not memorise

www.cncf.io/projects lists every hosted project and its current level. Levels change; the site is the source of truth.

6 The commonest exam error

Assuming a familiar tool is CNCF. Grafana, Loki, Terraform, Docker Engine and Ansible are not CNCF projects.

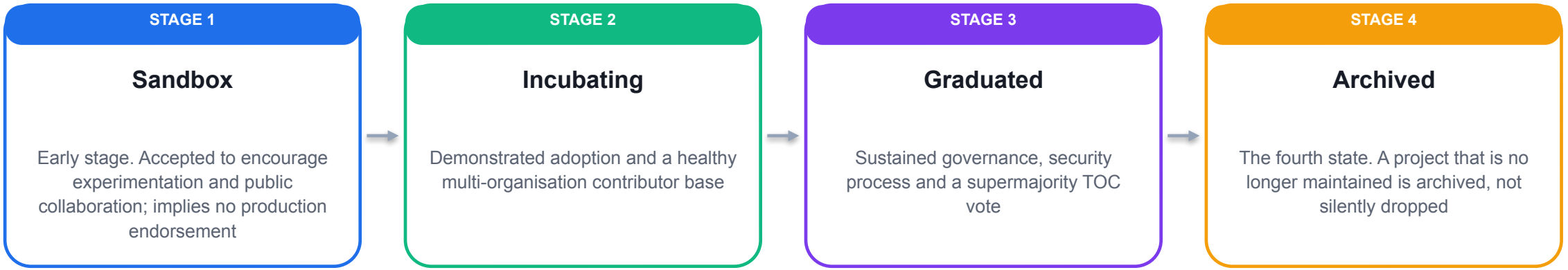
Source: CNCF project list and maturity levels: www.cncf.io/projects/ (checked 5 Sep 2026)

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

How A Project Moves Through CNCF Maturity

Promotion is a TOC decision against published criteria — not popularity, downloads or GitHub stars.



WHAT IT MEANS

Maturity describes the PROJECT's governance and adoption, not how good the software is. A sandbox project can be excellent; a graduated one can still be wrong for you.

Source: CNCF Technical Oversight Committee — project proposal and graduation criteria: github.com/cncf/toc

© 2026 Tertiary Infotech Academy Pte Ltd

ECOSYSTEM

What Each Promotion Actually Requires

Summarised from the published CNCF TOC criteria — the wording is revised over time, so cite the repository, not a slide.

	Sandbox	Incubating	Graduated
Signal	Potential and openness	Real production adoption	Sustainable, independent governance
Adopters	None required	At least three independent end users in production	Substantial, documented adoption
Contributors	An open, public repository	A healthy flow of committers from multiple organisations	Committers from at least two organisations
Process	Code of Conduct and a public licence	Documented, versioned releases and clear governance	Security disclosure process; OpenSSF Best Practices badge
Decision	TOC acceptance	TOC vote against the incubating criteria	TOC supermajority vote

CHOOSE

For KCNA: know the three levels, that archived is a real state, and that end-user adoption is the incubating gate.

Source: CNCF Technical Oversight Committee — project proposal and graduation criteria: github.com/cncf/toc

© 2026 Tertiary Infotech Academy Pte Ltd

COMMUNITY

Who Decides What — Governance In Practice

CNCF-wide guidance runs through the TOC and TAGs; Kubernetes runs its own SIGs. Do not conflate the two.

CNCF TOC

1 Technical Oversight Committee. Accepts projects, sets and applies the maturity criteria, and provides technical leadership across the foundation.

CNCF TAGs

2 Technical Advisory Groups (the successors to CNCF SIGs) advise the TOC in domains such as security, observability and runtime.

Kubernetes SIGs

3 Kubernetes' OWN structure: sig-network, sig-node, sig-storage and more, each with chairs, meetings and a public charter.

Kubernetes Steering

4 An elected committee that governs the project itself — process, subprojects and community, not code review.

Governing Board

5 The CNCF business side: budget, marketing and membership. Deliberately separated from technical decisions.

Code of Conduct

6 Adopting the CNCF Code of Conduct is a prerequisite for hosting, not an optional nicety.

Source: Kubernetes community governance — SIGs, Working Groups and the Steering Committee: github.com/kubernetes/community

© 2026 Tertiary Infotech Academy Pte Ltd

COMMUNITY

How You Actually Participate

'Community and Collaboration' is a named KCNA sub-domain — it is examined, not decorative.

1 KubeCon + CloudNativeCon

The CNCF flagship conference, run across several regions each year, with co-located project events and a contributor summit.

2 CNCF Ambassadors

Recognised community advocates who run meetups, write and mentor. A programme you apply to, not a job title.

3 Start with good-first-issue

Kubernetes and most CNCF projects label entry-level work. Documentation and test fixes are real contributions.

4 The contributor ladder

Kubernetes progresses member -> reviewer -> approver -> maintainer, recorded in OWNERS files, earned through review history.

5 DCO and CLA

Every commit must be signed off (DCO) or covered by a CLA. A contribution that is not is not merged, however good.

6 Slack, mailing lists, meetings

SIG meetings are public and recorded. Reading the notes before you propose anything is the unwritten rule.

Source: Kubernetes community governance — SIGs, Working Groups and the Steering Committee: github.com/kubernetes/community

© 2026 Tertiary Infotech Academy Pte Ltd

PRINCIPLES

Making An Accurate Claim About A Tool

Precision about the ecosystem is a professional habit and an examinable skill.

1 CNCF Technology Radar

An End User Community publication that places tools in rings (Adopt / Trial / Assess) based on an end-user survey at a point in time.

2 A radar is not a maturity level

It reports what surveyed end users do. It is not a TOC decision and confers no status on a project.

3 Three separate questions

Is it CNCF-hosted? At what maturity? And what do end users report doing with it? Answer them separately.

4 Vendor-neutral is the principle

Open governance, an open licence and no single-vendor control — the reason a standard like OTLP can be trusted.

5 Interoperability over integration

Cloud native design favours declared interfaces — CRI, CNI, CSI, OCI, OTLP — so components can be replaced.

6 Date your claims

Maturity levels and radar rings both change. Cite the source and the date you checked it, exactly as this deck does.

Source: CNCF project list and maturity levels: www.cncf.io/projects/ (checked 5 Sep 2026)

© 2026 Tertiary Infotech Academy Pte Ltd

PRINCIPLES

Cost Is An Observability Signal Too

At KCNA depth: know why cost needs allocating in Kubernetes and that an open model exists for it.

1 FinOps

The practice of making cloud spend a shared engineering responsibility, with visibility, allocation and optimisation as the phases.

2 Kubernetes hides cost

A cluster bill arrives per node. Cost per team, namespace or workload has to be derived — nothing reports it natively.

3 Allocation by request

Cost models attribute node cost to workloads using resource requests and actual usage, then roll up by label or namespace.

4 OpenCost

A CNCF-hosted project that implements an open cost allocation model for Kubernetes and exports it as Prometheus metrics.

5 Requests are the lever

Over-requesting reserves capacity nobody uses and is billed anyway. Right-sizing requests is the single biggest saving.

6 Same pipeline, different question

Cost data flows through the same scrape-store-query pipeline as every other metric.

Source: OpenCost project documentation: opencost.io

© 2026 Tertiary Infotech Academy Pte Ltd

SESSION 3 · 1:20 – 2:50 PM

10

Course Recap and Exam Readiness

Consolidate five days against the registered outcomes, then map them onto the current CNCF KCNA blueprint and this afternoon's WSQ assessment.

© 2026 Tertiary Infotech Academy Pte Ltd

RECAP

Five Days, Consolidated

Everything today feeds LO6 — monitor the system, then troubleshoot it — and closes the KCNA blueprint.

DAYS 1-4 — WHAT YOU BUILT

- **Day 1** Pods, containers, OCI images, the CRI, and multi-container patterns
- **Day 2** ReplicaSets, Deployments and rollout strategy; DaemonSets, Jobs, CronJobs; scheduling, requests, limits, QoS, HPA
- **Day 3** Services and EndpointSlice, DNS, Ingress; namespaces, ServiceAccounts, RBAC, Secrets and NetworkPolicy
- **Day 4** Volumes, PV/PVC/StorageClass, StatefulSets; control-plane anatomy and etcd; Helm, Kustomize and GitOps

DAY 5 — WHAT YOU ADDED TODAY

- **Signals** Metrics, logs, traces, Events and profiles; the four golden signals, RED and USE; SLI/SLO and error budgets
- **Health** Liveness, readiness and startup probes read as time budgets, and the controllers that react to them
- **Evidence** Events with field selectors, logs with --previous, the resource metrics pipeline and its limits
- **Pipeline** Exposition format, scrape, TSDB, PromQL, alerting; OpenTelemetry's place beside Prometheus
- **Method** The five-question triage tree across application, node and control-plane failure

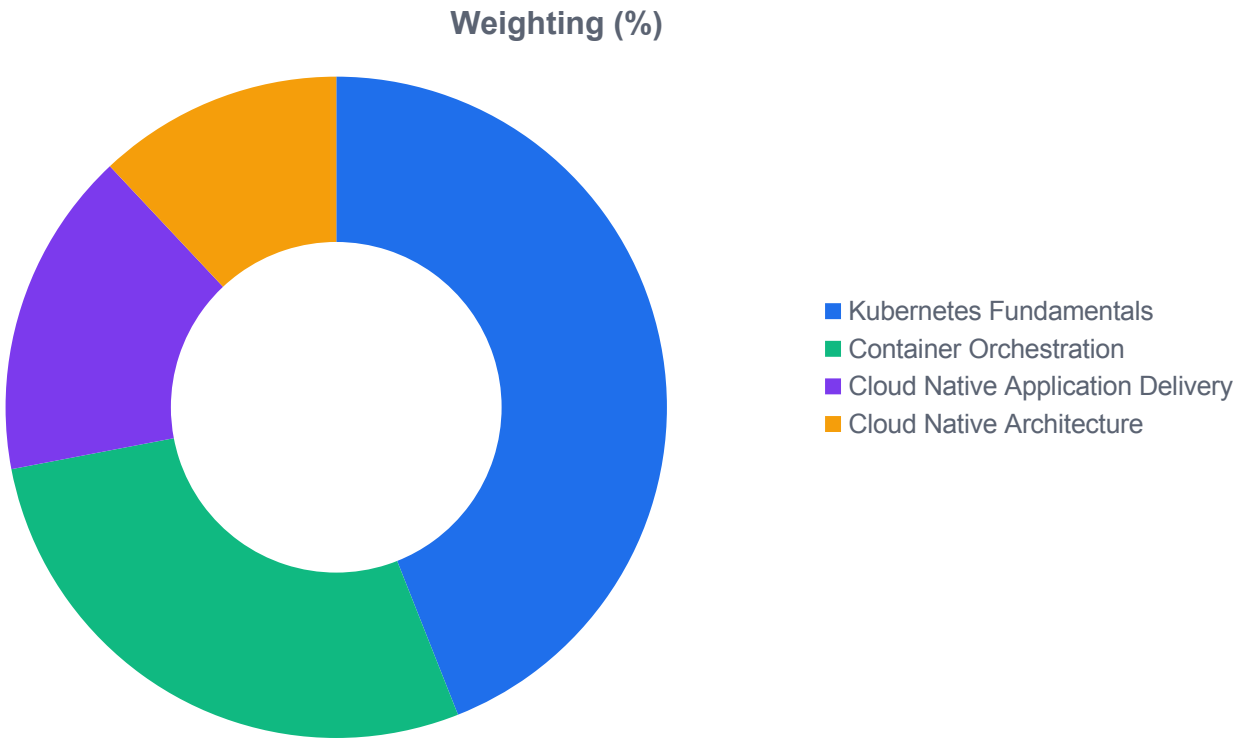
Source: Course outline, course_data.TOPICS and LEARNING_OUTCOMES

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM READINESS

The Current KCNA Blueprint

Effective 24 November 2025. The superseded five-domain 46/22/16/8/8 split is still circulating — do not study to it.



WHAT THE DATA SHOWS

Kubernetes Fundamentals 44%

Core concepts, administration, scheduling, containerization. Nearly half the paper, and Days 1-2 of this course.

Container Orchestration 28%

Networking, security, troubleshooting and storage — Days 3-5.

Application Delivery 16%

Application delivery and debugging — Day 4's Helm, Kustomize and GitOps, plus today's triage method.

Cloud Native Architecture 12%

Observability, ecosystem and principles, community and collaboration. All three were taught this afternoon.

Source: CNCF Curriculum repository, KCNA_Curriculum.pdf (github.com/cncf/curriculum), revision effective 24 Nov 2025. Retrieved 10 Sep 2026.

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM READINESS

KCNA Exam Facts

Verified facts from course_data.KCNA_EXAM_FACTS, plus the logistics learners consistently ask about.

Format	Online, proctored, multiple-choice
Questions	60
Duration	90 minutes
Passing score	75%
Validity	2 years
Prerequisites	None

Retake	One retake included with the exam purchase
Eligibility window	12 months from purchase
Delivery	Remote proctored; a webcam, a quiet room and a clear desk are required
Curriculum	Published in the CNCF curriculum repository and versioned — check the effective date
Relationship to WSQ	KCNA is a separate Linux Foundation certification; this course prepares for it and is assessed under WSQ
Confirm before booking	Terms are revised — verify on the official certification page

Source: Linux Foundation, Kubernetes and Cloud Native Associate (KCNA) certification page: training.linuxfoundation.org/certification/kubernetes-cloud-native-associate/

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM READINESS

Practice as a Learning Diagnostic

A practice resource supports diagnosis; the official curriculum defines coverage.

https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna

```
Official KCNA blueprint

  Kubernetes Fundamentals           44%
  Container Orchestration           28%
  Cloud Native Application Delivery 16%
  Cloud Native Architecture         12%

  Official exam: 60 questions / 90 minutes
```

RESOURCE OVERVIEW

- Timed practice reveals pace as well as concept gaps.
- Domain-level error patterns indicate revision priorities.
- Explanations demonstrate understanding beyond answer recognition.
- The official CNCF curriculum defines current topic coverage.

The Learner Guide contains the detailed practice and revision approach.

https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM READINESS

Technique For A 60-Question, 90-Minute Paper

75% of 60 is 45 correct answers. Time management, not obscure knowledge, is what usually costs the pass.

90 seconds per question

1

5,400 seconds over 60 questions. Flag anything past two minutes and move — the last ten questions are worth the same as the first ten.

Read the stem twice

2

Distractors are usually correct statements that do not answer the question asked. Identify the question before reading options.

Eliminate on consequence

3

Probe questions turn on what FAILS, not on syntax. Component questions turn on what BREAKS when it is gone.

Watch for superseded facts

4

v1 Endpoints is deprecated; ComponentStatus is deprecated; community ingress-nginx retired in March 2026 — the Ingress API did not.

Be exact about CNCF status

5

Graduated, incubating, sandbox, or not a CNCF project at all. This is a whole sub-domain of easy marks.

Answer everything

6

There is no negative marking. An unanswered question and a wrong one score identically.

Source: Linux Foundation, Kubernetes and Cloud Native Associate (KCNA) certification page: training.linuxfoundation.org/certification/kubernetes-cloud-native-associate/

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

Where KCNA Sits, And What Comes After

Confirm current formats, durations and prerequisites on the Linux Foundation certification pages before you book anything.

Certification	Format	Focus	Sensible order
KCNA	60 multiple-choice, 90 min, 75%	Breadth across Kubernetes and the cloud native ecosystem	First — no prerequisites
KCSA	Multiple-choice	Cloud native security fundamentals, at the same associate level	Alongside or just after KCNA
CKAD	Performance-based, hands-on	Building and running applications on Kubernetes	Next, if you write and ship workloads
CKA	Performance-based, hands-on	Operating and troubleshooting clusters	Next, if you operate clusters
CKS	Performance-based, hands-on	Cluster hardening; requires a current CKA	Last — CKA is a prerequisite

CHOOSE

Take KCNA for the breadth, then choose CKAD or CKA by what you actually do at work — not by which sounds harder.

Source: Linux Foundation, Kubernetes and Cloud Native Associate (KCNA) certification page: training.linuxfoundation.org/certification/kubernetes-cloud-native-associate/

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

The Thirty Days After This Course

A concrete plan beats intention. This one assumes a few hours a week, not a sabbatical.

Week 1 — rebuild from memory

1

Recreate Labs 22-25 in a fresh kind cluster with the READMEs closed. Recall, not recognition, is what the exam tests.

Week 2 — close the weak domain

2

Use your practice-bank domain report. Re-read the Kubernetes documentation page behind each wrong answer.

Week 3 — read primary sources

3

The CNCF curriculum PDF, cncf.io/projects, and the Kubernetes concepts section. Nothing second-hand.

Week 4 — two timed runs

4

Full 90-minute simulations on separate days. Book the exam only after both clear 75% comfortably.

Keep a triage card

5

The five questions, the symptom strings and the exit codes on one page. It is the most reusable thing you built this week.

Renewal, when the time comes

6

KCNA is valid for two years. CARE can renew an already-earned KCNA after a qualifying CKA or CKAD achievement or recertification. Check the current official terms.

Source: Linux Foundation, Kubernetes and Cloud Native Associate (KCNA) certification page: training.linuxfoundation.org/certification/kubernetes-cloud-native-associate/; CARE: training.linuxfoundation.org/care-program/

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT BRIEFING

This Afternoon's WSQ Assessment

Two instruments, three hours, mapped to the registered outcomes you have been working against all week.

WRITTEN ASSESSMENT (SAQ) · 3:30 – 4:30 PM

- **Format** 7 short-answer questions in 60 minutes
- **Coverage** One question per knowledge code, K1 through K7, in the registered order
- **K7 is today** Interactions among IT components — probes, Events, the metrics pipeline and triage
- **Technique** Answer the question that was asked; name the API object, field or command rather than describing it loosely

PRACTICAL PERFORMANCE (PP) · 4:30 – 6:30 PM

- **Format** 6 tasks in 120 minutes, one per learning outcome
- **Mapping** T1/A1, T2/A3, T3/A2, T4/A4, T5/A5, T6/A6. Tasks follow LO1 –LO6; LO6 covers monitoring and troubleshooting
- **Method** Work the five triage questions in order and record the phase before the cause
- **Evidence** Show the command AND its output. An assertion with no cluster evidence behind it is not competent performance

Source: course_data.ASSESSMENT, mirroring the approved legacy papers

© 2026 Tertiary Infotech Academy Pte Ltd

Afternoon Break

3:20 – 3:30 PM

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP UP

11

Course Summary and Next Steps

What you achieved this week, and where to go next.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU ACHIEVED

Five Days, Twenty-Five Labs

Map this back to the six registered learning outcomes — the assessment measures exactly these.

1 Built and read Kubernetes objects

You authored Pods, Deployments, Services, Ingresses, PVCs and NetworkPolicies by hand and can read any manifest field by field.

2 Explained the architecture

You can describe the control plane, the node plane and the reconciliation loop that connects them.

3 Operated workloads

You scaled, rolled out, rolled back, scheduled and constrained real workloads with resources and affinity.

4 Secured a cluster

You applied RBAC, ServiceAccounts, Secrets and NetworkPolicy, and can explain the 4C's model.

5 Delivered applications

You packaged with Helm and Kustomize and can explain the GitOps reconciliation model.

6 Diagnosed failures

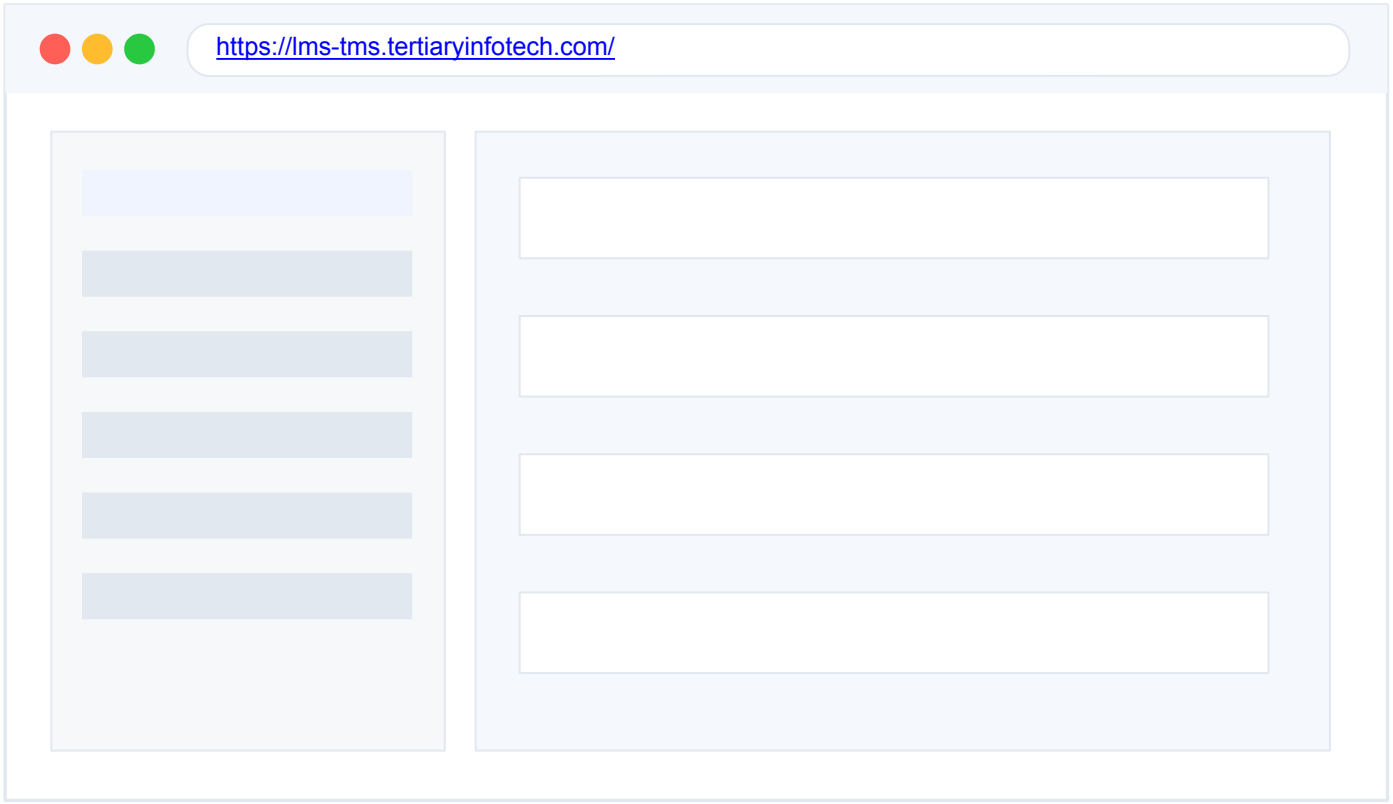
You have a repeatable triage method and have read real CrashLoopBackOff, Pending and no-endpoint failures.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

Courseware and Assessment on the LMS

The enrolled course brings the learning and assessment resources together.



RESOURCE OVERVIEW

- Current slides and Learner Guide.
- Lab resources and verification files.
- Written and practical candidate papers.
- Assessment submissions and assessor records.

https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

KCNA Practice and Readiness

The Learner Guide provides the study and resource-access instructions.



RESOURCE OVERVIEW

- Timed practice supports exam familiarity.
- The official KCNA format is multiple choice.
- Coverage follows the current CNCF blueprint.
- KCNA certification and WSQ assessment are separate.

https://exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-kcna

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT

Assessment — Final Reminder

Three hours total. Read every question fully before you start writing.

1 Written Assessment (SAQ)

7 open-ended short-answer questions. 60 minutes. Answer on the document provided.

2 Practical Performance (PP)

6 hands-on tasks against one scenario. 120 minutes. Capture evidence of each command.

3 Open book

Learner Guide, lab folders and kubernetes.io are all permitted. Your notes are permitted.

4 Take the attendance first

Complete the Assessment digital attendance before you begin, not after.

5 Submit on the LMS

Upload both answer documents before you leave the room.

6 Sign the record

Sign the Assessment Summary Record with your assessor to close the file.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT OVERVIEW

Assessment Records and Evidence

The Learner Guide contains the detailed assessment procedure.

TRAQOM feedback

1 Course quality and outcomes feedback; separate from attendance.

Attendance

2 Course participation and assessment-session records.

Assessment

3 Written Assessment: 60 minutes. Practical Performance: 120 minutes.

Submitted evidence

4 Candidate responses and practical evidence on the LMS.

Assessor record

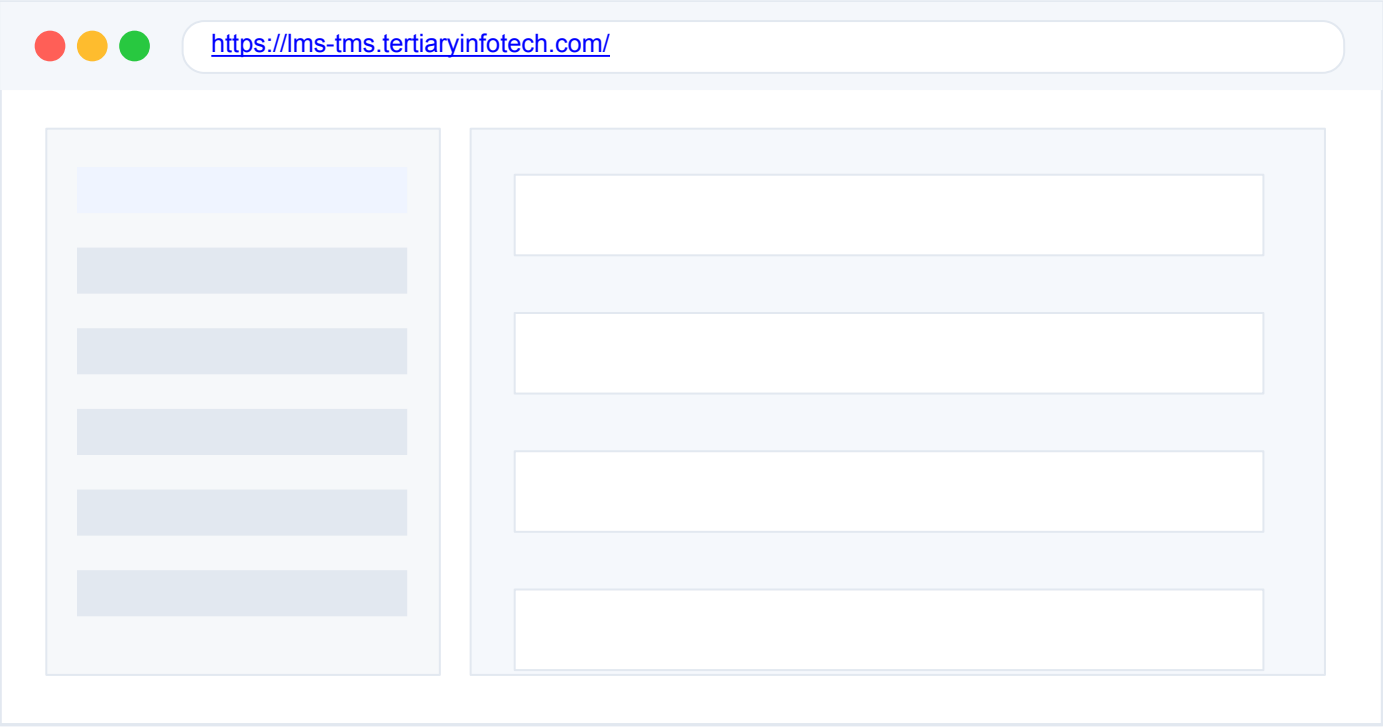
5 Competency decisions and the Assessment Summary Record.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE RESOURCES

Attendance and Feedback Records

Attendance, assessment evidence and course feedback are distinct records.



RESOURCE OVERVIEW

- Course attendance documents participation.
- Assessment attendance records the assessment session.
- TRAQOM is a learner feedback survey.
- The trainer confirms the required records for this course run.

TRAQOM: TPGateway Training Quality and Outcomes Measurement initiative.

https://lms-tms.tertiaryinfotech.com/

© 2026 Tertiary Infotech Academy Pte Ltd

Thank You

All the best for the KCNA exam — and for what you build next.

EMAIL
enquiry@tertiaryinfotech.com

PHONE
+65 6100 0613

WEB
www.tertiarycourses.com.sg

LMS
https://lms-tms.tertiaryinfotech.com/

Tertiary Infotech Academy Pte Ltd · UEN 201200696W



© 2026 Tertiary Infotech Academy Pte Ltd